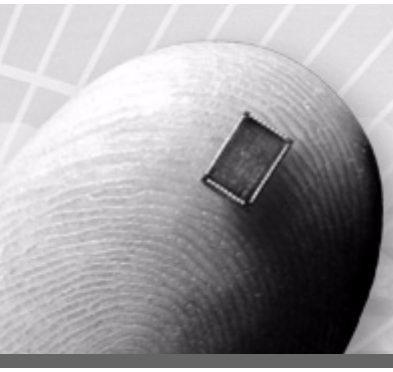
ON Semiconductor®



Hardware Reference Manual

for BelaSigna® 300

M-20741-005

ON Semiconductor

September 2011

This document may not be redistributed or transmitted in any form.

Copyright © 2011 Semiconductor Components Industries, LLC d/b/a ON Semiconductor.

ON Semiconductor and the ON logo are registered trademarks of Semiconductor Components Industries, LLC (ON Semiconductor). BELASIGNA and EZAIRO are either trademarks or registered trademarks of ON Semiconductor. All other brands, product names and company names mentioned herein may be trademarks or registered trademarks of their respective holders.

ON Semiconductor reserves the right to make changes without further notice to any products herein. ON Semiconductor makes no warranty, representation or guarantee regarding its products including suitability for any particular purpose, nor does ON Semiconductor assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. "Typical" parameters which may be provided in ON Semiconductor data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. ON Semiconductor does not convey any license under its patent rights nor the rights of others. ON Semiconductor products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the ON Semiconductor product could create a situation where personal injury or death may occur, unless otherwise expressly agreed to in writing by ON Semiconductor and BUYER. Should Buyer purchase or use ON Semiconductor products for any such unintended or unauthorized application, Buyer shall indemnify and hold ON Semiconductor and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that ON Semiconductor was negligent regarding the design or manufacture of the part. ON Semiconductor is an Equal Opportunity/Affirmative Action Employer. Literature is subject to all applicable copyright laws and is not for resale in any manner.

To maintain the quality of our publications, we request your comments on the accuracy, clarity, organization, and value of this document.

Address correspondence to:

Americas and Asia
ON Semiconductor
200-611 Kumpf Drive
Waterloo, Ontario, Canada N2V 1K8
Tel: +1 519 884 9696

Europe, Middle East and Africa
ON Semiconductor
Champs-Montants 12a
2074 Marin, Switzerland
Tel: +41 32 755 74 30

Website
www.onsemi.com

E-mail
dsp.support@onsemi.com

TABLE OF CONTENTS

1. Introduction 1

1.1 Purpose 1
1.2 Intended Audience 1
1.3 Conventions 1
1.4 Manual Organization 2
1.5 Further Reading 2

2. System Overview 5

2.1 System Architecture Introduction 5
2.2 CFX Digital Signal Processor 6
2.2.1 CFX Memory 6
2.3 HEAR Configurable Accelerator 7
2.3.1 HEAR Memory 7
2.4 FIFO Controller 7
2.5 Peripherals 8
2.6 Interfaces 8
2.7 Inputs and Outputs 8
2.7.1 Input/Output Controller (IOC) 8
2.7.2 Input Stage 9
2.7.3 Output Stage 9
2.7.4 Shared I2S and PCM Interface 9
2.8 Analog Support Blocks 9
2.8.1 Oscillator and Clocking 9
2.8.2 Power Supply and Power Management 10
2.9 Device Identification 10

3. CFX Digital Signal Processor 11

3.1 Processor Overview 11
3.2 Application Overview 11

4. HEAR Configurable Accelerator 15

4.1 Accelerator Overview 15
4.1.1 Function Chains 15
4.1.2 HEAR Control 15
4.1.3 Control and Configuration Registers 16
4.1.3.1 D_SYSTEM_CTRL Settings 16
4.2 Function Chain Controller 16
4.2.1 Control and Configuration Registers 17
4.2.1.1 D_HEAR_FC_ENABLE Settings 18
4.2.1.2 D_HEAR_FC_PRIORITY Settings 19
4.2.1.3 D_HEAR_FC_STATUS Settings 22
4.3 Block Floating-Point 23
4.3.1 Control and Configuration Registers 24
4.3.1.1 D_HEAR_BFP Settings 24

5. Memory 25

5.1 Memory Architecture 25
5.1.1 Memory Instances 26
5.1.2 Memory Bus Structures 28
5.1.3 Memory Controllers 29
5.1.3.1 Shared Memory Bus Controller 29
5.1.3.1.1 Control and Configuration Registers 29
5.1.4 Memory Interface Write Buffers 30
5.1.4.1 Buffering and Flushing of Write Operations 30
5.1.4.2 Visible Effects of Write Buffering 30
5.2 Program and Microcode Memory 31
5.2.1 P Memory Space 31
5.2.2 Program Memory (RAM and ROM) 32
5.2.3 Microcode Memory (RAM) 32
5.2.4 P Memory Space Memory Map 33
5.2.4.1 Interrupt Vector Mapping 34
5.2.4.2 Configuration Register Mapping 35
5.2.5 Program Memory Cache 35

5.3 Data Memory 36
5.3.1 CFX Data Memories (RAM and ROM) 36
5.3.2 Shared Memory (RAM and ROM) 36
5.3.3 Data Memory Space Memory Maps 38
5.3.3.1 HEAR and FIFO Configuration Register Mapping 39

6. FIFO Controller *41*

6.1 Introduction to the FIFO Controller 41
6.2 Defining Memory for Use as a FIFO 41
6.2.1 Control and Configuration Registers 43
6.3 FIFO Buffer Data and Configuration 44
6.3.1 FIFO Input and Output Block 45
6.3.2 FIFO Processing Block 46
6.3.3 Typical FIFO Configurations. 47
6.3.3.1 Input FIFO Configuration 47
6.3.3.2 Output FIFO Configuration 48
6.3.4 Control and Configuration Registers 49
6.4 Access Priority 50

7. Input/Output Controller (IOC) *51*

7.1 Introduction 51
7.2 IOC Input 51
7.2.1 Control and Configuration Registers 53
7.2.1.1 D_IOC_INPUT_CFG Settings 53
7.2.1.2 D_IOC_FIFO_CFG Settings 55
7.3 IOC Output 57
7.3.1 Control and Configuration Registers 58
7.3.1.1 D_IOC_FIFO_CFG Settings 58
7.4 IOC Data Transfer Timing 59
7.5 IOC Data Synchronization 60
7.5.1 Control and Configuration Registers 61
7.5.1.1 D_IOC_OUTPUT_CFG Settings 61

8.1 Input Stage Architecture 63
8.2 Input Selection 64
8.2.1 Input Dynamic Range Extension 65
8.2.2 Control and Configuration Registers 66
8.2.2.1 A_INPUT_CTRL Settings 66
8.2.2.2 A_IDRX0_CFG and A_IDRX1_CFG Settings 67
8.3 Preamplifiers 68
8.3.1 Control and Configuration Registers 69
8.3.1.1 A_IN_GAIN_CTRL Settings 69
8.4 Low-pass Filters 71
8.4.1 Control and Configuration Registers 71
8.4.1.1 A_ADC_CFG Settings 71
8.5 Line-out 72
8.5.1 Control and Configuration Registers 72
8.5.1.1 A_ADC_CFG Settings 72
8.6 A/D Converters 73
8.6.1 Sampling Delay 73
8.6.2 Control and Configuration Registers 74
8.6.2.1 A_ADC_CFG Settings 75
8.6.2.2 A_ADC_CUR_CTRL Settings 75
8.6.2.3 A_FRAC_DELAY_0_CTRL Settings 76
8.6.2.4 A_FRAC_DELAY_1_CTRL Settings 77
8.6.2.5 A_FRAC_DELAY_2_CTRL Settings 78
8.6.2.6 A_FRAC_DELAY_3_CTRL Settings 78
8.7 Decimation Filtering 79
8.7.1 Input Data Registers 80
8.7.2 DC Removal Filters 80
8.7.3 Multi-Chip Synchronization 80
8.7.4 Frequency-Shift Keying 81
8.7.5 Control and Configuration Registers 81
8.7.5.1 A_ADC_CFG Settings 82
8.7.5.2 A_BAND_SELECT_CTRL Settings 83

9. Output Stage 85

9.1 Output Stage Architecture 85
9.2 Interpolation Filtering 85
9.2.1 Control and Configuration Registers 86
9.2.1.1 D_IOC_OUTPUT_CFG Settings 86
9.2.1.2 A_O_DAC_GF_CTRL Settings 89
9.3 Sigma-Delta Modulator 89
9.3.1 Control and Configuration Registers 90
9.3.1.1 A_O_DAC_CTRL Settings 90
9.4 Direct-Digital Output Driver 93
9.4.1 High-Power Output Mode 93
9.4.2 Control and Configuration Registers 94
9.4.2.1 A_OUTPUT_CTRL Settings 94

10. Shared PCM and I2S Interface 97

10.1 PCM Interface Introduction 97
10.2 PCM Bus Configuration 97
10.2.1 Control and Configuration Registers 98
10.2.1.1 D_PCM_CFG Settings 98
10.3 PCM Signal Configuration 98
10.3.1 Frame Signal Configuration and Timing 99
10.3.2 Data Serial Input and Output Configuration 102
10.3.3 Control and Configuration Registers 103
10.3.3.1 D_PCM_CFG Settings 103
10.4 PCM Communications Management 105
10.4.1 PCM Device Configuration 105
10.4.2 PCM Data Management 105
10.4.3 PCM Interrupt Configuration 106
10.4.4 Control and Configuration Registers 107
10.4.4.1 D_CLK_CFG_1 Settings 107
10.4.4.2 D_PCM_CFG Settings 107
10.4.4.3 D_PCM_CTRL Settings 108
10.5 I2S Configuration and Usage 108

10.6 NOAHlink Digital Audio Streaming Configuration and Usage . 111

11. Peripherals *113*

11.1 Interrupt Controller 113
11.1.1 Control and Configuration Registers 116
11.1.1.1 D_INT_ENABLE Settings 116
11.1.1.2 D_INT_STATUS Settings 118
11.1.1.3 D_INT_PRIORITY Settings 120
11.2 General Purpose Timers 123
11.2.1 Control and Configuration Registers 124
11.2.1.1 D_SYSTEM_CTRL Settings 124
11.2.1.2 D_TIMER_1_CFG and D_TIMER_2_CFG Settings 125
11.3 Watchdog Timer 125
11.3.1 Watchdog Interrupt 126
11.3.2 Control and Configuration Registers 126
11.3.2.1 D_DSS_CTRL Settings 126
11.3.2.2 D_WATCHDOG_CFG Settings 127
11.4 Cyclic Redundancy Code (CRC) Generator 127
11.4.1 Control and Configuration Registers 128
11.5 Access Mode Security Blocks 128

12. External Interfaces *131*

12.1 Inter-IC Communications (I2C) Interfaces 131
12.1.1 Debug Port I2C Interface 133
12.1.1.1 Debug Port Configuration 134
12.1.1.2 Breakpoints 134
12.1.1.3 External Breakpoint Configuration 135
12.1.1.3.1 Data Modes 135
12.1.1.3.2 Double-Address Mode 136
12.1.1.3.3 HEAR Memory Conflict Mode 137
12.1.1.4 Debug Port Specific Control and Configuration Registers . . 137
12.1.1.4.1 D_DEBUG_CFG Settings 137
12.1.1.4.2 D_EXT_BREAK_CFG Settings 138

12.1.2 General Purpose I2C Interface 139
12.1.2.1 General Purpose I2C Interface - Slave Mode Specific Configuration 140
12.1.2.2 General Purpose I2C Interface - Master Mode Specific Configuration 141
12.1.2.3 General Purpose I2C Interrupts 142
12.1.2.3.1 Operation Using Manual Acknowledgement . . . 142
12.1.2.3.2 Operation Using Auto Acknowledgement 143
12.1.2.4 General Purpose I2C Interface Specific Control and Configuration Registers 144
12.1.2.4.1 D_I2C_CFG Settings 144
12.1.2.4.2 D_I2C_CTRL Settings 145
12.1.2.4.3 D_I2C_ADDR_START Settings 147
12.1.3 Control and Configuration Registers 147
12.1.3.1 D_I2C_PIN_CFG Settings 147
12.1.3.2 D_I2C_WATCHDOG_CFG Settings 148
12.2 General-Purpose I/O (GPIO) Interface 148
12.2.1 GPIO Pad Configuration 148
12.2.2 GPIO Interrupt 149
12.2.3 Control and Configuration Registers 150
12.2.3.1 D_GPIO_CFG Settings 150
12.2.3.2 D_GPIO_PIN_CFG Settings 151
12.3 Low-Speed Analog-to-Digital (LSAD) Converter Interface . 152
12.3.1 Control and Configuration Registers 153
12.3.1.1 A_LSAD_CTRL Settings 154
12.4 Serial Peripheral Interface (SPI) 155
12.4.1 SPI Interrupt 156
12.4.2 Control and Configuration Registers 156
12.4.2.1 D_SPI_CFG Settings 157
12.4.2.2 D_SPI_CTRL Settings 158
12.5 Universal Asynchronous Receiver-Transmitter (UART) Interface 158
12.5.1 UART Interrupts 159
12.5.2 Control and Configuration Registers 159
12.5.2.1 D_UART_CFG Settings 159

13. Clock Components 161

13.1 Clock Generation 161
13.1.1 Primary Oscillator 161
13.1.2 Standby Oscillator 162
13.1.3 Control and Configuration Registers 162
13.1.3.1 A_CLK_CTRL Settings 162
13.2 Clock Distribution 163
13.2.1 System Clock (SYSCLK) 164
13.2.2 Slow Clock (SLOWCLK) 164
13.2.3 Accelerator Clock (HEARCLK) 164
13.2.4 External Clock (EXTCLK) 165
13.2.5 User Clock (UCLK) 165
13.2.6 Control and Configuration Registers 166
13.2.6.1 D_CLK_CFG_0 Settings 166
13.2.6.2 D_CLK_CFG_1 Settings 168

14. Power Components 171

14.1 Power Supply 171
14.1.1 Battery Supply Voltage (VBAT) 172
14.1.2 Digital Output Supply Voltages (VDDO and VDDO_SPI) 172
14.1.3 Internal Band Gap Reference Voltage 172
14.1.4 Internal Digital Supply Voltage (VDDC) 173
14.1.5 Regulated Supply Voltage (VREG) and Regulated Output Supply Voltage (VREG_MIC) 173
14.1.6 Regulated Doubled Supply Voltage (VDBL) 173
14.1.7 Output Receiver Supply Voltage (VBATRCVR) 174
14.1.8 Control and Configuration Registers 174
14.1.8.1 A_PSU_CFG Settings 174
14.2 Power Management Unit 176
14.2.1 Power Supervisory 177
14.2.2 Power-On-Reset (POR) 177
14.3 Battery Monitor 178
14.3.1 Control and Configuration Registers 178

14.4 Soft Power Down Mode Tools 178
14.4.1 Control and Configuration Registers 179
14.4.1.1 A_ANALOG_PWR_CTRL Settings 179

A. Control and Configuration Registers *181*

A.1 Analog Block Registers 182
A.2 Digital Block Registers 185
A.3 HEAR Digital Block Registers 194
A.4 FIFO Configuration Registers 197

B. Sampling Frequency Reference *201*

C. Glossary *203*

This page left blank for printing purposes.

1 Introduction

1.1 PURPOSE

This manual provides a reference to the system hardware for application developers working with BELASIGNA® 300. This manual describes the components belonging to the BelaSigna 300 chip including providing information on the data processing and control components, memory components, peripherals and external interfaces. The data processing and control component information provided by this manual complements the CFX DSP description in the *CFX DSP Architecture Manual* and the HEAR description in the *HEAR Configurable Accelerator Reference Manual*. It further describes how each component of the BelaSigna 300 chip can be used in the implementation of an application and provides information for users about the configuration of the various system components.

The *Hardware Reference Manual* is a part of the BelaSigna 300 Evaluation and Development Kit (BelaSigna 300 EDK).

1.2 INTENDED AUDIENCE

This manual is primarily intended for engineers and other individuals who are responsible for developing and/or maintaining signal processing applications that make use of a BelaSigna 300 device. Individuals that are developing communications between an external device and BelaSigna 300 as well as those interested in the details of the audio components will find this manual particularly helpful as this manual focuses on the configuration and use of system components such as the HEAR Accelerator core in the implementation of a signal processing application by the CFX DSP core.

This manual assumes that a reader will have some familiarity with assembly-level programming, digital signal processing (DSP) application concepts, the design of basic audio systems and DSP tools.

1.3 CONVENTIONS

This book uses the following conventions:

- Control and configuration registers are shown in a `special font`. All control register names are prefixed with the letter “A” or the letter “D”. The letter “A” indicates that the control register is associated with control or configuration of an analog entity on the chip whereas the letter “D” indicates that the control register is associated with control or configuration of a digital entity on the chip.
- Angle brackets “<” and “>” identify an optional parameter or indicate a placeholder for specific information. To use an optional parameter or replace a placeholder, specify the information within the brackets; do not include the brackets themselves.
- Default settings for registers and bit fields are marked with an asterisk (*).
- All sample rates specified are the final decimated sample rates, unless stated otherwise.
- In general, numbers are presented in decimal notation. In cases where hexadecimal or binary notation is more convenient, these numbers are identified by the prefixes “0x” and “0b”

respectively. For example, the decimal number 123456 can also be represented as 0x1E240 or 0b11110001001000000.

1.4 MANUAL ORGANIZATION

The *Hardware Reference Manual* contains the following chapters and appendices:

- **Chapter 1: Introduction**, introduces this manual—explaining the purpose, intended audience, conventions used and organization. It also contains a list of further reading materials that provides information on other aspects of application development using BelaSigna 300.
- **Chapter 2: System Overview**, provides a high-level view of all of the hardware components of the BelaSigna 300 system and how these components interact.
- **Chapter 3: CFX Digital Signal Processor**, focusses on the interaction of the CFX DSP with the rest of the system and provides a model of data flow through the BelaSigna 300 system.
- **Chapter 4: HEAR Configurable Accelerator**, focusses on the configuration of the HEAR Accelerator using microcode and details of the hardware features of the HEAR that may need to be configured through the CFX DSP to perform a variety of signal processing tasks.
- **Chapter 5: Memory**, provides details on the various memory instances, buses and other memory structures that are associated with the CFX and HEAR cores.
- **Chapter 6: FIFO Controller**, provides details about the FIFO controller, including FIFO buffer configuration information and how the components of the BelaSigna 300 system use the FIFO buffers.
- **Chapter 7: Input/Output Controller (IOC)**, describes how to configure the IOC. This chapter also provides detailed information on the use of the IOC in acquiring input data and distributing output data through the input stage, output stage and PCM/I^2S interface.
- **Chapter 8: Input Stage**, provides information on the analog audio input stage of the BelaSigna 300 system.
- **Chapter 9: Output Stage**, provides information on the audio output stage of the BelaSigna 300 system.
- **Chapter 10: Shared PCM and I2S Interface**, provides detailed information on the shared I^2S and PCM interface that can be used by either the IOC or the CFX to move signal data into and out of the BelaSigna 300 system.
- **Chapter 11: Peripherals**, provides information about the interrupt controller, timers and other peripherals that are provided on a BelaSigna 300 device.
- **Chapter 12: External Interfaces**, provides information about the external interfaces (other than the PCM/I^2S interface) available to the CFX that may be used to pass control and configuration data between the BelaSigna 300 system and external components.
- **Chapter 13: Clock Components**, explains the configuration of all of the clocking components and distribution of clock signals through the BelaSigna 300 system.
- **Chapter 14: Power Components**, provides information about the power supply and power management features of the BelaSigna 300 system.
- **Appendix A: Control and Configuration Registers**, contains a summary table listing all of the hardware control and configuration registers and their settings.
- **Appendix B: Sampling Frequency Reference**, provides a reference lookup table for use in selecting a sampling frequency configuration for a given clock configuration and desired sampling frequency.
- **Appendix C: Glossary**, provides definitions of many of the terms defined and used throughout this document.

1.5 FURTHER READING

For more information, refer to the following documents:

- *Integrated Development Environment User's Guide*
- *Firmware Reference Manual*
- *CFX Toolchain Reference*
- *CFX DSP Architecture Manual*
- *HEAR Configurable Accelerator Reference Manual*
- *Input Dynamic Range Extension of the Ezairo 5900 and BelaSigna 300 Series*, ON Semiconductor application note 018

This page left blank for printing purposes.

2 System Overview

2.1 System Architecture Introduction

The BelaSigna 300 system is an asymmetric dual-core architecture, mixed-signal system-on-chip designed specifically for audio processing. The BelaSigna 300 system is centered around two processing cores: the CFX Digital Signal Processor (DSP) and the HEAR Configurable Accelerator. The CFX DSP Core is used to configure the system and coordinate the flow of signal data progressing through the system. The CFX DSP can also be used for custom signal processing applications that cannot be handled by the HEAR Core. The HEAR Core is a microcode configurable signal processing engine that works with the CFX DSP Core to execute a variety of signal processing functions. The HEAR is designed to perform both common signal processing operations and complex standard filterbanks such as the WOLA filterbank, reducing the loading on the CFX DSP Core.

The CFX and HEAR cores have a number of local and shared memories available to them. The two cores process data brought into system memory by the Input/Output Controller (IOC). The IOC can handle inputs from either an analog input stage or PCM interface input, and can handle outputs to a digital direct-drive output stage or PCM interface output. The FIFO controller simplifies access to data by the CFX DSP, the HEAR and the IOC by providing configurable hardware-based first-in, first-out (FIFO) buffers.

A top-level block overview of the components of the BelaSigna 300 system is provided in Figure 1. More information on the various component blocks is provided in the following subsections.

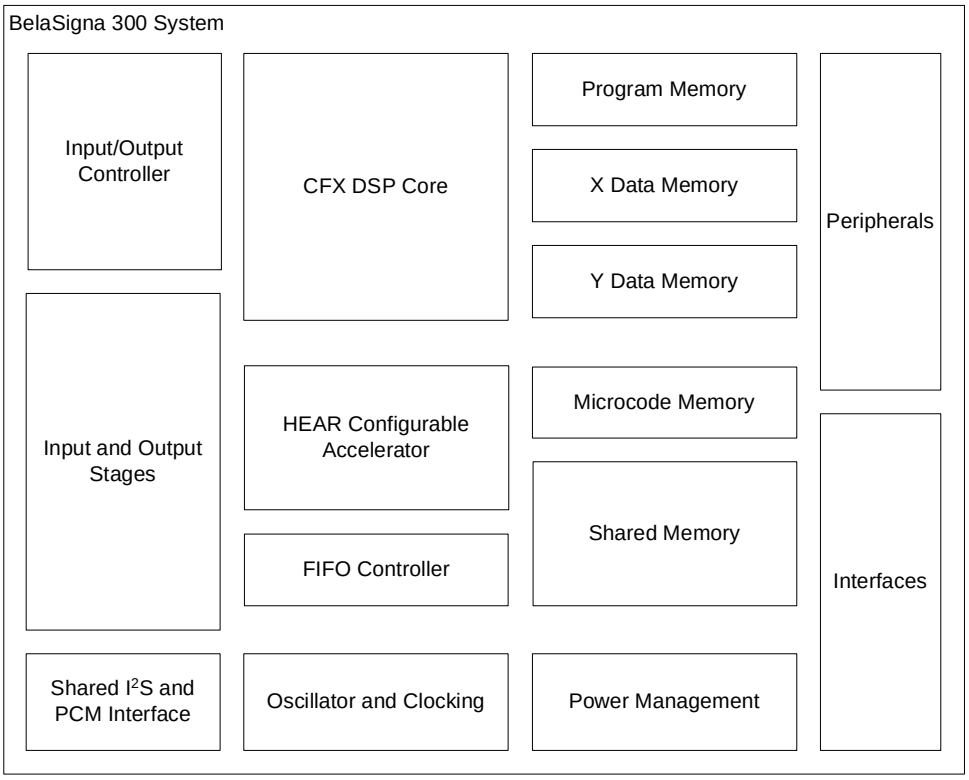


Figure 1: Top-Level System Block Diagram

2.2 CFX DIGITAL SIGNAL PROCESSOR

The CFX DSP is the primary digital processor in the system. It is a user-programmable, general-purpose DSP. The CFX serves many roles in the system: it acts as the primary controller in the system; it interacts closely with all peripherals and external interfaces; and it is used to coordinate and assist in all data processing tasks. The CFX DSP is closely tied to its local memories, but also has access to the HEAR Accelerator's memories.

More detailed information on the CFX processor is available in Chapter 3, "CFX Digital Signal Processor" on page 11 of this manual and in the *CFX DSP Architecture Manual*.

2.2.1 CFX Memory

The CFX DSP uses a dual-Harvard memory bus architecture to provide separate buses for a program memory and two data memories.

The program memory space available to the CFX includes a program ROM as well as program RAM space to store a user application and interrupt vectors. This memory is accessible as instructions to the CFX through the instruction memory bus and as data for the CFX through the P memory bus. Additionally the P memory space also provides access for the CFX to the HEAR core's microcode memory, as well as the system control and configuration registers.

The data memory areas available to the CFX are divided into two memory spaces that may be accessed either separately through the X memory and Y memory buses as 24-bit data, or together through these buses as 48-bit data in XY memory. The X memory bus also provides access for the CFX DSP to the HEAR shared memory and configuration registers.

More detailed information on the memories and associated memory buses is available in Chapter 5, “Memory” on page 25 of this manual.

2.3 HEAR CONFIGURABLE ACCELERATOR

The HEAR Configurable Accelerator provides additional digital signal processing functionality within the BelaSigna 300 system. The signal processing functionality of the HEAR is configured through a set of pre-defined microcode modules. The microcode modules provide functions that can be linked together to form function chains. These function chains are triggered through the integrated function chain controller of the HEAR via commands issued by the CFX or events from the FIFO controller.

The HEAR also has a number of other configurable components and communicates with the CFX DSP through its shared memories and CFX interrupts. The HEAR includes integrated block floating-point support for processing large dynamic range data such as subband signals.

More detailed information on the HEAR Configurable Accelerator is available in Chapter 4, “HEAR Configurable Accelerator” on page 15 of this manual and in the *HEAR Configurable Accelerator Reference Manual*.

2.3.1 HEAR Memory

The HEAR Accelerator core uses a Harvard memory bus architecture that provides separate memories for the microcode modules and data.

Microcode memory contains the user-configured microcode modules for the HEAR. This memory is accessible from the CFX processor through the P memory bus.

The HEAR shared memory is actually a set of distinct physical memories that are used to store temporary processing data, algorithm coefficients and signal data at all points during its processing (from input, to partially processed signal data, to output). This memory is provided to the CFX DSP through the X memory bus and is controlled by the shared memory bus controller. Access to some portions of this shared memory are further controlled by the FIFO controller.

More detailed information on the microcode and shared memories is available in Chapter 4, “HEAR Configurable Accelerator” on page 15 of this manual. Additional information about the FIFO controller and how it affects memory accesses is available in Chapter 6, “FIFO Controller” on page 41.

2.4 FIFO CONTROLLER

The FIFO controller controls access to the subset of shared memory that can be configured for use as FIFO buffers. This controller has three purposes in managing access to shared memory:

- It remaps portions of the address space so that a hardware circular buffer is implemented, and the most recent samples in the buffer can always be accessed at the same logical address.

- It resolves access conflicts between the input/output controller (IOC) and the two system cores that access these memory areas through the shared memory bus.
- It tracks incoming data to a buffer or outgoing data from a buffer. Changes to these buffers can trigger function chains on the HEAR or interrupts on the CFX.

More detailed information configuring FIFOs and using the FIFO controller is available in Chapter 6, “FIFO Controller” on page 41.

2.5 PERIPHERALS

The BelaSigna 300 system is provided with a number of peripherals to support various tasks that the system needs to perform. Each of these peripherals are controlled through the CFX DSP. The system peripherals include:

- An interrupt controller
- 2 general-purpose timers
- A watchdog timer
- A CRC calculation unit
- System security blocks that are used to restrict external access to application code and data

More detailed information on configuring and using the peripherals is available in Chapter 11, “Peripherals” on page 113 of this manual.

2.6 INTERFACES

The BelaSigna 300 system provides a number of external interfaces which can be controlled through the CFX processor. These interfaces provide support for a variety of types of external communications, as well as signalling through general-purpose input/output (GPIO) pads and low-speed analog-to-digital (LSAD) converters. The external interfaces provided are:

- An I^2C debug interface
- An I^2C general-purpose interface
- A SPI interface
- A UART interface
- 5 LSAD inputs
- 5 GPIO pads

More detailed information on configuring and using these external interfaces is available in Chapter 12, “External Interfaces” on page 131 of this manual.

2.7 INPUTS AND OUTPUTS

2.7.1 Input/Output Controller (IOC)

The Input/Output Controller (IOC) controls audio signal data input and output from the system. The IOC can be directed to acquire input data from any combination of the four input A/D channels belonging to the input stage and the two shared PCM/I^2S input channels. The IOC is also used to direct output data, sending the data to a combination of the single output channel and two output shared PCM/I^2S channels. The IOC routes this data to and from the shared memory via

the FIFO controller. Input data from and output data to the IOC are grouped into some combination of single-channel and multi-channel FIFO buffers.

More detailed information on the configuration and operation of the IOC can be found in Chapter 7, “Input/Output Controller (IOC)” on page 51.

2.7.2 Input Stage

The audio input stage on BelaSigna 300 provides four input channels multiplexed from four (CID 201) or five (CID 102) analog inputs. Each of these input channels uses a number of components including a preamplifier, several filters and a 16-bit oversampling sigma-delta A/D converter to provide audio input data to the IOC for use by the rest of the BelaSigna 300 system.

More detailed information on the configuration of the input stage and using the input stage in the acquisition of data can be found in Chapter 8, “Input Stage” on page 63.

2.7.3 Output Stage

The audio output stage on BelaSigna 300 provides a single output channel. This output channel uses a 16-bit oversampling sigma-delta D/A converter as the source for a direct digital output which may be used to drive zero-bias hearing aid receivers.

More detailed information on the configuration of the output stage and the distribution of data through the output stage can be found in Chapter 9, “Output Stage” on page 85.

2.7.4 Shared I^2S and PCM Interface

The BelaSigna 300 system includes a shared I^2S and PCM interface that is designed to allow streaming of digitized signal data directly into or out of the system through either the IOC or CFX core. This interface may be configured to communicate with any I^2S device, many PCM devices and the NOAHlink (audio streaming) device.

More detailed configuration and use information can be found in Chapter 10, “Shared PCM and I2S Interface” on page 97.

2.8 ANALOG SUPPORT BLOCKS

2.8.1 Oscillator and Clocking

The BelaSigna 300 system defaults to supplying a clock signal to the system from an external clock (CID 201) or from an on-chip oscillator (CID 102). The system provides an on-chip oscillator supporting system clock frequencies up to 10.24MHz, and a standby oscillator that can also be used to clock the system. Additionally, the BelaSigna 300 system contains both analog and digital components that allow for the selection of other clock sources as the clock for the system, and provide a number of clock dividers that are used to produce a set of derived clock domains needed by the various components of the BelaSigna 300 system.

Detailed information on the operation and configuration of the oscillator and clocking components can be found in Chapter 13, “Clock Components” on page 161.

2.8.2 Power Supply and Power Management

The BelaSigna 300 system includes integrated power supply and power management components. These components supply power to the system and cover all configuration aspects of managing power consumption through regular and stand-by power modes. These components also provide a regulated power supply to system components that require voltage regulation, monitor the power supplied to the system as a whole, and provide soft power-down tools to limit power consumption by the device.

Details on the power supply and power management features can be found in Chapter 14, "Power Components" on page 171.

2.9 DEVICE IDENTIFICATION

The BelaSigna 300 device provides a number of registers that contain static values, which can be used to identify a device and provide version information about the device when read. These registers can be used to verify compatibility for both user applications and external applications communicating using the debug port. A list of these registers along with a brief description of each are in Table 1.

TABLE 1: DEVICE IDENTIFICATION REGISTERS

Register	Description	Address
D_CHIP_FAMILY	The chip technology family to which the device belongs.	0xE030
D_CHIP_VERSION	Version number for the device. Used to indicate major updates which might not be backwards compatible. BelaSigna 300 devices use a chip version of 0x01 or 0x02.	0xE031
D_CHIP_REVISION	Revision number for the device. • An update of the minor revision number (bits [7:0]) indicates minor backwards-compatible or non-functional updates to the system that require no update to source code or binary objects. • An update of the major revision number (bits [15:8]) indicates updates that could affect source code or binary objects, and thus could require firmware library or software updates. When the major revision number is updated, the minor revision number is reset.	0xE032
D_DEBUG_VERSION	Version number of the debug port communications protocol. Used to indicate which communication options are available for the device.	0xE033

The chip version is commonly written as X.YY.ZZ (for example, 1.02.03), where X is the chip version number, YY is the major revision number, and ZZ is the minor revision number. Devices are also identified using just the chip version number and major revision number as a chip identifier (CID) value. For a chip version of X.YY.ZZ, the CID would be XYY (for example, 102).

3 CFX Digital Signal Processor

3.1 Processor Overview

The CFX DSP Core is a programmable, pipelined DSP that uses a dual-Harvard, dual-MAC architecture to efficiently process 24-bit signal data. For information on the hardware features of this processor and details of the CFX instruction set, refer to the *CFX DSP Architecture Manual*.

The CFX also plays a role as the central controller for the BelaSigna 300 system. As the only programmable processing component in the system it provides users with an interface for configuration and control of all of the other system components. Following a power-on reset (POR), the system starts executing the Boot ROM on the CFX DSP using this functionality to initialize a system application. It is also closely coupled to many of the memory components, and as a result, has access to all of the signal data that is being processed by the system.

All applications executing on the CFX DSP Core can be debugged through the debug controller. It provides access to all the CFX registers and the memory available through the memory buses attached to the CFX DSP. This transparent view of the system enables a user to review the current state of system components through their associated memory mapped control registers, and use this information to troubleshoot most issues in their application. Further details on the debug controller and its interaction with the CFX DSP can be found in Section 12.1.1, “Debug Port I2C Interface” on page 133.

3.2 Application Overview

The CFX DSP typically contains an application that implements a signal processing algorithm and uses one or more external interfaces for communications and/or control.

As part of implementing a signal processing algorithm the CFX DSP will perform a number of tasks. These tasks include:

- Configuration of the IOC, PCM/I^2S interface and input stage to obtain input signal information
- Configuration of the IOC, PCM/I^2S interface and output stage to distribute output signal information
- Configuration of FIFO buffers for use in handling input, output and intermediate data
- Configuration of the HEAR Configurable Accelerator by setting configuration registers and loading microcode to perform regular signal processing algorithms on the available data
- Execution of custom signal processing algorithms
- Communication with and/or signalling external devices through the debug port and other external interfaces.

Figure 2 provides an overview of possible paths that signal data will typically follow through the BelaSigna 300 system.

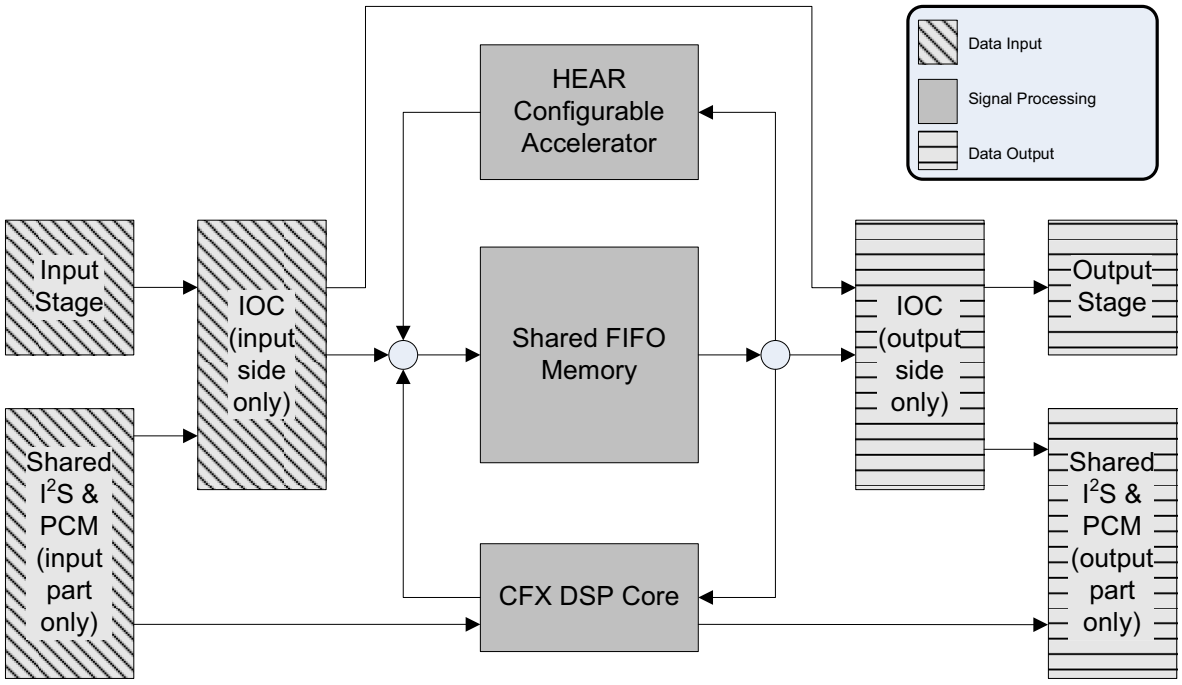


Figure 2: Signal Data Flow in a Typical CFX Digital Signal Processor Controlled Application

Signal data flow can be divided into three steps as shown in Figure 2: data input, signal processing and data output.

Data Input For input, the signal data to be processed will enter the system either through the input stage or the shared I^2S and PCM interface to the system. This data can be processed through the input side of the IOC directly to FIFOs in shared memory, or in the case of the PCM/I^2S interface can be passed directly to the CFX DSP through PCM interface data registers.

Signal Processing The signal data stored in the FIFOs in shared memory can then be processed through any number of data processing steps through functions implemented for the CFX processor or function chains available through the microcode configuration supplied for the HEAR. After each processing step the data will typically be written back to either an intermediate or output FIFO buffer.

Data Output Once all of the desired data processing has been performed, the data will be distributed from the system either directly through the shared PCM/I^2S interface or through the output side of the IOC. As shown, the IOC can then direct this output information either to the PCM/I^2S interface or the output stage.

Along with the implementation of the processing algorithm itself, the CFX processor can use communications over the interfaces and sensed data to update algorithm coefficients and even

algorithm behavior. For example the CFX might have a signal attached to an LSAD pin that is used to control the gain, and hence the output volume, of the output signal provided by the system.

Finally, the CFX DSP can configure the support components and/or use the external interfaces to send and receive control, configuration and signal data. These components of an application might be used for a variety of things, from supplying information to an external device to configuring the clock and power components. This portion of an application is usually highly specific based on the overall product needs and as a result, this document does not attempt to provide a generic view of this application component.

This page left blank for printing purposes.

4 HEAR Configurable Accelerator

4.1 Accelerator Overview

The HEAR Configurable Accelerator is a core that provides additional signal processing capabilities for the BelaSigna 300 system. Unlike the CFX DSP, the HEAR is not programmable, does not interact directly with any external interfaces, and does not include direct debugging access for applications that make use of the core. Although you cannot debug the HEAR, the debug controller can view some information on the HEAR through memories available on the memory buses attached to the CFX.

Information on the HEAR's hardware features as well as available microcode modules and how to configure the HEAR Configurable Accelerator to work with a given microcode configuration may be found in the *HEAR Configurable Accelerator Reference Manual*. Further hardware details regarding configuring the function chain controller can be found in Section 4.2, "Function Chain Controller" and further details on configuring the block floating-point feature that may be used with some microcode configurations can be found in Section 4.3, "Block Floating-Point".

4.1.1 Function Chains

The HEAR is a configurable 24-bit processing unit that runs data processing functions defined in HEAR microcode configurations. These functions, which can be used to process signal data within the system, are linked together into sequences of one or more functions known as microcode function chains. These function chains are then executed by the HEAR coprocessor through triggered events such as a command sent by the CFX DSP or an interrupt from one of the configured FIFOs. These chains are typically used to process data stored in shared memory: usually an Hx memory instance, an input FIFO or an intermediate FIFO. Output from these chains is typically stored to another location in shared memory: usually another Hx memory instance, an output FIFO or an intermediate FIFO.

Upon the completion of any function or at the end of a complete function chain, the HEAR processor can send the CFX processor an interrupt through one of eight HEAR interrupts. These interrupts can be used to notify the CFX that data is ready to be used for additional processing or to signal that the CFX can issue another HEAR command.

4.1.2 HEAR Control

To pause and resume the HEAR processing, set the `SYSTEM_CTRL_HEAR_PAUSE` and `SYSTEM_CTRL_HEAR_RESUME` bits respectively in the `D_SYSTEM_CTRL` register. When the HEAR is paused the accelerator clock is stopped, preventing the execution of any functions and all responses to the function chain controller. All uncompleted accesses to shared memory by the HEAR are put into a non-blocking wait state and these will be completed once processing resumes on the HEAR. While the HEAR is paused the CFX core and IOC can access shared memory at any addresses without additional delay being introduced by the HEAR Accelerator.

As bits in the `D_SYSTEM_CTRL` register only change state when set, a user does not need to worry about the state of the other bits of this register when controlling the HEAR. However, the behavior of the system is undefined if both of these bits are set by the same instruction. Reading the

SYSTEM_CTRL_HEAR_RUNNING bit (which shares a bit position with the SYSTEM_CTRL_HEAR_RESUME bit) provides the user with the current status of the HEAR.

IMPORTANT: While the HEAR is paused, function chain triggers are still collected in the D_HEAR_FC_STATUS register. When processing on the HEAR is resumed, these triggers will cause their associated function chains to be executed as described in Section 4.2, "Function Chain Controller".

The HEAR can also be reset by setting the SYSTEM_CTRL_HEAR_RESET bit in the D_SYSTEM_CTRL register. This resets all of the HEAR core, including the block-floating point unit and the function chain controller.

IMPORTANT: The HEAR reset additionally resets the FIFO controllers, clearing the current FIFO register settings.

4.1.3 Control and Configuration Registers

Register Name	Register Description	Address
D_SYSTEM_CTRL	System Control	0xE000

4.1.3.1 D_SYSTEM_CTRL Settings

Bit Field	Field Name	Field Description
5	SYSTEM_CTRL_HEAR_PAUSE	Pause processing on the HEAR Configurable Accelerator
4	SYSTEM_CTRL_RESUME	Resume processing by the HEAR Configurable Accelerator
4	SYSTEM_CTRL_RUNNING	Check the status of the HEAR Configurable Accelerator
3	SYSTEM_CTRL_RESET	Reset the HEAR Configurable Accelerator

Field Name	Value Symbol	Value Description	Hex Value
SYSTEM_CTRL_HEAR_PAUSE	SYSTEM_HEAR_PAUSE	Pause processing on the HEAR	0x1
SYSTEM_CTRL_RESUME	SYSTEM_HEAR_RESUME	Resume processing on the HEAR	0x1
SYSTEM_CTRL_RESET	SYSTEM_HEAR_RESET	Reset the HEAR	0x1

4.2 FUNCTION CHAIN CONTROLLER

The CFX DSP configures the function chain controller of the HEAR to control the execution of the HEAR's configurable microcode function chains.

Function chains are typically associated with triggers that can be used to execute them. The function chain controller is directed to trigger specific function chains within a microcode configuration by setting bits in the D_HEAR_FC_ENABLE register.

The CFX DSP sends commands to start the execution of one or more of the HEAR function chains by setting the appropriate bit or bits in the D_HEAR_CMDS_0_TO_7 and D_HEAR_CMDS_8_TO_15 registers. The processing of function chains can also be triggered by FIFO interrupts in the system. In both cases, the microcode functions to be executed in the function chain depend on the microcode configuration used by the user application. Each function chain has an entry in a vector table stored at D_HEAR_FC_VECT_BASE located in shared memory at address 0xBFE0 in the X memory address space. These function chain vector table entries can be modified during an application's execution to change which function chains are mapped to the CFX commands and FIFO interrupts. More information on the HEAR microcode, function chains and possible microcode configurations are available in the *HEAR Configurable Accelerator Reference Manual*.

Setting the corresponding bit in the D_HEAR_FC_PRIORITY register prioritizes enabled function chains as either normal or high priority. This allows a user application to select the function chains to process first if more than one function chain is pending when the function chain controller initiates the processing of a function chain. If multiple function chains are pending, all high priority function chains are handled first. If multiple pending function chains share the same priority level, then function chains are executed with the following precedence:

1. Function chains triggered by CFX commands, starting with the lowest numbered command
2. Function chains triggered by FIFO A interrupts, starting with the lowest numbered FIFO
3. Function chains triggered by FIFO B interrupts, starting with the lowest numbered FIFO

The processing status of function chains can be observed through the D_HEAR_FC_STATUS register. The bit corresponding to a particular enabled function chain is set whenever the execution of the function chain is pending.

IMPORTANT: Writing to the D_HEAR_FC_STATUS register might result in undefined application behavior.

4.2.1 Control and Configuration Registers

Register Name	Register Description	Address
D_HEAR_FC_ENABLE	Enable triggers for the HEAR function chains that will be used in an application	0x9F80
D_HEAR_FC_PRIORITY	Sets priority of HEAR function chain configurations	0x9F81
D_HEAR_FC_STATUS	Indicate which HEAR function chain configurations have been triggered and are pending execution	0x9F87
D_HEAR_CMDS_0_TO_7	Issue a command triggering the execution of one or more HEAR function chains (this register can be used to issue commands for function chains 0 to 7 by setting the corresponding appropriate bits in the range 0 to 7)	0xE004
D_HEAR_CMDS_8_TO_15	Issue a command triggering the execution of one or more HEAR function chains (this register can be used to issue commands for function chains 8 to 15 by setting the appropriate bits in the range 0 to 7)	0xE005

4.2.1.1 D_HEAR_FC_ENABLE Settings

Bit Field	Field Name	Field Description
23	HEAR_FC_FIFO_B3	Enable/disable the function chain for FIFO B3
22	HEAR_FC_FIFO_B2	Enable/disable the function chain for FIFO B2
21	HEAR_FC_FIFO_B1	Enable/disable the function chain for FIFO B1
20	HEAR_FC_FIFO_B0	Enable/disable the function chain for FIFO B0
19	HEAR_FC_FIFO_A3	Enable/disable the function chain for FIFO A3
18	HEAR_FC_FIFO_A2	Enable/disable the function chain for FIFO A2
17	HEAR_FC_FIFO_A1	Enable/disable the function chain for FIFO A1
16	HEAR_FC_FIFO_A0	Enable/disable the function chain for FIFO A0
15	HEAR_FC_CMD15	Enable/disable the function chain for CFX command 15
14	HEAR_FC_CMD14	Enable/disable the function chain for CFX command 14
13	HEAR_FC_CMD13	Enable/disable the function chain for CFX command 13
12	HEAR_FC_CMD12	Enable/disable the function chain for CFX command 12
11	HEAR_FC_CMD11	Enable/disable the function chain for CFX command 11
10	HEAR_FC_CMD10	Enable/disable the function chain for CFX command 10
9	HEAR_FC_CMD9	Enable/disable the function chain for CFX command 9
8	HEAR_FC_CMD8	Enable/disable the function chain for CFX command 8
7	HEAR_FC_CMD7	Enable/disable the function chain for CFX command 7
6	HEAR_FC_CMD6	Enable/disable the function chain for CFX command 6
5	HEAR_FC_CMD5	Enable/disable the function chain for CFX command 5
4	HEAR_FC_CMD4	Enable/disable the function chain for CFX command 4
3	HEAR_FC_CMD3	Enable/disable the function chain for CFX command 3
2	HEAR_FC_CMD2	Enable/disable the function chain for CFX command 2
1	HEAR_FC_CMD1	Enable/disable the function chain for CFX command 1
0	HEAR_FC_CMD0	Enable/disable the function chain for CFX command 0

Field Name	Value Symbol	Value Description	Hex Value
HEAR_FC_FIFO_B3	HEAR_FC_EBL_FIFO_B3	Enable the FIFO B3 function chain	0x1
HEAR_FC_FIFO_B2	HEAR_FC_EBL_FIFO_B2	Enable the FIFO B2 function chain	0x1
HEAR_FC_FIFO_B1	HEAR_FC_EBL_FIFO_B1	Enable the FIFO B1 function chain	0x1
HEAR_FC_FIFO_B0	HEAR_FC_EBL_FIFO_B0	Enable the FIFO B0 function chain	0x1
HEAR_FC_FIFO_A3	HEAR_FC_EBL_FIFO_A3	Enable the FIFO A3 function chain	0x1
HEAR_FC_FIFO_A2	HEAR_FC_EBL_FIFO_A2	Enable the FIFO A2 function chain	0x1
HEAR_FC_FIFO_A1	HEAR_FC_EBL_FIFO_A1	Enable the FIFO A1 function chain	0x1

Field Name	Value Symbol	Value Description	Hex Value
HEAR_FC_FIFO_A0	HEAR_FC_EBL_FIFO_A0	Enable the FIFO A0 function chain	0x1
HEAR_FC_CMD15	HEAR_FC_EBL_CMD15	Enable the CFX 15 function chain	0x1
HEAR_FC_CMD14	HEAR_FC_EBL_CMD14	Enable the CFX 14 function chain	0x1
HEAR_FC_CMD13	HEAR_FC_EBL_CMD13	Enable the CFX 13 function chain	0x1
HEAR_FC_CMD12	HEAR_FC_EBL_CMD12	Enable the CFX 12 function chain	0x1
HEAR_FC_CMD11	HEAR_FC_EBL_CMD11	Enable the CFX 11 function chain	0x1
HEAR_FC_CMD10	HEAR_FC_EBL_CMD10	Enable the CFX 10 function chain	0x1
HEAR_FC_CMD9	HEAR_FC_EBL_CMD9	Enable the CFX 9 function chain	0x1
HEAR_FC_CMD8	HEAR_FC_EBL_CMD8	Enable the CFX 8 function chain	0x1
HEAR_FC_CMD7	HEAR_FC_EBL_CMD7	Enable the CFX 7 function chain	0x1
HEAR_FC_CMD6	HEAR_FC_EBL_CMD6	Enable the CFX 6 function chain	0x1
HEAR_FC_CMD5	HEAR_FC_EBL_CMD5	Enable the CFX 5 function chain	0x1
HEAR_FC_CMD4	HEAR_FC_EBL_CMD4	Enable the CFX 4 function chain	0x1
HEAR_FC_CMD3	HEAR_FC_EBL_CMD3	Enable the CFX 3 function chain	0x1
HEAR_FC_CMD2	HEAR_FC_EBL_CMD2	Enable the CFX 2 function chain	0x1
HEAR_FC_CMD1	HEAR_FC_EBL_CMD1	Enable the CFX 1 function chain	0x1
HEAR_FC_CMD0	HEAR_FC_EBL_CMD0	Enable the CFX 0 function chain	0x1

4.2.1.2 D_HEAR_FC_PRIORITY Settings

Bit Field	Field Name	Field Description
23	HEAR_FC_FIFO_B3	Select the priority of the function chain for FIFO B3
22	HEAR_FC_FIFO_B2	Select the priority of the function chain for FIFO B2
21	HEAR_FC_FIFO_B1	Select the priority of the function chain for FIFO B1
20	HEAR_FC_FIFO_B0	Select the priority of the function chain for FIFO B0
19	HEAR_FC_FIFO_A3	Select the priority of the function chain for FIFO A3
18	HEAR_FC_FIFO_A2	Select the priority of the function chain for FIFO A2
17	HEAR_FC_FIFO_A1	Select the priority of the function chain for FIFO A1
16	HEAR_FC_FIFO_A0	Select the priority of the function chain for FIFO A0
15	HEAR_FC_CMD15	Select the priority of the function chain for CFX command 15
14	HEAR_FC_CMD14	Select the priority of the function chain for CFX command 14
13	HEAR_FC_CMD13	Select the priority of the function chain for CFX command 13
12	HEAR_FC_CMD12	Select the priority of the function chain for CFX command 12
11	HEAR_FC_CMD11	Select the priority of the function chain for CFX command 11
10	HEAR_FC_CMD10	Select the priority of the function chain for CFX command 10

Bit Field	Field Name	Field Description
9	HEAR_FC_CMD9	Select the priority of the function chain for CFX command 9
8	HEAR_FC_CMD8	Select the priority of the function chain for CFX command 8
7	HEAR_FC_CMD7	Select the priority of the function chain for CFX command 7
6	HEAR_FC_CMD6	Select the priority of the function chain for CFX command 6
5	HEAR_FC_CMD5	Select the priority of the function chain for CFX command 5
4	HEAR_FC_CMD4	Select the priority of the function chain for CFX command 4
3	HEAR_FC_CMD3	Select the priority of the function chain for CFX command 3
2	HEAR_FC_CMD2	Select the priority of the function chain for CFX command 2
1	HEAR_FC_CMD1	Select the priority of the function chain for CFX command 1
0	HEAR_FC_CMD0	Select the priority of the function chain for CFX command 0

Field Name	Value Symbol	Value Description	Hex Value
HEAR_FC_FIFO_B3	HEAR_FC_HIGHPRIO_FIFO_B3	FIFO B3 function chain is high priority	0x1
	HEAR_FC_NORMALPRIO_FIFO_B3	FIFO B3 function chain is normal priority	0x0*
HEAR_FC_FIFO_B2	HEAR_FC_HIGHPRIO_FIFO_B2	FIFO B2 function chain is high priority	0x1
	HEAR_FC_NORMALPRIO_FIFO_B2	FIFO B2 function chain is normal priority	0x0*
HEAR_FC_FIFO_B1	HEAR_FC_HIGHPRIO_FIFO_B1	FIFO B1 function chain is high priority	0x1
	HEAR_FC_NORMALPRIO_FIFO_B1	FIFO B1 function chain is normal priority	0x0*
HEAR_FC_FIFO_B0	HEAR_FC_HIGHPRIO_FIFO_B0	FIFO B0 function chain is high priority	0x1
	HEAR_FC_NORMALPRIO_FIFO_B0	FIFO B0 function chain is normal priority	0x0*
HEAR_FC_FIFO_A3	HEAR_FC_HIGHPRIO_FIFO_A3	FIFO A3 function chain is high priority	0x1
	HEAR_FC_NORMALPRIO_FIFO_A3	FIFO A3 function chain is normal priority	0x0*
HEAR_FC_FIFO_A2	HEAR_FC_HIGHPRIO_FIFO_A2	FIFO A2 function chain is high priority	0x1
	HEAR_FC_NORMALPRIO_FIFO_A2	FIFO A2 function chain is normal priority	0x0*

Field Name	Value Symbol	Value Description	Hex Value
HEAR_FC_FIFO_A1	HEAR_FC_HIGHPRIO_FIFO_A1	FIFO A1 function chain is high priority	0x1
	HEAR_FC_NORMALPRIO_FIFO_A1	FIFO A1 function chain is normal priority	0x0*
HEAR_FC_FIFO_A0	HEAR_FC_HIGHPRIO_FIFO_A0	FIFO A0 function chain is high priority	0x1
	HEAR_FC_NORMALPRIO_FIFO_A0	FIFO A0 function chain is normal priority	0x0*
HEAR_FC_CMD15	HEAR_FC_HIGHPRIO_CMD15	CFX command 15 function chain is high priority	0x1
	HEAR_FC_NORMALPRIO_CMD15	CFX command 15 function chain is normal priority	0x0*
HEAR_FC_CMD14	HEAR_FC_HIGHPRIO_CMD14	CFX command 14 function chain is high priority	0x1
	HEAR_FC_NORMALPRIO_CMD14	CFX command 14 function chain is normal priority	0x0*
HEAR_FC_CMD13	HEAR_FC_HIGHPRIO_CMD13	CFX command 13 function chain is high priority	0x1
	HEAR_FC_NORMALPRIO_CMD13	CFX command 13 function chain is normal priority	0x0*
HEAR_FC_CMD12	HEAR_FC_HIGHPRIO_CMD12	CFX command 12 function chain is high priority	0x1
	HEAR_FC_NORMALPRIO_CMD12	CFX command 12 function chain is normal priority	0x0*
HEAR_FC_CMD11	HEAR_FC_HIGHPRIO_CMD11	CFX command 11 function chain is high priority	0x1
	HEAR_FC_NORMALPRIO_CMD11	CFX command 11 function chain is normal priority	0x0*
HEAR_FC_CMD10	HEAR_FC_HIGHPRIO_CMD10	CFX command 10 function chain is high priority	0x1
	HEAR_FC_NORMALPRIO_CMD10	CFX command 10 function chain is normal priority	0x0*
HEAR_FC_CMD9	HEAR_FC_HIGHPRIO_CMD9	CFX command 9 function chain is high priority	0x1
	HEAR_FC_NORMALPRIO_CMD9	CFX command 9 function chain is normal priority	0x0*
HEAR_FC_CMD8	HEAR_FC_HIGHPRIO_CMD8	CFX command 8 function chain is high priority	0x1
	HEAR_FC_NORMALPRIO_CMD8	CFX command 8 function chain is normal priority	0x0*

Field Name	Value Symbol	Value Description	Hex Value
HEAR_FC_CMD7	HEAR_FC_HIGHPRIO_CMD7	CFX command 7 function chain is high priority	0x1
	HEAR_FC_NORMALPRIO_CMD7	CFX command 7 function chain is normal priority	0x0*
HEAR_FC_CMD6	HEAR_FC_HIGHPRIO_CMD6	CFX command 6 function chain is high priority	0x1
	HEAR_FC_NORMALPRIO_CMD6	CFX command 6 function chain is normal priority	0x0*
HEAR_FC_CMD5	HEAR_FC_HIGHPRIO_CMD5	CFX command 5 function chain is high priority	0x1
	HEAR_FC_NORMALPRIO_CMD5	CFX command 5 function chain is normal priority	0x0*
HEAR_FC_CMD4	HEAR_FC_HIGHPRIO_CMD4	CFX command 4 function chain is high priority	0x1
	HEAR_FC_NORMALPRIO_CMD4	CFX command 4 function chain is normal priority	0x0*
HEAR_FC_CMD3	HEAR_FC_HIGHPRIO_CMD3	CFX command 3 function chain is high priority	0x1
	HEAR_FC_NORMALPRIO_CMD3	CFX command 3 function chain is normal priority	0x0*
HEAR_FC_CMD2	HEAR_FC_HIGHPRIO_CMD2	CFX command 2 function chain is high priority	0x1
	HEAR_FC_NORMALPRIO_CMD2	CFX command 2 function chain is normal priority	0x0*
HEAR_FC_CMD1	HEAR_FC_HIGHPRIO_CMD1	CFX command 1 function chain is high priority	0x1
	HEAR_FC_NORMALPRIO_CMD1	CFX command 1 function chain is normal priority	0x0*
HEAR_FC_CMD0	HEAR_FC_HIGHPRIO_CMD0	CFX command 0 function chain is high priority	0x1
	HEAR_FC_NORMALPRIO_CMD0	CFX command 0 function chain is normal priority	0x0*

4.2.1.3 D_HEAR_FC_STATUS Settings

Bit Field	Field Name	Field Description
23	HEAR_FC_FIFO_B3	The function chain for FIFO B3 is pending execution
22	HEAR_FC_FIFO_B2	The function chain for FIFO B2 is pending execution
21	HEAR_FC_FIFO_B1	The function chain for FIFO B1 is pending execution
20	HEAR_FC_FIFO_B0	The function chain for FIFO B0 is pending execution
19	HEAR_FC_FIFO_A3	The function chain for FIFO A3 is pending execution

Bit Field	Field Name	Field Description
18	HEAR_FC_FIFO_A2	The function chain for FIFO A2 is pending execution
17	HEAR_FC_FIFO_A1	The function chain for FIFO A1 is pending execution
16	HEAR_FC_FIFO_A0	The function chain for FIFO A0 is pending execution
15	HEAR_FC_CMD15	The function chain for CFX command 15 is pending execution
14	HEAR_FC_CMD14	The function chain for CFX command 14 is pending execution
13	HEAR_FC_CMD13	The function chain for CFX command 13 is pending execution
12	HEAR_FC_CMD12	The function chain for CFX command 12 is pending execution
11	HEAR_FC_CMD11	The function chain for CFX command 11 is pending execution
10	HEAR_FC_CMD10	The function chain for CFX command 10 is pending execution
9	HEAR_FC_CMD9	The function chain for CFX command 9 is pending execution
8	HEAR_FC_CMD8	The function chain for CFX command 8 is pending execution
7	HEAR_FC_CMD7	The function chain for CFX command 7 is pending execution
6	HEAR_FC_CMD6	The function chain for CFX command 6 is pending execution
5	HEAR_FC_CMD5	The function chain for CFX command 5 is pending execution
4	HEAR_FC_CMD4	The function chain for CFX command 4 is pending execution
3	HEAR_FC_CMD3	The function chain for CFX command 3 is pending execution
2	HEAR_FC_CMD2	The function chain for CFX command 2 is pending execution
1	HEAR_FC_CMD1	The function chain for CFX command 1 is pending execution
0	HEAR_FC_CMD0	The function chain for CFX command 0 is pending execution

4.3 BLOCK FLOATING-POINT

The HEAR Configurable Accelerator supports the use of a block floating-point data type for use in data calculations. Use of this feature of the HEAR allows:

1. The detection of the maximum magnitude of a block of data
2. Shifting of data by a previous magnitude factor
3. Storing of the total shift amount (referred to as the block exponent)

This feature of the HEAR is configured by the CFX through the D_HEAR_BFP register as part of the overall configuration of the HEAR. The block exponent bit field (HEAR_BFP_BLOCK_EXP) from the D_HEAR_BFP register has been mirrored into the low bits of D_HEAR_BFP_BLOCK_EXP register to allow for efficient access to this important field.

The block floating-point configuration is normally controlled by the microcode configuration running on the HEAR. As use of block floating-point is tightly coupled with a microcode configuration, only specific configuration details of the available bit fields and defined symbols for these bit fields are provided here. For more information and further details on using this feature of the HEAR, see the *HEAR Configurable Accelerator Reference Manual*.

4.3.1 Control and Configuration Registers

Register Name	Register Description	Address
D_HEAR_BFP	Configure block floating-point functionality of the HEAR	0x9F90
D_HEAR_BFP_BLOCK_EXP	Mirror of the block floating-point total shift/block exponent field from D_HEAR_BFP	0x9F91

4.3.1.1 D_HEAR_BFP Settings

Bit Field	Field Name	Field Description
21:16	HEAR_BFP_BLOCK_EXP	Value of the block exponent indicating the shift that was applied to a block
15	HEAR_BFP_FMD_MODE	Enable magnitude detection for the full range (not just 20 to 23)
12:8	HEAR_BFP_BLOCK_MAX_MAG	Maximum magnitude detected in the current block
5:4	HEAR_BFP_SCALE_FACTOR	Number of shifts that are applied to a data element when read
1:0	HEAR_BFP_FORMAT	Configure the maximum number of block floating-point shifts

Field Name	Value Symbol	Value Description	Hex Value
HEAR_BFP_FMD_MODE	HEAR_BFP_FMD_DISABLE	Disable full range magnitude detection	0x0*
	HEAR_BFP_FMD_ENABLE	Enable full range magnitude detection	0x1
HEAR_BFP_FORMAT	HEAR_BFP_FORMAT_24BITS	Allow no shifts	0x0*
	HEAR_BFP_FORMAT_23BITS	Allow 0 or 1 shifts	0x1
	HEAR_BFP_FORMAT_22BITS	Allow 0, 1 or 2 shifts	0x2
	HEAR_BFP_FORMAT_21BITS	Allow 0, 1, 2 or 3 shifts	0x3

5 Memory

5.1 Memory Architecture

The BelaSigna 300 memory architecture consists of a number of single-port memories interconnected with the CFX and HEAR as well as the IOC through a series of buses and data controllers. Memory is divided into memory attached to the CFX and memory attached to the HEAR, but all memory connected to the HEAR is also accessible by the CFX DSP through either the P memory bus or X memory bus. An overview of the layout of the memories and buses is illustrated in Figure 3.

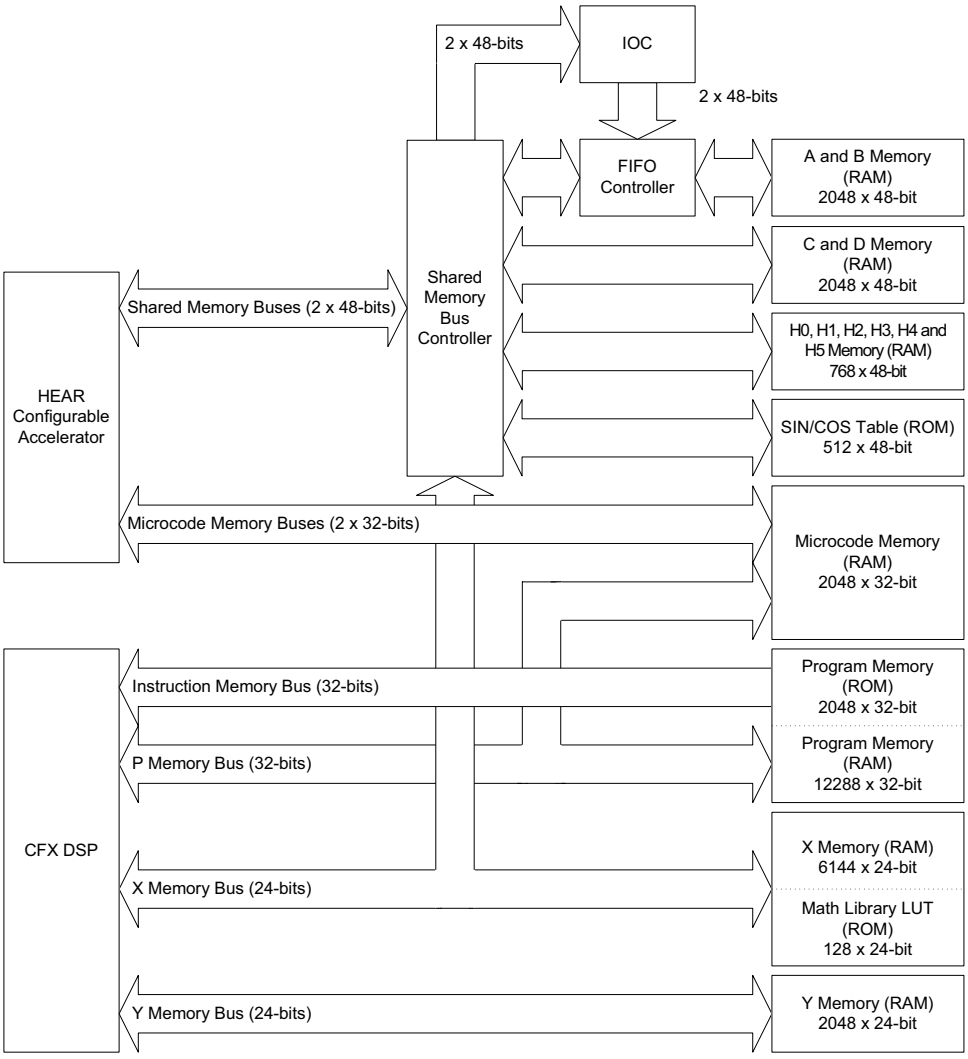


Figure 3: System Memory Architecture

5.1.1 Memory Instances

Table 2 lists the memory structures attached to the CFX, the size and width of each memory structure, as well as the bus or buses that the CFX DSP uses to access the memory.

TABLE 2: CFX MEMORY STRUCTURES

Memory Structure	Data Width	Memory Length	CFX Access Using
Program memory (ROM)	32	2048	P memory bus
Program memory (RAM)	32	12288	P memory bus
X memory (RAM)	24	6144	X, XY memory buses

TABLE 2: CFX MEMORY STRUCTURES (CONTINUED)

Memory Structure	Data Width	Memory Length	CFX Access Using
Math Library look up table (ROM)	24	128	X memory bus
Y memory (RAM)	24	2048	Y, XY memory buses

Table 3 lists the memory structures attached to the HEAR, the size and width of each memory structure, as well as the bus that the CFX DSP uses to access the memory.

TABLE 3: HEAR MEMORY STRUCTURES

Memory Structure	Data Width	Memory Length	CFX Access Using
Microcode memory (RAM)	32	2048	P memory bus
H0 memory (RAM)	48	128	X memory bus
H1 memory (RAM)	48	128	X memory bus
H2 memory (RAM)	48	128	X memory bus
H3 memory (RAM)	48	128	X memory bus
H4 memory (RAM)	48	128	X memory bus
H5 memory (RAM)	48	128	X memory bus
A memory (RAM)	48	1024	X memory bus
B memory (RAM)	48	1024	X memory bus
C memory (RAM)	48	1024	X memory bus
D memory (RAM)	48	1024	X memory bus
SIN/COS Table (ROM)	48	512	X memory bus

The microcode memory RAM structure that is attached to the HEAR can be broken down further into memory blocks, as shown in Table 4.

TABLE 4: MICROCODE MEMORY BLOCKS

Block	Start Address	End Address
Microcode memory 0	0x8000	0x83FF
Microcode memory 1	0x8400	0x87FF

The program memory (RAM) structure that is attached to the CFX can be broken down further into memory blocks, as shown in Table 5.

TABLE 5: PROGRAM MEMORY BLOCKS

Block	Start Address	End Address
P memory 0	0x1000	0x1FFF
P memory 1	0x2000	0x2FFF
P memory 2	0x3000	0x3FFF

The X memory RAM structure can be broken down further into memory blocks, as shown in Table 6.

TABLE 6: X MEMORY BLOCKS

Block	Start Address	End Address
X memory 0	0x0000	0x07FF
X memory 1	0x0800	0x0FFF
X memory 2	0x1000	0x17FF

5.1.2 Memory Bus Structures

Four memory buses connect the various memory instances to the CFX DSP. The primary connections and uses for these buses, which can be seen in Figure 3 on page 26, are as follows:

Instruction Bus: Bus used by the CFX DSP to fetch program instructions for execution. This bus additionally provides the CFX processor access to the interrupt vector table.

X Memory Bus: Bus used by the CFX DSP to access data in X memory, XY memory and shared memory. This bus also provides the CFX with access to the memory mapped control and configuration registers for the HEAR and for the FIFOs. This bus provides the least significant word of any accesses to the compound XY memory bus.

Y Memory Bus: Bus used by the CFX DSP to access data in Y memory and XY memory. This bus provides the most significant word of any accesses to the compound XY memory bus.

P Memory Bus: Bus used by the CFX DSP to access program and microcode memory as data. This bus also provides the CFX with access to the memory-mapped control and configuration registers used to configure analog and digital system components.

An additional four buses attached to the HEAR Accelerator provide access to microcode memory and shared memory for the HEAR. These buses, which can also be seen in Figure 3 on page 26, are set up in matched pairs to increase the data throughput of the HEAR:

Microcode Memory Buses:
These two buses allow the HEAR to fetch configuration information from the microcode memory instance.

Shared Memory Buses:
These two buses allow the HEAR access to all of shared memory through the shared memory bus controller.

5.1.3 Memory Controllers

As shown in Figure 3, the memory architecture contains a pair of controllers that operate between memory buses attached to the cores, and part or all of shared memory.

The shared memory bus controller coordinates access to the shared memories, directing data between each of the following separately:

- The shared memory instances and the X memory bus attached to the CFX
- The shared memory instances and the shared memory buses attached to the HEAR

For more information on this controller, see Section 5.1.3.1, “Shared Memory Bus Controller”.

The FIFO controller is connected to the IOC and the shared memory bus controller, and is used to provide FIFO access/addressing into A and B memory while coordinating and controlling memory access requests to these memories. More information on this controller is available in Chapter 6, “FIFO Controller” on page 41.

5.1.3.1 Shared Memory Bus Controller

The shared memory bus controller coordinates memory accesses and addressing into the underlying HEAR shared memories. For more information on the shared memory itself see Section 5.3.2, “Shared Memory (RAM and ROM)” on page 36.

In the case of a memory conflict being detected by this controller, access to shared memory is controlled through the `SYSTEM_CTRL_PRIORITY_HEAR` and `SYSTEM_CTRL_PRIORITY_CFX` bits of the `D_SYSTEM_CTRL` register. As only one of these bits can be set at any time, the core with priority retains access to the conflicting memory structure until the conflict is removed or resolved, while the other core is held in a wait state. Reading the value of the `SYSTEM_CTRL_PRIORITY_CFX` bit gives you the current configuration status and whether the HEAR or CFX has priority.

Note: Although A and B memory are also accessible to the IOC, resolution of memory access conflicts with the IOC are handled separately by the FIFO controller, as detailed in Section 6.4, “Access Priority” on page 50.

IMPORTANT: The access priority can be changed at any time, provided the CFX core does not execute a read or write to the shared memory in the clock cycle immediately preceding the change of priority (it is not possible to access shared memory in the same cycle that the access priority is updated).

5.1.3.1.1 Control and Configuration Registers

Register Name	Register Description	Address
`D_SYSTEM_CTRL`	System Control	0xE000

5.1.3.1.1.1 D_SYSTEM_CTRL Settings

Bit Field	Field Name	Field Description
7	SYSTEM_CTRL_PRIORITY_HEAR	Give the HEAR priority over the CFX on access to shared and microcode memory.
6	SYSTEM_CTRL_PRIORITY_CFX	Give the CFX priority over the HEAR on access to shared and microcode memory.

Field Name	Value Symbol	Value Description	Hex Value
SYSTEM_CTRL_PRIORITY_HEAR	SYSTEM_PRIORITY_HEAR	The HEAR Accelerator has priority	0x1*
SYSTEM_CTRL_PRIORITY_CFX	SYSTEM_PRIORITY_CFX	The CFX DSP has priority	0x1

5.1.4 Memory Interface Write Buffers

The CFX DSP core contains a write buffer for each of the P memory, X memory and Y memory buses. The buffers allow the CFX to perform any combination of memory-read and memory-write operations without stalling the instruction pipeline. A buffered memory-write operation generally does not affect program behavior and is transparent to the user application, but there are some situations where the CFX's write buffering is visible.

5.1.4.1 Buffering and Flushing of Write Operations

A memory-write operation is buffered by the CFX when a read operation from the same memory space as the write occurs in the clock cycle following the write. This is necessary because a write is performed in the **Execute** stage of the instruction pipeline while a read is performed in the **Decode** stage (see the *CFX DSP Architecture Manual* for details on the Program Control Unit's execution pipeline). Thus when a write is immediately followed by a read from the same memory, both operations need the bus at the same time. The read operation takes priority and the write operation is buffered.

A buffered write is flushed (stored to memory) as soon as the bus has a cycle during which it is not busy, which occurs for any instruction that is not a read from the same memory space. This includes another write operation (during its **Decode** stage the memory bus is free), memory operations to a different memory space, control-flow instructions (including interrupt handling), and register-based operations.

In summary, when a write operation is followed by a consecutive sequence of read operations from the same memory, the write is only stored to memory after all of the read operations have occurred. If any of the read operations reads from the same address to which the buffered write will store its value, the read operation returns the write operation's buffered value.

5.1.4.2 Visible Effects of Write Buffering

Write buffering can be visible to a user application for two reasons:

1. The delay in actually storing a value to memory.
2. Reading a buffered value instead of memory. In cases where a memory location does not have plain "copy all bits" behavior, a read operation gets a different result when the value comes from a write buffer instead of from memory.

Several control and configuration registers can behave unexpectedly for the second reason, if you write the register's P memory location and then immediately read it back so that you read the buffered value. Examples of special register behaviors that are not implemented by a buffered value include:

- Some bits might not be defined (for example, D_WATCHDOG_CFG).
- Writing a one to a bit clears the bit that was set, or otherwise changes the register values or initiates an action (for example, D_SYSTEM_CTRL).
- The bits in a register have different meanings when read than when written (for example, D_I2C_CTRL).
- Reading a register changes its value or initiates an action (for example, D_SPI_DATA when the SPI interface is configured for auto mode).

Visible write buffering due to a delayed memory store is less common. It is usually combined with writing a control register that automatically modifies another memory location (typically another control register). Since the store is delayed, the side-effect modification has not yet occurred if the other memory location is read immediately after the write. For example:

- Blocks such as the CRC Generator, where a read of D_CRC_VALUE would not include the value written to any D_CRC_ADD_x registers in the previous cycle
- Reading back the interrupt enable or priority bits in D_INT_ENABLE or D_INT_PRIORITY immediately after writing them through D_INT_EBL_x or D_INT_PRIO_x

Although all of the preceding examples involve control registers in P memory, effects of write buffering can also be seen with registers mapped into X memory. Examples include the FIFO registers, HEAR function chain controller registers, and HEAR block floating point registers. The value written to a register such as D_FIFO_xx_BLOCK_CNT might not match the value returned by that register, and the value of other registers in the same block could be updated by the value written to D_FIFO_xx_BLOCK_CNT.

IMPORTANT: In all of the above examples, you can avoid the undesirable side effects of write buffering by inserting (after the write operation) any instruction which does not read the same memory space. If you cannot move an existing instruction into place after the write, a NOP instruction will suffice.

Since there are no control registers with special behavior in the Y memory space, the write buffer for the Y memory bus is always transparent to the user application.

5.2 PROGRAM AND MICROCODE MEMORY

Program memory provides the CFX DSP with program instructions to execute and microcode memory provides the HEAR Accelerator with configuration information. This section provides detailed information on these two important memory components in addition to other components that are accessible through the dedicated instruction bus and the P memory bus.

5.2.1 P Memory Space

The P memory space (P memory) allows the CFX to access a proper superset of the program memory space for use as data. The P memory bus provides access to both the program memory instances for the CFX and the microcode memory instances for the HEAR. It also provides access to the memory mapped control and configuration registers for the majority of analog and digital components, and access to interrupt vectors for use by the interrupt controller and CFX.

When the P memory bus is used to read or write, a bus conflict occurs if the instruction bus or microcode buses are currently using the memory block to be accessed. When a bus conflict occurs the CFX DSP is held in a wait state, incurring a delay in the completion of the instruction accessing P memory. The wait state holds the CFX's clock, so the delay is not visible to the CFX in terms of instruction cycle counts. Thus a P memory access is valid in a single delay slot, for example, regardless of whether the memory access might cause a bus conflict and thus be delayed.

Note that a bus conflict does *not* occur, and P memory access proceeds without introducing a delay, if any of the following conditions hold:

- The CFX instruction and data are located in different memory blocks (see Section 5.1.1, "Memory Instances" on page 26 for a list of memory blocks).
- The instruction for the CFX is loaded from the program cache (see Section 5.2.5, "Program Memory Cache" on page 35).
- The CFX is currently in the first execution cycle of a multi-cycle instruction, including *Move-Move* instructions, so that it is not pre-fetching the next instruction.

Similar bus conflicts can also occur between the CFX DSP core and HEAR Accelerator core when both are accessing the microcode memory. Handling this possible conflict uses the `SYSTEM_CTRL_PRIORITY_HEAR` and `SYSTEM_CTRL_PRIORITY_CFX` bits of the `D_SYSTEM_CTRL` register. As only one of these bits can be set at any time, the processor with priority retains access to the conflicting memory structure until the conflict is removed or resolved, while the other core is held in a wait state. For configuration information about these bits, see Section 5.1.3.1, "Shared Memory Bus Controller" on page 29 on control and configuration registers.

5.2.2 Program Memory (RAM and ROM)

Instructions for execution on the CFX are fetched from the program memory through the dedicated instruction bus. Access on this bus is restricted to this use; therefore all data access to the program memory areas occurs over the P memory bus.

The program ROM is located from 0x0000 to 0x0800 of the addressable program memory space. This ROM contains code for booting the device as well as system-level routines for performing common tasks. For detailed information about the contents of the program ROM, see the *Firmware Reference Manual*.

The program RAM segments are located from 0x1000 to 0x3FFF of the addressable program memory space. This memory is used for user applications and for any 32-bit application data that a user application needs. The block of memory from 0x3000 to 0x3FFF is also mirrored at the top of the address space located at 0xF000 to 0xFFFF.

5.2.3 Microcode Memory (RAM)

The microcode memory is located in the addressable P memory space from 0x8000 to 0x8800. This memory is loaded by the CFX with a microcode configuration for the HEAR. Each function chain's microcode is loaded by the HEAR Configurable Accelerator using the microcode memory bus when that function chain is triggered.

5.2.4 P Memory Space Memory Map

The specific organization of the P memory space is shown in Figure 4. For reference by a program, many of the start and end addresses for memory areas in these memory spaces are associated with `#define` statements that can be found in the `sk3_hw.cfi` file.

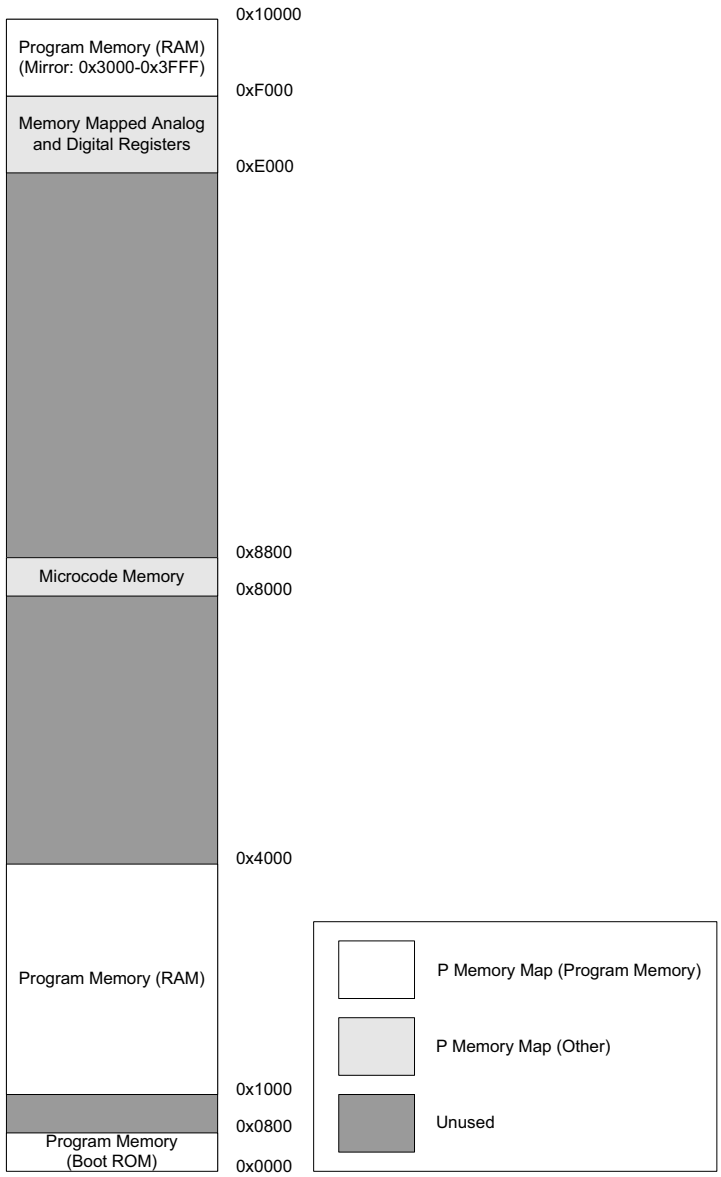


Figure 4: P Memory Space Memory Map

5.2.4.1 Interrupt Vector Mapping

The interrupt vector table, which is used by the interrupt controller and the CFX DSP, is mapped into the program memory between addresses 0xFFE0 and 0xFFFF (which is memory that has been mirrored from between addresses 0x3FE0 and 0x3FFF). The specific interrupt vectors and their locations in P memory are shown in Table 7.

TABLE 7: INTERRUPT VECTOR MAP

Interrupt	Vector	Address
System Reset	D_VECT_SYS_RESET	0x0000
Watchdog	D_VECT_WATCHDOG	0xFFE1
Timer 1	D_VECT_TIMER_1	0xFFE2
Timer 2	D_VECT_TIMER_2	0xFFE3
SPI	D_VECT_SPI	0xFFE4
I^2C	D_VECT_I2C	0xFFE5
Shared I^2S & PCM	D_VECT_PCM	0xFFE6
GPIO	D_VECT_GPIO	0xFFE7
HEAR Event 0	D_VECT_HEAR_0	0xFFE8
HEAR Event 1	D_VECT_HEAR_1	0xFFE9
HEAR Event 2	D_VECT_HEAR_2	0xFFEA
HEAR Event 3	D_VECT_HEAR_3	0xFFEB
HEAR Event 4	D_VECT_HEAR_4	0xFFEC
HEAR Event 5	D_VECT_HEAR_5	0xFFED
HEAR Event 6	D_VECT_HEAR_6	0xFFEE
HEAR Event 7	D_VECT_HEAR_7	0xFFEF
FIFO A0	D_VECT_FIFO_A0	0xFFF0
FIFO A1	D_VECT_FIFO_A1	0xFFF1
FIFO A2	D_VECT_FIFO_A2	0xFFF2
FIFO A3	D_VECT_FIFO_A3	0xFFF3
FIFO B0	D_VECT_FIFO_B0	0xFFF4
FIFO B1	D_VECT_FIFO_B1	0xFFF5
FIFO B2	D_VECT_FIFO_B2	0xFFF6
FIFO B3	D_VECT_FIFO_B3	0xFFF7
UART Receive	D_VECT_UART_RX	0xFFF8
UART Transmit	D_VECT_UART_TX	0xFFF9
Reserved	N/A	0xFFFA - 0xFFFF

5.2.4.2 Configuration Register Mapping

The analog and digital control and configuration registers are mapped into the P memory space between addresses 0xE000 and 0xEFFF. These registers are organized by their area of control and mapped as blocks into the P memory space as shown in Table 8.

TABLE 8: ANALOG AND DIGITAL CONTROL AND CONFIGURATION REGISTER BLOCK MAPPING

Controlled/Configured Area	Area Type	Block Start Address
General System Configuration	Digital	0xE000
Debug Port Related Configuration	Digital	0xE020
Interrupt Controller	Digital	0xE040
Clock Configuration	Digital	0xE080
I/O Controller (IOC)	Digital	0xE0C0
Digital A/D and D/A Converter Data	Digital	0xE0D0
Clock Generation (Oscillator)	Analog	0xE100
Power Components	Analog	0xE101
Input Stage	Analog	0xE140
Output Stage	Analog	0xE150
Low Speed A/D Converters (LSAD)	Analog	0xE180
Power Components (Battery Monitor)	Analog	0xE190
SPI Interface	Digital	0xE1C0
I^2C Interfaces	Digital	0xE200
Shared PCM/I^2S Interface	Digital	0xE240
GPIO Interface	Digital	0xE280
UART Interface	Digital	0xE2C0

5.2.5 Program Memory Cache

To reduce the power consumed by the CFX DSP, a small 16-word read-only instruction cache has been included on the instruction bus between the CFX DSP and the program memory. This cache is designed to reduce the power consumed by constantly accessing program memory to fetch subsequent instructions for the CFX DSP. You should not be able to detect any difference in functional code execution when using the cache, except for a slight change in timing due to the special case side effect discussed in the note below.

The program memory cache automatically reconfigures whenever a backwards branch of less than 256 words indicates a loop has been detected. At this point the cache aligns itself to ensure that the largest possible number of instructions from the loop are stored in the cache.

Note: As a side effect of the program memory cache, access to P memory while the CFX is fetching cached instructions from a loop does not result in a memory access conflict. As a result, when the program memory cache is being used for

a loop that uses program memory as data, the core does not incur delays as described in Section 5.2.1, “P Memory Space” on page 31.

5.3 DATA MEMORY

Data memory is used to store all types of signal and control data. The two kinds of memory are memory attached to the CFX DSP and memory attached to the HEAR. The CFX has direct access to a number of data memories that a user application can use as needed. All data memory attached to the HEAR Accelerator core is also available to the CFX as shared memory, although both cores access the memory through the shared memory bus controller. This section provides detailed information on these memory components; all of the related components that are accessible through the X, Y and shared memory buses; and memory components and related components that are located in the X, Y and XY memory maps.

5.3.1 CFX Data Memories (RAM and ROM)

Data memory attached to the CFX DSP primarily consists of two banks of 24-bit memory that can be accessed either individually as 24-bit memory or in combination as a single bank of 48-bit memory. These two individual memories are referred to as X memory and Y memory respectively. X memory is located from 0x0000 to 0x1800 in the X memory space and is accessed through the X memory bus. Y memory is located from 0x0000 to 0x0800 in the Y memory space and is accessed through the Y memory bus.

XY memory is a combined memory space that is addressable from 0x0000 to 0x0800. Each addressable element is a double-word 48-bit value using the data at the corresponding X memory address as the low word and at the corresponding Y memory address as the high word. XY memory is accessed through both the X and Y memory buses together as the XY memory bus.

Note: As XY memory uses physical memory containing both the X memory instance and Y memory instance located between 0x0000 and 0x07FF, any space reserved for XY memory should usually also be reserved in both X memory and Y memory.

X memory also contains a set of four look up tables (LUT) used by the math library firmware. These LUTs are collected into one 128-word math LUT ROM that is used in math library calculations that make use of the functions: 2^x, $\log_2 x$, $\frac{1}{x}$, and $\sqrt{x}$.

5.3.2 Shared Memory (RAM and ROM)

All of the shared memories are double-word structured 48-bit memories that allow 24-bit word addressing by both the CFX and HEAR. These memories and their locations in the X memory map are outlined in Table 9. All accesses to these memories are controlled by the shared memory bus controller, which provides access for the CFX DSP through the X memory space and access for the HEAR through the core’s 48-bit shared memory buses.

TABLE 9: MEMORY INSTANCES IN SHARED MEMORY

Memory Name	Start Address	End Address
H0 memory	0x8600	0x86FF
H1 memory	0x8700	0x87FF
H2 memory	0x8800	0x88FF
H3 memory	0x8900	0x89FF
H4 memory	0x8A00	0x8AFF
H5 memory	0x8B00	0x8BFF
SIN/COS Data Table (ROM)	0x9400	0x97FF
A memory	0xE000	0xE7FF
B memory	0xE800	0xEFFF
C memory	0xF000	0xF7FF
D memory	0xF800	0xFFFF

The shared memory instances are accessible both independently, as listed in Table 9, and interleaved with other shared memory instances. All interleaved combinations of shared memory instances are listed in Table 10. Both independent instances and interleaved memory instances are shown as appropriate in the X memory map (Figure 5 on page 39).

TABLE 10: INTERLEAVED MEMORY INSTANCES IN SHARED MEMORY

Memory	First Memory	Second Memory	Start Address	End Address
H01 memory	H0 memory	H1 memory	0x8000	0x81FF
H23 memory	H2 memory	H3 memory	0x8200	0x83FF
H45 memory	H4 memory	H5 memory	0x8400	0x85FF
H02 memory	H0 memory	H2 memory	0x8C00	0x8DFF
H13 memory	H1 memory	H3 memory	0x8E00	0x8FFF
H03 memory	H0 memory	H3 memory	0x9000	0x91FF
H12 memory	H1 memory	H2 memory	0x9200	0x93FF
AB memory	A memory	B memory	0xA000	0xAFFF
CD memory	C memory	D memory	0xB000	0xBFFF
AC memory	A memory	C memory	0xC000	0xCFFF
BD memory	B memory	D memory	0xD000	0xDFFF

The interleaving of memory is in 48-bit double words, resulting in two addressable addresses into each memory at each step. For example, the memory addressed between 0xA000 and 0xAFFF is an interleaved memory mapping that provides access to data in both A and B memory. Reading from 0xA000-0xA001 (the first double-word of the interleaved AB Memory) provides a user with the first double-word of A memory. Reading from 0xA002-0xA003 (the second double-word of the interleaved AB Memory) provides a user with the first double-word of B memory.

When using shared memory with both the CFX and HEAR cores, bus conflicts occur if both cores attempt to access the same memory instance in the same cycle. If a bus conflict occurs, access is granted to the CFX or HEAR based on the settings of the `D_SYSTEM_CTRL` register, as described in Section 5.1.3.1, “Shared Memory Bus Controller” on page 29, while the other core is held in a wait state. When the CFX DSP is held in a wait state, the delay is not visible to instructions running on the DSP, as explained in Section 5.2.1, “P Memory Space” on page 31.

5.3.3 Data Memory Space Memory Maps

The specific organization of the various data memory spaces is shown in Figure 5. For program references, many of the start and end addresses for memory areas in these memory spaces are associated with `#define` statements that are located in the `sk3_hw.cfi` file.

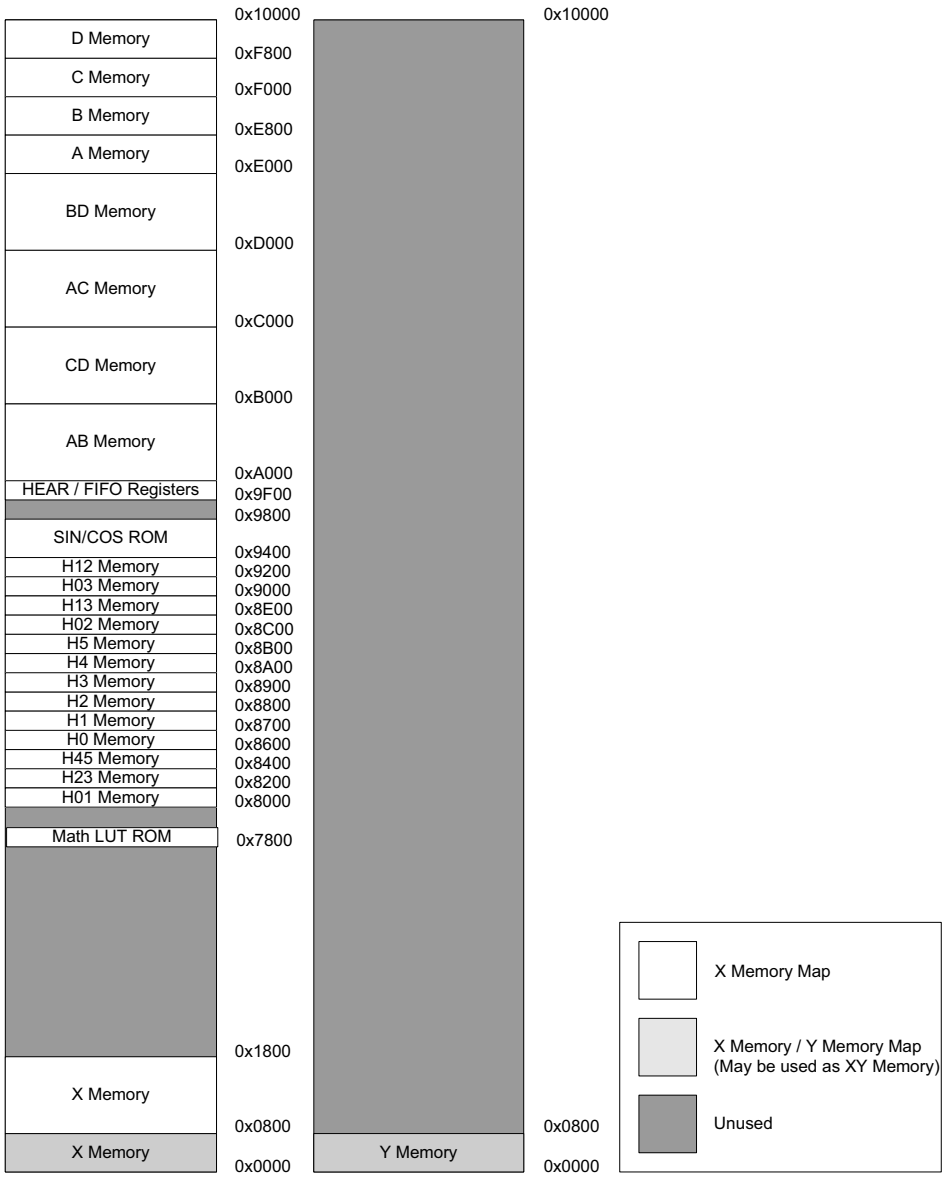


Figure 5: Data Memory Space Memory Maps

5.3.3.1 HEAR and FIFO Configuration Register Mapping

Mapped into the X memory space between addresses 0x9F00 and 0x9FFF are the HEAR and FIFO control and configuration registers. These registers are organized by their area of control and mapped as blocks into the X memory space as shown in Table 11.

TABLE 11: HEAR AND FIFO CONTROL AND CONFIGURATION REGISTER BLOCK MAPPING

Controlled/Configured Area	Block Start Address
FIFO A0	0x9F00
FIFO A1	0x9F10
FIFO A2	0x9F20
FIFO A3	0x9F30
FIFO B0	0x9F40
FIFO B1	0x9F50
FIFO B2	0x9F60
FIFO B3	0x9F70
Function Chain Controller	0x9F80
Block Floating-Point Configuration	0x9F90

6 FIFO Controller

6.1 Introduction to the FIFO Controller

The BelaSigna 300 system's FIFO controller provides the ability to define up to eight FIFO buffers. These FIFOs are defined as up to 4 FIFOs in each of A memory and B memory. To simplify discussion of FIFOs, the FIFOs are denoted as FIFO A0, FIFO A1, FIFO A2, FIFO A3, FIFO B0, FIFO B1, FIFO B2 and FIFO B3 in documentation and defines. These FIFOs may be used as input or output FIFOs for the system as used by the IOC; however they may also be used as software FIFOs through the use of access registers.

Each individual FIFO is associated with a set of FIFO configuration registers that are mapped into X memory starting at 0x9F00 (as defined in Table 11 on page 40). Each FIFO is also associated with an interrupt for the CFX DSP and a FIFO event for the HEAR.

IMPORTANT: Using the HEAR reset by setting the `SYSTEM_CTRL_HEAR_RESET` bit in the `D_SYSTEM_CTRL` register resets the FIFO controllers, clearing the current FIFO register settings.

6.2 Defining Memory for Use as a FIFO

A FIFO is a section of memory that is treated as a circular buffer in memory where components that use the memory use pointers to indicate the next location to be read or written.

For each FIFO in memory, define a start and end point for the FIFO, as well as specify the size of the FIFO. Figure 6 shows how the FIFO start and end pointers, along with the FIFO size indicator, are used to define an area of memory for use as a FIFO.

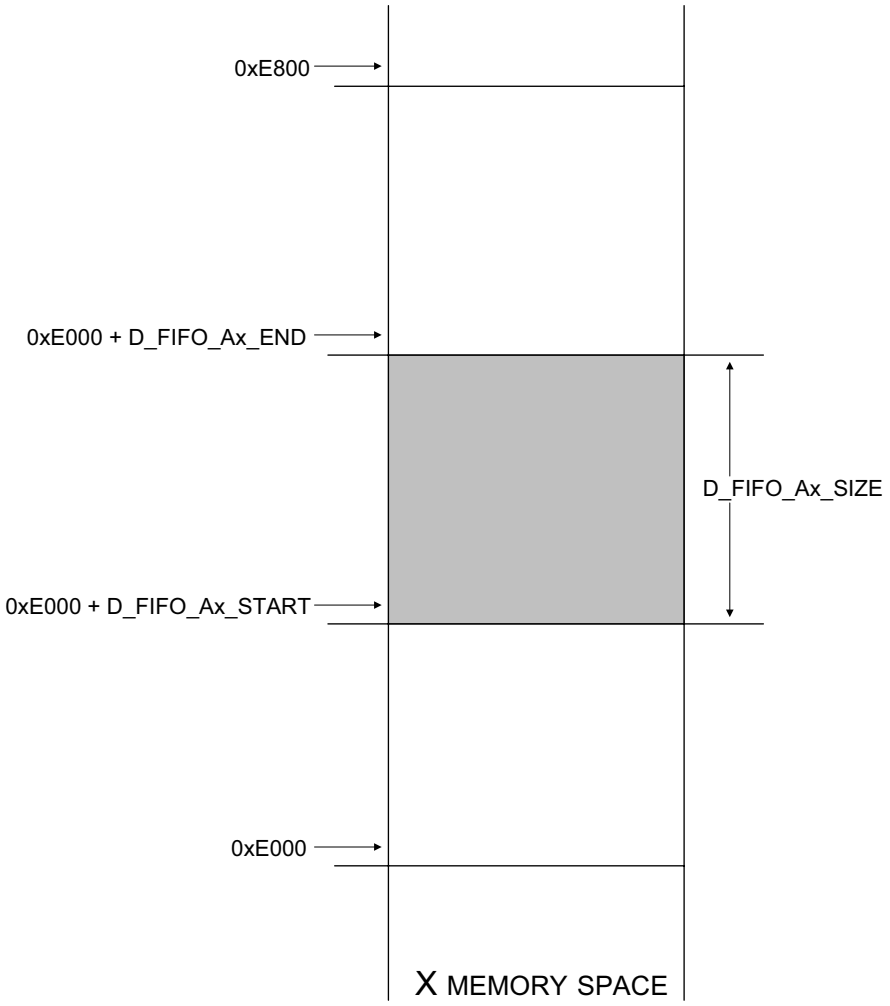


Figure 6: Defining an Area of A Memory for Use as a FIFO

The start address for a FIFO is defined by the associated `D_FIFO_xx_START` register. This register defines what the offset from the start of the memory (either 0xE000 for A memory or 0xE800 for B memory depending on which FIFO is being configured) to the FIFO should be. This start address is limited to steps of 32 words; therefore the associated bit field of this register is aligned to truncate values that do not meet this requirement. The end address for a FIFO is similarly defined by an offset specified in the associated `D_FIFO_xx_END` register, and this address is also limited to steps of 32 words by truncation. If a FIFO is enabled, the size of a FIFO must be defined by setting the `D_FIFO_xx_SIZE` register to the value of $(\text{D_FIFO_xx_END} - \text{D_FIFO_xx_START})$. The address pointed to by the `D_FIFO_xx_END` register is not considered part of the area of memory allocated to the FIFO.

Note: Setting the D_FIFO_xx_SIZE register for a FIFO to zero disables a FIFO. All accesses from the IOC or through the associated D_FIFO_xx_ACCESS register are ignored.

CAUTION: Defining two or more FIFOs that overlap in memory will result in undefined behavior. Further, a start address and end address that reserve less memory than the defined size or that define an invalid area (as would be the case if the end address offset is lower than the start address offset) will also result in undefined behavior.

The FIFO controller provides no address manipulation to areas of A memory and B memory that are not included in the defined FIFO ranges. As a result, a user application can access these areas of memory linearly.

6.2.1 Control and Configuration Registers

Register Name	Register Description	Address
D_FIFO_A0_START	Select the offset in A memory from the base of A memory to the start of FIFO A0	0x9F00
D_FIFO_A0_END	Select the offset in A memory from the base of A memory to the end of FIFO A0	0x9F01
D_FIFO_A0_SIZE	Disable FIFO A0 or indicate the size of FIFO A0	0x9F02
D_FIFO_A1_START	Select the offset in A memory from the base of A memory to the start of FIFO A1	0x9F10
D_FIFO_A1_END	Select the offset in A memory from the base of A memory to the end of FIFO A1	0x9F11
D_FIFO_A1_SIZE	Disable FIFO A1 or indicate the size of FIFO A1	0x9F12
D_FIFO_A2_START	Select the offset in A memory from the base of A memory to the start of FIFO A2	0x9F20
D_FIFO_A2_END	Select the offset in A memory from the base of A memory to the end of FIFO A2	0x9F21
D_FIFO_A2_SIZE	Disable FIFO A2 or indicate the size of FIFO A2	0x9F22
D_FIFO_A3_START	Select the offset in A memory from the base of A memory to the start of FIFO A3	0x9F30
D_FIFO_A3_END	Select the offset in A memory from the base of A memory to the end of FIFO A3	0x9F31
D_FIFO_A3_SIZE	Disable FIFO A3 or indicate the size of FIFO A3	0x9F32
D_FIFO_B0_START	Select the offset in B memory from the base of B memory to the start of FIFO B0	0x9F40
D_FIFO_B0_END	Select the offset in B memory from the base of B memory to the end of FIFO B0	0x9F41
D_FIFO_B0_SIZE	Disable FIFO B0 or indicate the size of FIFO B0	0x9F42
D_FIFO_B1_START	Select the offset in B memory from the base of B memory to the start of FIFO B1	0x9F50
D_FIFO_B1_END	Select the offset in B memory from the base of B memory to the end of FIFO B1	0x9F51

Register Name	Register Description	Address
D_FIFO_B1_SIZE	Disable FIFO B1 or indicate the size of FIFO B1	0x9F52
D_FIFO_B2_START	Select the offset in B memory from the base of B memory to the start of FIFO B2	0x9F60
D_FIFO_B2_END	Select the offset in B memory from the base of B memory to the end of FIFO B2	0x9F61
D_FIFO_B2_SIZE	Disable FIFO B2 or indicate the size of FIFO B2	0x9F62
D_FIFO_B3_START	Select the offset in B memory from the base of B memory to the start of FIFO B3	0x9F70
D_FIFO_B3_END	Select the offset in B memory from the base of B memory to the end of FIFO B3	0x9F71
D_FIFO_B3_SIZE	Disable FIFO B3 or indicate the size of FIFO B3	0x9F72

6.3 FIFO BUFFER DATA AND CONFIGURATION

There are three primary uses for a FIFO in the BelaSigna 300 system:

1. To collect blocks of input data from the IOC
2. To provide blocks of output data to the IOC
3. In software, to supply an application with a hardware FIFO buffer

When describing how a FIFO buffer is used for input from the IOC, it is useful to divide the FIFO buffer into two areas: an area to store the input block of data that is currently being received, and an area containing previously received blocks of data. Received data is collected into a block until a complete block of samples has been received.

IMPORTANT: Using the HEAR reset also resets the FIFO controllers which clears current FIFO register settings.

When describing how a FIFO buffer is used for output to the IOC, it is useful to divide the FIFO buffer into two areas: an area from which the IOC is currently reading output data, and an area to store future output blocks of samples. This second area contains previously read blocks of output data until this data is overwritten by new (future) output samples.

Finally, a FIFO can be used in software as an input FIFO, an output FIFO, or both—except that data is stored or loaded through the appropriate access register (D_FIFO_xx_ACCESS) rather than storing or loading data through the IOC. Alternatively, if not using the access register, a user application can read or write directly to the appropriate part of the FIFO and then force a block update by writing the block counter directly to advance the FIFO pointers.

CAUTION: As the FIFOs are located in memory that has been organized into a 48-bit double word configuration, the access register is actually a 48-bit register that allows reading or writing of up to 2 adjacent words of a FIFO within one cycle. As a result, since the CFX processor and related components do not have 48-bit access to this register, do not read or write the address immediately following an access register as this will be registered as a data access without reading or writing any useful data.

Note: All pointer updates, IOC accesses and accesses through the access register use the FIFO buffer as a circular buffer. As a result, any update calculation that would result in a pointer or access outside of the FIFO itself is reduced modulo the size of the FIFO.

6.3.1 FIFO Input and Output Block

In each use case, the input/output block (IOBLOCK or IO block) pointer and block size defines the area of input/output that the IOC or access register is currently accessing. The current block of input or output samples, indicated by the IO block pointer, is commonly referred to as the *IO block*. The elements related to the IO block currently being used are shown in Figure 7.

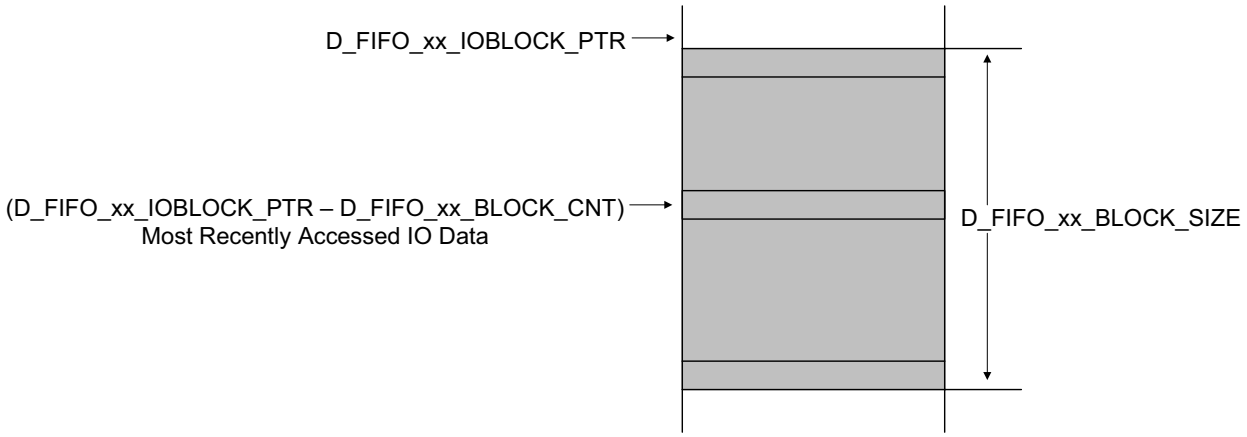


Figure 7: Current Input or Output Data Block for a FIFO

The block size for each IO block is defined by the `D_FIFO_xx_BLOCK_SIZE` register. The block size for a FIFO must not exceed the defined size of the FIFO. The block size might be further limited to the following maximum words:

- 512 words if smart FIFO accesses are used (either by the IOC or through the D_FIFO_xx_ACCESS register)
- 1023 words if smart FIFO accesses are not used

The IO block is an area of the block size that is filled starting at the top of the block (shaded area of Figure 7). The IO block pointer, which is accessible through the appropriate `D_FIFO_xx_IOBLOCK_PTR` register, provides an offset within the FIFO to the first address above the top of this block, indicating that the first data item stored in the block should be located one address below the pointer.

Finally, a block counter is maintained by the FIFO controller to indicate to the FIFO controller where the next data access should be located within the IO block. This counter, which you can see through the appropriate `D_FIFO_xx_BLOCK_CNT` register, indicates how many words have already been read or written in the current data block. This value is incremented following an access to the FIFO data by the IOC or through the access register. The most recently accessed input or output data in the FIFO is located at

(D_FIFO_xx_IOBLOCK_PTR – D_FIFO_xx_BLOCK_CNT) . The next location that will be accessed to store data written to the FIFO or provide data read from the FIFO is (D_FIFO_xx_IOBLOCK_PTR – (D_FIFO_xx_BLOCK_CNT + 1)) .

If at any point the block count is equal to or exceeds the block size, the FIFO configuration is updated to indicate that use of the current input/output data block is complete. This can occur when the block count is updated through an IOC FIFO access, an access register read/write, or by explicitly writing a value to the block count register from the CFX or the HEAR. When this happens, the following updates occur:

- The block count resets to 0.
- Both the IO block pointer and base pointer are decremented by the block size. As both pointers must point to an even address, if the block size was odd then the IO block pointer and base pointer are decremented by the (block size + 1).
- The interrupt associated with this FIFO is triggered. For more information on configuring interrupts and using interrupts within an application, see Section 11.1, “Interrupt Controller” on page 113.
- The HEAR receives a function chain controller event triggering the function chain linked to the FIFO. For more information about the function chain controller, see Section 4.2, “Function Chain Controller” on page 16. For more information indicating which FIFO configurations to use with a HEAR microcode configuration, see the *HEAR Configurable Accelerator Reference Manual*.

6.3.2 FIFO Processing Block

The data not currently being used for input or output from the FIFO is available for signal processing operations. A section of this area is normally reserved for use by these operations and is referred to as the processing block or processing area. The beginning of the processing block is defined by the base pointer, which you access through the appropriate `D_FIFO_xx_BASE_PTR` register. The base pointer provides an offset into the FIFO that is updated along with the IO block pointer. The base pointer is used to specify what actual hardware data will be mapped to the base of the FIFO from a software application perspective. Normally the mapping is used so that the base pointer indicates where in the physical memory the most recent input blocks or output buffer area is located. The FIFO controller in turn uses this pointer to remap the data when addressed by software so that when a software application accesses memory within a FIFO, the memory addressed at `D_FIFO_xx_START` matches the physical memory pointed to by the base pointer.

Figure 8 shows the area of one FIFO (shaded area) as it is bounded by the FIFO start and end points. The hardware memory mapping on the left shows where in memory the `D_FIFO_xx_BASE_PTR` actually points and how memory would be aligned if the FIFO were disabled by setting its size to zero (see Section 6.2, “Defining Memory for Use as a FIFO” on page 41 above). The software memory mapping on the right shows how memory appears to be aligned to software accessing the FIFO. This aligns the memory pointed to by `D_FIFO_xx_BASE_PTR` to the base of the FIFO (located at the offset of `D_FIFO_xx_START`).

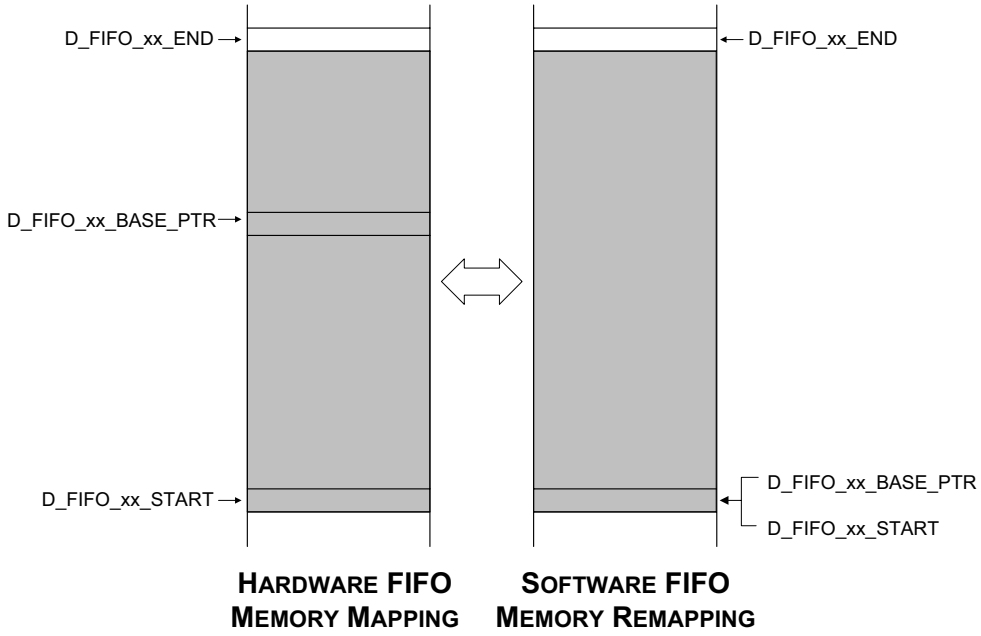


Figure 8: Hardware Memory Mapping and Software View of FIFO Memory Remapping

6.3.3 Typical FIFO Configurations

6.3.3.1 Input FIFO Configuration

When using a FIFO for input, a typical initial configuration might be:

- Base pointer set to zero
- IO block pointer set to zero

This will result in the latest full block of input samples always being located at the base of memory and provides the maximum amount of historical FIFO data to draw upon for algorithm calculations. Figure 9 illustrates a typical input FIFO configuration.

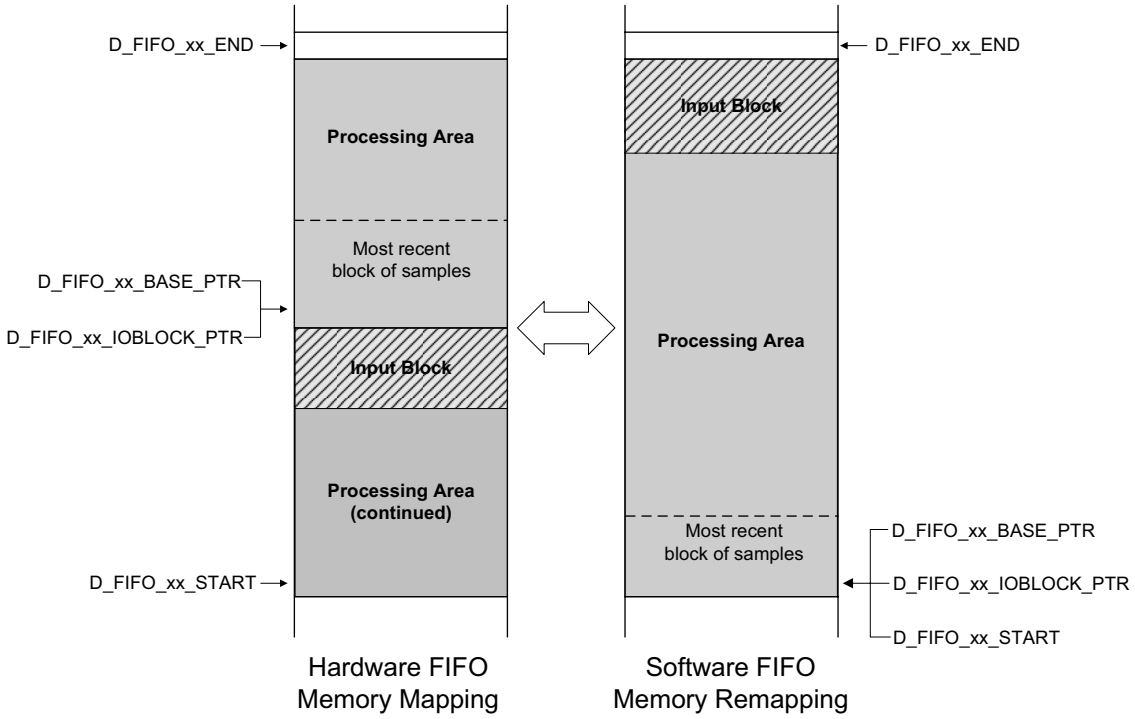


Figure 9: Input FIFO Hardware Memory Mapping and Software View of Memory Remapping

6.3.3.2 Output FIFO Configuration

When using a FIFO for output, a typical initial configuration might be:

- Base pointer set to zero
- IO block pointer set to (processing area + block size)

This will result in the top block of data from the processing area becoming the next output block each time the pointers are updated. It will also ensure that the processing area will be located at the start of the FIFO for convenient application access. Figure 10 illustrates a typical output FIFO configuration.

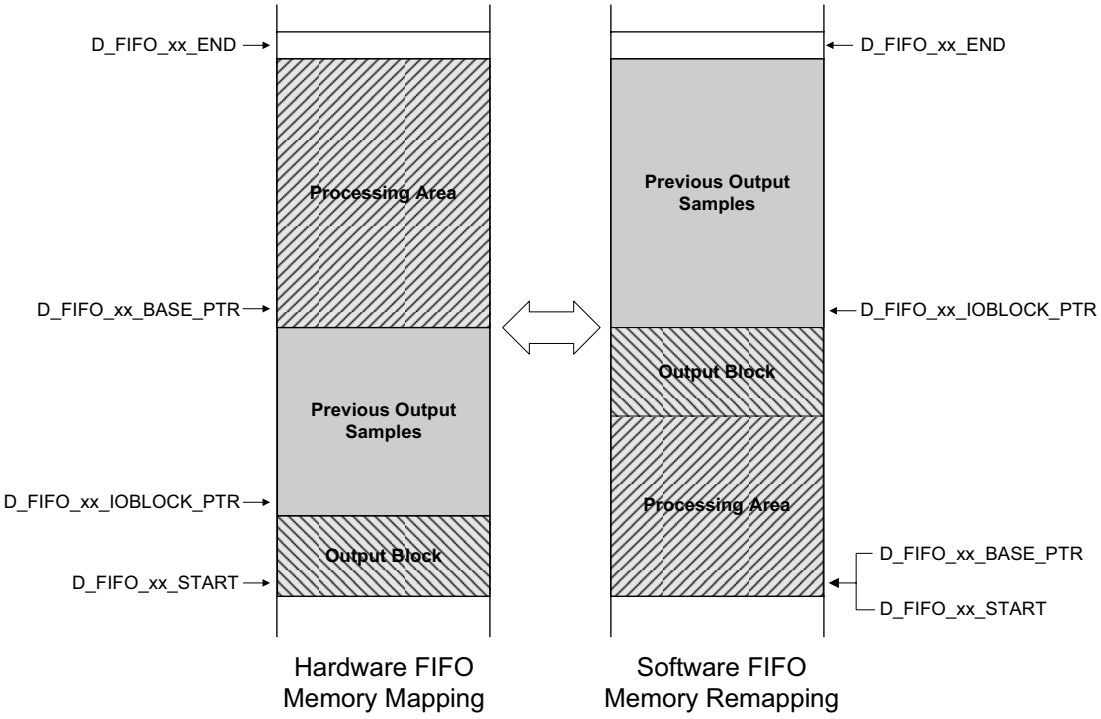


Figure 10: Output FIFO Hardware Memory Mapping and Software View of Memory Remapping

6.3.4 Control and Configuration Registers

Register Name	Register Description	Address
D_FIFO_A0_BASE_PTR	Offset within FIFO A0 to the data accessible at the FIFO start address	0x9F04
D_FIFO_A0_IOBLOCK_PTR	Offset within FIFO A0 to the top of the current IO data block	0x9F05
D_FIFO_A0_BLOCK_SIZE	Select the size of an IO data block for FIFO A0	0x9F08
D_FIFO_A0_BLOCK_CNT	Current read/write location in the IO data block for FIFO A0	0x9F0C
D_FIFO_A0_ACCESS	Access register for reading/writing the IO data block for FIFO A0	0x9F0E
D_FIFO_A1_BASE_PTR	Offset within FIFO A1 to the data accessible at the FIFO start address	0x9F14
D_FIFO_A1_IOBLOCK_PTR	Offset within FIFO A1 to the top of the current IO data block	0x9F15
D_FIFO_A1_BLOCK_SIZE	Select the size of an IO data block for FIFO A1	0x9F18
D_FIFO_A1_BLOCK_CNT	Current read/write location in the IO data block for FIFO A1	0x9F1C
D_FIFO_A1_ACCESS	Access register for reading/writing the IO data block for FIFO A1	0x9F1E
D_FIFO_A2_BASE_PTR	Offset within FIFO A2 to the data accessible at the FIFO start address	0x9F24
D_FIFO_A2_IOBLOCK_PTR	Offset within FIFO A2 to the top of the current IO data block	0x9F25
D_FIFO_A2_BLOCK_SIZE	Select the size of an IO data block for FIFO A2	0x9F28
D_FIFO_A2_BLOCK_CNT	Current read/write location in the IO data block for FIFO A2	0x9F2C
D_FIFO_A2_ACCESS	Access register for reading/writing the IO data block for FIFO A2	0x9F2E

Register Name	Register Description	Address
D_FIFO_A3_BASE_PTR	Offset within FIFO A3 to the data accessible at the FIFO start address	0x9F34
D_FIFO_A3_IOBLOCK_PTR	Offset within FIFO A3 to the top of the current IO data block	0x9F35
D_FIFO_A3_BLOCK_SIZE	Select the size of an IO data block for FIFO A3	0x9F38
D_FIFO_A3_BLOCK_CNT	Current read/write location in the IO data block for FIFO A3	0x9F3C
D_FIFO_A3_ACCESS	Access register for reading/writing the IO data block for FIFO A3	0x9F3E
D_FIFO_B0_BASE_PTR	Offset within FIFO B0 to the data accessible at the FIFO start address	0x9F44
D_FIFO_B0_IOBLOCK_PTR	Offset within FIFO B0 to the top of the current IO data block	0x9F45
D_FIFO_B0_BLOCK_SIZE	Select the size of an IO data block for FIFO B0	0x9F48
D_FIFO_B0_BLOCK_CNT	Current read/write location in the IO data block for FIFO B0	0x9F4C
D_FIFO_B0_ACCESS	Access register for reading/writing the IO data block for FIFO B0	0x9F4E
D_FIFO_B1_BASE_PTR	Offset within FIFO B1 to the data accessible at the FIFO start address	0x9F54
D_FIFO_B1_IOBLOCK_PTR	Offset within FIFO B1 to the top of the current IO data block	0x9F55
D_FIFO_B1_BLOCK_SIZE	Select the size of an IO data block for FIFO B1	0x9F58
D_FIFO_B1_BLOCK_CNT	Current read/write location in the IO data block for FIFO B1	0x9F5C
D_FIFO_B1_ACCESS	Access register for reading/writing the IO data block for FIFO B1	0x9F5E
D_FIFO_B2_BASE_PTR	Offset within FIFO B2 to the data accessible at the FIFO start address	0x9F64
D_FIFO_B2_IOBLOCK_PTR	Offset within FIFO B2 to the top of the current IO data block	0x9F65
D_FIFO_B2_BLOCK_SIZE	Select the size of an IO data block for FIFO B2	0x9F68
D_FIFO_B2_BLOCK_CNT	Current read/write location in the IO data block for FIFO B2	0x9F6C
D_FIFO_B2_ACCESS	Access register for reading/writing the IO data block for FIFO B2	0x9F6E
D_FIFO_B3_BASE_PTR	Offset within FIFO B3 to the data accessible at the FIFO start address	0x9F74
D_FIFO_B3_IOBLOCK_PTR	Offset within FIFO B3 to the top of the current IO data block	0x9F75
D_FIFO_B3_BLOCK_SIZE	Select the size of an IO data block for FIFO B3	0x9F78
D_FIFO_B3_BLOCK_CNT	Current read/write location in the IO data block for FIFO B3	0x9F7C
D_FIFO_B3_ACCESS	Access register for reading/writing the IO data block for FIFO B3	0x9F7E

6.4 ACCESS PRIORITY

All access to A memory and B memory as well as the FIFO configuration registers goes through the FIFO controller. To control which access has priority and to order the accesses received, the FIFO controller uses a fixed access priority scheme. The fixed access priority which is enforced by the FIFO controller is:

1. Direct read/write access by the IOC.
2. Read/write access using an access register (D_FIFO_xx_ACCESS) for a FIFO through the shared memory bus controller.
3. Write access to a block counter register (D_FIFO_xx_BLOCK_CNT) for a FIFO through the shared memory bus controller.
4. Read/write access to any remaining FIFO register through the shared memory bus controller.
5. Read/write access to any area outside of a defined FIFO buffer through the shared memory bus controller (linear access).
6. Read/write access to an area within a FIFO buffer through the shared memory bus controller.

7 Input/Output Controller (IOC)

7.1 INTRODUCTION

The Input/Output Controller (IOC) moves signal data from the input stage and PCM interface into system memory to be processed, and out of system memory to the output stage or PCM interface as audio output. To do this, the IOC is connected appropriately between each of the interfaces, and enjoys privileged access to the FIFOs through the FIFO controller.

The IOC interacts directly with each of the following components:

- The input stage
- The output stage
- The PCM interface
- The FIFOs and the FIFO controller

As a result, these components must be referenced when discussing the IOC. For more details on any of these other system components, refer to their respective chapters in this document.

7.2 IOC INPUT

On the input side, the IOC is attached to the four input channels and A/D converters of the input stage. These channels provide 18-bit input data and are denoted as ADC0, ADC1, ADC2 and ADC3 in constants and documentation. This 18-bit data can be aligned to the lower or upper 18 bits of the 24-bit input word by clearing or setting the appropriate `IOC_FIFO_CFG_ADCx_SHIFT` bit in the `D_IOC_FIFO_CFG` register. If aligned to the lower 18 bits, the data is sign extended to 24 bits. If aligned to the upper 18 bits, the data is zero padded to 24 bits. Additionally, the IOC can be attached to up to two channels of input from the PCM interface. Denoted as PCM0 and PCM1 in constants and documentation, these channels are nominally considered as 24-bit input data, as the PCM interface configuration can be used to align data that is shorter than 24 bits.

The IOC (if configured correctly) can direct each of the six input sources into individual FIFOs that have been configured. You can select which FIFO each channel is connected to by writing the desired configuration to the `IOC_INPUT_CFG_xxx_STORE` fields in the `D_IOC_INPUT_CFG` register. Each of these bit fields allows you to select from one of the eight FIFOs or to not store the input at all.

IMPORTANT: Connecting IOC input sources to FIFOs that have not been configured results in undefined behavior.

Additionally, the input channels that are attached to the IOC can be pair-wise configured to provide a dynamic range extended (IDRX) input channel of 23-bit input data from each pair of inputs. In constants and documentation, the input channel formed by combining ADC0 with ADC1 is denoted as IDRX0, and the input channel formed by combining ADC2 with ADC3 is denoted as IDRX1.

Input configuration and system configuration requirements related to the use of one or both dynamic range extended input channels is described in Section 8.2.1, “Input Dynamic Range Extension” on page 65.

IMPORTANT: When using the dynamic range extended input IDRX0, component input channel ADC0 must be directed to the desired FIFO and ADC1 must be configured to not store the input. When using the dynamic range extended input IDRX1, component input channel ADC2 must be directed to the desired FIFO and ADC3 must be configured to not store the input. Input data is not defined for other configurations.

Data from more than one input source can be stored in the same FIFO. In the case that more than one channel of the input stage is directed into a FIFO, the data received is guaranteed to be written to the FIFO ordered (in time sequence) ADC0, ADC2, ADC1 and ADC3. Similarly, if data is received from more than one PCM input channel, the data is guaranteed to be written to the FIFO ordered PCM0, PCM1. For details about how input FIFO data is written to memory, see Section 6.3.1, “FIFO Input and Output Block” on page 45.

Data that is received from dynamic range extended inputs are received aligned in memory where the later of their two component input channel sources would be. As a result, data received from IDRX0 will be placed where the data would have been received by ADC1. Similarly, data received from IDRX1 will be placed where the data would have been received by ADC3.

Storing the data from the PCM interface to the same location as the input channels is more difficult. As a result, in most cases we recommend that you configure the input channels and the PCM inputs to be received in different FIFOs. However, if PCM data must share the same FIFO as ADC data, one of the following situations will occur:

- If the IOC has synchronized the ADC input channels and PCM interface (as described in Section 7.5, “IOC Data Synchronization”) the PCM data will be written to the FIFO after the input channel data. In this configuration, the PCM data is received at the same time as the data from the input channels, and data is written to the FIFO from (in order) ADC0, ADC2, ADC1/IDRX0, ADC3/IDRX1, PCM0 and PCM1.
- If the IOC has not synchronized the input channels with the PCM interface, and an even number of input channels and both PCM sources are being directed to one FIFO, the IOC will insert the data from the PCM when received following all of the input channels. As a result when PCM input data is available, data is written to the FIFO from (in order) ADC0, ADC2, ADC1/IDRX0, ADC3/IDRX1, PCM0 and PCM1. When PCM data is not available, the data in the FIFO is ordered ADC0, ADC2, ADC1/IDRX0 and ADC3/IDRX1.
- If the IOC has not been configured for synchronization, a FIFO is being shared between a single input channel and a single PCM input, and the FIFO is not configured for double access, any data received simultaneously will be ordered ADCx/IDRXx first and then PCMx.
- If none of the above cases hold, the ordering of data stored in FIFOs from the IOC is not guaranteed.

IMPORTANT: If an application is not using data synchronization, we recommend not sharing any FIFOs between the PCM input channels and any other input source. Further, if an application is using data synchronization, all inputs from PCM input channels controlled by the IOC must be targeted to a FIFO shared with another input source.

IMPORTANT: Data synchronization between the ADCs and the PCM interface is only possible if these interfaces are using synchronous clock sources. Therefore, if clocking the PCM interface externally, the ADCs and system should be clocked externally by this same clock source.

Data from a single source being written to a FIFO may be configured to be written to the FIFO either one word at a time or two words at a time. Two words are written if the FIFO is configured for double access by setting the appropriate IOC_FIFO_CFG_FIFOxx_DBL_ACC bit in the D_IOC_FIFO_CFG register; otherwise each sample is written as it is received. Using double access mode reduces the number of times the IOC accesses the FIFOs, thus reducing the possibility of FIFO access conflicts with the CFX or the HEAR.

Data from an even number of sources are always written to a FIFO in pairs, regardless of the double access setting for the FIFO. Data from an odd number of sources (not including the single source case described above) are always written individually, again regardless of the double access setting for the FIFO.

Note: Writing the D_IOC_INPUT_CFG or the D_IOC_FIFO_CFG register resets the input state machine of the IOC.

7.2.1 Control and Configuration Registers

Register Name	Register Description	Address
D_IOC_INPUT_CFG	Configuration of where data from input sources is stored	0xE0C0
D_IOC_FIFO_CFG	Configuration of data formatting and FIFO behavior for the IOC	0xE0C1

7.2.1.1 D_IOC_INPUT_CFG Settings

Bit Field	Field Name	Field Description
23:20	IOC_INPUT_CFG_PCM1_STORE	Select where to store data from PCM1
19:16	IOC_INPUT_CFG_PCM0_STORE	Select where to store data from PCM0
15:12	IOC_INPUT_CFG_ADC3_STORE	Select where to store data from ADC3
11:8	IOC_INPUT_CFG_ADC2_STORE	Select where to store data from ADC2
7:4	IOC_INPUT_CFG_ADC1_STORE	Select where to store data from ADC1
3:0	IOC_INPUT_CFG_ADC0_STORE	Select where to store data from ADC0

Field Name	Value Symbol	Value Description	Hex Value
IOC_INPUT_CFG_PCM1_STORE	IOC_INPUT_CFG_PCM1_FA0	Store data from PCM1 to FIFO A0	0x0
	IOC_INPUT_CFG_PCM1_FA1	Store data from PCM1 to FIFO A1	0x1
	IOC_INPUT_CFG_PCM1_FA2	Store data from PCM1 to FIFO A2	0x2
	IOC_INPUT_CFG_PCM1_FA3	Store data from PCM1 to FIFO A3	0x3
	IOC_INPUT_CFG_PCM1_FB0	Store data from PCM1 to FIFO B0	0x4
	IOC_INPUT_CFG_PCM1_FB1	Store data from PCM1 to FIFO B1	0x5
	IOC_INPUT_CFG_PCM1_FB2	Store data from PCM1 to FIFO B2	0x6
	IOC_INPUT_CFG_PCM1_FB3	Store data from PCM1 to FIFO B3	0x7
	IOC_INPUT_CFG_PCM1_NONE	Do not store data from PCM1	0xF*

Field Name	Value Symbol	Value Description	Hex Value
IOC_INPUT_CFG_PCM0_STORE	IOC_INPUT_CFG_PCM0_FA0	Store data from PCM0 to FIFO A0	0x0
	IOC_INPUT_CFG_PCM0_FA1	Store data from PCM0 to FIFO A1	0x1
	IOC_INPUT_CFG_PCM0_FA2	Store data from PCM0 to FIFO A2	0x2
	IOC_INPUT_CFG_PCM0_FA3	Store data from PCM0 to FIFO A3	0x3
	IOC_INPUT_CFG_PCM0_FB0	Store data from PCM0 to FIFO B0	0x4
	IOC_INPUT_CFG_PCM0_FB1	Store data from PCM0 to FIFO B1	0x5
	IOC_INPUT_CFG_PCM0_FB2	Store data from PCM0 to FIFO B2	0x6
	IOC_INPUT_CFG_PCM0_FB3	Store data from PCM0 to FIFO B3	0x7
	IOC_INPUT_CFG_PCM0_NONE	Do not store data from PCM0	0xF*
IOC_INPUT_CFG_ADC3_STORE	IOC_INPUT_CFG_ADC3_FA0	Store data from ADC3 to FIFO A0	0x0
	IOC_INPUT_CFG_ADC3_FA1	Store data from ADC3 to FIFO A1	0x1
	IOC_INPUT_CFG_ADC3_FA2	Store data from ADC3 to FIFO A2	0x2
	IOC_INPUT_CFG_ADC3_FA3	Store data from ADC3 to FIFO A3	0x3
	IOC_INPUT_CFG_ADC3_FB0	Store data from ADC3 to FIFO B0	0x4
	IOC_INPUT_CFG_ADC3_FB1	Store data from ADC3 to FIFO B1	0x5
	IOC_INPUT_CFG_ADC3_FB2	Store data from ADC3 to FIFO B2	0x6
	IOC_INPUT_CFG_ADC3_FB3	Store data from ADC3 to FIFO B3	0x7
	IOC_INPUT_CFG_ADC3_NONE	Do not store data from ADC3	0xF*
IOC_INPUT_CFG_ADC2_STORE	IOC_INPUT_CFG_ADC2_FA0	Store data from ADC2 to FIFO A0	0x0
	IOC_INPUT_CFG_ADC2_FA1	Store data from ADC2 to FIFO A1	0x1
	IOC_INPUT_CFG_ADC2_FA2	Store data from ADC2 to FIFO A2	0x2
	IOC_INPUT_CFG_ADC2_FA3	Store data from ADC2 to FIFO A3	0x3
	IOC_INPUT_CFG_ADC2_FB0	Store data from ADC2 to FIFO B0	0x4
	IOC_INPUT_CFG_ADC2_FB1	Store data from ADC2 to FIFO B1	0x5
	IOC_INPUT_CFG_ADC2_FB2	Store data from ADC2 to FIFO B2	0x6
	IOC_INPUT_CFG_ADC2_FB3	Store data from ADC2 to FIFO B3	0x7
	IOC_INPUT_CFG_ADC2_NONE	Do not store data from ADC2	0xF*
IOC_INPUT_CFG_ADC1_STORE	IOC_INPUT_CFG_ADC1_FA0	Store data from ADC1 to FIFO A0	0x0
	IOC_INPUT_CFG_ADC1_FA1	Store data from ADC1 to FIFO A1	0x1
	IOC_INPUT_CFG_ADC1_FA2	Store data from ADC1 to FIFO A2	0x2
	IOC_INPUT_CFG_ADC1_FA3	Store data from ADC1 to FIFO A3	0x3
	IOC_INPUT_CFG_ADC1_FB0	Store data from ADC1 to FIFO B0	0x4
	IOC_INPUT_CFG_ADC1_FB1	Store data from ADC1 to FIFO B1	0x5
	IOC_INPUT_CFG_ADC1_FB2	Store data from ADC1 to FIFO B2	0x6
	IOC_INPUT_CFG_ADC1_FB3	Store data from ADC1 to FIFO B3	0x7
	IOC_INPUT_CFG_ADC1_NONE	Do not store data from ADC1	0xF*

Field Name	Value Symbol	Value Description	Hex Value
IOC_INPUT_CFG_ADC0_STORE	IOC_INPUT_CFG_ADC0_FA0	Store data from ADC0 to FIFO A0	0x0
	IOC_INPUT_CFG_ADC0_FA1	Store data from ADC0 to FIFO A1	0x1
	IOC_INPUT_CFG_ADC0_FA2	Store data from ADC0 to FIFO A2	0x2
	IOC_INPUT_CFG_ADC0_FA3	Store data from ADC0 to FIFO A3	0x3
	IOC_INPUT_CFG_ADC0_FB0	Store data from ADC0 to FIFO B0	0x4
	IOC_INPUT_CFG_ADC0_FB1	Store data from ADC0 to FIFO B1	0x5
	IOC_INPUT_CFG_ADC0_FB2	Store data from ADC0 to FIFO B2	0x6
	IOC_INPUT_CFG_ADC0_FB3	Store data from ADC0 to FIFO B3	0x7
	IOC_INPUT_CFG_ADC0_NONE	Do not store data from ADC0	0xF*

7.2.1.2 D_IOC_FIFO_CFG Settings

Bit Field	Field Name	Field Description
11	IOC_FIFO_CFG_ADC3_SHIFT	Select the format for the input data from ADC3
10	IOC_FIFO_CFG_ADC2_SHIFT	Select the format for the input data from ADC2
9	IOC_FIFO_CFG_ADC1_SHIFT	Select the format for the input data from ADC1
8	IOC_FIFO_CFG_ADC0_SHIFT	Select the format for the input data from ADC0
7	IOC_FIFO_CFG_FIFOB3_DBL_ACC	Configure FIFO B3 for double-access mode
6	IOC_FIFO_CFG_FIFOB2_DBL_ACC	Configure FIFO B2 for double-access mode
5	IOC_FIFO_CFG_FIFOB1_DBL_ACC	Configure FIFO B1 for double-access mode
4	IOC_FIFO_CFG_FIFOB0_DBL_ACC	Configure FIFO B0 for double-access mode
3	IOC_FIFO_CFG_FIFOA3_DBL_ACC	Configure FIFO A3 for double-access mode
2	IOC_FIFO_CFG_FIFOA2_DBL_ACC	Configure FIFO A2 for double-access mode
1	IOC_FIFO_CFG_FIFOA1_DBL_ACC	Configure FIFO A1 for double-access mode
0	IOC_FIFO_CFG_FIFOA0_DBL_ACC	Configure FIFO A0 for double-access mode

Field Name	Value Symbol	Value Description	Hex Value
IOC_FIFO_CFG_ADC3_SHIFT	IOC_FIFO_CFG_ADC3_SHIFT_RIGHT	Align the ADC3 data in the lower 18 bits and sign extend to 24 bits	0x0*
	IOC_FIFO_CFG_ADC3_SHIFT_LEFT	Align the ADC3 data in the high 18 bits and zero pad to 24 bits	0x1
IOC_FIFO_CFG_ADC2_SHIFT	IOC_FIFO_CFG_ADC2_SHIFT_RIGHT	Align the ADC2 data in the lower 18 bits and sign extend to 24 bits	0x0*
	IOC_FIFO_CFG_ADC2_SHIFT_LEFT	Align the ADC2 data in the high 18 bits and zero pad to 24 bits	0x1

Field Name	Value Symbol	Value Description	Hex Value
IOC_FIFO_CFG_ADC1_SHIFT	IOC_FIFO_CFG_ADC1_SHIFT_RIGHT	Align the ADC1 data in the lower 18 bits and sign extend to 24 bits	0x0*
	IOC_FIFO_CFG_ADC1_SHIFT_LEFT	Align the ADC1 data in the high 18 bits and zero pad to 24 bits	0x1
IOC_FIFO_CFG_ADC0_SHIFT	IOC_FIFO_CFG_ADC0_SHIFT_RIGHT	Align the ADC0 data in the lower 18 bits and sign extend to 24 bits	0x0*
	IOC_FIFO_CFG_ADC0_SHIFT_LEFT	Align the ADC0 data in the high 18 bits and zero pad to 24 bits	0x1
IOC_FIFO_CFG_FIFOB3_DBL_ACC	IOC_FIFO_CFG_FB3_DBL_ACC_DIS	Disable double-access mode for FIFO B3	0x0*
	IOC_FIFO_CFG_FB3_DBL_ACC_EN	Enable double-access mode for FIFO B3	0x1
IOC_FIFO_CFG_FIFOB2_DBL_ACC	IOC_FIFO_CFG_FB2_DBL_ACC_DIS	Disable double-access mode for FIFO B2	0x0*
	IOC_FIFO_CFG_FB2_DBL_ACC_EN	Enable double-access mode for FIFO B2	0x1
IOC_FIFO_CFG_FIFOB1_DBL_ACC	IOC_FIFO_CFG_FB1_DBL_ACC_DIS	Disable double-access mode for FIFO B1	0x0*
	IOC_FIFO_CFG_FB1_DBL_ACC_EN	Enable double-access mode for FIFO B1	0x1
IOC_FIFO_CFG_FIFOB0_DBL_ACC	IOC_FIFO_CFG_FB0_DBL_ACC_DIS	Disable double-access mode for FIFO B0	0x0*
	IOC_FIFO_CFG_FB0_DBL_ACC_EN	Enable double-access mode for FIFO B0	0x1
IOC_FIFO_CFG_FIFOA3_DBL_ACC	IOC_FIFO_CFG_FA3_DBL_ACC_DIS	Disable double-access mode for FIFO A3	0x0*
	IOC_FIFO_CFG_FA3_DBL_ACC_EN	Enable double-access mode for FIFO A3	0x1
IOC_FIFO_CFG_FIFOA2_DBL_ACC	IOC_FIFO_CFG_FA2_DBL_ACC_DIS	Disable double-access mode for FIFO A2	0x0*
	IOC_FIFO_CFG_FA2_DBL_ACC_EN	Enable double-access mode for FIFO A2	0x1
IOC_FIFO_CFG_FIFOA1_DBL_ACC	IOC_FIFO_CFG_FA1_DBL_ACC_DIS	Disable double-access mode for FIFO A1	0x0*
	IOC_FIFO_CFG_FA1_DBL_ACC_EN	Enable double-access mode for FIFO A1	0x1
IOC_FIFO_CFG_FIFOA0_DBL_ACC	IOC_FIFO_CFG_FA0_DBL_ACC_DIS	Disable double-access mode for FIFO A0	0x0*
	IOC_FIFO_CFG_FA0_DBL_ACC_EN	Enable double-access mode for FIFO A0	0x1

7.3 IOC OUTPUT

On the output side, the IOC is attached to the single output channel of the output stage. This channel uses 18 bits of data for output through the digital output and is denoted as DAC in constants and documentation. You can attach the IOC to up to two output channels on the PCM interface (denoted again as PCM0, PCM1 in constants and documentation). The data for PCM outputs is interpreted as 24-bit data and post-processed for use on the PCM interface as necessary through the PCM configuration.

Each of the outputs can use output data from any of the eight FIFOs, from any of the input sources directly, or no data at all. Select a configuration by writing to the IOC_OUTPUT_CFG_xxx_LOAD bit fields in the D_IOC_OUTPUT_CFG register. For a description of the various configurations, see Table 12.

IMPORTANT: Connecting IOC output sinks to FIFOs that have not been configured results in undefined behavior.

TABLE 12: IOC OUTPUT CONFIGURATIONS

Source(s)	Sink(s)	Description
FIFO or one of the PCM interface inputs	Output stage	Choose either the high or low bits of the 18 bits of output data from 24-bit FIFO data or 24-bit PCM data by using the IOC_FIFO_CFG_DAC_SHIFT bit from the D_IOC_FIFO_CFG register. When using the low 18 bits of output data, the input data can be configured to saturate if it exceeds the 18 bit range by setting the IOC_FIFO_CFG_DAC_SATURATE bit in the D_IOC_FIFO_CFG register. If using a PCM interface input as the source for the output stage, do not synchronize the PCM interface to the output stage (as described in Section 7.5, “IOC Data Synchronization”).
Input channel	Output stage	18 bits of input data are mapped directly to the 18 bits of output data.
Input channel	Output PCM channel	Data is aligned to the lower 18 bits then sign-extended to 24 bits.
Input PCM channel	Output PCM channel	Data is used directly as 24-bit data. To select less than 24 bits of PCM output data, use the PCM interface configuration.
One FIFO	Two PCM outputs	You can order the data into a single output FIFO. FIFO data can be read simultaneously (guaranteeing the even numbered words being used for PCM0 and odd numbered words for PCM1) by setting the IOC_OUTPUT_CFG_PCM_STEREO bit in the D_IOC_OUTPUT_CFG register. Setting the IOC_OUTPUT_CFG_PCM_DBL_ACC bit in the D_IOC_OUTPUT_CFG register allows the IOC to read data for both PCM outputs simultaneously. If the appropriate bit is not set or if only one PCM interface output is enabled, PCM output data is read from the FIFO one word at a time.

TABLE 12: IOC OUTPUT CONFIGURATIONS

Source(s)	Sink(s)	Description
Two different FIFOs	Two PCM outputs	PCM output data is read from the FIFOs one word at a time.
One FIFO	Output stage and one or both of the PCM interface outputs	If the PCM interface and DAC are synchronized (as described in Section 7.5, “IOC Data Synchronization”), then all samples are sent to all sinks. In this configuration, the resolution of the PCM output data is limited to the 18 most significant or least significant bits as configured by the IOC_FIFO_CFG_DAC_SHIFT bit from the D_IOC_FIFO_CFG register for the DAC output. The supplied 18-bit data is padded with zeros if the PCM data requires a greater data width.

When reading data to the output source, configuring the IOC_OUTPUT_CFG_DAC_DBL_ACC bit in the D_IOC_OUTPUT_CFG register selects between loading words individually or in pairs for use as output. This configuration only works if the associated FIFO uses an even block size.

CAUTION: If the output stage and PCM interface are not synchronized, sharing one FIFO as the source for data to both the output stage and one or both of the PCM interface outputs results in undefined behavior.

Note: Writing the D_IOC_OUTPUT_CFG or the D_IOC_FIFO_CFG register resets the output state machine of the IOC.

7.3.1 Control and Configuration Registers

Register Name	Register Description	Address
D_IOC_FIFO_CFG	Configuration of data formatting and FIFO behavior for the IOC	0xE0C1
D_IOC_OUTPUT_CFG	Configuration of where data for output sources is acquired and output behavior. For more information about the bit fields of this register, see Section 9.2.1.1, “D_IOC_OUTPUT_CFG Settings” on page 86.	0xE0C2

7.3.1.1 D_IOC_FIFO_CFG Settings

Bit Field	Field Name	Field Description
13	IOC_FIFO_CFG_DAC_SATURATE	Configure saturation of the input data to the DAC output
12	IOC_FIFO_CFG_DAC_SHIFT	Select the format for the output data provided to the DAC

Field Name	Value Symbol	Value Description	Hex Value
IOC_FIFO_CFG_DAC_SATURATE	IOC_FIFO_CFG_DAC_UNSATURATE	DAC input data is not saturated	0x0*
	IOC_FIFO_CFG_DAC_SATURATE	DAC input data points exceeding 18 bits of resolution are saturated	0x1
IOC_FIFO_CFG_DAC_SHIFT	IOC_FIFO_CFG_DAC_SHIFT_RIGHT	DAC input data is aligned to the lower 18 bits and sign extended to 24 bits	0x0*
	IOC_FIFO_CFG_DAC_SHIFT_LEFT	DAC input data is aligned to the high 18 bits and zero padded to 24 bits	0x1

7.4 IOC DATA TRANSFER TIMING

The IOC coordinates the sequential movement of data to and from the input and output components. The IOC creates a fixed ordering of data transfers to and from the FIFOs that results in the associated FIFO interrupts being generated in the same fixed order.

IMPORTANT: To provide a fixed ordering of data reception in the FIFOs (and consequently a fixed ordering of FIFO interrupts), the IOC must not be providing data to or requesting data from a FIFO while that FIFO is being configured.

If the PCM interface has been synchronized with the ADC and DAC components, as described in Section 7.5, “IOC Data Synchronization” on page 60, interrupts generated by IOC data transfers between the PCM interface and the FIFOs have a guaranteed timing relative to the other FIFO interrupts. When unsynchronized, the relative timing of interrupts generated for PCM interface data to other FIFO interrupts cannot be guaranteed.

The timing of software FIFO interrupts depends on the application using the software FIFO and is not controlled by the IOC.

The ordering and relative timing of the IOC data transfers to and from the FIFOs is found in Table 13. The timing of the FIFO interrupts is based on this ordering and relative timing, as well as the configuration of each FIFO’s block size and pointers.

TABLE 13: RELATIVE IOC DATA TRANSFER TIMING BETWEEN FIFOS AND SYSTEM COMPONENTS

Ordering	Components	Relative Timing
1	ADC0, ADC2	Data is transferred from ADC0 and ADC2 at the beginning of each sample period.
2	DAC	Data is transferred after a delay of approximately 1/4 of the sampling period from the beginning of the sampling period.

TABLE 13: RELATIVE IOC DATA TRANSFER TIMING BETWEEN FIFOS AND SYSTEM COMPONENTS

Ordering	Components	Relative Timing
3	ADC1, ADC3, IDRX0, IDRX1	Data is transferred after a delay of approximately 1/2 of the sampling period from the beginning of the sampling period.
4	PCM0, PCM1 (Receive) and PCM0, PCM1 (Transmit)	Data transfers due to PCM input data occur at the same time within a sample period as interrupts due to PCM output data with both occurring at the later of: • Immediately following the last enabled ADC or DAC • When data for the PCM interface arrives or is needed by the interface for transmission.

These timing guarantees and this sequencing is important for applications that process data synchronously, triggering their input and output processing on a single interrupt (typically a FIFO interrupt). In these situations, the application normally waits for the last interrupt to arrive before beginning to process the data. For example, timing must be considered when a FIFO interrupt is generated for a block of data transferred from ADC0, since a block of data from ADC1 that should be received and processed synchronously with this data would not yet have been completely received, and a previous output result block for the DAC would not yet have been completely transmitted.

To ensure that the IOC has completed transferring the current block of data for all of the relevant components, the interrupt associated with the last scheduled data transfer should be used for synchronization. As a result, a user application using ADC0 for input and the DAC for output would use the FIFO interrupt associated with the DAC to synchronize processing. Similarly, if the same application instead used ADC1 and the DAC, the FIFO interrupt associated with ADC1 would be used.

7.5 IOC DATA SYNCHRONIZATION

One important configuration that can be done using the IOC is the synchronization of the input and output stages with the PCM interface. To synchronize these components:

- Configure the PCM interface to operate at the sampling frequency that is used by the input and output stages.
- Set the `IOC_OUTPUT_CFG_PCM_SYNC_ADCDAC` bit in the `IOC_OUTPUT_CFG` register.

This configuration is referenced in the descriptions of IOC input and output behavior in the previous sections.

IMPORTANT: Synchronization of the PCM interface with the ADC/DAC should only be used if a user application will share one FIFO between input PCM and ADC data (providing interleaved input data) or will share one FIFO between output PCM and DAC data (providing interleaved output data). Otherwise, do not use synchronization because if it is enabled and PCM input data is being loaded to a FIFO that is not shared, the IOC will load duplicate samples from the PCM into the associated FIFO.

7.5.1 Control and Configuration Registers

Register Name	Register Description	Address
D_IOC_OUTPUT_CFG	Configuration of where data for output sources is acquired and output behavior	0xE0C2

7.5.1.1 D_IOC_OUTPUT_CFG Settings

Bit Field	Field Name	Field Description
15	IOC_OUTPUT_CFG_PCM_SYNC_ADCDAC	Enable/disable synchronization between the PCM interface and the input and output stages

Field Name	Value Symbol	Value Description	Hex Value
IOC_OUTPUT_CFG_PCM_SYNC_ADCDAC	IOC_OUTPUT_CFG_PCM_SYNC_ADCDAC_DIS	Disable synchronization between PCM and the input and output stages	0x0*
	IOC_OUTPUT_CFG_PCM_SYNC_ADCDAC_EN	Enable synchronization between PCM and the input and output stages	0x1

This page left blank for printing purposes.

8 Input Stage

8.1 Input Stage Architecture

The input stage of a BelaSigna 300 device provides four audio input channels that supply signal data to the rest of the BelaSigna 300 system through the IOC. Each input channel includes:

- Input selection using an analog multiplexer to select between 4 input sources for each channel from the available 5 input sources (Note: analog input source 3 is grounded for BelaSigna 300 devices with a CID of 201)
- Programmable input preamplification to adjust the selected raw input signal to a suitable level for the A/D converters
- An anti-aliasing low-pass filter with a selectable 20kHz or 50kHz (bypass) -3dB cut-off frequency
- Available line-out for the preamplified and filtered input signal
- An over-sampling analog-to-digital converter (ADC) which uses a programmable sampling frequency and provides a configurable sampling delay
- High-quality decimation filtering at all selectable sampling rates
- Selectable input muting
- Configurable power consumption for the ADC components

Each of these components are shown in Figure 11, which shows the overall architecture of the input stage.

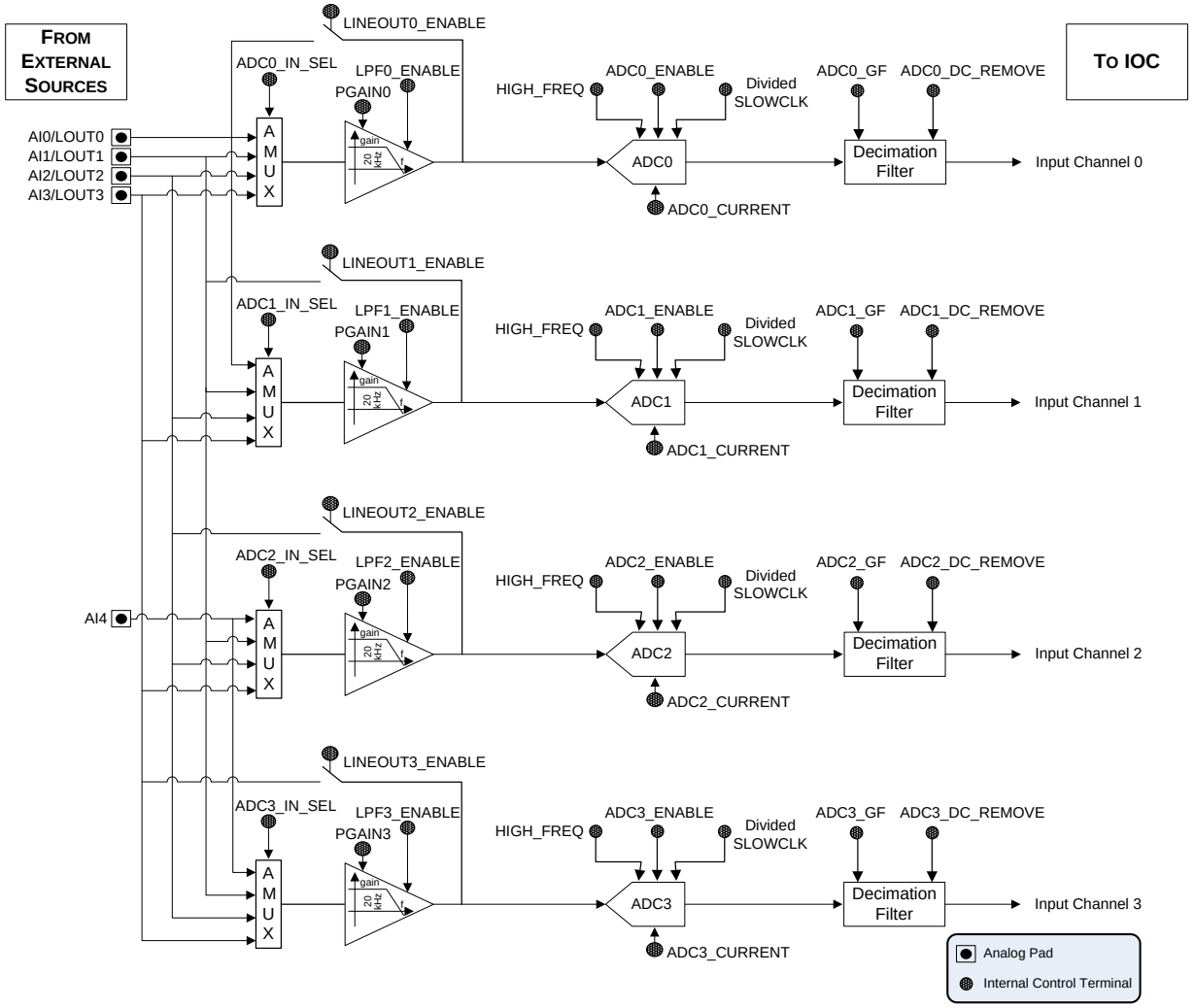


Figure 11: Input Stage Block Diagram

Note: In BelaSigna 300 devices with CID 201, the AI3/LOUT3 pad is internally grounded, limiting the available input signal sources for the input channels and eliminating line-out 3.

8.2 INPUT SELECTION

The four input channels select their signal source from four of the five input pads through the `A_INPUT_CTRL` register. Channels 0 and 1 use the `INPUT_CTRL_ADC0_IN_SEL` and `INPUT_CTRL_ADC1_IN_SEL` bit fields respectively to select one of the analog input pins 0, 1, 2 or 3 as the input source for the channel. Channels 2 and 3 use the `INPUT_CTRL_ADC2_IN_SEL` and `INPUT_CTRL_ADC3_IN_SEL` bit fields respectively to select one of analog input pins 1, 2, 3 or 4 as the input source for the channel.

Note: In BelaSigna 300 devices with CID 201, the AI3/LOUT3 pad is internally grounded, limiting the useful input signal sources that can be selected as the source for the BelaSigna 300 input channels.

The system also has the INPUT_CTRL_ADCx_ENABLE bits from the A_INPUT_CTRL register that allow you to selectively enable or disable the receipt of data on each of the 4 input stage channels.

CAUTION: Do not use the same input source for multiple input channels or a reduction in the signal quality for all channels occurs due to the associated reduction of input impedance. This is an issue even if only one of the input channels sharing an input source is active.

External de-coupling capacitors are required for all inputs that are used in the system. Ground unused input pins.

8.2.1 Input Dynamic Range Extension

The four input channels on the BelaSigna 300 device can be combined into pairs to increase the input dynamic range available to an application. This can be used to avoid design trade-offs between the sufficient amplification of low-level input signals and the saturation of high-level signals. The extension of the input dynamic range provides a single channel of 23-bit input data from each pair of inputs by combining channel 0 with 1 and channel 2 with 3.

The INPUT_CTRL_IDRXx_ENABLE bits from the A_INPUT_CTRL register allow you to selectively enable or disable the use of these channel pairs as dynamic range extended input stage channels.

When configured to provide an extended input dynamic range, the following settings are controlled internally and overwritten by the hardware:

- The gain setting for the preamplifier belonging to each component input channel
- The decimation filter gain factor setting belonging to each component input channel
- The sampling delay associated with channel 0 for IDRX0, and channel 2 for IDRX1

To use a dynamic range extended input channel, the inputs to that channel must be connected together externally to the BelaSigna 300 device. However, a separate external coupling capacitor is still needed for each of the inputs connected this way, as the input impedances will differ between channels based on the preamplifier gain configuration selected by the system for each component input channel.

IMPORTANT: If the DC removal filters provided by the decimation filters for either component channel of a dynamic range extended input channel are disabled, the combined input signal channel will be corrupted.

Configuration of the extended dynamic range input channels is performed through the A_IDRX0_CFG and A_IDRX1_CFG configuration registers. These registers provide bit fields that allow an application to configure the timers, signal limits and available preamp gain settings belonging to the underlying input controller for each extended dynamic range input channel. Information about these configuration parameters and limitations on configuring the input channels is available in the *Input Dynamic Range Extension of the Ezairo 5900 and BelaSigna 300 Series* application note, which you can obtain from ON Semiconductor Customer Support.

The IDRX_CFG_IDRX_SEL bit from the IDRX configuration registers indicates the underlying channel that is currently providing the data to the combined input channel. The combined input channel can be configured to disable an ADC from an input channel when it is not providing data to the combined channel by setting the IDRX_CFG_ADC_OFF_EN bit in the configuration register.

8.2.2 Control and Configuration Registers

Register Name	Register Description	Address
A_INPUT_CTRL	General analog input control	0xE141
A_IDRX0_CFG	Configure the dynamic range extended input channel 0	0xE14C
A_IDRX1_CFG	Configure the dynamic range extended input channel 1	0xE14D

8.2.2.1 A_INPUT_CTRL Settings

Bit Field	Field Name	Field Description
15	INPUT_CTRL_IDRX1_ENABLE	Enable/disable dynamic range extended input using channels 2 and 3
14:13	INPUT_CTRL_ADC3_IN_SEL	Select the input source for input channel 3
12	INPUT_CTRL_ADC3_ENABLE	Enable/disable input on input channel 3
10:9	INPUT_CTRL_ADC2_IN_SEL	Select the input source for input channel 2
8	INPUT_CTRL_ADC2_ENABLE	Enable/disable input on input channel 2
7	INPUT_CTRL_IDRX0_ENABLE	Enable/disable dynamic range extended input using channels 0 and 1
6:5	INPUT_CTRL_AD1_IN_SEL	Select the input source for input channel 1
4	INPUT_CTRL_ADC1_ENABLE	Enable/disable input on input channel 1
2:1	INPUT_CTRL_ADC0_IN_SEL	Select the input source for input channel 0
0	INPUT_CTRL_ADC0_ENABLE	Enable/disable input on input channel 0

Field Name	Value Symbol	Value Description	Hex Value
INPUT_CTRL_IDRX1_ENABLE	IDRX1_DISABLE	Disable dynamic range extended input channel 1	0x0*
	IDRX1_ENABLE	Enable dynamic range extended input channel 1	0x1
INPUT_CTRL_ADC3_IN_SEL	ADC3_SEL_AI4	Select AI4 for input channel 3	0x0
	ADC3_SEL_AI1	Select AI1 for input channel 3	0x1
	ADC3_SEL_AI2	Select AI2 for input channel 3	0x2
	ADC3_SEL_AI3	Select AI3 for input channel 3 (for CID=201 this input source is grounded)	0x3*

Field Name	Value Symbol	Value Description	Hex Value
INPUT_CTRL_ADC3_ENABLE	ADC3_DISABLE	Disable input channel 3	0x0*
	ADC3_ENABLE	Enable input channel 3	0x1
INPUT_CTRL_ADC2_IN_SEL	ADC2_SEL_AI4	Select AI4 for input channel 2	0x0
	ADC2_SEL_AI1	Select AI1 for input channel 2	0x1
	ADC2_SEL_AI2	Select AI2 for input channel 2	0x2*
	ADC2_SEL_AI3	Select AI3 for input channel 2 (for CID=201 this input source is grounded)	0x3
INPUT_CTRL_ADC2_ENABLE	ADC2_DISABLE	Disable input channel 2	0x0*
	ADC2_ENABLE	Enable input channel 2	0x1
INPUT_CTRL_IDRX0_ENABLE	IDRX0_DISABLE	Disable dynamic range extended input channel 0	0x0*
	IDRX0_ENABLE	Enable dynamic range extended input channel 0	0x1
INPUT_CTRL_AD1_IN_SEL	ADC1_SEL_AI0	Select AI0 for input channel 1	0x0
	ADC1_SEL_AI1	Select AI1 for input channel 1	0x1*
	ADC1_SEL_AI2	Select AI2 for input channel 1	0x2
	ADC1_SEL_AI3	Select AI3 for input channel 1 (for CID=201 this input source is grounded)	0x3
INPUT_CTRL_ADC1_ENABLE	ADC1_DISABLE	Disable input channel 1	0x0*
	ADC1_ENABLE	Enable input channel 1	0x1
INPUT_CTRL_ADC0_IN_SEL	ADC0_SEL_AI0	Select AI0 for input channel 0	0x0*
	ADC0_SEL_AI1	Select AI1 for input channel 0	0x1
	ADC0_SEL_AI2	Select AI2 for input channel 0	0x2
	ADC0_SEL_AI3	Select AI3 for input channel 0 (for CID=201 this input source is grounded)	0x3
INPUT_CTRL_ADC0_ENABLE	ADC0_DISABLE	Disable input channel 0	0x0*
	ADC0_ENABLE	Enable input channel 0	0x1

8.2.2.2 A_IDRX0_CFG and A_IDRX1_CFG Settings

Bit Field	Field Name	Field Description
23	IDRX_CFG_IDRX_SEL	Indicates the input channel that is actively supplying input for the combined channel
22	IDRX_CFG_ADC_OFF_EN	Enable/disable the channel not actively supplying input
21:20	IDRX_CFG_MAX_PG	Configure the maximum preamp gain setting for the input channels

Bit Field	Field Name	Field Description
17:16	IDRX_CFG_MIN_PG	Configure the minimum preamp gain setting for the input channels
15:12	IDRX_CFG_HIGH_LIMIT	Configure the high limit for pre-amplified input signals
11:8	IDRX_CFG_LOW_LIMIT	Configure the low limit for pre-amplified input signals
7:4	IDRX_CFG_TIMER2	Delay for timer 2, in multiples of 256 samples
3:0	IDRX_CFG_TIMER1	Delay for timer 1, in multiples of 256 samples

Field Name	Value Symbol	Value Description	Hex Value
IDRX_CFG_ADC_OFF_EN	IDRX_INACTIVE_ADC_ENABLED	Keep the inactive dynamic range extended input ADC enabled	0x0*
	IDRX_INACTIVE_ADC_DISABLED	Allow the inactive dynamic range extended input ADC to be disabled	0x1
IDRX_CFG_MAX_PG	IDRX_MAX_PREAMP_GAIN_12	Select a maximum preamp gain of 12dB	0x0
	IDRX_MAX_PREAMP_GAIN_18	Select a maximum preamp gain of 18dB	0x1
	IDRX_MAX_PREAMP_GAIN_24	Select a maximum preamp gain of 24dB	0x2
	IDRX_MAX_PREAMP_GAIN_30	Select a maximum preamp gain of 30dB	0x3*
IDRX_CFG_MIN_PG	IDRX_MIN_PREAMP_GAIN_0	Select a minimum preamp gain of 0dB	0x0*
	IDRX_MIN_PREAMP_GAIN_12	Select a minimum preamp gain of 12dB	0x1
	IDRX_MIN_PREAMP_GAIN_18	Select a minimum preamp gain of 18dB	0x2
	IDRX_MIN_PREAMP_GAIN_24	Select a minimum preamp gain of 24dB	0x3

8.3 PREAMPLIFIERS

The input stage provides a preamplifier for each of the four input channels. When enabled these preamplifiers can amplify the input signal between 12 and 30dB in 3dB steps. Use the IN_GAIN_PGAINx bit fields from the A_IN_GAIN_CTRL register to select a preamplifier gain for each channel.

The preamplifiers used as part of the input stage introduce a delay in sampling the input signal by the system. This delay varies depending on the configuration of the IN_GAIN_PGAINx bit fields from the A_IN_GAIN_CTRL register, whether the input low-pass filter (LPF) is enabled and the selection of input coupling capacitor. The approximate amount of delay introduced by each configuration using a 10nF coupling capacitor and operating from a standard 1.28MHz slow clock is given in Table 14.

TABLE 14: APPROXIMATE SAMPLING DELAY INTRODUCED BY THE PREAMPLIFIER COMPONENT

Preamplifier Setting	Added Sampling Delay with Input LPF Disabled (μs)	Added Sampling Delay with Input LPF Enabled (μs)
PGAINx_BYPASS	0.0	2.0
PGAINx_12DB	6.0	11.0
PGAINx_15DB	6.5	12.0
PGAINx_18DB	7.0	13.0
PGAINx_21DB	7.5	13.5
PGAINx_24DB	8.0	14.0
PGAINx_27DB	8.5	15.0
PGAINx_30DB	9.0	16.0

The input stage uses an external reference signal for each input channel through the associated AIRxy pads which each provide a clean ground reference signal for the attached channels. We recommend that any input microphones be connected between the supplied microphone VREG pad and microphone GND pad, as the associated AIRxy reference is internally connected to the microphone GND pad. This results in a voltage drop of 1V across the microphone.

For optimal noise performance, calibrate the input stage after configuring the preamplifier gains. This calibration is performed by the `Sys_Set_IN_GAIN_CTRL`, `Sys_SetReg_IN_GAIN_CTRL` and `Sys_CalibrateADCs` macros. For more information about these calibration support macros, see the *Firmware Reference Manual*.

8.3.1 Control and Configuration Registers

Register Name	Register Description	Address
A_IN_GAIN_CTRL	Preamplifier gain control	0xE142

8.3.1.1 A_IN_GAIN_CTRL Settings

Bit Field	Field Name	Field Description
14:12	IN_GAIN_PGAIN3	Select the preamplifier gain for input channel 3
10:8	IN_GAIN_PGAIN2	Select the preamplifier gain for input channel 2
6:4	IN_GAIN_PGAIN1	Select the preamplifier gain for input channel 1
2:0	IN_GAIN_PGAIN0	Select the preamplifier gain for input channel 0

Field Name	Value Symbol	Value Description	Hex Value
IN_GAIN_PGAIN3	PGAIN3_BYPASS	Bypass the preamplifier gain	0x0*
	PGAIN3_12DB	Set the preamplifier gain to 12dB	0x1
	PGAIN3_15DB	Set the preamplifier gain to 15dB	0x2
	PGAIN3_18DB	Set the preamplifier gain to 18dB	0x3
	PGAIN3_21DB	Set the preamplifier gain to 21dB	0x4
	PGAIN3_24DB	Set the preamplifier gain to 24dB	0x5
	PGAIN3_27DB	Set the preamplifier gain to 27dB	0x6
	PGAIN3_30DB	Set the preamplifier gain to 30dB	0x7
IN_GAIN_PGAIN2	PGAIN2_BYPASS	Bypass the preamplifier gain	0x0*
	PGAIN2_12DB	Set the preamplifier gain to 12dB	0x1
	PGAIN2_15DB	Set the preamplifier gain to 15dB	0x2
	PGAIN2_18DB	Set the preamplifier gain to 18dB	0x3
	PGAIN2_21DB	Set the preamplifier gain to 21dB	0x4
	PGAIN2_24DB	Set the preamplifier gain to 24dB	0x5
	PGAIN2_27DB	Set the preamplifier gain to 27dB	0x6
	PGAIN2_30DB	Set the preamplifier gain to 30dB	0x7
IN_GAIN_PGAIN1	PGAIN1_BYPASS	Bypass the preamplifier gain	0x0*
	PGAIN1_12DB	Set the preamplifier gain to 12dB	0x1
	PGAIN1_15DB	Set the preamplifier gain to 15dB	0x2
	PGAIN1_18DB	Set the preamplifier gain to 18dB	0x3
	PGAIN1_21DB	Set the preamplifier gain to 21dB	0x4
	PGAIN1_24DB	Set the preamplifier gain to 24dB	0x5
	PGAIN1_27DB	Set the preamplifier gain to 27dB	0x6
	PGAIN1_30DB	Set the preamplifier gain to 30dB	0x7
IN_GAIN_PGAIN0	PGAIN0_BYPASS	Bypass the preamplifier gain	0x0*
	PGAIN0_12DB	Set the preamplifier gain to 12dB	0x1
	PGAIN0_15DB	Set the preamplifier gain to 15dB	0x2
	PGAIN0_18DB	Set the preamplifier gain to 18dB	0x3
	PGAIN0_21DB	Set the preamplifier gain to 21dB	0x4
	PGAIN0_24DB	Set the preamplifier gain to 24dB	0x5
	PGAIN0_27DB	Set the preamplifier gain to 27dB	0x6
	PGAIN0_30DB	Set the preamplifier gain to 30dB	0x7

8.4 LOW-PASS FILTERS

At the input, the signal can be passed through one of the anti-aliasing low-pass filters that have been combined with the preamplifiers. These filters can be used to attenuate high-frequency aliasing components of the input signals, partially eliminating this side-effect of digital sampling.

IMPORTANT: As the anti-aliasing low-pass filters are combined with the preamplification component, bypassing a preamplifier also bypasses the low-pass filter.

Each filter is a 2nd-order, active low-pass filter with a -3dB cut-off frequency at 20kHz. To enable the filter for a set of channels, use the appropriate ADC_CFG_LPFx_ENABLE bits in the A_ADC_CFG register.

The A/D converters oversample the input signal by a factor of eight. As a result, if a 16kHz sampling frequency is selected with the slow clock set to 1.28MHz, no aliased images appear for input frequencies below 120kHz. As a result, if considering the combined roll-off of the low-pass filters at a cut-off frequency of 20kHz and the roll-off present in a typical microphone, this configuration provides sufficient aliasing protection.

The low-pass filters add a delay component to the overall sampling delay for a channel. For further information on the approximate effects of enabling the low-pass filter on sampling delay, see Table 14 in Section 8.3, “Preamplifiers” on page 68.

8.4.1 Control and Configuration Registers

Register Name	Register Description	Address
A_ADC_CFG	Analog-to-digital converter configuration	0xE140

8.4.1.1 A_ADC_CFG Settings

Bit Field	Field Name	Field Description
13	ADC_CFG_LPF3_ENABLE	Select the low-pass filter for channel 3
12	ADC_CFG_LPF2_ENABLE	Select the low-pass filter for channel 2
11	ADC_CFG_LPF1_ENABLE	Select the low-pass filter for channel 1
10	ADC_CFG_LPF0_ENABLE	Select the low-pass filter for channel 0

Field Name	Value Symbol	Value Description	Hex Value
ADC_CFG_LPF3_ENABLE	LPF3_IN_BYPASS	Select a 50kHz cut-off frequency	0x0
	LPF3_IN_ACTIVE_20KHZ	Select a 20kHz cut-off frequency	0x1*
ADC_CFG_LPF2_ENABLE	LPF2_IN_BYPASS	Select a 50kHz cut-off frequency	0x0
	LPF2_IN_ACTIVE_20KHZ	Select a 20kHz cut-off frequency	0x1*
ADC_CFG_LPF1_ENABLE	LPF1_IN_BYPASS	Select a 50kHz cut-off frequency	0x0
	LPF1_IN_ACTIVE_20KHZ	Select a 20kHz cut-off frequency	0x1*

Field Name	Value Symbol	Value Description	Hex Value
ADC_CFG_LPF0_ENABLE	LPF0_IN_BYPASS	Select a 50kHz cut-off frequency	0x0
	LPF0_IN_ACTIVE_20KHZ	Select a 20kHz cut-off frequency	0x1*

8.5 LINE-OUT

The preamplified and low-pass-filtered input signals from channels 0 to 3 are available as output on the input pads AI0/LOUT0 to AI3/LOUT3. To configure an input pad for use as a line-out signal set the appropriate ADC_CFG_LINEOUTx_EBL bit in the A_ADC_CFG register.

IMPORTANT: AI3 is grounded for BelaSigna 300 devices with a CID of 201; therefore, for those devices, a developer must ensure that LOUT3 is not enabled for output.

8.5.1 Control and Configuration Registers

Register Name	Register Description	Address
A_ADC_CFG	Analog-to-digital converter configuration	0xE140

8.5.1.1 A_ADC_CFG Settings

Bit Field	Field Name	Field Description
9	ADC_CFG_LINEOUT3_ENABLE	Enable/disable line-out output of channel 3 data on AI3/LOUT3
8	ADC_CFG_LINEOUT2_ENABLE	Enable/disable line-out output of channel 2 data on AI2/LOUT2
7	ADC_CFG_LINEOUT1_ENABLE	Enable/disable line-out output of channel 1 data on AI1/LOUT1
6	ADC_CFG_LINEOUT0_ENABLE	Enable/disable line-out output of channel 0 data on AI0/LOUT0

Field Name	Value Symbol	Value Description	Hex Value
ADC_CFG_LINEOUT3_ENABLE	LINE_OUT3_DISABLE	Disable line-out on AI3	0x0*
	LINE_OUT3_ENABLE	Enable line-out on AI3 (for CID=201 this pad is grounded and LOUT3 must not be enabled)	0x1
ADC_CFG_LINEOUT2_ENABLE	LINE_OUT2_DISABLE	Disable line-out on AI2	0x0*
	LINE_OUT2_ENABLE	Enable line-out on AI2	0x1
ADC_CFG_LINEOUT1_ENABLE	LINE_OUT1_DISABLE	Disable line-out on AI1	0x0*
	LINE_OUT1_ENABLE	Enable line-out on AI1	0x1
ADC_CFG_LINEOUT0_ENABLE	LINE_OUT0_DISABLE	Disable line-out on AI0	0x0*
	LINE_OUT0_ENABLE	Enable line-out on AI0	0x1

8.6 A/D CONVERTERS

The analog-to-digital (A/D) converters supplied on the BelaSigna 300 device can be used to supply the system with data obtained at a configurable sampling rate. These A/D converters oversample the input signal by a factor of 8 to obtain a higher quality signal, which is then filtered by the decimation filter to obtain the desired sampling frequency.

Note: The high oversampling of the A/D converters shifts aliased signal components into a significantly higher frequency band. For example, if a sampling frequency of 16kHz is selected, no aliased images appears for input frequencies below 120kHz.

Select the sampling rate for the A/D converters by setting the ADC_CFG_SAMPLE_FREQ bit field from the A_ADC_CFG register. The A/D converter oversamples the incoming signal at 8 times this requested data rate. The sampling frequency can be calculated using the following equation:

$$f_{sampling} = \frac{f_{SLOWCLK}}{(ADC_CFG_SAMPLE_FREQ + 8) \cdot 8}$$

A table providing the sampling frequency obtained for each setting and slow clock configuration is in Appendix B, "Sampling Frequency Reference" on page 201. As specifying the sampling frequency largely depends on the slow clock configuration, refer to Section 13.2.2, "Slow Clock (SLOWCLK)" on page 164 for more information on configuring the slow clock.

The input and output stages of the BelaSigna 300 system both use the same sampling rate (more information about the output stage is in Chapter 9, "Output Stage" on page 85).

IMPORTANT: When using the A/D converters with a slow clock configuration that results in the supplied clock frequency exceeding 1.5MHz, set the ADC_CFG_HIGH_FREQ bit in the A_ADC_CFG register.

The current supplied to the A/D converters is configured by setting the ADC_CUR_CTRL_ADCx_CURRENT bit fields in the A_ADC_CUR_CTRL control register. In general, when using sampling frequencies below 20kHz, use the nominal value for these bit fields. For higher sampling frequencies, apply a higher current to prevent any reduction in fidelity. The nominal current (I_{nom}) is approximately 35µA for an A/D converter operating at 1.28MHz, with a supply voltage of 1.25V.

8.6.1 Sampling Delay

The A/D converters are designed so that you can precisely set the timing of when a signal is sampled. The specification of a sampling delay between channels is defined in terms of two components: an integer sampling delay component and a fractional sampling delay component.

Note: The time at which the signal on ADC0 would be sampled by default is the point of reference for all delays.

The integer component of the sampling delay specifies the delay in terms of the time between samples in the oversample domain. This can be seen in Figure 12, which shows the relationship between the sampling frequency of the A/D converter, and the integer component and fractional component of the sampling delay.

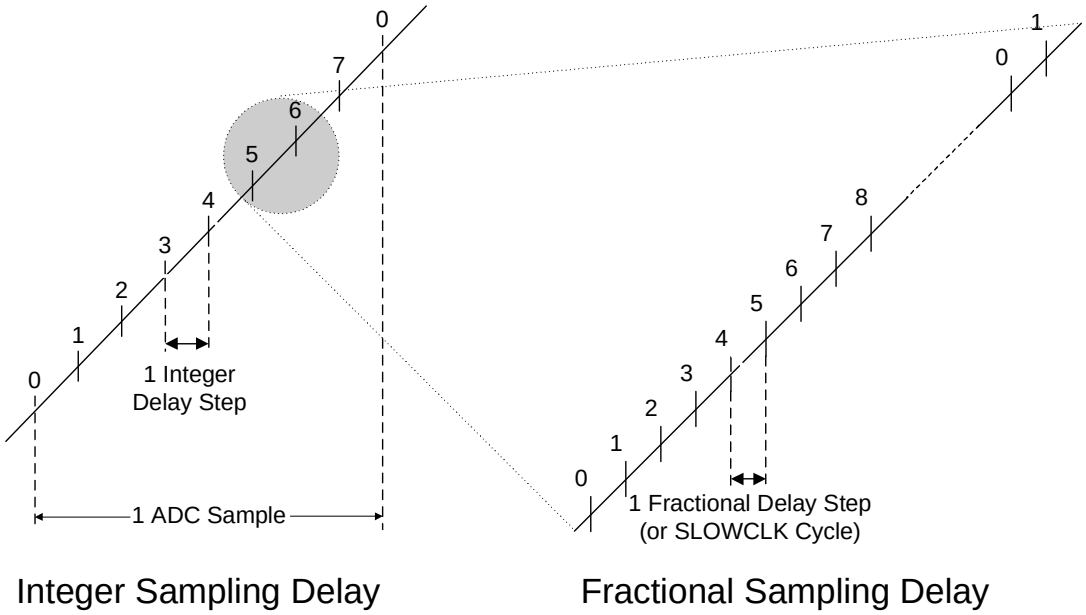


Figure 12: Sampling Delay Components

To set the integer delay for each channel, set the FRAC_DELAY_INTEGER bit fields of the A_FRAC_DELAY_x_CTRL registers. The integer delay allows for a delay of up to 7 additional oversampling periods or a delay of just under one full sample.

The fractional component of the sampling delay allows you to specify an additional delay in the timing of obtaining samples. This delay is measured in terms of additional slow clock cycles before the system takes a sample measurement in the oversample domain. You can see this in Figure 12 and specify it by setting the FRAC_DELAY_FRACTIONAL bit fields, again in the A_FRAC_DELAY_x_CTRL registers. This fine tuning of the sampling delay allows you to add to the integer delay component from 0 to $(\text{ADC_CFG_SAMPLE_FREQ} + 8)$ slow clock cycles of fractional delay.

CAUTION: The preamplifier stage and preamplifier configuration settings also contribute to the sampling delay. In particular if you are using different configurations for preamplifiers in your application, refer to Section 8.3, “Preamplifiers” on page 68 for more information.

8.6.2 Control and Configuration Registers

Register Name	Register Description	Address
D_ADC0_DATA	Most recent input data received on input channel 0	0xE0D0
D_ADC1_DATA	Most recent input data received on input channel 1	0xE0D1
D_ADC2_DATA	Most recent input data received on input channel 2	0xE0D2
D_ADC3_DATA	Most recent input data received on input channel 3	0xE0D3
A_ADC_CFG	Analog-to-digital converter configuration	0xE140

Register Name	Register Description	Address
A_ADC_CUR_CTRL	Control current supplied to the AD converters	0xE144
A_FRAC_DELAY_0_CTRL	Configure the fractional sampling delay for channel 0	0xE147
A_FRAC_DELAY_1_CTRL	Configure the fractional sampling delay for channel 1	0xE148
A_FRAC_DELAY_2_CTRL	Configure the fractional sampling delay for channel 2	0xE149
A_FRAC_DELAY_3_CTRL	Configure the fractional sampling delay for channel 3	0xE14A

8.6.2.1 A_ADC_CFG Settings

Bit Field	Field Name	Field Description
14	ADC_CFG_HIGH_FREQ	Select high or low frequency operation
5:0	ADC_CFG_SAMPLE_FREQ	Configure the input sampling frequency

Field Name	Value Symbol	Value Description	Hex Value
ADC_CFG_HIGH_FREQ	ADC_HIGH_FREQ_DISABLE	Select low frequency operation	0x0*
	ADC_HIGH_FREQ_ENABLE	Select high frequency operation	0x1

8.6.2.2 A_ADC_CUR_CTRL Settings

Bit Field	Field Name	Field Description
14:12	ADC_CUR_CTRL_ADC3_CURRENT	Select the current supplied to ADC 3
10:8	ADC_CUR_CTRL_ADC2_CURRENT	Select the current supplied to ADC 2
6:4	ADC_CUR_CTRL_ADC1_CURRENT	Select the current supplied to ADC 1
2:0	ADC_CUR_CTRL_ADC0_CURRENT	Select the current supplied to ADC 0

Field Name	Value Symbol	Value Description	Hex Value
ADC_CUR_CTRL_ADC3_CURRENT	ADC3_CURRENT_4_0X	Use 400% of the nominal current	0x0
	ADC3_CURRENT_2_0X	Use 200% of the nominal current	0x1
	ADC3_CURRENT_1_3X	Use 130% of the nominal current	0x2
	ADC3_CURRENT_1_0X	Use the nominal current	0x3*
	ADC3_CURRENT_0_8X	Use 80% of the nominal current	0x4
	ADC3_CURRENT_0_7X	Use 70% of the nominal current	0x5
	ADC3_CURRENT_0_6X	Use 60% of the nominal current	0x6
	ADC3_CURRENT_0_5X	Use 50% of the nominal current	0x7

Field Name	Value Symbol	Value Description	Hex Value
ADC_CUR_CTRL_ADC2_CURRENT	ADC2_CURRENT_4_0X	Use 400% of the nominal current	0x0
	ADC2_CURRENT_2_0X	Use 200% of the nominal current	0x1
	ADC2_CURRENT_1_3X	Use 130% of the nominal current	0x2
	ADC2_CURRENT_1_0X	Use the nominal current	0x3*
	ADC2_CURRENT_0_8X	Use 80% of the nominal current	0x4
	ADC2_CURRENT_0_7X	Use 70% of the nominal current	0x5
	ADC2_CURRENT_0_6X	Use 60% of the nominal current	0x6
	ADC2_CURRENT_0_5X	Use 50% of the nominal current	0x7
ADC_CUR_CTRL_ADC1_CURRENT	ADC1_CURRENT_4_0X	Use 400% of the nominal current	0x0
	ADC1_CURRENT_2_0X	Use 200% of the nominal current	0x1
	ADC1_CURRENT_1_3X	Use 130% of the nominal current	0x2
	ADC1_CURRENT_1_0X	Use the nominal current	0x3*
	ADC1_CURRENT_0_8X	Use 80% of the nominal current	0x4
	ADC1_CURRENT_0_7X	Use 70% of the nominal current	0x5
	ADC1_CURRENT_0_6X	Use 60% of the nominal current	0x6
	ADC1_CURRENT_0_5X	Use 50% of the nominal current	0x7
ADC_CUR_CTRL_ADC0_CURRENT	ADC0_CURRENT_4_0X	Use 400% of the nominal current	0x0
	ADC0_CURRENT_2_0X	Use 200% of the nominal current	0x1
	ADC0_CURRENT_1_3X	Use 130% of the nominal current	0x2
	ADC0_CURRENT_1_0X	Use the nominal current	0x3*
	ADC0_CURRENT_0_8X	Use 80% of the nominal current	0x4
	ADC0_CURRENT_0_7X	Use 70% of the nominal current	0x5
	ADC0_CURRENT_0_6X	Use 60% of the nominal current	0x6
	ADC0_CURRENT_0_5X	Use 50% of the nominal current	0x7

8.6.2.3 A_FRAC_DELAY_0_CTRL Settings

Bit Field	Field Name	Field Description
10:8	FRAC_DELAY_INTEGER	Select the coarse fractional sampling delay
5:0	FRAC_DELAY_FRACTIONAL	Select the fine fractional sampling delay

Field Name	Value Symbol	Value Description	Hex Value
FRAC_DELAY_INTEGER	ADC0_INTEGER_DELAY_0	Select no coarse fractional delay	0x0*
	ADC0_INTEGER_DELAY_1	Delay the sampling of ADC0 by 1/8 of the sampling frequency	0x1
	ADC0_INTEGER_DELAY_2	Delay the sampling of ADC0 by 2/8 of the sampling frequency	0x2
	ADC0_INTEGER_DELAY_3	Delay the sampling of ADC0 by 3/8 of the sampling frequency	0x3
	ADC0_INTEGER_DELAY_4	Delay the sampling of ADC0 by 4/8 of the sampling frequency	0x4
	ADC0_INTEGER_DELAY_5	Delay the sampling of ADC0 by 5/8 of the sampling frequency	0x5
	ADC0_INTEGER_DELAY_6	Delay the sampling of ADC0 by 6/8 of the sampling frequency	0x6
	ADC0_INTEGER_DELAY_7	Delay the sampling of ADC0 by 7/8 of the sampling frequency	0x7

8.6.2.4 A_FRAC_DELAY_1_CTRL Settings

Bit Field	Field Name	Field Description
10:8	FRAC_DELAY_INTEGER	Select the coarse fractional sampling delay
5:0	FRAC_DELAY_FRACTIONAL	Select the fine fractional sampling delay

Field Name	Value Symbol	Value Description	Hex Value
FRAC_DELAY_INTEGER	ADC1_INTEGER_DELAY_0	Select no coarse fractional delay	0x0*
	ADC1_INTEGER_DELAY_1	Delay the sampling of ADC1 by 1/8 of the sampling frequency	0x1
	ADC1_INTEGER_DELAY_2	Delay the sampling of ADC1 by 2/8 of the sampling frequency	0x2
	ADC1_INTEGER_DELAY_3	Delay the sampling of ADC1 by 3/8 of the sampling frequency	0x3
	ADC1_INTEGER_DELAY_4	Delay the sampling of ADC1 by 4/8 of the sampling frequency	0x4
	ADC1_INTEGER_DELAY_5	Delay the sampling of ADC1 by 5/8 of the sampling frequency	0x5
	ADC1_INTEGER_DELAY_6	Delay the sampling of ADC1 by 6/8 of the sampling frequency	0x6
	ADC1_INTEGER_DELAY_7	Delay the sampling of ADC1 by 7/8 of the sampling frequency	0x7

8.6.2.5 A_FRAC_DELAY_2_CTRL Settings

Bit Field	Field Name	Field Description
10:8	FRAC_DELAY_INTEGER	Select the coarse fractional sampling delay
5:0	FRAC_DELAY_FRACTIONAL	Select the fine fractional sampling delay

Field Name	Value Symbol	Value Description	Hex Value
FRAC_DELAY_INTEGER	ADC2_INTEGER_DELAY_0	Select no coarse fractional delay	0x0*
	ADC2_INTEGER_DELAY_1	Delay the sampling of ADC2 by 1/8 of the sampling frequency	0x1
	ADC2_INTEGER_DELAY_2	Delay the sampling of ADC2 by 2/8 of the sampling frequency	0x2
	ADC2_INTEGER_DELAY_3	Delay the sampling of ADC2 by 3/8 of the sampling frequency	0x3
	ADC2_INTEGER_DELAY_4	Delay the sampling of ADC2 by 4/8 of the sampling frequency	0x4
	ADC2_INTEGER_DELAY_5	Delay the sampling of ADC2 by 5/8 of the sampling frequency	0x5
	ADC2_INTEGER_DELAY_6	Delay the sampling of ADC2 by 6/8 of the sampling frequency	0x6
	ADC2_INTEGER_DELAY_7	Delay the sampling of ADC2 by 7/8 of the sampling frequency	0x7

8.6.2.6 A_FRAC_DELAY_3_CTRL Settings

Bit Field	Field Name	Field Description
10:8	FRAC_DELAY_INTEGER	Select the coarse fractional sampling delay
5:0	FRAC_DELAY_FRACTIONAL	Select the fine fractional sampling delay

Field Name	Value Symbol	Value Description	Hex Value
FRAC_DELAY_INTEGER	ADC3_INTEGER_DELAY_0	Select no coarse fractional delay	0x0*
	ADC3_INTEGER_DELAY_1	Delay the sampling of ADC3 by 1/8 of the sampling frequency	0x1
	ADC3_INTEGER_DELAY_2	Delay the sampling of ADC3 by 2/8 of the sampling frequency	0x2
	ADC3_INTEGER_DELAY_3	Delay the sampling of ADC3 by 3/8 of the sampling frequency	0x3
	ADC3_INTEGER_DELAY_4	Delay the sampling of ADC3 by 4/8 of the sampling frequency	0x4
	ADC3_INTEGER_DELAY_5	Delay the sampling of ADC3 by 5/8 of the sampling frequency	0x5
	ADC3_INTEGER_DELAY_6	Delay the sampling of ADC3 by 6/8 of the sampling frequency	0x6
	ADC3_INTEGER_DELAY_7	Delay the sampling of ADC3 by 7/8 of the sampling frequency	0x7

8.7 DECIMATION FILTERING

The decimation filters downsample and filter the oversampled signal from the A/D converters to obtain a final non-oversampled stream of input data. This low-pass filtering is done using 11th-order wave digital filters.

During the decimation filtering step a configurable gain factor can be added to the signal from each of the system channels. To set these gains, select the appropriate value for the ADC_GF_CTRL_ADCx_GF bit fields of the A_ADC_GF01_CTRL and A_ADC_GF23_CTRL registers. Each of these four fields contains a 12-bit two's-complement signed value which can be used to configure a gain for the corresponding decimation filter output. The actual range of the gain depends on the value of ADC_CFG_SAMPLE_FREQ. The smallest range that can be configured is from -1 to $(1-2^{-11})$ when ADC_CFG_SAMPLE_FREQ is 0. Selecting a negative gain inverts the phase of the signal. A gain of 0 mutes the channel.

The gain that is added through this gain value, is scaled by a factor based on the range of ADC_CFG_SAMPLE_FREQ as described in Table 15.

TABLE 15: SAMPLE FREQUENCY GAIN SCALING FACTOR

Range for **ADC_CFG_SAMPLE_FREQ**	Value of **SAMPLE_FREQ_SCALING_FACTOR**
0 to 7	1
8 to 23	2
Greater than or equal to 24	4

We recommend that the gain value for each filter is set in such a way that the signal level, after downsampling and filtering, is the same as it was before downsampling and filtering (i.e., no gain or attenuation performed). These gain factors are useful to fine tune the gain realized in the input

stage and are commonly used to compensate for changes in sampling frequency or to match the gain between microphones in a multiple-microphone system. To obtain a signal that has no gain or attenuation added during this step, set the gain in the following equality to 1:

$$\text{GAIN} = \frac{\text{ADC_GF_CTRL_ADCx_GF} \times (\text{ADC_CFG_SAMPLE_FREQ} + 8)}{(16384 \times \text{SAMPLE_FREQ_SCALING_FACTOR})}$$

Note: Setting any of the ADC_GF_CTRL_ADCx_GF bit field values to zero effectively mutes the associated input channel.

The default value for the decimation filter gain factor, 0x665 results in approximately unity gain for the default ADC_CFG_SAMPLE_FREQ bit-field configuration.

Although each ADC is only a 16-bit converter, the output generated by the decimation filter when downsampling the provided 16-bit oversampled data is 18-bit output data. As a result, all input data provided by the input stage to the system is handled as 18-bit data.

8.7.1 Input Data Registers

The input data as received by the rest of the system from the input stage is interpreted as 18-bit data by the Input/Output Controller (IOC) when written to input FIFOs and output sources. This same 18-bit data that the IOC uses can also be read by the CFX DSP directly from the D_ADCx_DATA registers, which contain the most recent input data available for each input channel.

8.7.2 DC Removal Filters

The decimation filtering component of the input stage includes a high-pass filter for each channel designed to remove any DC signal component from the input signal belonging to each channel. The DC removal filters are based on first-order digital filter structures with programmable cut-off frequencies. Configuration of these filters use the ADC_CFG_ADCx_DC_REMOVE bit fields of the A_ADC_CFG register to select between cut-off frequencies of 5Hz, 10Hz, 20Hz or to bypass the DC removal filter.

The filter cut-off frequency scales linearly with the sampling frequency, and cut-off frequencies for the filters have been defined in terms of a 16kHz sampling frequency. For sampling frequencies other than 16kHz, scale the cut-off frequency along with any increase or decrease in sampling frequency. For example, the cut-off frequencies for a sampling frequency of 32kHz are 10Hz, 20Hz and 40Hz respectively.

8.7.3 Multi-Chip Synchronization

The decimation filters provide multi-chip synchronization by generating a synchronization signal that the remote device can use to synchronize the filter of one BelaSigna 300 device with other BelaSigna 300 devices. This signal, which may be transmitted or received on the multiplexed GPIO 3 pin, synchronizes the clock slow clock divider and decimation filters on the rising edge of the signal allowing an application to ensure that all signals received are sampled at the same time.

Selecting the configuration GPIO3_SELECT_SYNC_OUT for the GPIO_PIN_CFG_GPIO3 bit field and configuring the GPIO 3 pad for output allows a device to act as the master device in a multi-chip synchronization configuration. In this configuration the device is producing an output synchronizing signal to which other devices can be synchronized.

Selecting the configuration GPIO3_SELECT_SYNC_IN for the GPIO_PIN_CFG_GPIO3 bit field and configuring the GPIO 3 pad for input allows a device to act as a slave device in a multi-chip synchronization configuration. In this configuration the device is receiving and synchronizing to the synchronizing signal that the device using the "out" configuration provided.

For information on configuring the multiplexed GPIO 3 pin for multi-chip synchronization functionality, refer to Section 12.2, "General-Purpose I/O (GPIO) Interface" on page 148.

IMPORTANT: When using multi-chip synchronization, it is essential that all devices that are being synchronized operate using the same system clock source. Typical configurations include using the system clock generated by one of the devices in the multi-chip configuration, or using an external clock source from all the devices as the source to SYSCLK through the external clock of the remaining devices. For more information about configurations that provide or receive an external clock, see Section 13.2.4, "External Clock (EXTCLK)" on page 165.

8.7.4 Frequency-Shift Keying

The decimation filters provide support for receiving data encoded with frequency-shift keying. This allows an interface to implement a receive-only remote control interface.

Frequency-shift keying is a form of signal modulation that typically uses signals at two discrete frequencies to transmit binary data. This feature, which is available only on input channels 2 and 3, allows you to define a signal band that you can use to limit the received signal to a specific transmission band without filtering out the frequency-shift keying carrier frequency.

The pass band for the frequency-shift keying feature is configurable in steps of one half of the configured sampling frequency up to four times the sampling frequency. The pass band for each of the band-pass filters is configurable using the BAND_SELECT_ADCx bit fields from the A_BAND_SELECT_CTRL register.

Note: The supplied defines for the BAND_SELECT_ADCx bit fields have been specified in terms of a 16kHz sampling frequency. If your application uses a different sampling frequency, appropriately scale the values used to define the pass-band to obtain the correct input band for the filter.

Support for handling frequency-shift keyed input data is not provided directly by the BelaSigna 300 device. As a result either the CFX DSP or the HEAR Accelerator core has to perform any analysis of the frequency-shift keyed signal.

WARNING: The band pass filter of the Frequency Shift Keying feature does not function correctly unless the DC removal filter is configured for bypass operation.

8.7.5 Control and Configuration Registers

Register Name	Register Description	Address
A_ADC_CFG	Configure the AD converters	0xE140
A_ADC_GF01_CTRL	Configure the gain factors for input channels 0 and 1	0xE145

Register Name	Register Description	Address
A_ADC_GF23_CTRL	Configure the gain factors for input channels 2 and 3	0xE146
A_BAND_SELECT_CTRL	Select which signal bands are filtered by the decimation filter for ADC 2 and ADC 3	0xE14B

8.7.5.1 A_ADC_CFG Settings

Bit Field	Field Name	Field Description
22:21	ADC_CFG_ADC3_DC_REMOVE	Configure DC removal filter for input channel 3
20:19	ADC_CFG_ADC2_DC_REMOVE	Configure DC removal filter for input channel 2
18:17	ADC_CFG_ADC1_DC_REMOVE	Configure DC removal filter for input channel 1
16:15	ADC_CFG_ADC0_DC_REMOVE	Configure DC removal filter for input channel 0

Field Name	Value Symbol	Value Description	Hex Value
ADC_CFG_ADC3_DC_REMOVE	ADC3_DC_REMOVE_CUTOFF_5HZ	Enable a 5Hz high-pass filter for DC removal on ADC 3	0x0
	ADC3_DC_REMOVE_CUTOFF_10HZ	Enable a 10Hz high-pass filter for DC removal on ADC 3	0x1
	ADC3_DC_REMOVE_CUTOFF_20HZ	Enable a 20Hz high-pass filter for DC removal on ADC 3	0x2*
	ADC3_DC_REMOVE_CUTOFF_BYPASS	Disable DC removal filtering on ADC 3	0x3
ADC_CFG_ADC2_DC_REMOVE	ADC2_DC_REMOVE_CUTOFF_5HZ	Enable a 5Hz high-pass filter for DC removal on ADC 2	0x0
	ADC2_DC_REMOVE_CUTOFF_10HZ	Enable a 10Hz high-pass filter for DC removal on ADC 2	0x1
	ADC2_DC_REMOVE_CUTOFF_20HZ	Enable a 20Hz high-pass filter for DC removal on ADC 2	0x2*
	ADC2_DC_REMOVE_CUTOFF_BYPASS	Disable DC removal filtering on ADC 2	0x3
ADC_CFG_ADC1_DC_REMOVE	ADC1_DC_REMOVE_CUTOFF_5HZ	Enable a 5Hz high-pass filter for DC removal on ADC 1	0x0
	ADC1_DC_REMOVE_CUTOFF_10HZ	Enable a 10Hz high-pass filter for DC removal on ADC 1	0x1
	ADC1_DC_REMOVE_CUTOFF_20HZ	Enable a 20Hz high-pass filter for DC removal on ADC 1	0x2*
	ADC1_DC_REMOVE_CUTOFF_BYPASS	Disable DC removal filtering on ADC 1	0x3

Field Name	Value Symbol	Value Description	Hex Value
ADC_CFG_ADC0_DC_REMOVE	ADC0_DC_REMOVE_CUTOFF_5HZ	Enable a 5Hz high-pass filter for DC removal on ADC 0	0x0
	ADC0_DC_REMOVE_CUTOFF_10HZ	Enable a 10Hz high-pass filter for DC removal on ADC 0	0x1
	ADC0_DC_REMOVE_CUTOFF_20HZ	Enable a 20Hz high-pass filter for DC removal on ADC 0	0x2*
	ADC0_DC_REMOVE_CUTOFF_BYPASS	Disable DC removal filtering on ADC 0	0x3

8.7.5.2 A_BAND_SELECT_CTRL Settings

Bit Field	Field Name	Field Description
6:4	BAND_SELECT_ADC3	Select the pass-band of the filtered signal on input channel 3
2:0	BAND_SELECT_ADC2	Select the pass-band of the filtered signal on input channel 2

Field Name	Value Symbol	Value Description	Hex Value
BAND_SELECT_ADC3	BAND_SELECT_ADC3_0K_TO_8K	Select a pass-band for ADC 3 of 0kHz to 8kHz for 16kHz sampling	0x0*
	BAND_SELECT_ADC3_8K_TO_16K	Select a pass-band for ADC 3 of 8kHz to 16kHz for 16kHz sampling	0x1
	BAND_SELECT_ADC3_16K_TO_24K	Select a pass-band for ADC 3 of 16kHz to 24kHz for 16kHz sampling	0x2
	BAND_SELECT_ADC3_24K_TO_32K	Select a pass-band for ADC 3 of 24kHz to 32kHz for 16kHz sampling	0x3
	BAND_SELECT_ADC3_32K_TO_40K	Select a pass-band for ADC 3 of 32kHz to 40kHz for 16kHz sampling	0x4
	BAND_SELECT_ADC3_40K_TO_48K	Select a pass-band for ADC 3 of 40kHz to 48kHz for 16kHz sampling	0x5
	BAND_SELECT_ADC3_48K_TO_56K	Select a pass-band for ADC 3 of 48kHz to 56kHz for 16kHz sampling	0x6
	BAND_SELECT_ADC3_56K_TO_64K	Select a pass-band for ADC 3 of 56kHz to 64kHz for 16kHz sampling	0x7

Field Name	Value Symbol	Value Description	Hex Value
BAND_SELECT_ADC2	BAND_SELECT_ADC2_0K_TO_8K	Select a pass-band for ADC 2 of 0kHz to 8kHz for 16kHz sampling	0x0*
	BAND_SELECT_ADC2_8K_TO_16K	Select a pass-band for ADC 2 of 8kHz to 16kHz for 16kHz sampling	0x1
	BAND_SELECT_ADC2_16K_TO_24K	Select a pass-band for ADC 2 of 16kHz to 24kHz for 16kHz sampling	0x2
	BAND_SELECT_ADC2_24K_TO_32K	Select a pass-band for ADC 2 of 24kHz to 32kHz for 16kHz sampling	0x3
	BAND_SELECT_ADC2_32K_TO_40K	Select a pass-band for ADC 2 of 32kHz to 40kHz for 16kHz sampling	0x4
	BAND_SELECT_ADC2_40K_TO_48K	Select a pass-band for ADC 2 of 40kHz to 48kHz for 16kHz sampling	0x5
	BAND_SELECT_ADC2_48K_TO_56K	Select a pass-band for ADC 2 of 48kHz to 56kHz for 16kHz sampling	0x6
	BAND_SELECT_ADC2_56K_TO_64K	Select a pass-band for ADC 2 of 56kHz to 64kHz for 16kHz sampling	0x7

9 Output Stage

9.1 Output Stage Architecture

The output stage of a BelaSigna 300 device provides a single audio output channel that post-processes signal data from the rest of the BelaSigna 300 system and provides it to an external receiver or speaker. The output channel includes:

- High-quality interpolation filtering that automatically tracks the selected sample rate of the sigma-delta modulator
- Ultra low-power, high fidelity, over-sampled sigma-delta D/A conversion with programmable sample rates
- A direct digital output driver for zero-bias hearing aid receivers and standard loudspeakers with a selectable high-power mode that combines multiple output pads for maximum output signal power

Each of these components of the output channel is shown in Figure 13, which shows the overall architecture of the output stage.

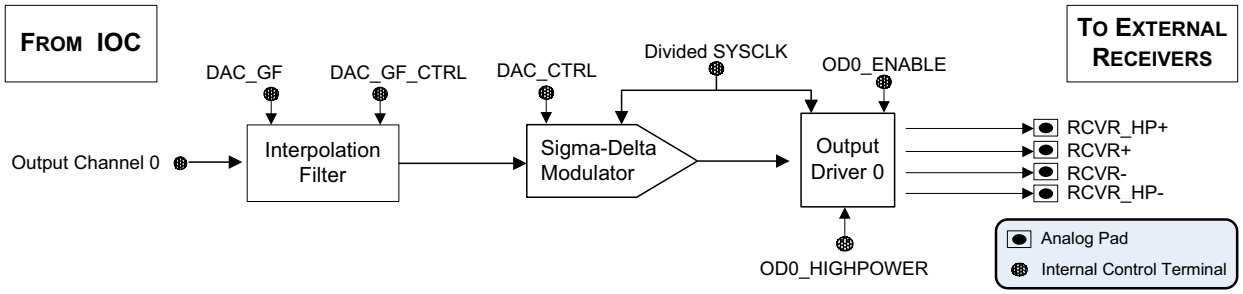


Figure 13: Output Stage Block Diagram

9.2 Interpolation Filtering

The output stage contains an interpolation filter for the output channel which upsamples and filters the signal that is being passed to the sigma-delta modulator. This filter is part of the decimation filtering block of the input stage and is based on an 11th-order low-pass wave digital filter structure. As a result of this link to the decimation filters, the interpolation filter automatically tracks the selected sample rate and upsampled frequency.

The interpolation filter component of the output stage provides the ability to apply a gain to the output signal. By default this gain or is disabled; however a user application can configure the `O_DAC_GF_CTRL` bit field in the `A_O_DAC_GF_CTRL` register to change this setting. This bit field allows you to select whether the application decides if the gain is enabled/disabled, or uses the setting of GPIO 4 to enable/disable the gain. If the system is configured to use GPIO 4 to control the application of gain, select the bit-field setting `GPIO4_SELECT_DAC_SCALE` as the desired

functionality for GPIO 4 and configure GPIO 4 as an input. For more information about configuring GPIO pads for alternate functionality, see Section 12.2, “General-Purpose I/O (GPIO) Interface” on page 148.

The amount of gain applied by the interpolation filter is configured using the O_DAC_GF bit field in the A_O_DAC_GF_CTRL register. This bit field uses a 12-bit signed value, with 2 integer bits and 10 fractional bits. This can be used to configure a gain in the range -2 to $(2 - 2^{-10})$. The default setting of 0x400 for this bit field corresponds to the application of no gain across the interpolation filter.

Similar to the input stage, the output stage uses 18-bit data supplied to the interpolation filter to produce the 16-bit data provided to the sigma-delta converter.

9.2.1 Control and Configuration Registers

Register Name	Register Description	Address
D_IOC_OUTPUT_CFG	Configuration of where data for output sources is acquired and configuration of output behavior.	0xE0C2
A_O_DAC_GF_CTRL	Control the gain added to the signal during digital-to-analog conversion	0xE152

9.2.1.1 D_IOC_OUTPUT_CFG Settings

Bit Field	Field Name	Field Description
14	IOC_OUTPUT_CFG_PCM_DBL_ACC	Enable/disable double-access mode for PCM output
13	IOC_OUTPUT_CFG_PCM_STEREO	Enable/disable stereo configuration for PCM output
12	IOC_OUTPUT_CFG_DAC_DBL_ACC	Enable/disable double-access mode for the output stage channel
11:8	IOC_OUTPUT_CFG_PCM1_LOAD	Select the source for data used by the PCM1 output
7:4	IOC_OUTPUT_CFG_PCM0_LOAD	Select the source for data used by the PCM0 output
3:0	IOC_OUTPUT_CFG_DAC_LOAD	Select the source for data used by the output stage channel

Field Name	Value Symbol	Value Description	Hex Value
IOC_OUTPUT_CFG_PCM_DBL_ACC	IOC_OUTPUT_CFG_PCM_DBL_ACC_DIS	Disable double-access for PCM output	0x0*
	IOC_OUTPUT_CFG_PCM_DBL_ACC_EN	Enable double-access for PCM output	0x1
IOC_OUTPUT_CFG_PCM_STEREO	IOC_OUTPUT_CFG_PCM_STEREO_DIS	Disable PCM output stereo mode	0x0*
	IOC_OUTPUT_CFG_PCM_STEREO_EN	Enable PCM output stereo mode	0x1

Field Name	Value Symbol	Value Description	Hex Value
IOC_OUTPUT_CFG_DAC_DBL_ACC	IOC_OUTPUT_CFG_DAC_DBL_ACC_DIS	Disable double-access for the output stage	0x0*
	IOC_OUTPUT_CFG_DAC_DBL_ACC_EN	Enable double-access for the output stage	0x1
IOC_OUTPUT_CFG_PCM1_LOAD	IOC_OUTPUT_CFG_PCM1_FA0	PCM1 uses data from FIFO A0	0x0
	IOC_OUTPUT_CFG_PCM1_FA1	PCM1 uses data from FIFO A1	0x1
	IOC_OUTPUT_CFG_PCM1_FA2	PCM1 uses data from FIFO A2	0x2
	IOC_OUTPUT_CFG_PCM1_FA3	PCM1 uses data from FIFO A3	0x3
	IOC_OUTPUT_CFG_PCM1_FB0	PCM1 uses data from FIFO B0	0x4
	IOC_OUTPUT_CFG_PCM1_FB1	PCM1 uses data from FIFO B1	0x5
	IOC_OUTPUT_CFG_PCM1_FB2	PCM1 uses data from FIFO B2	0x6
	IOC_OUTPUT_CFG_PCM1_FB3	PCM1 uses data from FIFO B3	0x7
	IOC_OUTPUT_CFG_PCM1_PCMRX0	PCM1 uses data from the PCM0 input	0x8
	IOC_OUTPUT_CFG_PCM1_PCMRX1	PCM1 uses data from the PCM1 input	0x9
	IOC_OUTPUT_CFG_PCM1_ADC0	PCM1 uses data from ADC0	0xA
	IOC_OUTPUT_CFG_PCM1_ADC1	PCM1 uses data from ADC1	0xB
	IOC_OUTPUT_CFG_PCM1_ADC2	PCM1 uses data from ADC2	0xC
	IOC_OUTPUT_CFG_PCM1_ADC3	PCM1 uses data from ADC3	0xD
	IOC_OUTPUT_CFG_PCM1_NONE	Disable output from PCM1	0xF*

Field Name	Value Symbol	Value Description	Hex Value
IOC_OUTPUT_CFG_PCM0_LOAD	IOC_OUTPUT_CFG_PCM0_FA0	PCM0 uses data from FIFO A0	0x0
	IOC_OUTPUT_CFG_PCM0_FA1	PCM0 uses data from FIFO A1	0x1
	IOC_OUTPUT_CFG_PCM0_FA2	PCM0 uses data from FIFO A2	0x2
	IOC_OUTPUT_CFG_PCM0_FA3	PCM0 uses data from FIFO A3	0x3
	IOC_OUTPUT_CFG_PCM0_FB0	PCM0 uses data from FIFO B0	0x4
	IOC_OUTPUT_CFG_PCM0_FB1	PCM0 uses data from FIFO B1	0x5
	IOC_OUTPUT_CFG_PCM0_FB2	PCM0 uses data from FIFO B2	0x6
	IOC_OUTPUT_CFG_PCM0_FB3	PCM0 uses data from FIFO B3	0x7
	IOC_OUTPUT_CFG_PCM0_PCMRX0	PCM0 uses data from the PCM0 input	0x8
	IOC_OUTPUT_CFG_PCM0_PCMRX1	PCM0 uses data from the PCM1 input	0x9
	IOC_OUTPUT_CFG_PCM0_ADC0	PCM0 uses data from ADC0	0xA
	IOC_OUTPUT_CFG_PCM0_ADC1	PCM0 uses data from ADC1	0xB
	IOC_OUTPUT_CFG_PCM0_ADC2	PCM0 uses data from ADC2	0xC
	IOC_OUTPUT_CFG_PCM0_ADC3	PCM0 uses data from ADC3	0xD
	IOC_OUTPUT_CFG_PCM0_NONE	Disable output from PCM0	0xF*

Field Name	Value Symbol	Value Description	Hex Value
IOC_OUTPUT_CFG_DAC_LOAD	IOC_OUTPUT_CFG_DAC_FA0	DAC uses data from FIFO A0	0x0
	IOC_OUTPUT_CFG_DAC_FA1	DAC uses data from FIFO A1	0x1
	IOC_OUTPUT_CFG_DAC_FA2	DAC uses data from FIFO A2	0x2
	IOC_OUTPUT_CFG_DAC_FA3	DAC uses data from FIFO A3	0x3
	IOC_OUTPUT_CFG_DAC_FB0	DAC uses data from FIFO B0	0x4
	IOC_OUTPUT_CFG_DAC_FB1	DAC uses data from FIFO B1	0x5
	IOC_OUTPUT_CFG_DAC_FB2	DAC uses data from FIFO B2	0x6
	IOC_OUTPUT_CFG_DAC_FB3	DAC uses data from FIFO B3	0x7
	IOC_OUTPUT_CFG_DAC_PCMRX0	DAC uses data from the PCM0 input	0x8
	IOC_OUTPUT_CFG_DAC_PCMRX1	DAC uses data from the PCM1 input	0x9
	IOC_OUTPUT_CFG_DAC_ADC0	DAC uses data from ADC0	0xA
	IOC_OUTPUT_CFG_DAC_ADC1	DAC uses data from ADC1	0xB
	IOC_OUTPUT_CFG_DAC_ADC2	DAC uses data from ADC2	0xC
	IOC_OUTPUT_CFG_DAC_ADC3	DAC uses data from ADC3	0xD
	IOC_OUTPUT_CFG_DAC_NONE	Disable output from DAC	0xF*

9.2.1.2 A_O_DAC_GF_CTRL Settings

Bit Field	Field Name	Field Description
17:16	O_DAC_GF_CTRL	Configure when the output DAC gain factor is used
11:0	O_DAC_GF	Select an output DAC gain factor

Field Name	Value Symbol	Value Description	Hex Value
O_DAC_GF_CTRL	O_DAC_GF_CTRL_DISABLE	Disable the output DAC gain factor	0x0*
	O_DAC_GF_CTRL_ENABLE	Enable the output DAC gain factor	0x2
	O_DAC_GF_CTRL_BY_GPIO4	Select whether the output DAC gain factor is enabled through GPIO 4	0x3

9.3 SIGMA-DELTA MODULATOR

The BelaSigna 300 device includes a 3rd-order three level sigma-delta modulator to produce a pulse density modulated (PDM) output signal from the oversampled signal data provided by the interpolation filter.

When the oversampled signal is received from the interpolation filters by the sigma-delta modulator, it is quantized to 1 bit by the modulator. This process makes use of noise shaping of the output signal. As a result, the noise floor is no longer flat, but rather is flat for a pass-band before the noise climbs at higher frequencies. The frequency band where the noise floor is flat depends on the frequency of SYSCLK and the configuration of the O_DAC_CTRL_OSRDAC bit field from the A_O_DAC_CTRL register. The useful range for the modulator, where the noise floor remains flat, ranges from a lower bound of 20Hz to an upper bound defined by:

$$F_{\text{UPPER BOUND}} = \frac{F_{\text{SYSCLK}}}{128 \cdot (\text{O_DAC_CTRL_OSRDAC} + 1)}$$

For information on configuring the system clock frequency, see Section 13.2.1, “System Clock (SYSCLK)” on page 164.

9.3.1 Control and Configuration Registers

Register Name	Register Description	Address
A_O_DAC_CTRL	Control the digital-to-analog conversion in the sigma-delta modulator	0xE151

9.3.1.1 A_O_DAC_CTRL Settings

Bit Field	Field Name	Field Description
5:0	O_DAC_CTRL_OSRDAC	Prescaler for the system clock to select the desired sampling rate

Field Name	Value Symbol	Value Description	Hex Value
O_DAC_CTRL_OSRDAC	OSR_PRESCALE_1	Use a prescaled value of SYSCLK/1	0x0*
	OSR_PRESCALE_2	Use a prescaled value of SYSCLK/2	0x1
	OSR_PRESCALE_3	Use a prescaled value of SYSCLK/3	0x2
	OSR_PRESCALE_4	Use a prescaled value of SYSCLK/4	0x3
	OSR_PRESCALE_5	Use a prescaled value of SYSCLK/5	0x4
	OSR_PRESCALE_6	Use a prescaled value of SYSCLK/6	0x5
	OSR_PRESCALE_7	Use a prescaled value of SYSCLK/7	0x6
	OSR_PRESCALE_8	Use a prescaled value of SYSCLK/8	0x7
	OSR_PRESCALE_9	Use a prescaled value of SYSCLK/9	0x8
	OSR_PRESCALE_10	Use a prescaled value of SYSCLK/10	0x9
	OSR_PRESCALE_11	Use a prescaled value of SYSCLK/11	0xA
	OSR_PRESCALE_12	Use a prescaled value of SYSCLK/12	0xB
	OSR_PRESCALE_13	Use a prescaled value of SYSCLK/13	0xC
	OSR_PRESCALE_14	Use a prescaled value of SYSCLK/14	0xD
	OSR_PRESCALE_15	Use a prescaled value of SYSCLK/15	0xE
	OSR_PRESCALE_16	Use a prescaled value of SYSCLK/16	0xF
	OSR_PRESCALE_17	Use a prescaled value of SYSCLK/17	0x10
	OSR_PRESCALE_18	Use a prescaled value of SYSCLK/18	0x11
	OSR_PRESCALE_19	Use a prescaled value of SYSCLK/19	0x12
	OSR_PRESCALE_20	Use a prescaled value of SYSCLK/20	0x13
	OSR_PRESCALE_21	Use a prescaled value of SYSCLK/21	0x14
	OSR_PRESCALE_22	Use a prescaled value of SYSCLK/22	0x15
	OSR_PRESCALE_23	Use a prescaled value of SYSCLK/23	0x16
	OSR_PRESCALE_24	Use a prescaled value of SYSCLK/24	0x17
	OSR_PRESCALE_25	Use a prescaled value of SYSCLK/25	0x18
	OSR_PRESCALE_26	Use a prescaled value of SYSCLK/26	0x19
	OSR_PRESCALE_27	Use a prescaled value of SYSCLK/27	0x1A
	OSR_PRESCALE_28	Use a prescaled value of SYSCLK/28	0x1B
	OSR_PRESCALE_29	Use a prescaled value of SYSCLK/29	0x1C
	OSR_PRESCALE_30	Use a prescaled value of SYSCLK/30	0x1D
	OSR_PRESCALE_31	Use a prescaled value of SYSCLK/31	0x1E
	OSR_PRESCALE_32	Use a prescaled value of SYSCLK/32	0x1F

Field Name	Value Symbol	Value Description	Hex Value
O_DAC_CTRL_OSRDAC (Continued)	OSR_PRESCALE_33	Use a prescaled value of SYSCLK/33	0x20
	OSR_PRESCALE_34	Use a prescaled value of SYSCLK/34	0x21
	OSR_PRESCALE_35	Use a prescaled value of SYSCLK/35	0x22
	OSR_PRESCALE_36	Use a prescaled value of SYSCLK/36	0x23
	OSR_PRESCALE_37	Use a prescaled value of SYSCLK/37	0x24
	OSR_PRESCALE_38	Use a prescaled value of SYSCLK/38	0x25
	OSR_PRESCALE_39	Use a prescaled value of SYSCLK/39	0x26
	OSR_PRESCALE_40	Use a prescaled value of SYSCLK/40	0x27
	OSR_PRESCALE_41	Use a prescaled value of SYSCLK/41	0x28
	OSR_PRESCALE_42	Use a prescaled value of SYSCLK/42	0x29
	OSR_PRESCALE_43	Use a prescaled value of SYSCLK/43	0x2A
	OSR_PRESCALE_44	Use a prescaled value of SYSCLK/44	0x2B
	OSR_PRESCALE_45	Use a prescaled value of SYSCLK/45	0x2C
	OSR_PRESCALE_46	Use a prescaled value of SYSCLK/46	0x2D
	OSR_PRESCALE_47	Use a prescaled value of SYSCLK/47	0x2E
	OSR_PRESCALE_48	Use a prescaled value of SYSCLK/48	0x2F
	OSR_PRESCALE_49	Use a prescaled value of SYSCLK/49	0x30
	OSR_PRESCALE_50	Use a prescaled value of SYSCLK/50	0x31
	OSR_PRESCALE_51	Use a prescaled value of SYSCLK/51	0x32
	OSR_PRESCALE_52	Use a prescaled value of SYSCLK/52	0x33
	OSR_PRESCALE_53	Use a prescaled value of SYSCLK/53	0x34
	OSR_PRESCALE_54	Use a prescaled value of SYSCLK/54	0x35
	OSR_PRESCALE_55	Use a prescaled value of SYSCLK/55	0x36
	OSR_PRESCALE_56	Use a prescaled value of SYSCLK/56	0x37
	OSR_PRESCALE_57	Use a prescaled value of SYSCLK/57	0x38
	OSR_PRESCALE_58	Use a prescaled value of SYSCLK/58	0x39
	OSR_PRESCALE_59	Use a prescaled value of SYSCLK/59	0x3A
	OSR_PRESCALE_60	Use a prescaled value of SYSCLK/60	0x3B
	OSR_PRESCALE_61	Use a prescaled value of SYSCLK/61	0x3C
	OSR_PRESCALE_62	Use a prescaled value of SYSCLK/62	0x3D
	OSR_PRESCALE_63	Use a prescaled value of SYSCLK/63	0x3E
	OSR_PRESCALE_64	Use a prescaled value of SYSCLK/64	0x3F

9.4 DIRECT-DIGITAL OUTPUT DRIVER

The direct-digital output driver provided by the BelaSigna 300 device is designed to drive a typical zero-bias receiver without the need for an external amplifier.

The sigma-delta modulators supply the pulse density modulated (PDM) output data to an output H-bridge structure that can be used to directly drive a receiver with the digital output signal.

To enable or disable the digital output driver, configure the OUTPUT_CTRL_OD0_ENABLE bit in the A_OUTPUT_CTRL register.

9.4.1 High-Power Output Mode

The output driver provided on the BelaSigna 300 device includes a mode for high-power operation. To configure the output driver to operate in either normal or high-power mode, use the OUTPUT_CTRL_OD0_HIGHPOWER bit in the A_OUTPUT_CTRL register.

If high-power operation mode is selected, the output signal that is normally provided to the RCVR+ and RCVR- pads of the output driver is also provided to the RCVR_HP+ and RCVR_HP- pads respectively. This configuration reduces the impedance of the output stage allowing for a higher overall output power, particularly for low impedance loads.

When the output driver is configured to operate in high-power mode, you must combine the synchronized output signals externally to the BelaSigna 300 device. Connect both RCVR+ and RCVR_HP+ to a single terminal on an output transducer, and connect both RCVR- and RCVR_HP- to the other terminal. Figure 14 shows this typical signal routing of connections for the output driver in high-power mode.

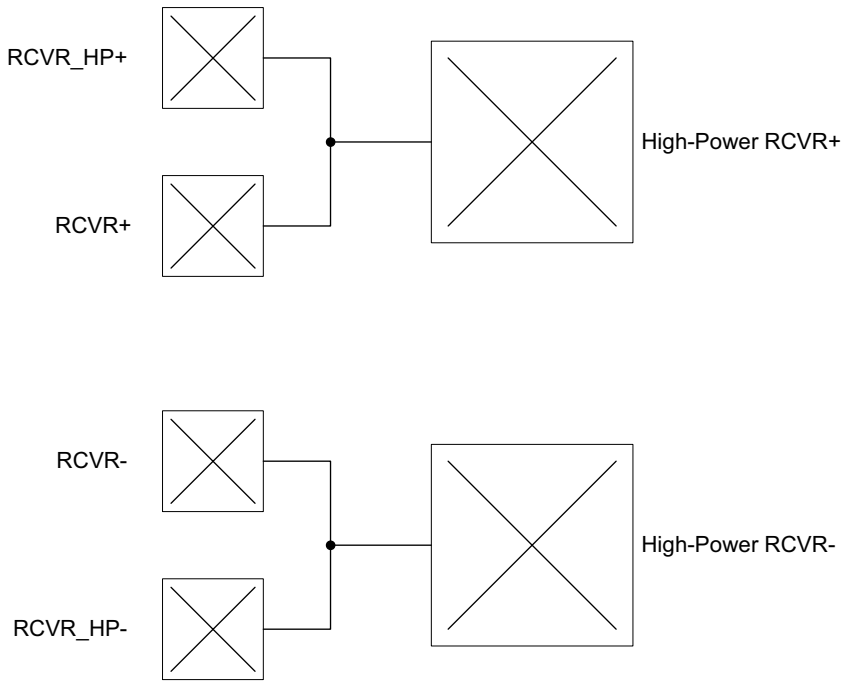


Figure 14: External Signal Routing of Connections for High-Power Output Mode

WARNING: The audio fidelity when in high-power mode can be slightly degraded compared to normal mode depending on the application. Prior to building a prototype with the configuration shown in Figure 14, developers are invited to contact ON Semiconductor for further implementation details.

9.4.2 Control and Configuration Registers

Register Name	Register Description	Address
A_OUTPUT_CTRL	Control the output driver	0xE150

9.4.2.1 A_OUTPUT_CTRL Settings

Bit Field	Field Name	Field Description
2	OUTPUT_CTRL_OD0_HIGHPOWER	Enable/disable the high-power mode of the output driver
0	OUTPUT_CTRL_OD0_ENABLE	Enable/disable the output driver

Field Name	Value Symbol	Value Description	Hex Value
OUTPUT_CTRL_OD0_HIGHPOWER	OD0_HIGHPOWER_DISABLE	Use the normal output configuration	0x0*
	OD0_HIGHPOWER_ENABLE	Enable high-power output configuration	0x1
OUTPUT_CTRL_OD0_ENABLE	OD0_DISABLE	Disable the output driver	0x0*
	OD0_ENABLE	Enable the output driver	0x1

This page left blank for printing purposes.

10 Shared PCM and I²S Interface

10.1 PCM Interface Introduction

The BelaSigna 300 system includes a highly configurable pulse code modulation (PCM) interface that can be used to stream control, configuration or signal data into and out of the device.

This interface makes use of four external pads in communications. These pins are:

- A bidirectional interface clock pin (PCM_CLK)
- A bidirectional frame pin (PCM_FRAME)
- A serial output data pin (PCM_SERO)
- A serial input data pin (PCM_SERI)

One of the advantages of this interface is the high communication speed that allows the interface to operate at the system clock frequency. Additionally, you can configure the interface to operate in either full-duplex mode or half-duplex mode (either transmitting or receiving) because of the separate serial data lines.

The BelaSigna 300 system's PCM interface can be configured to be compatible with communication over the inter IC sound (I^2S) protocol. For information about configuring and using the PCM interface for I^2S communications, see Section 10.5, "I2S Configuration and Usage" on page 108. It also supports a configuration that is compatible with digital audio streaming from a NOAHlink device. For detailed information about configuring and using the PCM interface in NOAHlink digital audio streaming mode, see Section 10.6, "NOAHlink Digital Audio Streaming Configuration and Usage" on page 111.

10.2 PCM Bus Configuration

The PCM bus has two configuration bits to control the electrical behavior of the bus pads.

The `PCM_CLK_LPF` bit in the `D_PCM_CFG` register enables a low-pass filter connected to the PCM clock. This reduces the effect of cross talk and grounding issues on the PCM interface in an application.

You can use the `PCM_PULLUP_DIS` bit in the `D_PCM_CFG` register to disable all of the pull-up resistors that are supplied for the input pads of the PCM interface.

CAUTION: Using the `PCM_PULLUP_DIS` bit to disable the bus pull-up resistors is not recommended under most conditions. We recommend that if you want to use this configuration, consult with ON Semiconductor first, as attempting to use this interface without the pull-up resistors might result in unintended behaviors.

10.2.1 Control and Configuration Registers

Register Name	Register Description	Address
D_PCM_CFG	Shared PCM and I^2S interface configuration	0xE240

10.2.1.1 D_PCM_CFG Settings

Bit Field	Field Name	Field Description
23	PCM_CLK_LPF	Enable/disable the low-pass filter for the PCM clock signal
22	PCM_PULLUP_DISABLE	Disable/enable the pull-up resistors for the PCM input signals

Field Name	Value Symbol	Value Description	Hex Value
PCM_CLK_LPF	PCM_CLK_FILTER_DISABLED	Disable the low-pass filter for the PCM clock signal	0x0*
	PCM_CLK_FILTER_ENABLED	Enable the low-pass filter for the PCM clock signal	0x1
PCM_PULLUP_DISABLE	PCM_PULLUP_ENABLE	Enable the pull-up resistors for the PCM input signals	0x0*
	PCM_PULLUP_DISABLE	Disable the pull-up resistors for the PCM input signals	0x1

10.3 PCM SIGNAL CONFIGURATION

The shared PCM and I^2S interface uses a wide number of configuration options to help define a set of signals that may be interpreted by the system. For the purpose of clarifying the following explanations, the signals generated by the system in the default PCM configuration are shown in Figure 15.

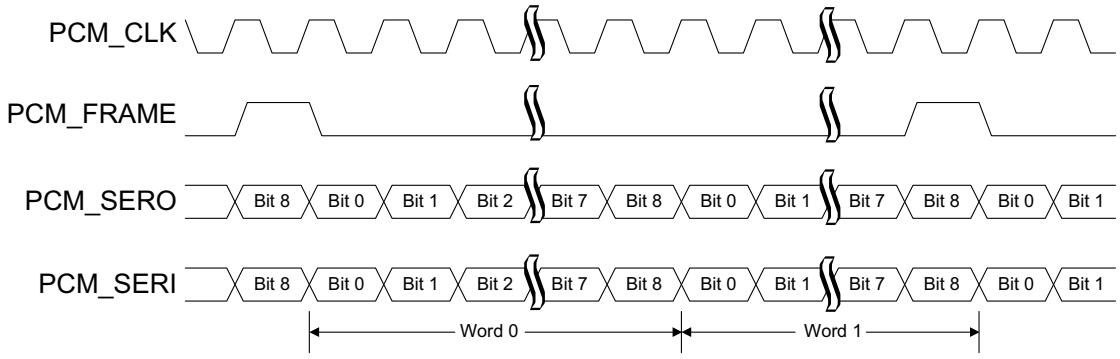


Figure 15: Signal Timing Diagram: Default PCM Configuration

Further information on the required signal configuration and other interface configuration that is needed to allow the interface to be compatible with I^2S, including an example timing diagram, is presented in Section 10.5, "I2S Configuration and Usage" on page 108.

Further information on the required signal configuration and hardware configuration that allows this interface to be compatible with NOAHlink digital PCM signals, including an example timing diagram, is presented in Section 10.6, "NOAHlink Digital Audio Streaming Configuration and Usage" on page 111. This section also briefly discusses the hardware limitations of this interface configuration.

IMPORTANT: If the PCM interface is not idle when changing any configuration settings then changing the PCM configuration during a transaction will result in undefined behavior on the associated PCM signal lines.

10.3.1 Frame Signal Configuration and Timing

The PCM interface uses a signal, called a frame signal, to realign transmission over the PCM interface between two devices with every data frame transmitted.

Note: In all configurations, the PCM interface samples the frame signal on the falling edge of the PCM clock signal.

The frame signal divides the communications on the PCM interface into data frames (or sub-frames, if PCM_SUBFRAME is enabled). Each aspect of the PCM frame signal can be configured as follows:

Interval In the default configuration of PCM_MULTIWORD_CTRL (configured for two words per frame), and PCM_SUBFRAME (configured to disable a frame signal being generated for each word within a frame), the frame signal is generated once for every two-word frame. Changing the PCM_MULTIWORD_CTRL field from the D_PCM_CFG register allows you to select the number of words input per frame, incrementing in multiples of two words, and hence the number of words per data frame. Alternately, if the PCM_SUBFRAME field of the D_PCM_CFG register is configured to enable subframes, a frame signal is generated with every subframe word of each data frame.

IMPORTANT: Configuring the PCM_MULTIWORD_CTRL bit field to something other than two words per frame while setting the PCM_SUBFRAME bit results in a configuration where you will receive a PCM frame signal for every subframe word. Despite the well-defined behavior of the frame signal generated for this configuration, the handling of data and interrupts by the PCM interface might not appear sensible and will likely result in undefined or unexpected data.

Shape The frame signal can be configured to take one of two shapes by setting the PCM_FRAME_WIDTH bit in the D_PCM_CFG register. By default, the frame signal is configured to produce a short width frame signal which produces a single cycle pulse in the frame signal at the beginning of each frame. Alternately, you can configure the signal to produce a long width frame signal which uses a 50% duty cycle square wave and is spaced to be aligned with the specified width between signal events.

Alignment

You can configure the PCM frame signal by using the PCM_FRAME_ALIGN bit from the D_PCM_CFG register so that the first bit of data sampled is sampled at the same time as the frame signal or one cycle after the frame signal. In this way the frame signal is aligned with the first bit of the new data frame or with the last bit of the previous data frame.

Note: Regardless of the PCM frame signal alignment configuration, the PCM frame signal occurs at the beginning of each frame and only one frame signal occurs for each frame received. As a result, the PCM frame signal, if aligned with the last bit, occurs one bit before the first data bit transmitted at the start of a transaction, and does not occur on the last bit of the last frame of a transaction. Similarly, if aligned to the first bit, the first frame signal is aligned with the first data bit transmitted at the start of a transaction, and no trailing frame signal occurs for a transaction.

Note: To properly terminate a transmission, an expected frame signal pulse must be missed. When the frame signal is configured to be aligned with the first bit of a transmission, this requires one additional PCM clock pulse following the transmission of the last bit of the last frame.

For clarification of the above PCM frame signal timing configuration explanations, several examples of different PCM frame signal traces are shown in Figure 16. In all cases the PCM_FRAME_ALIGN bit has been set to PCM_FRAME_ALIGN_FIRST_BIT (which differs from the default configuration shown in Figure 15 on page 98) to prevent any confusion as to what are the effect of the other frame signal configuration bits. The frame signal configurations shown are:

1. Configured to indicate the subframe words of a frame with a short pulse frame signal.
2. Configured to indicate the subframe words of a frame with a long width frame signal.
3. Configured to use a 2-word frame with a short pulse frame signal.
4. Configured to use a 2-word frame with a long width frame signal.
5. Configured to use a 4-word frame with a short pulse frame signal.
6. Configured to use a 4-word frame with a long width frame signal.

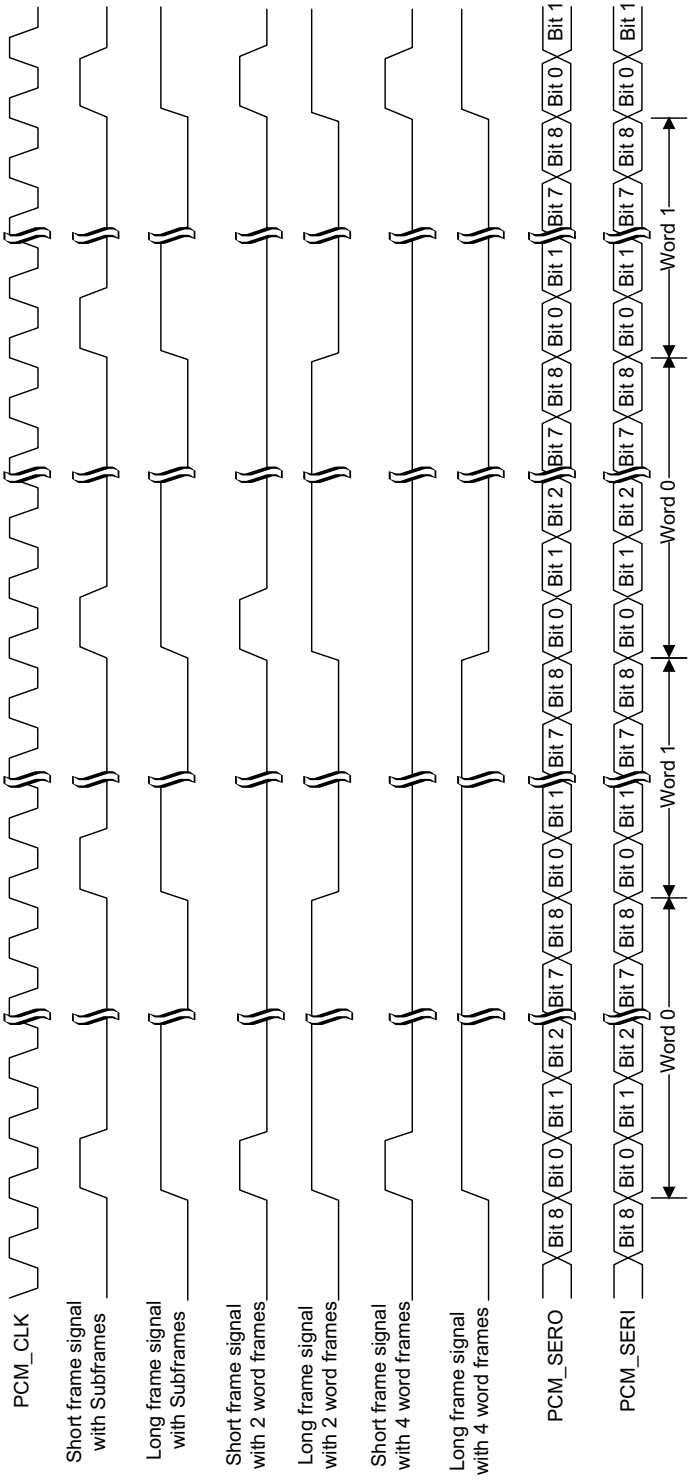


Figure 16: Frame Signal Timing Examples

10.3.2 Data Serial Input and Output Configuration

The PCM interface allows for data bits to be configured for transmission and reception in either an MSB to LSB ordering or an LSB to MSB ordering. To select between these two configurations, set the PCM_BIT_ORDER bit in the D_PCM_CFG register. You can also set the length of each word of a PCM transaction to be between 9 and 24 bits per word by setting the PCM_WORD_SIZE bit field in the D_PCM_CFG register.

IMPORTANT: In all configurations, the PCM data serial input and output lines are sampled on the falling edge of the PCM clock signal. When starting a new transmission, at least two falling edges must occur on the PCM clock line before the first bit can be transmitted or received on either of the PCM data lines.

Additionally, the PCM interface also allows for a number of different data configurations for both data being received and data being transmitted over the PCM interface.

For receive data, the data received is stored into a 24-bit data type. The data bits that are actually received can be left-shifted to place them in the MSB positions of the data to be stored. Alternatively, they can be right-shifted into the LSB positions of the data stored. To select between these two configurations, use the PCM_SHIFT_RX_DATA bit in the D_PCM_CFG register. An example of the resulting input data for each configuration is shown in Table 16.

TABLE 16: RECEIVE DATA CONFIGURATION EXAMPLE FOR A 16-BIT TRANSMISSION

PCM Data Received (Using PCM_WORD_SIZE_16)	PCM Receive Data Configuration	PCM Data Input
0x1234	PCM_SHIFT_RX_DATA_LEFT	0x123400
	PCM_SHIFT_RX_DATA_RIGHT	0x001234

Similarly the output data transmitted by the PCM interface is loaded from 24-bit data. As the word size may not be 24 bits in length, we provided some possible configurations of the transmit data.

First of all, setting the PCM_FIELD_TX_DATA bit in the D_PCM_CFG register selects the bit field within the data to be transmitted. This bit allows you to specify that the data to be sent is loaded from the most significant portion or least significant portion of the data.

If you selected the least significant portion of the data for transmission, you can also configure the interface to either saturate or truncate the data that is to be sent by setting the PCM_SATURATE_TX_DATA bit in the D_PCM_CFG register. Truncating the data to be transmitted results in the exact value of the lower data bits. Saturating the data to be transmitted takes the original data; but if that data exceeds the size that can fit in the bit-field data selected, the data that is sent is set to the maximum or minimum value that can be stored in the bit field (as appropriate).

Note: Selecting data saturation by setting the PCM_SATURATE_TX_DATA bit when the PCM_FIELD_TX_DATA bit has been configured to select the most significant data bits of the data for transmission results in undefined behavior.

An example of the effects that these configuration bits have on data supplied to the PCM interface for transmission of 16-bit words is provided in Table 17.

TABLE 17: TRANSMIT DATA CONFIGURATION EXAMPLE FOR A 16-BIT TRANSMISSION

PCM Data Supplied	PCM Transmit Data Configuration	PCM Data Output (Using PCM_WORD_SIZE_16)
0x123456	PCM_FIELD_TX_DATA_MSB (with PCM_TX_DATA_UNSATURATED or PCM_TX_DATA_SATURATED)	0x1234
	PCM_FIELD_TX_DATA_LSB, PCM_TX_DATA_UNSATURATED	0x3456
	PCM_FIELD_TX_DATA_LSB, PCM_TX_DATA_SATURATED	0x7FFF

10.3.3 Control and Configuration Registers

Register Name	Register Description	Address
D_PCM_CFG	Shared PCM and I^2S interface configuration	0xE240

10.3.3.1 D_PCM_CFG Settings

Bit Field	Field Name	Field Description
21	PCM_SATURATE_TX_DATA	If using the low bit field for PCM transmissions, saturate the data if the data is too large
20	PCM_FIELD_TX_DATA	Select the high or low bit field from the 24-bit data for transmission
19	PCM_SHIFT_RX_DATA	Left or right shift the data received within the 24-bit data
(18:15)	PCM_WORD_SIZE	Select the number of bits per PCM word
(14:12)	PCM_MULTIWORD_CTRL	Configure the number of words per PCM frame
5	PCM_FRAME_WIDTH	Select between a short (pulse) or a long PCM frame signal
4	PCM_FRAME_ALIGN	Select the relative alignment of the PCM frame and data signals
3	PCM_BIT_ORDER	Select a data bit ordering for data that is sent and received, selecting whether data starts with the MSB or the LSB
2	PCM_SUBFRAME	Select between a frame signal with every word and a frame signal with every frame

Field Name	Value Symbol	Value Description	Hex Value
PCM_SATURATE_TX_DATA	PCM_TX_DATA_UNSATURATED	Do not saturate transmit data values	0x0*
	PCM_TX_DATA_SATURATED	Saturate transmit data values	0x1

Field Name	Value Symbol	Value Description	Hex Value
PCM_FIELD_TX_DATA	PCM_FIELD_TX_DATA_MSB	Select the most significant bit field for transmission	0x0*
	PCM_FIELD_TX_DATA_LSB	Select the least significant bit field for transmission	0x1
PCM_SHIFT_RX_DATA	PCM_SHIFT_RX_DATA_LEFT	Shift the received data left	0x0*
	PCM_SHIFT_RX_DATA_RIGHT	Shift the received data right	0x1
PCM_WORD_SIZE	PCM_WORD_SIZE_9	Each PCM word contains 9 bits	0x0*
	PCM_WORD_SIZE_10	Each PCM word contains 10 bits	0x1
	PCM_WORD_SIZE_11	Each PCM word contains 11 bits	0x2
	PCM_WORD_SIZE_12	Each PCM word contains 12 bits	0x3
	PCM_WORD_SIZE_13	Each PCM word contains 13 bits	0x4
	PCM_WORD_SIZE_14	Each PCM word contains 14 bits	0x5
	PCM_WORD_SIZE_15	Each PCM word contains 15 bits	0x6
	PCM_WORD_SIZE_16	Each PCM word contains 16 bits	0x7
	PCM_WORD_SIZE_17	Each PCM word contains 17 bits	0x8
	PCM_WORD_SIZE_18	Each PCM word contains 18 bits	0x9
	PCM_WORD_SIZE_19	Each PCM word contains 19 bits	0xA
	PCM_WORD_SIZE_20	Each PCM word contains 20 bits	0xB
	PCM_WORD_SIZE_21	Each PCM word contains 21 bits	0xC
	PCM_WORD_SIZE_22	Each PCM word contains 22 bits	0xD
	PCM_WORD_SIZE_23	Each PCM word contains 23 bits	0xE
	PCM_WORD_SIZE_24	Each PCM word contains 24 bits	0xF
PCM_MULTIWORD_CTRL	PCM_MULTIWORD_2	PCM frames contain 2 words	0x0*
	PCM_MULTIWORD_4	PCM frames contain 4 words	0x1
	PCM_MULTIWORD_6	PCM frames contain 6 words	0x2
	PCM_MULTIWORD_8	PCM frames contain 8 words	0x3
	PCM_MULTIWORD_10	PCM frames contain 10 words	0x4
	PCM_MULTIWORD_12	PCM frames contain 12 words	0x5
	PCM_MULTIWORD_14	PCM frames contain 14 words	0x6
	PCM_MULTIWORD_16	PCM frames contain 16 words	0x7
PCM_FRAME_WIDTH	PCM_FRAME_WIDTH_SHORT	The frame signal is a one cycle pulse	0x0*
	PCM_FRAME_WIDTH_LONG	The frame signal is a 50% duty cycle square wave	0x1
PCM_FRAME_ALIGN	PCM_FRAME_ALIGN_LAST_BIT	Align the frame signal with the last bit of the previous frame	0x0*
	PCM_FRAME_ALIGN_FIRST_BIT	Align the frame signal with the first bit of the current frame	0x1

Field Name	Value Symbol	Value Description	Hex Value
PCM_BIT_ORDER	PCM_BIT_ORDER_LSB_FIRST	Data is sent/received starting with the LSB	0x0*
	PCM_BIT_ORDER_MSB_FIRST	Data is sent/received starting with the MSB	0x1
PCM_SUBFRAME	PCM_SUBFRAME_DISABLE	Generate a frame signal every frame	0x0*
	PCM_SUBFRAME_ENABLE	Generate a frame signal every word	0x1

10.4 PCM COMMUNICATIONS MANAGEMENT

10.4.1 PCM Device Configuration

The PCM interface uses the PCM_ENABLE bit in the D_PCM_CFG register to enable and disable the PCM interface entirely. When using the PCM interface in any mode, enable this bit.

You can configure the PCM interface as a either a master device (controlling the frame signal and usually the interface clocking), or as a slave device (receiving the frame signal and usually being clocked from an external device). To select between these configurations, set or clear the PCM_SLAVE bit as appropriate in the D_PCM_CFG register.

You can also configure the PCM interface so that the PCM frame signal is driven from UCLK: set the CLK_CFG1_PCM_FRAME_PAD_SEL bit in the D_CLK_CFG_1 register.

CAUTION: If UCLK is the source for the PCM frame signal, configure the PCM interface to use long width frame signals (by setting PCM_FRAME_WIDTH to PCM_FRAME_WIDTH_LONG). Also configure the PCM interface as a slave PCM device so that the interface does not try to drive the PCM frame signal at the same time it is being driven by UCLK.

To configure the PCM clock, use the CLK_CFG1_PCM_CLK_PAD_SEL bit field in the D_CLK_CFG_1 register. By default it is configured as input, which is the desired configuration if the device uses an externally generated clock signal as the source for PCM clock. If you want the device to supply the clock internally instead, configure the device to use SYSCLK, SLOWCLK or UCLK as the source for the PCM clock signal.

10.4.2 PCM Data Management

Either the CFX DSP or the IOC can control the data received or transmitted by the PCM interface. To select between these two configurations, use the PCM_DATA_MGT bit in the D_PCM_CFG register.

In both configurations any data received is interpreted as being received on two separate channels. In all underlying signal configurations, the interface treats all data that is to be sent or that has been received in pairs of words—the first being assigned to PCM channel 0 and the second being assigned to PCM channel 1. The most recent pair of words received is always stored into the

D_PCM_DATA_0 (storing data for channel 0) and D_PCM_DATA_1 (storing data for channel 1) registers.

IMPORTANT: In all timing diagrams included in this chapter, the data assigned to channel 0 has been marked as "Word 0" and the data assigned to channel 1 has been marked as "Word 1".

When you configure the interface to allow the IOC to manage PCM interface data, data input using the interface is stored into a FIFO, and data output using the interface is read from a FIFO as per the IOC configuration. For more information on configuring the IOC to use the PCM interface for data communications, see Chapter 7, "Input/Output Controller (IOC)" on page 51.

When you configure the interface to allow the CFX to manage PCM interface data, the core should use the PCM transmit and PCM receive interrupts to control data flow. All data sent and received by the PCM interface is only accessible through the D_PCM_DATA_0 and D_PCM_DATA_1 registers.

10.4.3 PCM Interrupt Configuration

The PCM interface provides an interrupt which can be configured to trigger on both the transmission and reception of data. The triggers for this interrupt are independently configurable, and interrupts due to the transmission of data or the reception of data can be received separately or together, depending on the interface configuration. See Section 11.1, "Interrupt Controller" on page 113 for information regarding interrupt configuration and handling.

Enable or disable the PCM transmit interrupt with the PCM_TX_INT bit in the D_PCM_CFG register. When transmit interrupts are enabled, interrupts are generated for every two words of data transmitted regardless of the length of each PCM frame. These interrupts occur when the data has been loaded from the D_PCM_DATA_x registers to the internal interface registers. As a result, after a transmit interrupt, you can reload the PCM data registers with the next words to be transmitted without incurring a delay between words transmitted over the PCM interface.

When a PCM interrupt has been generated due to the transmission of data, the PCM_TX_INT_STATUS bit in the D_PCM_CTRL register is set. When one or both of the D_PCM_DATA_x registers is written, providing the interface with the requested data, this bit is cleared.

Enable or disable the PCM receive interrupt with the PCM_RX_INT bit in the D_PCM_CFG register. When receive interrupts are enabled, interrupts are generated for every two words of data received regardless of the length of each PCM frame.

When a PCM interrupt has been generated due to the reception of data, the PCM_RX_INT_STATUS bit in the D_PCM_CTRL register is set. When one or both of the D_PCM_DATA_x registers is used to read the received data that caused this interrupt, this bit is cleared.

CAUTION: Take extra care when configuring the PCM_TX_INT and PCM_RX_INT bits without using defines since both of these bits enable their respective interrupt triggers when cleared (and disable those interrupt triggers when set).

10.4.4 Control and Configuration Registers

Register Name	Register Description	Address
D_CLK_CFG_1	PCM clock and related configuration	0xE081
D_PCM_CFG	Shared PCM and I^2S interface configuration	0xE240
D_PCM_DATA_0	Location where word 0 of the data is loaded from/stored to when using the CFX to manage data	0xE241
D_PCM_DATA_1	Location where word 1 of the data is loaded from/stored to when using the CFX to manage data	0xE242
D_PCM_CTRL	Control bit indicators for the PCM and I^2S interface	0xE243

10.4.4.1 D_CLK_CFG_1 Settings

Bit Field	Field Name	Field Description
23:22	CLK_CFG1_PCM_CLK_PAD_SEL	Select whether the CFX or the IOC will manage PCM data
21	CLK_CFG1_PCM_FRAME_PAD_SEL	Configure how often receive interrupts are generated

Field Name	Value Symbol	Value Description	Hex Value
CLK_CFG1_PCM_CLK_PAD_SEL	PCM_CLK_PAD_AS_INPUT	Use an external PCM clock source	0x0*
	PCM_CLK_PAD_SEL_SYSCLK	Use SYSCLK for the PCM clock	0x1
	PCM_CLK_PAD_SEL_SLOWCLK	Use SLOWCLK for the PCM clock	0x2
	PCM_CLK_PAD_SEL_UCLK	Use UCLK for the PCM clock	0x3
CLK_CFG1_PCM_FRAME_PAD_SEL	PCM_FRAME_PAD_SEL_DIS	Allow the PCM interface or an external interface to drive the PCM frame pad	0x0*
	PCM_FRAME_PAD_SEL_UCLK	Use UCLK for the PCM frame signal	0x1

10.4.4.2 D_PCM_CFG Settings

Bit Field	Field Name	Field Description
11	PCM_DATA_MGT	Select whether the CFX or the IOC will manage PCM data
7	PCM_TX_INT	Enable/disable transmit interrupts
6	PCM_RX_INT	Enable/disable receive interrupts
1	PCM_ENABLE	Enable/disable the PCM interface
0	PCM_SLAVE	Select master or slave mode operation for the PCM interface

Field Name	Value Symbol	Value Description	Hex Value
PCM_DATA_MGT	PCM_DATA_MGT_IOC	The IOC will handle data received/transmitted	0x0*
	PCM_DATA_MGT_CFX	The CFX will handle data received/transmitted	0x1
PCM_TX_INT	PCM_TX_INT_ENABLE	Enable PCM transmit interrupts	0x0*
	PCM_TX_INT_DISABLE	Disable PCM transmit interrupts	0x1
PCM_RX_INT	PCM_RX_INT_ENABLE	Enable PCM receive interrupts	0x0*
	PCM_RX_INT_DISABLE	Disable PCM receive interrupts	0x1
PCM_ENABLE	PCM_DISABLE	Disable the PCM interface	0x0*
	PCM_ENABLE	Enable the PCM interface	0x1
PCM_SLAVE	PCM_MASTER	Use the PCM interface in master mode	0x0
	PCM_SLAVE	Use the PCM interface in slave mode	0x1*

10.4.4.3 D_PCM_CTRL Settings

Bit Field	Field Name	Field Description
1	PCM_TX_INT_STATUS	Indicate when a PCM interface transmit interrupt has occurred
0	PCM_RX_INT_STATUS	Indicate when a PCM interface receive interrupt has occurred

10.5 I²S CONFIGURATION AND USAGE

The I²S protocol is designed to allow a device to share data digitally between devices. When using the PCM interface on BelaSigna 300 for I²S communications there are several configuration requirements. Table 18 lists the required signal and data management settings for the PCM interface to enable I²S communication. As a side effect of the configuration requiring that the device operate in PCM slave mode, the frame signal must be used as an input for I²S communications.

TABLE 18: REQUIRED CONFIGURATION SETTINGS FOR I²S CONFIGURATION

Configuration Bit or Bit Field	Setting
PCM_WORD_SIZE	PCM_WORD_SIZE_16
PCM_MULTIWORD_CTRL	PCM_MULTIWORD_2
PCM_FRAME_WIDTH	PCM_FRAME_WIDTH_LONG
PCM_FRAME_ALIGN	PCM_FRAME_ALIGN_LAST_BIT
PCM_BIT_ORDER	PCM_BIT_ORDER_MSB_FIRST

TABLE 18: REQUIRED CONFIGURATION SETTINGS FOR I^2S CONFIGURATION

Configuration Bit or Bit Field	Setting
PCM_SUBFRAME	PCM_SUBFRAME_DISABLE
PCM_SLAVE	PCM_SLAVE

Note that when operating in this configuration, the behavior of the PCM interface is slightly different than in any other configuration. As shown in the I^2S signal timing diagram (Figure 17) the device allows bit 15 to be repeatedly transmitted until a rising or falling edge on the frame signal is detected. When configured for slave mode with a word size of 16 and a long frame width the PCM interface synchronizes communications with both the rising edge and falling edge of the frame signal (instead of only synchronizing with the rising edge as it does in all other configurations). In this way, the interface is made to be compatible with I^2S signals that use a width greater than 16 bits of data.

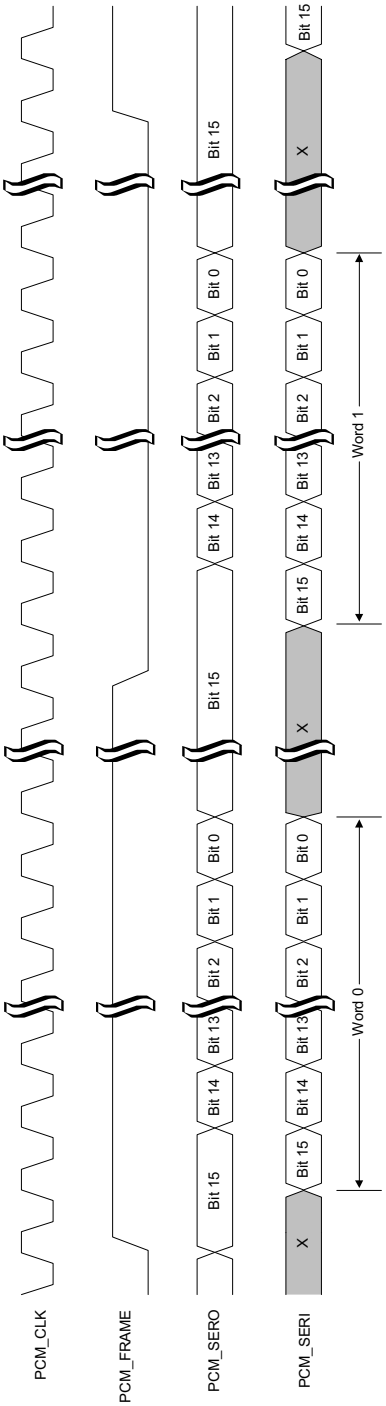


Figure 17: Signal Timing Diagram: I^2S Configuration

This I^2S implementation is compatible with the implementations for data that is exactly 16 bits in length and the frame signal has a duty cycle of exactly 50%.

10.6 NOAHLINK DIGITAL AUDIO STREAMING CONFIGURATION AND USAGE

You can configure the PCM interface to be compatible with inputting data using the digital audio streaming feature of a NOAHlink device. This configuration however requires a number of signal configuration settings as shown in Table 19. This configuration also requires several modifications to the hardware connections as described in Table 20.

TABLE 19: REQUIRED CONFIGURATION SETTINGS FOR NOAHLINK DIGITAL AUDIO STREAMING CONFIGURATION

Configuration Bit or Bit Field	Setting
PCM_WORD_SIZE	PCM_WORD_SIZE_16
PCM_MULTIWORD_CTRL	PCM_MULTIWORD_2
PCM_FRAME_WIDTH	PCM_FRAME_WIDTH_LONG
PCM_FRAME_ALIGN	PCM_FRAME_ALIGN_FIRST_BIT
PCM_BIT_ORDER	PCM_BIT_ORDER_LSB_FIRST
PCM_SUBFRAME	PCM_SUBFRAME_DISABLE

TABLE 20: HARDWARE CONNECTION CONFIGURATION FOR NOAHLINK DIGITAL AUDIO STREAMING

Pad	Connected To	Reason
PCM_CLK	UCLK (internally)	Signal output or input at PCM clock is ignored
PCM_FRAME	I^2C SDA line as output	Uses disconnected pull-ups on I^2C line to limit glitches in the connection to a NOAHlink audio streaming device
PCM_SERO	Unconnected	NOAHlink digital audio streaming is input only
PCM_SERI	I^2C SCL line as input	Uses disconnected pull-ups on I^2C line to limit glitches in the connection to a NOAHlink device

To enable audio streaming when connected to a NOAHlink device, configure the debug port to operate in audio streaming mode.

Note: Support for configuring the debug port for use with audio streaming can be obtained by contacting ON Semiconductor Customer Support.

When in audio streaming mode with the required PCM interface configuration the device will stream audio data from a NOAHlink device. After configuring the PCM interface in this way and

enabling audio streaming the resulting signals should match the timing diagram shown in Figure 18.

IMPORTANT: If you configured the PCM interface for NOAHlink digital audio streaming by connecting the relevant PCM and I^2C pads, analog audio streaming using the NOAHlink device and the I^2C interface is not possible.

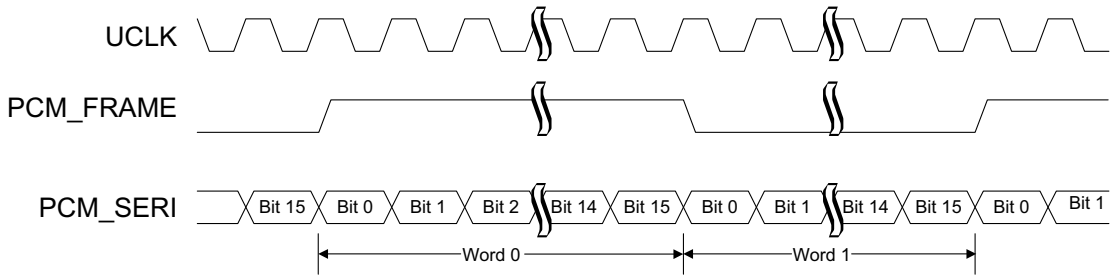


Figure 18: Signal Timing Diagram: NOAHlink Digital Audio Streaming PCM Configuration

11 Peripherals

11.1 Interrupt Controller

The BelaSigna 300 system contains an interrupt controller linked to the CFX DSP that services 25 interrupt sources. The controller primarily uses an external interrupt channel which is directly connected to the various interfaces, peripherals and other components that can trigger an interrupt. The controller also uses a software interrupt channel that is triggered by the `interrupt` instruction of the CFX DSP. For more information about the `interrupt` instruction see the *CFX DSP Architecture Manual*.

Each interrupt source is associated with an interrupt vector that has been mapped into program memory starting at 0xFFE0 (mirrored from 0x3FE0). These vectors must be filled with an unconditional branch to the desired interrupt handler for the interrupt source associated with the vector. The possible interrupt sources along with their interrupt vectors and the ordering of interrupts are listed in Table 21. You can find details specifying what events or conditions will trigger an interrupt condition for a specific interrupt source in the section of this manual that describes the interrupt source.

Table 21: Interrupt Sources and Vectors

Interrupt Source	Interrupt Vector	Number
Watchdog timer	`D_VECT_WATCHDOG`	1
General purpose timer 1	`D_VECT_TIMER_1`	2
General purpose timer 2	`D_VECT_TIMER_2`	3
SPI interface	`D_VECT_SPI`	4
I^2C interface	`D_VECT_I2C`	5
PCM interface	`D_VECT_PCM`	6
GPIO interface	`D_VECT_GPIO`	7
HEAR Accelerator (interrupt #0)	`D_VECT_HEAR_0`	8
HEAR Accelerator (interrupt #1)	`D_VECT_HEAR_1`	9
HEAR Accelerator (interrupt #2)	`D_VECT_HEAR_2`	10
HEAR Accelerator (interrupt #3)	`D_VECT_HEAR_3`	11
HEAR Accelerator (interrupt #4)	`D_VECT_HEAR_4`	12
HEAR Accelerator (interrupt #5)	`D_VECT_HEAR_5`	13
HEAR Accelerator (interrupt #6)	`D_VECT_HEAR_6`	14
HEAR Accelerator (interrupt #7)	`D_VECT_HEAR_7`	15
FIFO A0	`D_VECT_FIFO_A0`	16
FIFO A1	`D_VECT_FIFO_A1`	17
FIFO A2	`D_VECT_FIFO_A2`	18

TABLE 21: INTERRUPT SOURCES AND VECTORS (CONTINUED)

Interrupt Source	Interrupt Vector	Number
FIFO A3	D_VECT_FIFO_A3	19
FIFO B0	D_VECT_FIFO_B0	20
FIFO B1	D_VECT_FIFO_B1	21
FIFO B2	D_VECT_FIFO_B2	22
FIFO B3	D_VECT_FIFO_B3	23
UART interface data reception	D_VECT_UART_RX	24
UART interface data transmission	D_VECT_UART_TX	25

Even when more than one interrupt is enabled, only one interrupt can be issued for handling at a time. As a result you might find it valuable to prioritize certain interrupts over others. Assign either normal or high priority to an interrupt by setting or clearing the appropriate bits in the D_INT_PRIORITY register. This setting has also been mapped into the D_INT_PRIO_x registers in 8-bit blocks for efficient configuration of a subset of the interrupt sources. If multiple interrupts are pending, all high priority interrupts are handled before normal priority interrupts. As well, if multiple pending interrupts share a priority level, the lowest numbered interrupt (as numbered in Table 21) is handled first.

The interrupt enable register, D_INT_ENABLE, provides interrupt enable flags for each of the interrupt vectors. This register also contains the INT_EBL_MASTER bit, which acts as a master enable/disable for all interrupts regardless of their individual settings. These bits have also been mapped into the D_INT_EBL_x and D_INT_DIS_x registers. The D_INT_EBL_x registers provide an interface that allows a user only to enable interrupts, which renders reading the interrupt enable register to acquire the current settings before changing them unnecessary. Similarly, the D_INT_DIS_x registers provide an interface that allows a user only to disable interrupts. Finally, an interrupt can also be enabled through the D_INT_ACK_EBL_x registers. These registers act like the D_INT_EBL_x registers except that they also acknowledge the interrupts that are being enabled to clear out any interrupt conditions that might have been pending when the interrupt source was enabled.

When an interrupt source generates an interrupt event, the interrupt status bit that corresponds to the interrupt source is set in the D_INT_STATUS register. The bits in this register can be manually cleared to acknowledge their linked interrupt by writing a 1 to either this bit in the D_INT_STATUS register or to the corresponding bit in one of the D_INT_ACK_x registers. The define INT_ACK_ALL_INTS is available as a short form for use when writing to the D_INT_STATUS register to clear all interrupts. The bit corresponding to an interrupt is also cleared when the CFX DSP handles the associated interrupt. When the CFX DSP is acknowledging an interrupt, no new interrupt signal for that interrupt will be accepted until the following clock cycle.

Note: An interrupt that is not enabled is still marked in the status register because the interrupt enable register only masks interrupt notification and not generation. As a result, if you want to ensure that the interrupt triggers only

for new interrupt conditions, acknowledge interrupts when the interrupt is enabled.

IMPORTANT: Unlike the case where the CFX DSP is automatically acknowledging an interrupt, when an interrupt is acknowledged by a user application, an interrupt signal for it is accepted immediately.

In summary, the interrupt controller selects an interrupt to issue based on the following criteria:

1. The interrupt controller starts with those interrupts that are pending as marked in the interrupt status register (`D_INT_STATUS`).
2. This list of pending interrupts is culled by the interrupt enable settings in `D_INT_ENABLE`, with all pending interrupts being culled from the list if the `INT_MASTER_EBL` bit is not set.
3. The remaining list is split into high priority and normal priority interrupts.
4. The first interrupt from the remaining list with a high priority is issued. If no high priority interrupts are pending the first normal priority interrupt is issued.

This selection criteria is illustrated in Figure 19.

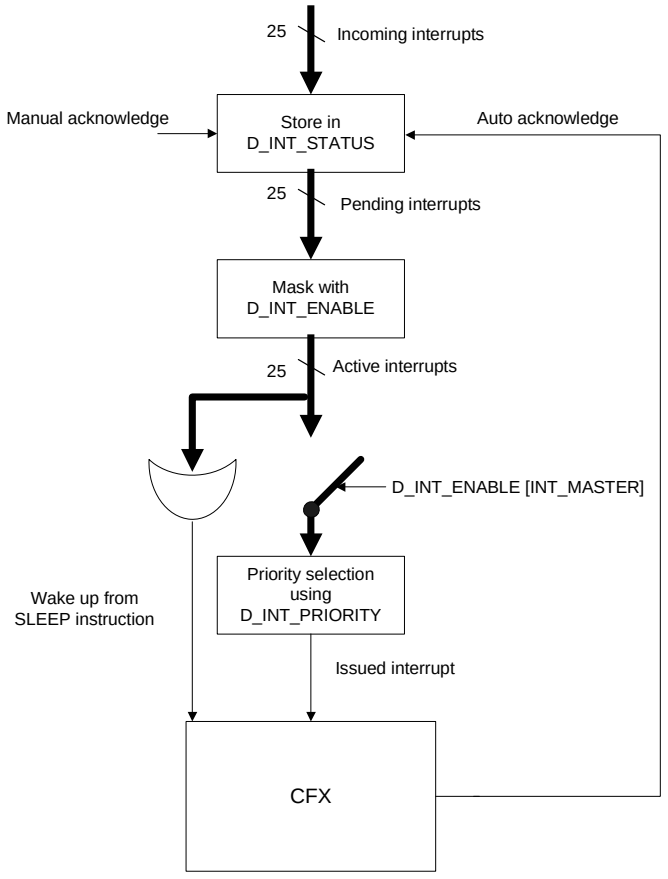


Figure 19: Criteria the Interrupt Controller Uses to Select an Interrupt to Issue

11.1.1 Control and Configuration Registers

Register Name	Register Description	Address
D_INT_ENABLE	Enable/disable interrupts based on interrupt source	0xE040
D_INT_STATUS	Check the status of/acknowledge interrupts based on interrupt source	0xE041
D_INT_PRIORITY	Set the priority of interrupts based on interrupt source	0xE042
D_INT_EBL_0	Enable interrupts 1 to 7 and the master interrupt enable	0xE050
D_INT_EBL_1	Enable interrupts 8 to 15	0xE051
D_INT_EBL_2	Enable interrupts 16 to 23	0xE052
D_INT_EBL_3	Enable interrupts 24 to 25	0xE053
D_INT_ACK_EBL_0	Acknowledge and enable interrupts 1 to 7 and the master interrupt enable	0xE054
D_INT_ACK_EBL_1	Acknowledge and enable interrupts 8 to 15	0xE055
D_INT_ACK_EBL_2	Acknowledge and enable interrupts 16 to 23	0xE056
D_INT_ACK_EBL_3	Acknowledge and enable interrupts 24 to 25	0xE057
D_INT_DIS_0	Disable interrupts 1 to 7 and the master interrupt enable	0xE058
D_INT_DIS_1	Disable interrupts 8 to 15	0xE059
D_INT_DIS_2	Disable interrupts 16 to 23	0xE05A
D_INT_DIS_3	Disable interrupts 24 to 25	0xE05B
D_INT_ACK_0	Acknowledge interrupts 1 to 7	0xE05C
D_INT_ACK_1	Acknowledge interrupts 8 to 15	0xE05D
D_INT_ACK_2	Acknowledge interrupts 16 to 23	0xE05E
D_INT_ACK_3	Acknowledge interrupts 24 to 25	0xE05F
D_INT_PRIO_0	Select between normal and high priority for interrupts 1 to 7	0xE060
D_INT_PRIO_1	Select between normal and high priority for interrupts 8 to 15	0xE061
D_INT_PRIO_2	Select between normal and high priority for interrupts 16 to 23	0xE062
D_INT_PRIO_3	Select between normal and high priority for interrupts 24 to 25	0xE063

11.1.1.1 D_INT_ENABLE Settings

Bit Field	Field Name	Field Description
25	INT_UART_TX	Enable/disable UART transmit interrupts
24	INT_UART_RX	Enable/disable UART receive interrupts
23	INT_FIFO_B3	Enable/disable interrupts for FIFO B3
22	INT_FIFO_B2	Enable/disable interrupts for FIFO B2
21	INT_FIFO_B1	Enable/disable interrupts for FIFO B1
20	INT_FIFO_B0	Enable/disable interrupts for FIFO B0
19	INT_FIFO_A3	Enable/disable interrupts for FIFO A3
18	INT_FIFO_A2	Enable/disable interrupts for FIFO A2

Bit Field	Field Name	Field Description
17	INT_FIFO_A1	Enable/disable interrupts for FIFO A1
16	INT_FIFO_A0	Enable/disable interrupts for FIFO A0
15	INT_HEAR_7	Enable/disable interrupt #7 from the HEAR
14	INT_HEAR_6	Enable/disable interrupt #6 from the HEAR
13	INT_HEAR_5	Enable/disable interrupt #5 from the HEAR
12	INT_HEAR_4	Enable/disable interrupt #4 from the HEAR
11	INT_HEAR_3	Enable/disable interrupt #3 from the HEAR
10	INT_HEAR_2	Enable/disable interrupt #2 from the HEAR
9	INT_HEAR_1	Enable/disable interrupt #1 from the HEAR
8	INT_HEAR_0	Enable/disable interrupt #0 from the HEAR
7	INT_GPIO	Enable/disable GPIO interrupts
6	INT_PCM	Enable/disable PCM interrupts
5	INT_I2C	Enable/disable I^2C interrupts
4	INT_SPI	Enable/disable SPI interrupts
3	INT_TIMER_2	Enable/disable interrupts for timer 2
2	INT_TIMER_1	Enable/disable interrupts for timer 1
1	INT_WATCHDOG	Enable/disable watchdog timer interrupts
0	INT_MASTER	Master enable/disable for all interrupts

Field Name	Value Symbol	Value Description	Hex Value
INT_UART_TX	INT_EBL_UART_TX	Enable UART transmit interrupts	0x1
INT_UART_RX	INT_EBL_UART_RX	Enable UART receive interrupts	0x1
INT_FIFO_B3	INT_EBL_FIFO_B3	Enable interrupts for FIFO B3	0x1
INT_FIFO_B2	INT_EBL_FIFO_B2	Enable interrupts for FIFO B2	0x1
INT_FIFO_B1	INT_EBL_FIFO_B1	Enable interrupts for FIFO B1	0x1
INT_FIFO_B0	INT_EBL_FIFO_B0	Enable interrupts for FIFO B0	0x1
INT_FIFO_A3	INT_EBL_FIFO_A3	Enable interrupts for FIFO A3	0x1
INT_FIFO_A2	INT_EBL_FIFO_A2	Enable interrupts for FIFO A2	0x1
INT_FIFO_A1	INT_EBL_FIFO_A1	Enable interrupts for FIFO A1	0x1
INT_FIFO_A0	INT_EBL_FIFO_A0	Enable interrupts for FIFO A0	0x1
INT_HEAR_7	INT_EBL_HEAR_7	Enable interrupt #7 from the HEAR	0x1
INT_HEAR_6	INT_EBL_HEAR_6	Enable interrupt #6 from the HEAR	0x1
INT_HEAR_5	INT_EBL_HEAR_5	Enable interrupt #5 from the HEAR	0x1

Field Name	Value Symbol	Value Description	Hex Value
INT_HEAR_4	INT_EBL_HEAR_4	Enable interrupt #4 from the HEAR	0x1
INT_HEAR_3	INT_EBL_HEAR_3	Enable interrupt #3 from the HEAR	0x1
INT_HEAR_2	INT_EBL_HEAR_2	Enable interrupt #2 from the HEAR	0x1
INT_HEAR_1	INT_EBL_HEAR_1	Enable interrupt #1 from the HEAR	0x1
INT_HEAR_0	INT_EBL_HEAR_0	Enable interrupt #0 from the HEAR	0x1
INT_GPIO	INT_EBL_GPIO	Enable GPIO interrupts	0x1
INT_PCM	INT_EBL_PCM	Enable PCM interrupts	0x1
INT_I2C	INT_EBL_I2C	Enable I^2C interrupts	0x1
INT_SPI	INT_EBL_SPI	Enable SPI interrupts	0x1
INT_TIMER_2	INT_EBL_TIMER_2	Enable interrupts for timer 2	0x1
INT_TIMER_1	INT_EBL_TIMER_1	Enable interrupts for timer 1	0x1
INT_WATCHDOG	INT_EBL_WATCHDOG	Enable watchdog timer interrupts	0x1
INT_MASTER	INT_EBL_MASTER	Master enable for all interrupts	0x1

11.1.1.2 D_INT_STATUS Settings

Bit Field	Field Name	Field Description
25	INT_UART_TX	Acknowledge/pending status of UART transmit interrupt
24	INT_UART_RX	Acknowledge/pending status of UART receive interrupt
23	INT_FIFO_B3	Acknowledge/pending status of interrupt for FIFO B3
22	INT_FIFO_B2	Acknowledge/pending status of interrupt for FIFO B2
21	INT_FIFO_B1	Acknowledge/pending status of interrupt for FIFO B1
20	INT_FIFO_B0	Acknowledge/pending status of interrupt for FIFO B0
19	INT_FIFO_A3	Acknowledge/pending status of interrupt for FIFO A3
18	INT_FIFO_A2	Acknowledge/pending status of interrupt for FIFO A2
17	INT_FIFO_A1	Acknowledge/pending status of interrupt for FIFO A1
16	INT_FIFO_A0	Acknowledge/pending status of interrupt for FIFO A0
15	INT_HEAR_7	Acknowledge/pending status of interrupt #7 from the HEAR
14	INT_HEAR_6	Acknowledge/pending status of interrupt #6 from the HEAR
13	INT_HEAR_5	Acknowledge/pending status of interrupt #5 from the HEAR
12	INT_HEAR_4	Acknowledge/pending status of interrupt #4 from the HEAR
11	INT_HEAR_3	Acknowledge/pending status of interrupt #3 from the HEAR
10	INT_HEAR_2	Acknowledge/pending status of interrupt #2 from the HEAR

Bit Field	Field Name	Field Description
9	INT_HEAR_1	Acknowledge/pending status of interrupt #1 from the HEAR
8	INT_HEAR_0	Acknowledge/pending status of interrupt #0 from the HEAR
7	INT_GPIO	Acknowledge/pending status of GPIO interrupt
6	INT_PCM	Acknowledge/pending status of PCM interrupt
5	INT_I2C	Acknowledge/pending status of I^2C interrupt
4	INT_SPI	Acknowledge/pending status of SPI interrupt
3	INT_TIMER_2	Acknowledge/pending status of interrupt for timer 2
2	INT_TIMER_1	Acknowledge/pending status of interrupt for timer 1
1	INT_WATCHDOG	Acknowledge/pending status of watchdog timer interrupt

Field Name	Value Symbol	Value Description	Hex Value
INT_UART_TX	INT_ACK_UART_TX	Acknowledge UART transmit interrupt	0x1
INT_UART_RX	INT_ACK_UART_RX	Acknowledge UART receive interrupt	0x1
INT_FIFO_B3	INT_ACK_FIFO_B3	Acknowledge interrupt for FIFO B3	0x1
INT_FIFO_B2	INT_ACK_FIFO_B2	Acknowledge interrupt for FIFO B2	0x1
INT_FIFO_B1	INT_ACK_FIFO_B1	Acknowledge interrupt for FIFO B1	0x1
INT_FIFO_B0	INT_ACK_FIFO_B0	Acknowledge interrupt for FIFO B0	0x1
INT_FIFO_A3	INT_ACK_FIFO_A3	Acknowledge interrupt for FIFO A3	0x1
INT_FIFO_A2	INT_ACK_FIFO_A2	Acknowledge interrupt for FIFO A2	0x1
INT_FIFO_A1	INT_ACK_FIFO_A1	Acknowledge interrupt for FIFO A1	0x1
INT_FIFO_A0	INT_ACK_FIFO_A0	Acknowledge interrupt for FIFO A0	0x1
INT_HEAR_7	INT_ACK_HEAR_7	Acknowledge interrupt #7 from the HEAR	0x1
INT_HEAR_6	INT_ACK_HEAR_6	Acknowledge interrupt #6 from the HEAR	0x1
INT_HEAR_5	INT_ACK_HEAR_5	Acknowledge interrupt #5 from the HEAR	0x1
INT_HEAR_4	INT_ACK_HEAR_4	Acknowledge interrupt #4 from the HEAR	0x1
INT_HEAR_3	INT_ACK_HEAR_3	Acknowledge interrupt #3 from the HEAR	0x1
INT_HEAR_2	INT_ACK_HEAR_2	Acknowledge interrupt #2 from the HEAR	0x1
INT_HEAR_1	INT_ACK_HEAR_1	Acknowledge interrupt #1 from the HEAR	0x1
INT_HEAR_0	INT_ACK_HEAR_0	Acknowledge interrupt #0 from the HEAR	0x1
INT_GPIO	INT_ACK_GPIO	Acknowledge GPIO interrupt	0x1
INT_PCM	INT_ACK_PCM	Acknowledge PCM interrupt	0x1
INT_I2C	INT_ACK_I2C	Acknowledge I^2C interrupt	0x1
INT_SPI	INT_ACK_SPI	Acknowledge SPI interrupt	0x1
INT_TIMER_2	INT_ACK_TIMER_2	Acknowledge interrupt for timer 2	0x1
INT_TIMER_1	INT_ACK_TIMER_1	Acknowledge interrupt for timer 1	0x1
INT_WATCHDOG	INT_ACK_WATCHDOG	Acknowledge watchdog timer interrupt	0x1

11.1.1.3 D_INT_PRIORITY Settings

Bit Field	Field Name	Field Description
25	INT_UART_TX	Select normal or high priority for UART transmit interrupt
24	INT_UART_RX	Select normal or high priority for UART receive interrupt
23	INT_FIFO_B3	Select normal or high priority for interrupt for FIFO B3
22	INT_FIFO_B2	Select normal or high priority for interrupt for FIFO B2
21	INT_FIFO_B1	Select normal or high priority for interrupt for FIFO B1
20	INT_FIFO_B0	Select normal or high priority for interrupt for FIFO B0
19	INT_FIFO_A3	Select normal or high priority for interrupt for FIFO A3
18	INT_FIFO_A2	Select normal or high priority for interrupt for FIFO A2
17	INT_FIFO_A1	Select normal or high priority for interrupt for FIFO A1
16	INT_FIFO_A0	Select normal or high priority for interrupt for FIFO A0
15	INT_HEAR_7	Select normal or high priority for interrupt #7 from the HEAR
14	INT_HEAR_6	Select normal or high priority for interrupt #6 from the HEAR
13	INT_HEAR_5	Select normal or high priority for interrupt #5 from the HEAR
12	INT_HEAR_4	Select normal or high priority for interrupt #4 from the HEAR
11	INT_HEAR_3	Select normal or high priority for interrupt #3 from the HEAR
10	INT_HEAR_2	Select normal or high priority for interrupt #2 from the HEAR
9	INT_HEAR_1	Select normal or high priority for interrupt #1 from the HEAR
8	INT_HEAR_0	Select normal or high priority for interrupt #0 from the HEAR
7	INT_GPIO	Select normal or high priority for GPIO interrupt
6	INT_PCM	Select normal or high priority for PCM interrupt
5	INT_I2C	Select normal or high priority for I^2C interrupt
4	INT_SPI	Select normal or high priority for SPI interrupt
3	INT_TIMER_2	Select normal or high priority for interrupt for timer 2
2	INT_TIMER_1	Select normal or high priority for interrupt for timer 1
1	INT_WATCHDOG	Select normal or high priority for watchdog timer interrupt

Field Name	Value Symbol	Value Description	Hex Value
INT_UART_TX	INT_NORMALPRIO_UART_TX	Normal interrupt priority for UART transmit	0x0*
	INT_HIGHPRIO_UART_TX	High interrupt priority for UART transmit	0x1
INT_UART_RX	INT_NORMALPRIO_UART_RX	Normal interrupt priority for UART receive	0x0*
	INT_HIGHPRIO_UART_RX	High interrupt priority for UART receive	0x1

Field Name	Value Symbol	Value Description	Hex Value
INT_FIFO_B3	INT_NORMALPRIO_FIFO_B3	Normal interrupt priority for interrupt for FIFO B3	0x0*
	INT_HIGHPRIO_FIFO_B3	High interrupt priority for interrupt for FIFO B3	0x1
INT_FIFO_B2	INT_NORMALPRIO_FIFO_B2	Normal interrupt priority for interrupt for FIFO B2	0x0*
	INT_HIGHPRIO_FIFO_B2	High interrupt priority for interrupt for FIFO B2	0x1
INT_FIFO_B1	INT_NORMALPRIO_FIFO_B1	Normal interrupt priority for interrupt for FIFO B1	0x0*
	INT_HIGHPRIO_FIFO_B1	High interrupt priority for interrupt for FIFO B1	0x1
INT_FIFO_B0	INT_NORMALPRIO_FIFO_B0	Normal interrupt priority for interrupt for FIFO B0	0x0*
	INT_HIGHPRIO_FIFO_B0	High interrupt priority for interrupt for FIFO B0	0x1
INT_FIFO_A3	INT_NORMALPRIO_FIFO_A3	Normal interrupt priority for interrupt for FIFO A3	0x0*
	INT_HIGHPRIO_FIFO_A3	High interrupt priority for interrupt for FIFO A3	0x1
INT_FIFO_A2	INT_NORMALPRIO_FIFO_A2	Normal interrupt priority for interrupt for FIFO A2	0x0*
	INT_HIGHPRIO_FIFO_A2	High interrupt priority for interrupt for FIFO A2	0x1
INT_FIFO_A1	INT_NORMALPRIO_FIFO_A1	Normal interrupt priority for interrupt for FIFO A1	0x0*
	INT_HIGHPRIO_FIFO_A1	High interrupt priority for interrupt for FIFO A1	0x1
INT_FIFO_A0	INT_NORMALPRIO_FIFO_A0	Normal interrupt priority for interrupt for FIFO A0	0x0*
	INT_HIGHPRIO_FIFO_A0	High interrupt priority for interrupt for FIFO A0	0x1
INT_HEAR_7	INT_NORMALPRIO_HEAR_7	Normal interrupt priority for interrupt #7 from the HEAR	0x0*
	INT_HIGHPRIO_HEAR_7	High interrupt priority for interrupt #7 from the HEAR	0x1
INT_HEAR_6	INT_NORMALPRIO_HEAR_6	Normal interrupt priority for interrupt #6 from the HEAR	0x0*
	INT_HIGHPRIO_HEAR_6	High interrupt priority for interrupt #6 from the HEAR	0x1

Field Name	Value Symbol	Value Description	Hex Value
INT_HEAR_5	INT_NORMALPRIO_HEAR_5	Normal interrupt priority for interrupt #5 from the HEAR	0x0*
	INT_HIGHPRIO_HEAR_5	High interrupt priority for interrupt #5 from the HEAR	0x1
INT_HEAR_4	INT_NORMALPRIO_HEAR_4	Normal interrupt priority for interrupt #4 from the HEAR	0x0*
	INT_HIGHPRIO_HEAR_4	High interrupt priority for interrupt #4 from the HEAR	0x1
INT_HEAR_3	INT_NORMALPRIO_HEAR_3	Normal interrupt priority for interrupt #3 from the HEAR	0x0*
	INT_HIGHPRIO_HEAR_3	High interrupt priority for interrupt #3 from the HEAR	0x1
INT_HEAR_2	INT_NORMALPRIO_HEAR_2	Normal interrupt priority for interrupt #2 from the HEAR	0x0*
	INT_HIGHPRIO_HEAR_2	High interrupt priority for interrupt #2 from the HEAR	0x1
INT_HEAR_1	INT_NORMALPRIO_HEAR_1	Normal interrupt priority for interrupt #1 from the HEAR	0x0*
	INT_HIGHPRIO_HEAR_1	High interrupt priority for interrupt #1 from the HEAR	0x1
INT_HEAR_0	INT_NORMALPRIO_HEAR_0	Normal interrupt priority for interrupt #0 from the HEAR	0x0*
	INT_HIGHPRIO_HEAR_0	High interrupt priority for interrupt #0 from the HEAR	0x1
INT_GPIO	INT_NORMALPRIO_GPIO	Normal interrupt priority for GPIO	0x0*
	INT_HIGHPRIO_GPIO	High interrupt priority for GPIO	0x1
INT_PCM	INT_NORMALPRIO_PCM	Normal interrupt priority for PCM	0x0*
	INT_HIGHPRIO_PCM	High interrupt priority for PCM	0x1
INT_I2C	INT_NORMALPRIO_I2C	Normal interrupt priority for I^2C	0x0*
	INT_HIGHPRIO_I2C	High interrupt priority for I^2C	0x1
INT_SPI	INT_NORMALPRIO_SPI	Normal interrupt priority for SPI	0x0*
	INT_HIGHPRIO_SPI	High interrupt priority for SPI	0x1
INT_TIMER_2	INT_NORMALPRIO_TIMER_2	Normal interrupt priority for timer 2	0x0*
	INT_HIGHPRIO_TIMER_2	High interrupt priority for timer 2	0x1
INT_TIMER_1	INT_NORMALPRIO_TIMER_1	Normal interrupt priority for timer 1	0x0*
	INT_HIGHPRIO_TIMER_1	High interrupt priority for timer 1	0x1
INT_WATCHDOG	INT_NORMALPRIO_WATCHDOG	Normal interrupt priority for the watchdog timer	0x0*
	INT_HIGHPRIO_WATCHDOG	High interrupt priority for the watchdog timer	0x1

11.2 GENERAL PURPOSE TIMERS

The BelaSigna 300 system contains two general purpose timers for use in a user's application. Each of these timers operates independently of the other and can be configured to trigger at a wide range of frequencies.

By default the general purpose timers operate at a division of 32 from the slow clock. To increase this prescaling division for a timer, set the `TIMER_CFG_PRESCALE` bit field in the appropriate `D_TIMER_x_CFG` register. Prescalers increase in multiples of 2 allowing the selection of a clock that is divided by as much as 4096 for each timer. As a result, the range of the timers can be increased; however the granularity of the steps at which the timer can be configured to trigger at increases in parallel.

After prescaling the slow clock, each timer can be configured to trigger after 1 to 4096 cycles of the prescaled clock by setting the `TIMER_CFG_DELAY` bit field in the appropriate `D_TIMER_x_CFG` register. The resulting timer delay is equal to:

$$\text{DELAY} = \frac{2^{(5 + \text{TIMER_CFG_PRESCALE})} \times (\text{TIMER_CFG_DELAY} + 1)}{\text{SLOWCLK}}$$

A summary of possible timer delays for 1.28MHz is in Table 22, and timer delays for 1.92MHz are in Table 23.

TABLE 22: GENERAL PURPOSE TIMER CONFIGURATION (1.28MHZ)

Prescaler Divisor (Bit Field setting)	Timer Granularity	Counter Range	Minimum Timeout	Maximum Timeout
32 (0)	0.025ms	1 to 4096	0.025ms	102.4ms
64 (1)	0.05ms	1 to 4096	0.05ms	204.8ms
128 (2)	0.1ms	1 to 4096	0.1ms	409.6ms
256 (3)	0.2ms	1 to 4096	0.2ms	819.2ms
512 (4)	0.4ms	1 to 4096	0.4ms	1.6384s
1024 (5)	0.8ms	1 to 4096	0.8ms	3.2768s
2056 (6)	1.6ms	1 to 4096	1.6ms	6.5536s
4096 (7)	3.2ms	1 to 4096	3.2ms	13.1072s

TABLE 23: GENERAL PURPOSE TIMER CONFIGURATION (1.92MHZ)

Prescaler Divisor (Bit Field setting)	Timer Granularity	Counter Range	Minimum Timeout	Maximum Timeout
32 (0)	0.017ms	1 to 4096	0.017ms	68.27ms
64 (1)	0.033ms	1 to 4096	0.033ms	136.53ms
128 (2)	0.067ms	1 to 4096	0.067ms	273.07ms
256 (3)	0.133ms	1 to 4096	0.133ms	546.13ms

TABLE 23: GENERAL PURPOSE TIMER CONFIGURATION (1.92MHZ)

Prescaler Divisor (Bit Field setting)	Timer Granularity	Counter Range	Minimum Timeout	Maximum Timeout
512 (4)	0.267ms	1 to 4096	0.267ms	1.092s
1024 (5)	0.533ms	1 to 4096	0.533ms	2.185s
2056 (6)	1.067ms	1 to 4096	1.067ms	4.369s
4096 (7)	2.133ms	1 to 4096	2.133ms	8.738s

The general purpose timers can be configured to either run continuously (free running mode) or to trigger a single time (one-shot mode). To select the timer mode, use the TIMER_CFG_MODE bit in the D_TIMER_x_CFG registers.

Start or stop each timer by writing to the TIMER_CFG_START_STOP bit in the D_TIMER_x_CFG registers. This method of starting/stopping the timer is typically used when the timer is started or stopped during configuration. Alternatively, the timers can be started (but not stopped) by setting the appropriate SYSTEM_CTRL_TIMER_x_START bit in the D_SYSTEM_CTRL register. As with all bits in the D_SYSTEM_CTRL register, setting a SYSTEM_CTRL_TIMER_x_START bit does not require knowledge of the configuration of the other register bits (as bit settings in the D_SYSTEM_CTRL register only change when explicitly set and not when cleared). Reading the SYSTEM_CTRL_TIMER_x_START bit indicates whether the timer is currently running or is stopped.

11.2.1 Control and Configuration Registers

Register Name	Register Description	Address
D_SYSTEM_CTRL	System Control	0xE000
D_TIMER_1_CFG	Configuration of general purpose timer 1	0xE002
D_TIMER_2_CFG	Configuration of general purpose timer 2	0xE003

11.2.1.1 D_SYSTEM_CTRL Settings

Bit Field	Field Name	Field Description
2	SYSTEM_CTRL_TIMER_2_START	Start general purpose timer 2
2	SYSTEM_CTRL_TIMER_2_BUSY	Check the status of general purpose timer 2
1	SYSTEM_CTRL_TIMER_1_START	Start general purpose timer 1
1	SYSTEM_CTRL_TIMER_1_BUSY	Check the status of general purpose timer 1

Field Name	Value Symbol	Value Description	Hex Value
SYSTEM_CTRL_TIMER_2_START	SYSTEM_TIMER_2_START	Start general purpose timer 2	0x1
SYSTEM_CTRL_TIMER_1_START	SYSTEM_TIMER_1_START	Start general purpose timer 1	0x1

11.2.1.2 D_TIMER_1_CFG and D_TIMER_2_CFG Settings

Bit Field	Field Name	Field Description
16	TIMER_CFG_START_STOP	Start or stop the general purpose timer
15	TIMER_CFG_MODE	Select between one-shot and free-running mode for the timer
14:12	TIMER_CFG_PRESCALE	Select a prescaling divider to the clock for the timer
11:0	TIMER_CFG_DELAY	Configure the counter cycles to delay for the timer

Field Name	Value Symbol	Value Description	Hex Value
TIMER_CFG_START_STOP	TIMER_STOP	Stop the timer	0x0*
	TIMER_START	Start the timer	0x1
TIMER_CFG_MODE	TIMER_ONESHOT	Select one-shot mode (timer triggers only once)	0x0*
	TIMER_FREERUN	Select free-run mode	0x1
TIMER_CFG_PRESCALE	TIMER_PRESCALE_32	Divide SLOWCLK by 32	0x0*
	TIMER_PRESCALE_64	Divide SLOWCLK by 64	0x1
	TIMER_PRESCALE_128	Divide SLOWCLK by 128	0x2
	TIMER_PRESCALE_256	Divide SLOWCLK by 256	0x3
	TIMER_PRESCALE_512	Divide SLOWCLK by 512	0x4
	TIMER_PRESCALE_1024	Divide SLOWCLK by 1024	0x5
	TIMER_PRESCALE_2048	Divide SLOWCLK by 2048	0x6
	TIMER_PRESCALE_4096	Divide SLOWCLK by 4096	0x7

11.3 WATCHDOG TIMER

The watchdog timer is a simple safety system that in most cases will reset a system that has malfunctioned. It is a countdown timer that must be periodically acknowledged by writing SYSTEM_WATCHDOG_REFERESH to the D_SYSTEM_CTRL register before it reaches zero. Setting this bit does not affect the other bits in this register as these bits only change when they are explicitly set. The system assumes that the application's failure to acknowledge this countdown timer before it reaches zero indicates that the system is malfunctioning and must be reset. The countdown timer value for the watchdog timer is not visible to the CFX DSP. The watchdog timer cannot be disabled.

The watchdog timer runs on a prescaled clock that has been derived from the slow clock. This clock is used to decrement the value in a 13-bit counter. The delay before this timer will time out can be configured using the WATCHDOG_CFG_TIMEOUT field of the D_WATCHDOG_CFG register. However, since this field is defined in terms of SLOWCLK the delay is different for different slow clocks (although the configuration defines for this timer are specified in terms of a 1.28MHz SLOWCLK by historical convention). Table 24 shows the delays associated with SLOWCLK settings of 1.28MHz and 1.92MHz.

TABLE 24: WATCHDOG TIMEOUT INTERVALS FOR DIFFERENT SLOWCLK SETTINGS

Watchdog Scaler Setting	Approximate Timeout: SLOWCLK=1.28MHz (ms)	Approximate Timeout: SLOWCLK=1.92MHz (ms)
WATCHDOG_TIMEOUT_1M6	1.6	1.1
WATCHDOG_TIMEOUT_3M2	3.2	2.1
WATCHDOG_TIMEOUT_6M4	6.4	4.3
WATCHDOG_TIMEOUT_12M8	12.8	8.5
WATCHDOG_TIMEOUT_25M6	25.6	17.1
WATCHDOG_TIMEOUT_51M2	51.2	34.1
WATCHDOG_TIMEOUT_102M4	102.4	68.3
WATCHDOG_TIMEOUT_204M8	204.8	136.5
WATCHDOG_TIMEOUT_409M6	409.6	273.1
WATCHDOG_TIMEOUT_819M2	819.2	546.1
WATCHDOG_TIMEOUT_1638M4	1638.4	1092.3
WATCHDOG_TIMEOUT_3276M8	3276.8	2184.5

11.3.1 Watchdog Interrupt

The watchdog has an associated interrupt that triggers whenever the watchdog times out. If the watchdog timer has not been reset and a second watchdog timeout occurs, the entire system resets. See Section 11.1, “Interrupt Controller” on page 113 for information regarding interrupt configuration and handling.

11.3.2 Control and Configuration Registers

Register Name	Register Description	Address
D_SYSTEM_CTRL	System Control	0xE000
D_WATCHDOG_CFG	Watchdog Timeout Configuration	0xE001

11.3.2.1 D_DSS_CTRL Settings

Bit Field	Field Name	Field Description
0	SYSTEM_CTRL_WATCHDOG_REFRESH	Acknowledge the watchdog timer.

Field Name	Value Symbol	Value Description	Hex Value
SYSTEM_CTRL_WATCHDOG_REFRESH	SYSTEM_WATCHDOG_REFRESH	Reset the watchdog timer	0x1

11.3.2.2 D_WATCHDOG_CFG Settings

Bit Field	Field Name	Field Description
3:0	WATCHDOG_CFG_TIMEOUT	Configure the delay before a watchdog timeout (all defines are relative to a SLOWCLK of 1.28MHz)

Field Name	Value Symbol	Value Description	Hex Value
WATCHDOG_CFG_TIMEOUT	WATCHDOG_TIMEOUT_1M6	Timeout after 1.6 ms	0x0
	WATCHDOG_TIMEOUT_3M2	Timeout after 3.2 ms	0x1
	WATCHDOG_TIMEOUT_6M4	Timeout after 6.4 ms	0x2
	WATCHDOG_TIMEOUT_12M8	Timeout after 12.8 ms	0x3
	WATCHDOG_TIMEOUT_25M6	Timeout after 25.6 ms	0x4
	WATCHDOG_TIMEOUT_51M2	Timeout after 51.2 ms	0x5
	WATCHDOG_TIMEOUT_102M4	Timeout after 102.4 ms	0x6
	WATCHDOG_TIMEOUT_204M8	Timeout after 204.8 ms	0x7
	WATCHDOG_TIMEOUT_409M6	Timeout after 409.6 ms	0x8
	WATCHDOG_TIMEOUT_819M2	Timeout after 819.2 ms	0x9
	WATCHDOG_TIMEOUT_1638M4	Timeout after 1638.4 ms	0xA
	WATCHDOG_TIMEOUT_3276M8	Timeout after 3276.8 ms	0xB*

11.4 CYCLIC REDUNDANCY CODE (CRC) GENERATOR

The BelaSigna 300 system uses a standard cyclic redundancy code (CRC) algorithm to ensure data integrity for the file system and all debug port communications. CRC has been selected to ensure data integrity as it will catch most data errors and in particular is guaranteed to catch all single-bit errors, two-bit errors, burst errors and any error containing an odd number of bits. As calculating a CRC is a common but moderately expensive software calculation, a hardware CRC generator is included as a peripheral accessible by the CFX DSP.

The selected implementation of the CRC algorithm used for BelaSigna 300 is the standard CRC-CCITT configuration. The parameters associated with this algorithm implementation are available in Table 25.

TABLE 25: CRC-CCITT ALGORITHM PARAMETERS

CRC Parameter	Parameter Value
Order	16
Polynomial	$x^{16} + x^{12} + x^5 + 1$
Polynomial (hex)	0x1021
Initial Value (hex)	0xFFFF

TABLE 25: CRC-CCITT ALGORITHM PARAMETERS

CRC Parameter	Parameter Value
Final XOR Value (hex)	0x0000
Notes	No data manipulation is required (i.e. no data byte reversal, reversal of the final result or other finalization)

To use the CRC generator:

1. Write an initial value of 0xFFFF to the D_CRC_VALUE register.
2. Write to the CRC input registers any data that needs to be included in the final CRC read from the generator.
3. At any time you can read the D_CRC_VALUE register to obtain the CRC for the data that has been added using the CRC input since the last time the register was initialized.

11.4.1 Control and Configuration Registers

Register Name	Register Description	Address
D_CRC_VALUE	The current value of the calculated CRC (based on data input)	0xE022
D_CRC_ADD_1	CRC input register for adding 1 bit of data to the CRC calculation	0xE023
D_CRC_ADD_8	CRC input register for adding 8 bits of data to the CRC calculation	0xE024
D_CRC_ADD_16	CRC input register for adding 16 bits of data to the CRC calculation	0xE025
D_CRC_ADD_24	CRC input register for adding 24 bits of data to the CRC calculation	0xE026
D_CRC_ADD_32	CRC input register for adding 32 bits of data to the CRC calculation	0xE027

11.5 ACCESS MODE SECURITY BLOCKS

The BelaSigna 300 device includes a few components that can be used to secure intellectual property stored on the device and attached EEPROM. To activate these protective measures, enable access mode protection on the debug port as described in Section 12.1.1.1, “Debug Port Configuration” on page 134. A description of the restrictions placed on debug port commands when restricted access is enabled can be found in the debug port reference materials provided by the *CFX Communication Protocols Manual*. A list of protected memory resources is provided in Table 26.

TABLE 26: PROTECTED MEMORY SECTIONS

Memory Space	Start Address	End Address	Protected Length (in Words)	Data Cleared Indicator Value
Program Memory	0x1000	0x3FFF	0x2FFF	0x00000000
Microcode Memory	0x8000	0x87FF	0x07FF	0x00000000
X Memory	0x20	0x179B	0x177C	0x000000
Y Memory	0x20	0x07FF	0x07DF	0x000000

Sensitive blocks of program code or data to be loaded to P memory can be stored as encrypted data in non-volatile memory to protect it from direct attacks. The BelaSigna 300 device can decrypt this data using non-externally accessible keys at startup, but to obtain access to potentially sensitive parts of the system, either the decoded application must unlock the device or you must use a "Jump ROM" command sequence, as described in the *CFX Communication Protocols Manual*, to unlock the device. In the case that the system is using the "Jump ROM" command sequence to unlock the device, the system is rendered incapable of further deciphering any information that is stored in protected areas of memory, and all data currently stored in protected areas of memory is cleared.

CAUTION: Although the debug port can be used to protect data, none of the other interfaces explicitly have any access restrictions placed on them. To guarantee IP protection, ensure that the application included on the device does not do something to make the system insecure. Examples of this are providing support for unlocking the device through another interface or providing access to memory through another interface—both of which could allow a user to read back the contents of protected memory areas.

This page left blank for printing purposes.

12 External Interfaces

12.1 Inter-IC Communications (I^2C) Interfaces

The BelaSigna 300 system contains two I^2C interfaces that share the same I^2C bus within the device. Both interfaces implement a subset of the Philips I^2C protocol as described in the *Philips I^2C Bus Specification* document.

The I^2C interface uses the rising edges of a serial clock signal (SCL) to clock in data from a serial data signal (SDA) in order to communicate between devices. The I^2C interface is designed to handle bus traffic operating at up to 400kHz; however for communications to proceed on the I^2C interface, the following relation relating the system clock (which is used internally to clock the I^2C interface) to SCL must hold:

$$F_{SYSCLK} \geq 3.125 \times F_{SCL}$$

The example timing diagram shown in Figure 20 provides a user with some information about the important elements in an I^2C transaction which are described in further detail in the previously mentioned bus specification. These elements are:

Start Condition

The SDA transitions from the idle high state to the low state while the SCL remains high. This can also happen during a transmission as a repeated start condition, which indicates that the transaction will start again without an intermediate stop condition.

Address Bits

During the first byte transmitted, the first 7 bits on the SDA (clocked in on a rising edge of SCL) provide the desired address of the device with which the master device wishes to communicate. A device that has been properly addressed should acknowledge this transaction.

Read/Write Direction Bit

During the first byte transmitted, the last bit on the SDA (clocked in on a rising edge of SCL) indicates the direction of the transaction. A zero for this bit indicates a write by the master, a one indicates a read by the master. If the master device has requested a read, the slave device will control the SDA line on subsequent bytes to transmit data.

Data Byte

All other bytes, excluding the addressing and read/write byte, that are transmitted on the SDA are considered as data bytes for the transaction.

Acknowledge or Not Acknowledge Bit

The acknowledge bit is used to tell the sender that the byte has been received. The device receiving data should acknowledge each data byte, including the address byte. During this time, the bus device that is sending

data on the data bus stops driving the SDA and allows the line to be pulled high. To not acknowledge a byte, the receiving device does not need to do anything. To acknowledge a byte, the receiving device needs to pull the SDA low.

A receiving slave device should not acknowledge a byte if the slave device is not the device that was addressed, or if the device cannot handle the byte received. A master device should not acknowledge a byte if the master is receiving and wishes to end the transaction. If a not acknowledge is encountered, the device transmitting data should generate a stop condition.

Stop Condition

The SDA transitions from the low state to high state while the SCL remains high. This ends an I²C transaction.

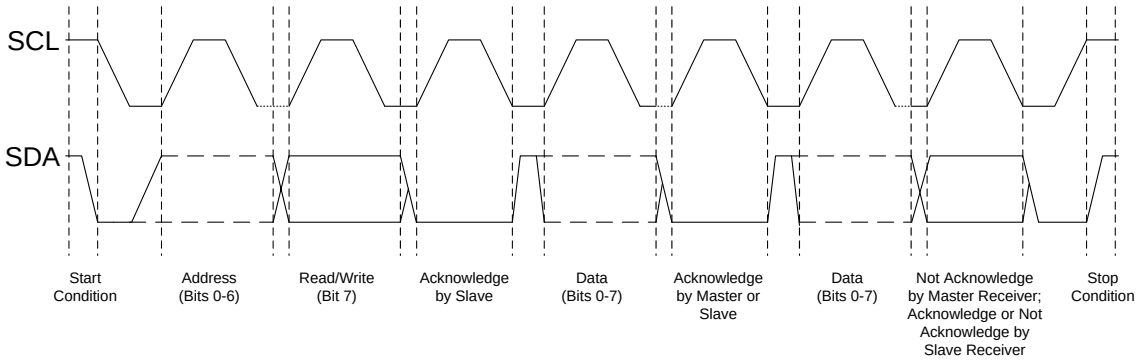


Figure 20: I²C Signal Timing Diagram

Since two interfaces share the same bus, addressing these interfaces becomes an important concern. Be careful not to address both the debug port I²C interface and general purpose I²C interface at the same address because the debug port I²C interface will take precedence for all communications with the shared address. When enabled, the debug interface will respond at both its internally-programmable and externally-programmable addresses (both of which default to 0x60). The general-purpose I²C interface, when enabled, responds at the I²C general call address (0x00) and can be configured to respond at an additional user-specified address.

CAUTION: If you are using the ON Semiconductor Communications Accelerator Adaptor (CAA) to communicate with the BelaSigna 300 device over either I²C interface, do not use the address 0x51 because it conflicts with the non-volatile memory storage of the CAA device.

For further information on addressing the I²C interfaces, see their respective subsections below.

The BelaSigna 300 device includes three configuration parameters for the I²C bus itself:

- The `I2C_CFG_PULL_UP` bit in the `D_I2C_PIN_CFG` register configures the value of the pull-up registers provided on the I²C bus.

- The I2C_CFG_FILTER_EN bit in the D_I2C_PIN_CFG register enables a pair of low-pass filters for the bus lines that can be used to clean up the I²C bus signals.
- The I2C_PIN_CFG_PULLUP_DIS bit in the D_I2C_PIN_CFG register allows you to disconnect the pull-ups on the I²C interface lines.

CAUTION: Using the I2C_PIN_CFG_PULLUP_DIS bit to disable the bus pull-up resistors is not recommended under most conditions. We recommend that you consult ON Semiconductor before using this configuration because attempting to use this interface without the pull-up resistors might result in unintended behaviors.

As multiple interfaces can communicate on this bus with one or more external devices, the I2C_CTRL_LINE_FREE bit in the D_I2C_CTRL register provides information on current bus traffic. This bit is only set when the internal I²C bus state machine is currently idle, and both bus lines are high.

For this same reason a watchdog timer is provided for the I²C bus lines. This timer, which has a timeout period configurable by setting the I2C_WATCHDOG_CFG_TIMEOUT bit field in the D_I2C_WATCHDOG_CFG register, watches the bus lines to ensure that the BelaSigna 300 device does not hold them low for an extended period of time; this might otherwise occur and lock out the device's debug port if noise or another I²C driver causes an error on the bus. The timer is cleared, preventing a timeout, when the interface is not holding the bus lines and there is no start condition pending. If a timeout does occur, the I²C interface is reset, the I2C_CTRL_ERROR bit is set, and the count in the D_I2C_ERROR_CNT register is incremented.

A bus error can be detected by the interface to allow for the use of multi-master bus configurations and to prevent issues arising when reading from multiple slaves at the same address. A bus error will be detected whenever the value written to the I²C data line (SDA) does not match the physical value of this line. When a bus error is detected, the I2C_CTRL_ERRORand I2C_CTRL_BUS_ERROR bits are set.

Reading the D_I2C_ERROR_CNT register resets the error count register to 0, and clears the I2C_CTRL_ERROR error status bit if the I2C_CTRL_BUS_ERROR status bit is not set to 1.

12.1.1 Debug Port I²C Interface

The debug port I²C interface is a hardware debugger for the BelaSigna 300 system that is always enabled regardless of the configuration of the general-purpose I²C interface.

The debug port I²C interface always responds to communications at two configurable addresses that default to an address of 0x60. These addresses are:

- The internally-programmable address that can be configured by writing the desired 7-bit address into the D_DEBUG_ADDRESS register.
- The externally-programmable address that can be configured using the **Write Address** and **Acquire Address** debug port commands (for more information, see the *CFX Communication Protocols Manual*).

The debug port implements the debug port protocol command set and is tightly coupled with the CFX DSP and the memory components attached to the CFX. As a result, the debug port only provides you with a peripheral view (through the contents of shared and microcode memory) of the HEAR processing of microcode configurations.

12.1.1.1 Debug Port Configuration

The debug port interface can be used with the PCM interface to provide digital audio streaming functionality. When operating in this mode, the I^2C interface is connected to the PCM interface and the debug port is flipped into audio streaming mode. The debug port indicates this mode to the system by setting the DEBUG_CFG_AUDIO_MODE in the D_DEBUG_CFG register. More information about digital audio streaming that is not supplied in the *CFX Communication Protocols Manual* is in Section 10.6, "NOAHlink Digital Audio Streaming Configuration and Usage" on page 111.

Note: Support for configuring the debug port for use with audio streaming can be obtained by contacting ON Semiconductor Customer Support.

While the DEBUG_CFG_ACCESS_MODE bit in the D_DEBUG_CFG register is set, the functionality of the debug port is limited. This is designed as part of the system security mechanism and a user application can bypass this security by clearing this bit. For more information about security in the BelaSigna 300 system, see Section 11.5, "Access Mode Security Blocks" on page 128, and security related information on the BelaSigna 300 file system in the *Firmware Reference Manual*.

The debug port uses the D_DEBUG_DATA register as a memory-mapped register for direct communication with the CFX DSP over the debug port while the system is running. When this register has been written by the debug port, the DEBUG_CFG_REG_WRITTEN bit in the D_DEBUG_CFG register remains set until the register is read. Similarly, when this register has been read by the debug port, the DEBUG_CFG_REG_READ bit in the D_DEBUG_CFG register remains set until the register is written.

12.1.1.2 Breakpoints

The primary troubleshooting components of the debug port are the different types of breakpoints that can be configured through the debug port protocol. These breakpoints include the following:

Software breakpoints

Included in code for the CFX processor as `breakpoint` instructions. See the *CFX DSP Architecture Manual* for details on the `breakpoint` instruction and the *CFX Communication Protocols Manual* for information on configuring the debug port to stop at software breakpoints.

Hardware breakpoints

Configured as pointers to P Memory that are triggered when CFX DSP issues the instruction pointed to by the hardware breakpoint for execution. See the *CFX Communication Protocols Manual* for details on configuring hardware breakpoints and configuring the debug port to stop at hardware breakpoints.

External breakpoints

Configured to trigger a breakpoint event based on accessed data at addresses corresponding to specified conditions and/or the value of the written data. See Section 12.1.1.3, "External Breakpoint Configuration" in this manual for details on configuring external breakpoints.

12.1.1.3 External Breakpoint Configuration

External breakpoints can be used to detect a variety of different memory access conditions.

IMPORTANT: When an external breakpoint condition is observed, the instructions currently loaded to the CFX DSP instruction pipeline will complete before the core is stopped. This typically results in a perceived delay of 3 cycles between an external breakpoint event and the CFX DSP stopping.

Different types of memory access conditions that can be tested for are grouped into comparison modes that are selectable using the EXT_BREAK_CFG_COMP_MODE bit field from the D_EXT_BREAK_CFG register. Each of these modes interprets the other external breakpoint configuration registers differently. External breakpoints can be configured into:

- One of six data modes, which are described in Section 12.1.1.3.1, “Data Modes”
- Double-address mode, which is described in Section 12.1.1.3.2, “Double-Address Mode”
- HEAR memory conflict mode, which is described in Section 12.1.1.3.3, “HEAR Memory Conflict Mode” on page 137

12.1.1.3.1 Data Modes

In these modes the 32-bit D_EXT_BREAK_DATA register is divided into address and data elements of various lengths. Interpretation details for this register for each of the data modes can be found in Table 27. As well, the EXT_BREAK_CFG_BUS_SELECT bit field of the D_EXT_BREAK_CFG register can be used in these modes to specify which memory bus is watched for the breakpoint condition.

TABLE 27: D_EXT_BREAK_DATA USAGE FOR EXTERNAL BREAKPOINT DATA MODES

Mode	Address Component	Data Component	Notes
32-bit data	-	31:0	Address condition is ignored. Data is not modified.
High 24-bit data	31:24	23:0	Address is padded to 16 bits with 8 zeros to the right. Data padded to 32 bits with 8 zeros to the right.
Low 24-bit data	31:24	23:0	Address is padded to 16 bits with 8 zeros to the right. Data sign-extended to 32 bits.
High 16-bit data	31:16	15:0	Address is not modified. Data is padded to 32 bits with 16 zeros to the right.
Mid 16-bit data	31:16	15:0	Address is not modified. Data is padded to 24 bits with 8 zeros to the right and then sign-extended to 32 bits.
Low 16-bit data	31:16	15:0	Address is not modified. Data is sign-extended to 32 bits.

IMPORTANT: Data and address conditions that use zero-padded data and addresses generally use “nominal equality”. A value is considered to be nominally equal to a condition if the value is equal to all bits in the condition that are not affected by the zero padding.

The address conditions, which are listed in Table 28, are used to restrict external breakpoint triggers based on the memory bus address where the access occurred. In 32-bit data mode no address component is specified, and this condition is ignored.

TABLE 28: ADDRESS CONDITIONS FOR DATA MODES

Condition	Description
Greater Than or Equal	Address is greater than or equal to the address component
Less Than	Address is less than the address component
Nominally Equal	Address is equal to the specified bits of the address component; bits of the address component that have been padded with zeros are ignored
Strictly Equal	Address is equal to the address component including padded zeros

For all data modes, the data conditions listed in Table 29 are used to further restrict external breakpoint events based on the value of data being written.

TABLE 29: DATA CONDITIONS FOR DATA MODES

Condition	Description
Equal	Data is nominally equal to the data component
Not Equal	Data is nominally not equal to the data component
Greater Than or Equal	Data is greater than or equal to the data component
Less Than	Data is nominally less than the data component

In this mode, an external breakpoint is triggered when a data access on the specified memory bus occurs that fits the access condition at an address that meets the address condition.

12.1.1.3.2 Double-Address Mode

The `D_EXT_BREAK_DATA` register is interpreted as two 16-bit addresses, with the high 16 bits being used as the first address and the low 16 bits used as the second address. As well, the `EXT_BREAK_CFG_BUS_SELECT` bit field of the `D_EXT_BREAK_CFG` register can be used in this mode to specify which memory bus's address lines are watched for the breakpoint condition.

The address conditions listed in Table 30 are used to restrict the addresses on which a data access will trigger a breakpoint. The access conditions listed in Table 31 are used to further restrict what types of data access will trigger a breakpoint.

TABLE 30: ADDRESS CONDITIONS FOR DOUBLE ADDRESS MODE

Condition	Description
Equal	Address is equal to address 1 or address 2
Bounded Range	Address is between address 1 and address 2
Greater Than or Equal	Address is greater than or equal to address 1, or is equal to address 2
Less Than or Equal	Address is equal to address 1, or is less than or equal to address 2

TABLE 31: ACCESS CONDITIONS FOR DOUBLE-ADDRESS MODE

Condition	Description
Disabled	External breakpoints are not generated
Read or Write Access	Address is considered when data is either read or written
Read Access Only	Address is only considered when data is read
Write Access Only	Address is only considered when data is written

In this mode, an external breakpoint is triggered when a data access on the specified memory bus occurs that fits the access condition at an address that meets the address condition.

12.1.1.3.3 HEAR Memory Conflict Mode

The D_EXT_BREAK_DATA register is not used in this mode. Further, in this mode the address condition, access condition and bus selection are all ignored.

In this mode, an external breakpoint is triggered when a memory conflict between the CFX DSP core and HEAR Accelerator core occurs.

12.1.1.4 Debug Port Specific Control and Configuration Registers

Register Name	Register Description	Address
D_DEBUG_CFG	Debug port status and security configuration	0xE034
D_DEBUG_DATA	Data register used by debug port for external communication	0xE035
D_DEBUG_ADDRESS	Selection of the debug port internally-programmable address	0xE204
D_EXT_BREAK_CFG	External breakpoint configuration (configuration component)	0xE020
D_EXT_BREAK_DATA	External breakpoint configuration (data component)	0xE021

For control and configuration registers that apply to both the debug port I^2C interface and the general purpose I^2C interface, see Section 12.1.3, “Control and Configuration Registers” on page 147.

12.1.1.4.1 D_DEBUG_CFG Settings

Bit Field	Field Name	Field Description
3	DEBUG_CFG_DBG_REG_WRITTEN	Indicator that the debug port data register has been written
2	DEBUG_CFG_DBG_REG_READ	Indicator that the debug port data register has been read
1	DEBUG_CFG_DBG_AUDIO_MODE	Indicator that the debug port is operating in digital audio streaming mode
0	DEBUG_CFG_DBG_ACCESS_MODE	Selection/indicator of the debug port access settings

Field Name	Value Symbol	Value Description	Hex Value
DEBUG_CFG_DBG_ACCESS_MODE	ACCESS_UNRESTRICTED	Access to the CFX and memory through the debug port is not restricted	0x0
	ACCESS_RESTRICTED	The debug port command set is restricted to a limited subset	0x1*

12.1.1.4.2 D_EXT_BREAK_CFG Settings

Bit Field	Field Name	Field Description
8:6	EXT_BREAK_CFG_COMP_MODE	Select the comparison mode
5:4	EXT_BREAK_CFG_ADDR_COND	Select the address condition
3:2	EXT_BREAK_CFG_ACCESS_COND	Select the access or data condition
1:0	EXT_BREAK_CFG_BUS_SELECT	Select the data bus to monitor for external breakpoints

Field Name	Value Symbol	Value Description	Hex Value
EXT_BREAK_CFG_COMP_MODE	EXTBREAK_DOUBLE_ADDR	Select double-address mode	0x0*
	EXTBREAK_32BIT_DATA	Select 32-bit data mode	0x1
	EXTBREAK_24BIT_DATA_HIGH	Select 8-bit address, 24-bit data mode (31:8)	0x2
	EXTBREAK_24BIT_DATA_LOW	Select 8-bit address, 24-bit data mode (23:0)	0x3
	EXTBREAK_16BIT_DATA_HIGH	Select 16-bit address, 16-bit data mode (31:16)	0x4
	EXTBREAK_16BIT_DATA_MID	Select 16-bit address, 16-bit data mode (23:8)	0x5
	EXTBREAK_16BIT_DATA_LOW	Select 16-bit address, 16-bit data mode (15:0)	0x6
	EXTBREAK_HEAR_CONFLICT	Select shared-memory conflict detection mode	0x7

Field Name	Value Symbol	Value Description	Hex Value
EXT_BREAK_CFG_ADDR_COND	EXTBREAK_ADDR_EQUAL	Double-address mode: Equal to address 1 or 2	0x0*
	EXTBREAK_ADDRDATA_GTE	Data mode: Greater than or equal to address	0x0*
	EXTBREAK_ADDR_RANGE	Double-address mode: Between address 1 and 2	0x1
	EXTBREAK_ADDRDATA_LT	Data mode: Less than address	0x1
	EXTBREAK_ADDR_GT	Double-address mode: Greater than address 1 or equal to address 1 or 2	0x2
	EXTBREAK_ADDRDATA_EQU	Data mode: Equal to address	0x2
	EXTBREAK_ADDR_LT	Double-address mode: Less than address 2 or equal to address 1 or 2	0x3
	EXTBREAK_ADDRDATA_SEQU	Data mode: Equal to address (including padding)	0x3
EXT_BREAK_CFG_ACCESS_COND	EXTBREAK_ACCESS_FALSE	Double-address mode: Disabled	0x0*
	EXTBREAK_ACCESSDATA_EQU	Data mode: Equal to data	0x0*
	EXTBREAK_ACCESS_RW	Double-address mode: Read or write access at address	0x1
	EXTBREAK_ACCESSDATA_NOTEQU	Data mode: Not equal to data	0x1
	EXTBREAK_ACCESS_READ	Double-address mode: Read access at address	0x2
	EXTBREAK_ACCESSDATA_GTE	Data mode: Greater than or equal to data	0x2
	EXTBREAK_ACCESS_WRITE	Double-address mode: Write access at address	0x3
	EXTBREAK_ACCESSDATA_LT	Data mode: Less than data	0x3
EXT_BREAK_CFG_BUS_SELECT	EXTBREAK_BUS_DISABLED	External breakpoints disabled	0x0*
	EXTBREAK_BUS_XMEM	Use the X memory bus	0x1
	EXTBREAK_BUS_YMEM	Use the Y memory bus	0x2
	EXTBREAK_BUS_PMEM	Use the P memory bus	0x3

12.1.2 General Purpose I²C Interface

The general purpose I²C interface provides a half-duplex communications channel that operates at speeds up to 400kHz using the I²C bus it shares with the debug port. During any transaction the I2C_CTRL_RW bit from the D_I2C_CTRL register indicates whether the current I²C transaction is a read or write transaction.

The D_I2C_CTRL register is the interface control register. Within D_I2C_CTRL, the bit fields are defined as either event bits, state bits, or write-only bits. The event bits are used to indicate that

an event has occurred, the state bits are used to indicate an interface operating state, and the write-only bits are used to trigger an event. All of the bit fields and their corresponding types are summarized in Section 12.1.2.4.2, “D_I2C_CTRL Settings” on page 145.

Note: The interface supports both master operation and slave operation. A user application can use the interface in both modes provided the application ensures that the I²C bus is not currently in use. Check the I2C_CTRL_LINE_FREE bit in the D_I2C_CTRL register to verify that the I²C bus is not currently in use when attempting to initialize a transfer as a master device.

The general purpose I²C interface uses the D_I2C_DATA register to transmit and receive data through a 1-byte internal buffer. As use of the D_I2C_DATA register has side effects that can advance the I²C interface state machine, this register is mirrored without side effects in the D_I2C_DATA_M register. If the internal buffer is full, the I2C_CTRL_BUFFER_FULL bit from the D_I2C_CTRL register is set.

Generally each byte of an I²C transaction must be acknowledged or not-acknowledged by a user application. Use the I2C_CFG_AUTO_ACK bit from the I2C_CFG register to select whether the application itself handles this acknowledgement manually or the interface handles it automatically. In either case, if the interface does not have needed data or a place to put data the transaction is paused by stretching the clock signal until the interface can continue. If the clock is being stretched, the I2C_CTRL_CLK_STRETCH bit from the D_I2C_CTRL register is set as an indicator.

Use of the above mentioned data registers and the general behavior of the I²C state machine is described in Section 12.1.2.3.1, “Operation Using Manual Acknowledgement” on page 142 for manual acknowledgement mode, and in Section 12.1.2.3.2, “Operation Using Auto Acknowledgement” on page 143 for automatic acknowledgement mode.

Note: If at any point when operating in either master or slave mode a user application needs to reset the I²C bus lines (for example, after encountering a bus error condition), the application should write the I2C_CTRL_RESET bit from the D_I2C_CTRL register. This resets both the I²C bus and the internal state machines of the I2C interface to allow the system to cleanly start again from a known state.

12.1.2.1 General Purpose I²C Interface - Slave Mode Specific Configuration

In slave mode the device is receiving the serial clock signal from an external master device, which also controls addressing, direction and the start/stop conditions for each I²C transfer.

To enable a device in slave mode, set the I2C_SLAVE_EN bit from the D_I2C_CFG register. This allows the device to respond to communications on both the I²C general call address (0x00) and at a programmable slave address that can be selected using the I2C_CFG_SLAVE_ID bit field in the D_I2C_CFG register. During a transaction, the I2C_CTRL_GEN_CALL bit in the D_I2C_CTRL register can be queried to identify which of the addresses was used to address the device.

IMPORTANT: If at any point the BelaSigna 300 device is addressed using the I²C bus at an address configured for use by both the debug port interface and the general purpose interface, the debug port interface takes precedence.

A device receiving data in a slave mode configuration can use the I2C_CTRL_LAST_DATA bit from the D_I2C_CTRL register to automatically NACK the last data byte as an indication from the slave device that the master device should send a stop condition.

Note: The I2C_CTRL_LAST_DATA bit is cleared when a new transfer is initiated on the I²C bus.

12.1.2.2 General Purpose I²C Interface - Master Mode Specific Configuration

In master mode, the device provides the serial clock signal and controls all transfer information. To configure the clock signal, select a prescaling division by 3 using the I2C_CFG_MASTER_SPEED bit field in the D_I2C_CFG register. The interface clock frequency for a given configuration can be calculated using:

$$F_{I2C\ MASTER} = \frac{SYSCLK}{3 \cdot (I2C_CFG_MASTER_SPEED + 1)}$$

As noted in Section 12.1, “Inter-IC Communications (I2C) Interfaces” on page 131, the I²C interface is designed to operate at a maximum speed of 400kHz.

A user application must manually control all components of a master mode transaction. To transmit or receive using master mode, a device must:

1. Check that the I²C bus lines are not currently in use by reading the I2C_CTRL_LINE_FREE bit in the D_I2C_CTRL register. It can only start a transaction if the lines are free.
2. Start a transaction, sending the start condition, address and direction, by loading the appropriate address and direction to the D_I2C_ADDR_START register.
   Note: Using a slave address that matches one of the debug port addresses is technically allowed for a master device; however, the local debug port I²C interface will not respond to communications originating from the general purpose I²C interface. Due to possible confusion related to this unusual configuration (since, for example, another master producing the same I²C bus traffic will observe different behavior), we recommend that users avoid using debug port assigned I²C addresses for I²C master mode transactions.
3. Handle data and interrupts as appropriate for the transaction (see Section 12.1.2.3, “General Purpose I2C Interrupts”).
4. Complete the transaction by sending a stop condition.

   A stop condition can be generated using either the I2C_CTRL_STOP_CMD bit or the I2C_CTRL_LAST_DATA bit in the D_I2C_CTRL register.

   When using the I2C_CTRL_LAST_DATA bit, a stop condition is automatically generated after the current data byte transfer is completed. If the interface is receiving data, this bit will also automatically generate the required NACK of the last data byte that is received.

   Note: If the I²C interface is used for transmitting, the last byte of data to be transmitted must be written to the D_I2C_DATA register before setting the I2C_CTRL_LAST_DATA bit, as this bit triggers a stop condition at the end of the current transfer.

Note: As previously noted, the I2C_CTRL_LAST_DATA bit is cleared when a new transfer is initiated on the I²C bus.

When using the I2C_CTRL_STOP_CMD bit, a stop condition is issued immediately. This bit will normally be set when the I2C_CTRL_ACK or I2C_CTRL_NACK bits in the D_I2C_CTRL register are set.

CAUTION: Using the stop event (STOP) immediately transmits a stop condition as soon as possible. This can result in a terminated transfer, and remote devices on the bus might detect I²C errors.

12.1.2.3 General Purpose I²C Interrupts

The general purpose I²C interface uses an associated interrupt which, when enabled, signals the receipt of a correct address byte and the completion of each data byte in the transaction.

When enabled, the I²C interrupt signals the stop condition following a transaction for a master transfer. The system can also be configured to receive this interrupt when operating in slave mode by setting the I2C_CFG_STOP_INT bit from the D_I2C_CFG register.

See Section 11.1, “Interrupt Controller” on page 113 for information regarding interrupt configuration and handling.

Several indicators can be used with the interrupt to identify the state of the general purpose I²C interface:

- If the I2C_CTRL_ADDR_DATA bit from the D_I2C_CTRL register is set, the interrupt was generated in response to a recognized sequence including a start condition and address on the general purpose I²C interface.
- If the I2C_CTRL_STOP_DETECTED bit from the D_I2C_CTRL register is set, the interrupt was generated in response to a stop interrupt being detected on the general purpose I²C interface. This bit is cleared immediately when a new transaction starts. No interrupt is generated for I²C transactions that use an address that was not recognized by the general purpose I²C interface.
- If the I2C_CTRL_DATA_EVENT bit from the D_I2C_CTRL register is set, the interrupt indicates that data has been received and can be read, or that data is needed to continue with the transmission of data.
- If the I2C_CTRL_BUS_ERROR bit from the D_I2C_CTRL register is set, then a bus error has occurred.
- If the I2C_CTRL_ERROR bit from the D_I2C_CTRL register is set, then either a bus error or an I²C watchdog timeout has occurred.

12.1.2.3.1 Operation Using Manual Acknowledgement

If a user application is using manual acknowledgement (i.e. the I2C_CFG_AUTO_ACK bit from the D_I2C_CFG register is disabled), it must use the I2C_CTRL_NACK and I2C_CTRL_ACK bits from the D_I2C_CTRL register to acknowledge (ACK) or not-acknowledge (NACK) transaction bytes.

When handling a transaction, an interrupt is issued whenever data or an acknowledgement is required (in addition to interrupts triggered by stop conditions).

Note: An interrupt is not generated following a NACK for any data condition. However, an interrupt is generated (if operating in master mode or stop interrupts are enabled) on the stop condition that is expected to follow the NACK.

An interrupt is also generated if the interface cannot send data that has been provided because the interface is idle or has already received a NACK from the slave device during the current transaction. As noted above, when the interface is waiting for data, the clock line is automatically stretched until the required data is provided.

A special interrupt is also generated for a master read once the acknowledge bit from the address byte has been received. This scenario is a special case because it does not require data nor acknowledgement to trigger an interrupt, but instead is designed to allow the interface to check whether or not a slave device has acknowledged the transfer at that point. Provided the slave device responded, the interface can issue either an ACK or a NACK command to continue the transfer.

Note: Use of the `I2C_CTRL_NACK`, `I2C_CTRL_ACK` and `I2C_CTRL_STOP_CMD` bits from the `D_I2C_CTRL` register is only defined when an acknowledgement is needed during a data transfer. Interface behavior in response to these control bits being set at other times is undefined.

12.1.2.3.2 Operation Using Auto Acknowledgement

When operating using auto acknowledgement (i.e. the `I2C_CFG_AUTO_ACK` bit from the `D_I2C_CFG` register is enabled), transfers are controlled by the availability of bytes for transmission, or a place to put a byte when receiving data. This allows transfers to occur without clock stretching, provided the firmware is able to handle each data byte fast enough.

When the interface handles a transaction, interrupts are issued whenever new data can be written or new data has been received. This allows a user application to maintain a transaction without introducing delay.

Note: An interrupt is not generated following a NACK for any data condition. However, an interrupt is generated (if operating in master mode or stop interrupts are enabled) on the stop condition that is expected to follow the NACK.

Additionally, interrupts are generated if data has been supplied but cannot be sent because the interface has received a NACK from the slave device or is currently idle (as the interface would be following a stop condition). If data is required but is not currently available because the buffer is empty, the clock is stretched and an additional interrupt is generated. If new data was received, but the previous data has not been read (resulting in the buffer not being available), the clock is stretched until the buffer becomes free.

IMPORTANT: When receiving data in master mode, the timing of interrupts for auto acknowledgement of data prevents a user application from cleanly terminating a transfer using the LAST_DATA event. As a result, if the interface is running in this mode, a dummy byte must be transferred to cleanly terminate the current transfer with a stop condition following this byte.

12.1.2.4 General Purpose I2C Interface Specific Control and Configuration Registers

Register Name	Register Description	Address
D_I2C_CFG	Configure the general-purpose I2C interface	0xE200
D_I2C_CTRL	Control signals on the general-purpose I2C interface	0xE201
D_I2C_DATA	Register used for storing received data and data to transmit	0xE202
D_I2C_DATA_M	Mirror of the data register that has no effect on the interface state	0xE203
D_I2C_ADDR_START	Initiate a transaction sending a start condition and address/direction byte over the I2C bus	0xE206

For control and configuration registers that apply to both the debug port I2C interface and the general purpose I2C interface, see Section 12.1.3, “Control and Configuration Registers” on page 147.

12.1.2.4.1 D_I2C_CFG Settings

Bit Field	Field Name	Field Description
19:12	I2C_CFG_MASTER_SPEED	Configure the speed of the SCL signal
10:4	I2C_CFG_SLAVE_ID	Selection of the slave variable address
3	I2C_CFG_STOP_INT	Enable/disable interrupts on stop conditions
2	I2C_CFG_AUTO_ACK	Enable/disable automatic acknowledge mode
0	I2C_CFG_SLAVE_EN	Enable/disable the I2C slave interface

Field Name	Value Symbol	Value Description	Hex Value
I2C_CFG_MASTER_SPEED[1]	I2C_MASTER_PRESCALE_3	I²C master speed is SYSCLK/3	0x0*
	I2C_MASTER_PRESCALE_6	I²C master speed is SYSCLK/6	0x1
	I2C_MASTER_PRESCALE_9	I²C master speed is SYSCLK/9	0x2
	I2C_MASTER_PRESCALE_12	I²C master speed is SYSCLK/12	0x3
	I2C_MASTER_PRESCALE_15	I²C master speed is SYSCLK/15	0x4
	I2C_MASTER_PRESCALE_18	I²C master speed is SYSCLK/18	0x5
	I2C_MASTER_PRESCALE_21	I²C master speed is SYSCLK/21	0x6
	I2C_MASTER_PRESCALE_24	I²C master speed is SYSCLK/24	0x7
	I2C_MASTER_PRESCALE_27	I²C master speed is SYSCLK/27	0x8
	I2C_MASTER_PRESCALE_30	I²C master speed is SYSCLK/30	0x9
	I2C_MASTER_PRESCALE_33	I²C master speed is SYSCLK/33	0xA
	I2C_MASTER_PRESCALE_36	I²C master speed is SYSCLK/36	0xB
	I2C_MASTER_PRESCALE_39	I²C master speed is SYSCLK/39	0xC
	I2C_MASTER_PRESCALE_42	I²C master speed is SYSCLK/42	0xD
	I2C_MASTER_PRESCALE_45	I²C master speed is SYSCLK/45	0xE
	I2C_MASTER_PRESCALE_48	I²C master speed is SYSCLK/48	0xF
	I2C_MASTER_PRESCALE_51	I²C master speed is SYSCLK/51	0x10
I2C_CFG_STOP_INT	I2C_CFG_STOP_INT_DISABLE	No interrupt on a stop condition	0x0*
	I2C_CFG_STOP_INT_ENABLE	Interrupt on a stop condition	0x1
I2C_CFG_AUTO_ACK	I2C_CFG_AUTO_ACK_DISABLE	Disable auto acknowledge mode	0x0*
	I2C_CFG_AUTO_ACK_ENABLE	Enable auto acknowledge mode	0x1
I2C_CFG_SLAVE_EN	I2C_CFG_SLAVE_DISABLE	Disable I²C slave mode	0x0*
	I2C_CFG_SLAVE_ENABLE	Enable I²C slave mode	0x1

1. Only a limited number of prescalers have been defined for I²C master speed; however prescalers ranging from 3 to 768 in steps of 3 are possible.

12.1.2.4.2 D_I2C_CTRL Settings

Bit Field	Field Name	Bit Type	Field Description
10	I2C_CTRL_STOP_DETECTED	Event	Indicate if a stop condition has been detected. Cleared after a new transaction starts, or after reading D_I2C_CTRL.
9	I2C_CTRL_DATA_EVENT	Event	Indicate if data has been received or is needed for transmission. Cleared after reading data from or writing data to D_I2C_DATA.

Bit Field	Field Name	Bit Type	Field Description
8	I2C_CTRL_ERROR	Event	Indicate if an I²C bus error has occurred since the last stop condition or if an I²C watchdog error has occurred. Cleared after reading D_I2C_ERROR_CNT. D_I2C_CTRL_ERROR is 0 when D_I2C_ERROR_CNT contains 0 and I2C_CTRL_BUS_ERROR is not 1.
7	I2C_CTRL_BUS_ERROR	Event	Indicate if an I²C bus error has occurred since the last stop condition. Automatically cleared when the bus error disappears, which happens when the I²C interface detects a stop condition.
6	I2C_CTRL_BUFFER_FULL	State	Indicate that the I²C buffer is full. Transition out of this state occurs after reading D_I2C_DATA or transmitting the next byte.
5	I2C_CTRL_CLK_STRETCH	State	Indicate that the SCL line of the I²C bus is being held due to an empty transmit buffer. Transition out of this state occurs after writing D_I2C_DATA.
5	I2C_CTRL_RESET	Write-Only	Abort any current transaction and reset the I²C interface.
4	I2C_CTRL_LINE_FREE	State	Indicate that both I²C bus lines are currently high. Transition out of this state occurs when either SDA or SCL are pulled or driven low.
4	I2C_CTRL_LAST_DATA	Write-Only	Indicate that a byte is the last byte in a transaction, and will automatically terminate an I²C transfer.
3	I2C_CTRL_ADDR_DATA	State	Indicate that an address has been sent or received. Transition out of this state occurs after a data byte has been sent or received.
3	I2C_CTRL_STOP_CMD	Write-Only	Issue a stop condition (master mode only).
2	I2C_CTRL_RW	State	Indicate the state of the Read/Write direction bit last sent or received. Transition out of this state occurs after a new Read/Write direction bit is sent or received.
1	I2C_CTRL_GEN_CALL	State	Indicate that the last address sent or received was the general call address (0x0). Transition out of this state occurs after a non-general call address is sent or received.
1	I2C_CTRL_NACK	Write-Only	Negative acknowledgement of a sent/received byte.
0	I2C_CTRL_ACK	Write-Only	Acknowledgement of a sent/received byte.

Field Name	Value Symbol	Value Description	Hex Value
I2C_CTRL_RESET	I2C_RESET	Reset the I²C interface	0x1
I2C_CTRL_LAST_DATA	I2C_LAST_DATA	Indicate that the current byte is the last byte to be transferred	0x1
I2C_CTRL_STOP_CMD	I2C_STOP_CMD	Manually issue a stop command	0x1

Field Name	Value Symbol	Value Description	Hex Value
I2C_CTRL_NACK	I2C_NACK	NACK sent/received byte	0x1
I2C_CTRL_ACK	I2C_ACK	ACK sent/received byte	0x1

12.1.2.4.3 D_I2C_ADDR_START Settings

Bit Field	Field Name	Field Description
7:1	I2C_START_ADDR	Select an I²C address with which to initiate communications
0	I2C_START_RW	Select the direction for the I²C transaction

Field Name	Value Symbol	Value Description	Hex Value
I2C_START_RW	I2C_START_WRITE	Initiate a write transfer	0x0
	I2C_START_READ	Initiate a read transfer	0x1

12.1.3 Control and Configuration Registers

Register Name	Register Description	Address
D_I2C_ERROR_CNT	I²C bus watchdog timer reset count	0xE205
D_I2C_PIN_CFG	Configure the pads for the I²C interfaces	0xE207
D_I2C_WATCHDOG_CFG	Configure the timeout period of the I²C bus watchdog timer	0xE208

12.1.3.1 D_I2C_PIN_CFG Settings

Bit Field	Field Name	Field Description
2	I2C_PIN_CFG_PULLUP_DIS	Disable/enable the pull-up resistors attached to the I²C interfaces
1	I2C_PIN_CFG_PULL_UP	Select the desired resistance of the pull-ups for the I²C interfaces
0	I2C_PIN_CFG_FILTER_EN	Enable/disable filtering on the I²C pads

Field Name	Value Symbol	Value Description	Hex Value
I2C_PIN_CFG_PULLUP_DIS	I2C_PIN_CFG_PULLUP_ENABLE	Enable the I²C bus pull-ups	0x0*
	I2C_PIN_CFG_PULLUP_DISABLE	Disable the I²C bus pull-ups	0x1
I2C_PIN_CFG_PULL_UP	I2C_PIN_CFG_PULL_UP_250K	Use a 250kΩ pull-up resistance	0x0
	I2C_PIN_CFG_PULL_UP_1K	Use a 1kΩ pull-up resistance	0x1*

Field Name	Value Symbol	Value Description	Hex Value
I2C_PIN_CFG_FILTER_EN	I2C_PIN_CFG_FILTER_DISABLE	Disable filtering at the I²C pads	0x0
	I2C_PIN_CFG_FILTER_ENABLE	Enable filtering at the I²C pads	0x1*

12.1.3.2 D_I2C_WATCHDOG_CFG Settings

Bit Field	Field Name	Field Description
2:0	I2C_WATCHDOG_CFG_TIMEOUT	Configure the timeout period for the I²C bus watchdog timer

Field Name	Value Symbol	Value Description	Hex Value
I2C_WATCHDOG_CFG_TIMEOUT	I2C_WATCHDOG_TIMEOUT_250M	Timeout period is 250ms	0x0
	I2C_WATCHDOG_TIMEOUT_500M	Timeout period is 500ms	0x1
	I2C_WATCHDOG_TIMEOUT_1000M	Timeout period is 1000ms	0x2*
	I2C_WATCHDOG_TIMEOUT_2000M	Timeout period is 2000ms	0x3
	I2C_WATCHDOG_TIMEOUT_4000M	Timeout period is 4000ms	0x4
	I2C_WATCHDOG_TIMEOUT_DISABLED	Watchdog timeout is disabled	0x5

12.2 GENERAL-PURPOSE I/O (GPIO) INTERFACE

BelaSigna 300 provides five software-controlled general-purpose digital I/O (GPIO) pads. The function of these GPIO pads is defined by a user application, which can use them for general input or output.

12.2.1 GPIO Pad Configuration

All of the GPIO pads are multiplexed with LSAD and other functionality to reduce the pin-out for the device. The desired functionality is selected through the GPIO_PIN_CFG_GPIOx bit fields of the D_GPIO_PIN_CFG register. Descriptions of the functionality available at these pins when configured for something other than generic GPIO functionality are in the corresponding sections elsewhere in this document.

Regardless of the selected functionality data, the direction of the GPIO pins can be selected from input or output through the GPIO_CFG_DIR bit field (and the underlying GPIO_CFG_DIR_CFG_GPIOx bits) of the D_GPIO_CFG register. It is important that the direction of signals is selected correctly to match the desired functionality. For example if you are using the UART functionality, set GPIO 0 for use as the UART_TX pin via the D_GPIO_PIN_CFG register and also set this pin as an output signal via the D_GPIO_CFG register.

Each of the GPIO pads has an attached pull-up resistor. These resistors are disabled whenever the GPIO is configured for output. This includes:

- GPIO output mode (pin configuration 0x0, with the GPIO direction configured for output)
- Output modes multiplexed onto the GPIO pad (pin configuration 0x1)

These resistors are also always disabled when using the GPIO pads for LSAD input (pin configuration 0x3). In other input modes, you can configure GPIO 0 to enable or disable the pull-up resistor by setting the associated GPIO_CFG_PULLUP_DIS_GPIO0 bit in the D_GPIO_CFG register. For all other GPIOs operating in input mode, pull-up resistors are always enabled and cannot be disabled.

To enable a digital input on GPIOs 1 to 4, the input must be enabled by loading GPIO_DIGITAL_INPUT_ENABLE to the appropriate GPIO_CFG_INPUT_DIS_GPIOx bit from the D_GPIO_CFG register. If GPIO_DIGITAL_INPUT_DISABLE is loaded to this bit instead, the GPIO input signal for that pin will be disabled.

The current value of GPIO signals can be read or written through the D_GPIO_DATA register. Writing to pins configured as inputs is permitted, but has no effect on the signal. Reading from the GPIO 0 pin always gives the actual value at the pin, regardless of the configuration. Reading other GPIO pins only gives the value of signals configured for input mode.

Note: When GPIO pins are configured as inputs, they have an internal pull-up resistor connected by default. If a GPIO pin is configured as an input with a pull-up and is left floating, a read from the GPIO results in a data value of 1. The data read from a floating input where the pull-up has been disabled is undefined.

12.2.2 GPIO Interrupt

The GPIO interface provides an associated configurable interrupt which, when enabled, signals the occurrence of an event or a condition on a specified GPIO pad. See Section 11.1, “Interrupt Controller” on page 113 for information regarding interrupt configuration and handling.

To select the GPIO pad to use as a trigger for the interrupt, set the GPIO_CFG_INT_SRC bit field in the D_GPIO_CFG register. To select the event to use as a trigger for the interrupt, use the GPIO_CFG_INT_EVENT bit field of the D_GPIO_CFG register. A list of the possible triggering events (including a description of each event) is listed in Table 32.

TABLE 32: GPIO INTERRUPT EVENTS

Event	Event Description
Disabled	An interrupt is never generated.
High level	An interrupt is generated whenever a logical 1 is detected on the GPIO pin. Interrupts continue to be generated while this condition remains true.
Low level	An interrupt is generated whenever a logical 0 is detected on the GPIO pin. Interrupts continue to be generated while this condition remains true.
Rising Edge	An interrupt is generated when a transition from a logical 0 to a logical 1 is detected.
Falling Edge	An interrupt is generated when a transition from a logical 1 to a logical 0 is detected.
Transition	An interrupt is generated when any transition between a logical 0 and a logical 1 is detected.

12.2.3 Control and Configuration Registers

Register Name	Register Description	Address
D_GPIO_CFG	Control and configuration for the GPIO pins	0xE280
D_GPIO_PIN_CFG	Selection of the functionality attached to each of the GPIO pins	0xE281
D_GPIO_DATA	GPIO input and output data	0xE282

12.2.3.1 D_GPIO_CFG Settings

Bit Field	Field Name	Field Description
(22:20)	GPIO_CFG_INT_SRC	Select the source for triggering the GPIO interrupt
(18:16)	GPIO_CFG_INT_EVENT	Select the event that will trigger a GPIO interrupt
12	GPIO_CFG_PULLUP_DIS_GPIO4	Manually disable the pull-up resistor for GPIO 4
11	GPIO_CFG_PULLUP_DIS_GPIO3	Manually disable the pull-up resistor for GPIO 3
10	GPIO_CFG_PULLUP_DIS_GPIO2	Manually disable the pull-up resistor for GPIO 2
9	GPIO_CFG_PULLUP_DIS_GPIO1	Manually disable the pull-up resistor for GPIO 1
8	GPIO_CFG_PULLUP_DIS_GPIO0	Manually disable the pull-up resistor for GPIO 0
4	GPIO_CFG_DIR_CFG_GPIO4	Select the direction of GPIO 4 (either input or output)
3	GPIO_CFG_DIR_CFG_GPIO3	Select the direction of GPIO 3 (either input or output)
2	GPIO_CFG_DIR_CFG_GPIO2	Select the direction of GPIO 2 (either input or output)
1	GPIO_CFG_DIR_CFG_GPIO1	Select the direction of GPIO 1 (either input or output)
0	GPIO_CFG_DIR_CFG_GPIO0	Select the direction of GPIO 0 (either input or output)

Field Name	Value Symbol	Value Description	Hex Value
GPIO_CFG_INT_SRC	GPIO_INT_SRC_GPIO0	Use GPIO 0 as the interrupt source	0x0*
	GPIO_INT_SRC_GPIO1	Use GPIO 1 as the interrupt source	0x1
	GPIO_INT_SRC_GPIO2	Use GPIO 2 as the interrupt source	0x2
	GPIO_INT_SRC_GPIO3	Use GPIO 3 as the interrupt source	0x3
	GPIO_INT_SRC_GPIO4	Use GPIO 4 as the interrupt source	0x4

Field Name	Value Symbol	Value Description	Hex Value
GPIO_CFG_INT_EVENT	GPIO_INT_EVENT_DISABLED	Disable GPIO interrupts	0x0*
	GPIO_INT_EVENT_HIGH_LEVEL	Trigger an interrupt on a logical 1	0x1
	GPIO_INT_EVENT_LOW_LEVEL	Trigger an interrupt on a logical 0	0x2
	GPIO_INT_EVENT_RISING_ EDGE	Trigger an interrupt on a rising edge	0x3
	GPIO_INT_EVENT_FALLING_ EDGE	Trigger an interrupt on a falling edge	0x4
	GPIO_INT_EVENT_TRANSITION	Trigger an interrupt on a transition	0x5
GPIO_CFG_PULLUP_DIS_GPIO4	GPIO4_PULLUP_ENABLE	Automatic pull-up configuration	0x0*
	GPIO4_PULLUP_DISABLE	Manually disable GPIO 4 pull-up	0x1
GPIO_CFG_PULLUP_DIS_GPIO3	GPIO3_PULLUP_ENABLE	Automatic pull-up configuration	0x0*
	GPIO3_PULLUP_DISABLE	Manually disable GPIO 3 pull-up	0x1
GPIO_CFG_PULLUP_DIS_GPIO2	GPIO2_PULLUP_ENABLE	Automatic pull-up configuration	0x0*
	GPIO2_PULLUP_DISABLE	Manually disable GPIO 2 pull-up	0x1
GPIO_CFG_PULLUP_DIS_GPIO1	GPIO1_PULLUP_ENABLE	Automatic pull-up configuration	0x0*
	GPIO1_PULLUP_DISABLE	Manually disable GPIO 1 pull-up	0x1
GPIO_CFG_PULLUP_DIS_GPIO0	GPIO0_PULLUP_ENABLE	Automatic pull-up configuration	0x0*
	GPIO0_PULLUP_DISABLE	Manually disable GPIO 0 pull-up	0x1
GPIO_CFG_DIR_CFG_GPIO4	GPIO4_OUTPUT	Configure GPIO 4 as output	0x0
	GPIO4_INPUT	Configure GPIO 4 as input	0x1*
GPIO_CFG_DIR_CFG_GPIO3	GPIO3_OUTPUT	Configure GPIO 3 as output	0x0
	GPIO3_INPUT	Configure GPIO 3 as input	0x1*
GPIO_CFG_DIR_CFG_GPIO2	GPIO2_OUTPUT	Configure GPIO 2 as output	0x0
	GPIO2_INPUT	Configure GPIO 2 as input	0x1*
GPIO_CFG_DIR_CFG_GPIO1	GPIO1_OUTPUT	Configure GPIO 1 as output	0x0
	GPIO1_INPUT	Configure GPIO 1 as input	0x1*
GPIO_CFG_DIR_CFG_GPIO0	GPIO0_OUTPUT	Configure GPIO 0 as output	0x0
	GPIO0_INPUT	Configure GPIO 0 as input	0x1*

12.2.3.2 D_GPIO_PIN_CFG Settings

Bit Field	Field Name	Field Description
(9:8)	GPIO_PIN_CFG_GPIO4	Select the functionality for GPIO 4
(7:6)	GPIO_PIN_CFG_GPIO3	Select the functionality for GPIO 3
(5:4)	GPIO_PIN_CFG_GPIO2	Select the functionality for GPIO 2

Bit Field	Field Name	Field Description
(3:2)	GPIO_PIN_CFG_GPIO1	Select the functionality for GPIO 1
(1:0)	GPIO_PIN_CFG_GPIO0	Select the functionality for GPIO 0

Field Name	Value Symbol	Value Description	Hex Value
GPIO_PIN_CFG_GPIO4	GPIO4_SELECT_GPIO	Use GPIO 4 as a GPIO pin	0x0*
	GPIO4_SELECT_NONE	Disable GPIO 4	0x1
	GPIO4_SELECT_DAC_SCALE	Use GPIO 4 to control DAC scaling	0x2
	GPIO4_SELECT_LSAD	Use GPIO 4 as an LSAD input	0x3
GPIO_PIN_CFG_GPIO3	GPIO3_SELECT_GPIO	Use GPIO 3 as a GPIO pin	0x0*
	GPIO3_SELECT_SYNC_OUT	Use GPIO 3 to create a synchronizing output signal	0x1
	GPIO3_SELECT_SYNC_IN	Use GPIO 3 to synchronize to an input synchronizing signal	0x2
	GPIO3_SELECT_LSAD	Use GPIO 3 as an LSAD input	0x3
GPIO_PIN_CFG_GPIO2	GPIO2_SELECT_GPIO	Use GPIO 2 as a GPIO pin	0x0*
	GPIO2_SELECT_NONE1	Disable GPIO 2	0x1
	GPIO2_SELECT_NONE2	Disable GPIO 2	0x2
	GPIO2_SELECT_LSAD	Use GPIO 2 as an LSAD input	0x3
GPIO_PIN_CFG_GPIO1	GPIO1_SELECT_GPIO	Use GPIO 1 as a GPIO pin	0x0*
	GPIO1_SELECT_NONE	Disable GPIO 1	0x1
	GPIO1_SELECT_UART_RX	Use GPIO 1 as UART receive	0x2
	GPIO1_SELECT_LSAD	Use GPIO 1 as an LSAD input	0x3
GPIO_PIN_CFG_GPIO0	GPIO0_SELECT_GPIO	Use GPIO 0 as a GPIO pin	0x0*
	GPIO0_SELECT_UART_TX	Use GPIO 0 as UART transmit	0x1
	GPIO0_SELECT_NONE	Disable GPIO 0	0x2
	GPIO0_SELECT_LSAD	Use GPIO 0 as an LSAD input	0x3

12.3 LOW-SPEED ANALOG-TO-DIGITAL (LSAD) CONVERTER INTERFACE

The Low-Speed Analog-to-Digital (LSAD) converters provide a low-frequency 10-bit A/D conversion of 5 external signals and 3 internal signals. The external signals are multiplexed onto the 5 GPIO pads and are numbered as channels 0 to 4. The converted output from the most recent conversion of each external signal can be found in the A_LSAD_CHx_DATA registers. The internal signals of the LSAD are used to monitor the internal digital supply voltage (VDDC), the battery voltage (VBAT) and LSAD reference voltage (VREG). The converted output from the most recent conversion of these internal signals can be found in the A_LSAD_VDDC_DATA, A_LSAD_VBAT_DATA and A_LSAD_VREG_DATA registers respectively.

IMPORTANT: GPIO 0 can be configured to LSAD 0 only if all the other GPIO pins are configured for LSAD functionality.

The purpose of the LSAD converter is to sample input voltages that typically change slowly—for example, the voltage associated with a potentiometer-based volume control. All LSAD converter inputs are multiplexed with other functionality. Refer to Section 12.2, "General-Purpose I/O (GPIO) Interface" on page 148 for information on how to configure these multiplexed pins for LSAD converter functionality.

The LSAD signals are sampled sequentially at a sampling rate defined by SLOWCLK and prescaled using the LSAD_CTRL_FREQ bit field in the A_LSAD_CTRL register. The overall sampling frequency is defined by:

$$f_{LSAD} = \frac{f_{SLOWCLK}}{20 \times (LSAD_CTRL_FREQ + 1)}$$

As a result, if the slow clock is operating at 1.28MHz, the default setting of the LSAD_CTRL_FREQ bit field results in an overall sampling rate of 12.8kHz and a sampling rate of 1.6kHz for each individual LSAD input.

Each LSAD can be configured to run in one of two modes by setting or clearing the appropriate LSAD_CTRL_xxx_MODE bits in the A_LSAD_CTRL register. In both modes the LSAD measures a nominal input range of 0V to 2V using a converter that measures signals in the range of -1V to 3V.

In trimmer mode The output data value from the LSAD uses a 9-bit digitally saturated unsigned result scaled between 0x000 and 0x1FF to represent a signal in the 0V to 2V range.

In audio mode The output data value from the LSAD uses a 10-bit unsaturated signed result scaled between 0x300 and 0x0FF to represent a signal in the 0V to 2V range. Results in the range 0x200 to 0x1FF are possible in this mode and correspond to voltages in the larger -1V to 3V range; however measurements outside the nominal range are limited by the protection diodes attached to the LSAD I/O pads.

12.3.1 Control and Configuration Registers

Register Name	Register Description	Address
A_LSAD_CTRL	Control and configuration for the LSAD converters	0xE180
A_LSAD_CH0_DATA	10-bit data result from the LSAD converter for channel 0	0xE181
A_LSAD_CH1_DATA	10-bit data result from the LSAD converter for channel 1	0xE182
A_LSAD_CH2_DATA	10-bit data result from the LSAD converter for channel 2	0xE183
A_LSAD_CH3_DATA	10-bit data result from the LSAD converter for channel 3	0xE184
A_LSAD_CH4_DATA	10-bit data result from the LSAD converter for channel 4	0xE185
A_LSAD_VDDC_DATA	10-bit data result from the LSAD converter for the internal digital supply voltage	0xE186
A_LSAD_VBAT_DATA	10-bit data result from the LSAD converter for the battery voltage	0xE187
A_LSAD_VREG_DATA	10-bit data result from the LSAD converter for the 1V regulated supply voltage	0xE188

12.3.1.1 A_LSAD_CTRL Settings

Bit Field	Field Name	Field Description
11	LSAD_CTRL_VREG_MODE	Select between trimmer and audio mode for the VREG LSAD
10	LSAD_CTRL_VBAT_MODE	Select between trimmer and audio mode for the VBAT LSAD
9	LSAD_CTRL_VDDC_MODE	Select between trimmer and audio mode for the VDDC LSAD
8	LSAD_CTRL_CH4_MODE	Select between trimmer and audio mode for the CH4 LSAD
7	LSAD_CTRL_CH3_MODE	Select between trimmer and audio mode for the CH3 LSAD
6	LSAD_CTRL_CH2_MODE	Select between trimmer and audio mode for the CH2 LSAD
5	LSAD_CTRL_CH1_MODE	Select between trimmer and audio mode for the CH1 LSAD
4	LSAD_CTRL_CH0_MODE	Select between trimmer and audio mode for the CH0 LSAD
3:0	LSAD_CTRL_FREQ	Prescaling divisor for SLOWCLK to obtain the LSAD sampling rate

Field Name	Value Symbol	Value Description	Hex Value
LSAD_CTRL_VREG_MODE	LSAD_CTRL_VREG_MODE_TRIMMER	Select LSAD trimmer mode for VREG	0x0*
	LSAD_CTRL_VREG_MODE_AUDIO_OUT	Select LSAD audio mode for VREG	0x1
LSAD_CTRL_VBAT_MODE	LSAD_CTRL_VBAT_MODE_TRIMMER	Select LSAD trimmer mode for VBAT	0x0*
	LSAD_CTRL_VBAT_MODE_AUDIO_OUT	Select LSAD audio mode for VBAT	0x1
LSAD_CTRL_VDDC_MODE	LSAD_CTRL_VDDC_MODE_TRIMMER	Select LSAD trimmer mode for VDDC	0x0*
	LSAD_CTRL_VDDC_MODE_AUDIO_OUT	Select LSAD audio mode for VDDC	0x1
LSAD_CTRL_CH4_MODE	LSAD_CTRL_CH4_MODE_TRIMMER	Select LSAD trimmer mode for channel 4	0x0*
	LSAD_CTRL_CH4_MODE_AUDIO_OUT	Select LSAD audio mode for channel 4	0x1
LSAD_CTRL_CH3_MODE	LSAD_CTRL_CH3_MODE_TRIMMER	Select LSAD trimmer mode for channel 3	0x0*
	LSAD_CTRL_CH3_MODE_AUDIO_OUT	Select LSAD audio mode for channel 3	0x1
LSAD_CTRL_CH2_MODE	LSAD_CTRL_CH2_MODE_TRIMMER	Select LSAD trimmer mode for channel 2	0x0*
	LSAD_CTRL_CH2_MODE_AUDIO_OUT	Select LSAD audio mode for channel 2	0x1
LSAD_CTRL_CH1_MODE	LSAD_CTRL_CH1_MODE_TRIMMER	Select LSAD trimmer mode for channel 1	0x0*
	LSAD_CTRL_CH1_MODE_AUDIO_OUT	Select LSAD audio mode for channel 1	0x1

Field Name	Value Symbol	Value Description	Hex Value
LSAD_CTRL_CH0_MODE	LSAD_CTRL_CH0_MODE_TRIMMER	Select LSAD trimmer mode for channel 0	0x0*
	LSAD_CTRL_CH0_MODE_AUDIO_OUT	Select LSAD audio mode for channel 0	0x1
LSAD_CTRL_FREQ	LSAD_CTRL_DISABLE	Disable the LSAD converters	0x0
	LSAD_CTRL_PRESCALE_40	Divide SLOWCLK by 40	0x1
	LSAD_CTRL_PRESCALE_60	Divide SLOWCLK by 60	0x2
	LSAD_CTRL_PRESCALE_80	Divide SLOWCLK by 80	0x3
	LSAD_CTRL_PRESCALE_100	Divide SLOWCLK by 100	0x4*
	LSAD_CTRL_PRESCALE_120	Divide SLOWCLK by 120	0x5
	LSAD_CTRL_PRESCALE_140	Divide SLOWCLK by 140	0x6
	LSAD_CTRL_PRESCALE_160	Divide SLOWCLK by 160	0x7
	LSAD_CTRL_PRESCALE_180	Divide SLOWCLK by 180	0x8
	LSAD_CTRL_PRESCALE_200	Divide SLOWCLK by 200	0x9
	LSAD_CTRL_PRESCALE_220	Divide SLOWCLK by 220	0xA
	LSAD_CTRL_PRESCALE_240	Divide SLOWCLK by 240	0xB
	LSAD_CTRL_PRESCALE_260	Divide SLOWCLK by 260	0xC
	LSAD_CTRL_PRESCALE_280	Divide SLOWCLK by 280	0xD
	LSAD_CTRL_PRESCALE_300	Divide SLOWCLK by 300	0xE
	LSAD_CTRL_PRESCALE_320	Divide SLOWCLK by 320	0xF

12.4 SERIAL PERIPHERAL INTERFACE (SPI)

The Serial Peripheral Interface (SPI) of the BelaSigna 300 system is designed to communicate with external devices with a particular focus on EEPROM and related non-volatile memory (NVM) devices. Setting the SPI_CFG_ENABLE bit in the D_SPI_CFG control register enables the SPI interface.

The EEPROM library from the firmware is designed to assist users wishing to use the SPI interface with an external EEPROM device. See the *Firmware Reference Manual* for more details on this library.

The SPI interface implementation operates in master mode only and all data is transmitted or received starting with the MSB of the word received. To configure the interface to transmit or receive between 1 and 32 bits of data for each word, set the SPI_CTRL_N_BITS bit field in the D_SPI_CTRL register (where in each transaction $\text{bits} = \text{SPI_CTRL_N_BITS} + 1$).

The direction of data transmission can be selected using the SPI_CTRL_RW field from the D_SPI_CTRL register. When transmitting data over the SPI interface, setting the SPI_CTRL_START bit in the D_SPI_CTRL register starts a transaction. This same bit (which is also known as the SPI_CTRL_BUSY bit) can also be read to indicate whether the SPI interface is currently processing a transaction.

If the SPI_CFG_AUTO bit in the D_SPI_CFG register is set, the SPI interface operates in auto mode. In this mode, reading or writing the data through the D_SPI_DATA register sets the SPI_CTRL_START bit, initiating the next transaction on the SPI interface.

Note: When the interface is disabled any data currently being transmitted is allowed to complete before the interface shuts down.

Data to be transmitted and data that has been received are both stored in the D_SPI_DATA transmit and receive register. The contents of the D_SPI_DATA register are also mirrored into the D_SPI_DATA_M register and other mirrored registers (as identified by the D_SPI_DATA_M_ register name prefix) that provide partial transaction results. Reading from these mirrored registers does not trigger an SPI transaction when running in auto mode.

The interface operates using a divided form of SYSCLK, operating at a power-of-2 division from this clock as defined by the SPI_CFG_PRESCALE bit field in the D_SPI_CFG register. The interface can also be configured to react to either clock polarity by clearing or setting the SPI_CFG_CLKPOL bit in the D_SPI_CFG register. If this bit is set, the serial output changes on the rising edge and the serial input is sampled on the falling edge. If this bit is cleared, the serial output changes on the falling edge and the serial input is sampled on the rising edge.

The SPI interface includes a configurable pull-up resistor that is attached to the serial input line of the interface. To disconnect this resistor from the pad, set the SPI_CFG_PULLUP_DIS bit in the D_SPI_CFG register.

WARNING: Using the SPI_CFG_PULLUP_DIS bit to disable the bus pull-up resistor for the SPI serial input line is not recommended under most conditions. We recommend that you consult with ON Semiconductor before using this configuration because attempting to use this interface without the pull-up resistor might result in unintended behavior.

12.4.1 SPI Interrupt

The SPI interface uses an associated interrupt which, when enabled, signals the completion of a transaction. See Section 11.1, “Interrupt Controller” on page 113 for information regarding interrupt configuration and handling.

When an interrupt is received while a user application is receiving data from the SPI interface, the new data received can be read from the D_SPI_DATA register. Alternately, if the application is transmitting data using the SPI interface, the D_SPI_DATA register can now be loaded with the next word of data to be transmitted.

12.4.2 Control and Configuration Registers

Register Name	Register Description	Address
D_SPI_CFG	Configuration of the SPI interface	0xE1C0
D_SPI_CTRL	Control SPI interface transactions	0xE1C1
D_SPI_DATA	SPI transmit and receive data	0xE1C2
D_SPI_DATA_M	Mirrored SPI transmit and receive data	0xE1C3
D_SPI_DATA_M_16LSB	Mirrored partial SPI transmit and receive data (bits 0 - 15)	0xE1C4
D_SPI_DATA_M_16MSB	Mirrored partial SPI transmit and receive data (bits 16 - 31)	0xE1C5

Register Name	Register Description	Address
D_SPI_DATA_M_8LSB	Mirrored partial SPI transmit and receive data (bits 0 - 7)	0xE1C6
D_SPI_DATA_M_8_15BITS	Mirrored partial SPI transmit and receive data (bits 8 - 15)	0xE1C7
D_SPI_DATA_M_16_23BITS	Mirrored partial SPI transmit and receive data (bits 16 - 23)	0xE1C8
D_SPI_DATA_M_8MSB	Mirrored partial SPI transmit and receive data (bits 24 - 32)	0xE1C9

12.4.2.1 D_SPI_CFG Settings

Bit Field	Field Name	Field Description
6	SPI_CFG_PULLUP_DIS	Disable/enable the pull-up resistor for the SPI serial input
5	SPI_CFG_CLKPOL	Select clock polarity for SPI communications
4	SPI_CFG_AUTO	Automatically start a new transaction when D_SPI_DATA is read or written
3	SPI_CFG_ENABLE	Enable/disable the SPI interface
2:0	SPI_CFG_PRESCALE	Pre-scaling divisor for SYSCLK to obtain the interface speed

Field Name	Value Symbol	Value Description	Hex Value
SPI_CFG_PULLUP_DIS	SPI_PULLUP_ENABLE	Enable the serial input pull-up	0x0*
	SPI_PULLUP_DISABLE	Disable the serial input pull-up	0x1
SPI_CFG_CLKPOL	SPI_CLKPOL_NORMAL	Use normal clock polarity	0x0*
	SPI_CLKPOL_INVERSE	Use inverse clock polarity	0x1
SPI_CFG_AUTO	SPI_PORT_AUTO_DISABLE	Disable SPI auto mode	0x0*
	SPI_PORT_AUTO_ENABLE	Enable SPI auto mode	0x1
SPI_CFG_ENABLE	SPI_PORT_DISABLE	Disable the SPI interface	0x0*
	SPI_PORT_ENABLE	Enable the SPI interface	0x1
SPI_CFG_PRESCALE	SPI_CLK_PRESCALE_2	Divide SYSCLK by 2	0x0*
	SPI_CLK_PRESCALE_4	Divide SYSCLK by 4	0x1
	SPI_CLK_PRESCALE_8	Divide SYSCLK by 8	0x2
	SPI_CLK_PRESCALE_16	Divide SYSCLK by 16	0x3
	SPI_CLK_PRESCALE_32	Divide SYSCLK by 32	0x4
	SPI_CLK_PRESCALE_64	Divide SYSCLK by 64	0x5
	SPI_CLK_PRESCALE_128	Divide SYSCLK by 128	0x6
	SPI_CLK_PRESCALE_256	Divide SYSCLK by 256	0x7

12.4.2.2 D_SPI_CTRL Settings

Bit Field	Field Name	Field Description
7 (Write)	SPI_CTRL_START	Control bit used to start transactions in normal mode
7 (Read)	SPI_CTRL_BUSY	Indicator of whether the SPI interface is in a transaction
6	SPI_CTRL_RW	Select either read or write transactions
5	SPI_CTRL_CS_N	Set the negative chip select output pin
4:0	SPI_CTRL_N_BITS	Select how many bits should be in a transaction

Field Name	Value Symbol	Value Description	Hex Value
SPI_CTRL_START	SPI_START	Start the next SPI transaction	0x1
SPI_CTRL_RW	SPI_WRITE	Configure the port for writing	0x0*
	SPI_READ	Configure the port for reading	0x1
SPI_CTRL_CS_N	SPI_CS_LOW	Set the CS pin low	0x0*
	SPI_CS_HIGH	Set the CS pin high	0x1

12.5 UNIVERSAL ASYNCHRONOUS RECEIVER-TRANSMITTER (UART) INTERFACE

The general-purpose UART interface is designed to support RS-232 communications with the BelaSigna 300 system. Setting the UART_CFG_ENABLE bit in the D_UART_CFG control register enables the UART interface.

The pins for this interface are multiplexed onto GPIO pins 0 and 1. Refer to Section 12.2, “General-Purpose I/O (GPIO) Interface” on page 148 for information on configuring these pins for UART interface functionality.

The UART interface operates in full-duplex mode using a standard data format of 1 start bit, 8 data bits and 1 stop bit. All data bytes being sent or received is interpreted as starting with the LSB. Figure 21 shows the waveform for a UART transmit or receive transaction.

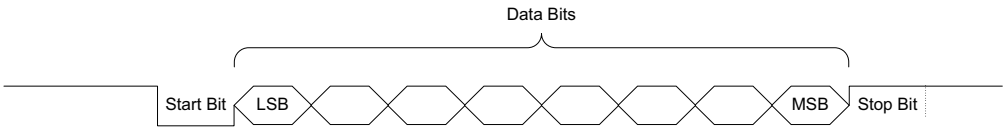


Figure 21: UART Transaction Waveform

Data to be transmitted and data that has been received are both stored in the D_UART_DATA transmit and receive register.

Note: When the interface is disabled, any data currently being transmitted are allowed to complete before the interface shuts down.

The baud rate (specified in bits per second) for the UART interface is defined in terms of SYSCLK and several configuration parameters as seen in the following equation for baud rate calculation:

$$\text{baud rate} = \frac{(\text{UART_CFG_PRESCALE} \cdot \text{SYSCLK})}{(2^{18} \times (1 + 11 \times \text{UART_CFG_PRESCALE_ENABLE}))}$$

Starting from SYSCLK, a constant division by 2^{18} (or $0.2048 \times [1.28 \times 10^{6}]$) is applied. Next, a pre-scaling division by 12 can be added by setting the UART_CFG_PRESCALE_ENABLE bit in the D_UART_CFG register. Finally, the UART_CFG_PRESCALE bit field from the same register provides a 16-bit multiplication of the resulting value to obtain the final baud rate. Together these settings allow the selection of a baud rate between 0.4 baud and 320 kbaud for a SYSCLK of 1.28MHz.

For example, to obtain a standard 115200 baud rate with a SYSCLK of 1.28MHz, clear the UART_CFG_PRESCALE_ENABLE bit and set the UART_CFG_PRESCALE bit field to 23593.

12.5.1 UART Interrupts

The UART interface uses a pair of associated interrupts that separately control transmission and reception of UART data. See Section 11.1, “Interrupt Controller” on page 113 for information regarding interrupt configuration and handling.

If a user application is receiving data from the UART interface, the UART_RX interrupt signals that a data byte has been successfully received and has been written to the D_UART_DATA register.

If a user application is transmitting data from the UART interface, the UART_TX interrupt signals that transmission of the data value in the D_UART_DATA register has started and that the application can now load the next byte to be transmitted.

12.5.2 Control and Configuration Registers

Register Name	Register Description	Address
D_UART_CFG	Configuration of the UART interface	0xE2C0
D_UART_DATA	UART transmit and receive data	0xE2C1

12.5.2.1 D_UART_CFG Settings

Bit Field	Field Name	Field Description
23:8	UART_CFG_PRESCALE	Pre-scaling multiplier in baud rate calculation
4	UART_CFG_PRESCALE_ENABLE	Enable/disable pre-scaling division by 12 in baud rate calculation
0	UART_CFG_ENABLE	Enable/disable the UART interface

Field Name	Value Symbol	Value Description	Hex Value
UART_CFG_PRESCALE_ENABLE	UART_PRESCALE_DISABLE	Disable pre-scaling division by 12	0x0*
	UART_PRESCALE_ENABLE	Enable pre-scaling division by 12	0x1
UART_CFG_ENABLE	UART_DISABLE	Disable UART interface	0x0*
	UART_ENABLE	Enable UART interface	0x1

13 Clock Components

13.1 Clock Generation

All clocks and clock domains in the BelaSigna 300 system are derived from the system clock (SYSCLK). Because of its importance to the functioning of BelaSigna 300 and the execution of applications, this clock signal can be generated in multiple ways for maximum flexibility. Choose one of the following sources:

- The internal primary oscillator (discussed in Section 13.1.1, “Primary Oscillator”); this clock is used as the default source for SYSCLK on CID 102 devices
- The internal standby oscillator (discussed in Section 13.1.2, “Standby Oscillator” on page 162)
- The external clock pad (discussed in Section 13.2.4, “External Clock (EXTCLK)” on page 165); after the boot system initialization this clock is used as the default source for SYSCLK on CID 201 devices

13.1.1 Primary Oscillator

The primary oscillator is an internal RC oscillator that can be configured to provide a clock signal to the BelaSigna 300 device. On CID 201 devices, this oscillator is used to initialize the system and ensure that it is in an appropriate state before being clocked by the EXTCLK. On CID 102 devices, this oscillator is used as the default clock for the system. If your application does not use the primary oscillator to clock the system, we recommend that you disable this oscillator by clearing the `CLK_CTRL_SYSCLK` bit field from the `A_CLK_CTRL` control register. Attempting to clear this register bit field while the primary oscillator is the source for the system clock results in a value of `SYSCLK_0M62` or 0x1 being stored in the `CLK_CTRL_SYSCLK` bit field.

Configure the coarse frequency of the on-chip primary oscillator by using the `CLK_CTRL_SYSCLK` bit field from the `A_CLK_CTRL` control register to select a nominal oscillator frequency. Next, to finely calibrate the frequency of this clock signal to obtain an exact clock frequency for use in the system, use the `CLK_CTRL_CCR_DATA` bit field from the `A_CLK_CTRL` control register.

The behavior of the primary oscillator with respect to the configured parameters is listed in Table 33.

Note: When the primary oscillator is configured, the nominal coarse oscillator frequency assumes the default fine calibration setting of 0x80 (a setting that is centered in the -20% to 20% range of fine calibration).

TABLE 33: PRIMARY OSCILLATOR BEHAVIOR

Parameter	Performance and Notes
Coarse Oscillator Frequency	-35% to +68% of the nominal coarse oscillator frequency
Fine Calibration Range	-20% to +20% of the actual coarse oscillator frequency
Calibrated Oscillator Frequency	-1% to +1% of calibrated frequency

13.1.2 Standby Oscillator

The BelaSigna 300 device includes a second oscillator to allow for low frequency operation in standby mode. When operating in this mode power consumption is greatly reduced; however the system is still executing instructions and can be woken up to a regular operation mode. When the standby clock is selected as the source for the system clock (see Section 13.2.1, “System Clock (SYSCLK)” on page 164), the standby oscillator provides a 17kHz clock signal.

13.1.3 Control and Configuration Registers

Register Name	Register Description	Address
D_CLK_CFG_0	Digital clock configuration (register 0)	0xE080
A_CLK_CTRL	Analog clock control and oscillator settings	0xE100

13.1.3.1 A_CLK_CTRL Settings

Bit Field	Field Name	Field Description
11:8	CLK_CTRL_SYSCLK	Select the coarse calibration setting for the primary oscillator
7:0	CLK_CTRL_CCR_DATA	Select the fine calibration setting for the primary oscillator

Field Name	Value Symbol	Value Description	Hex Value
CLK_CTRL_SYSCLK	SYSCLK_OFF	Disable the primary oscillator	0x0
	SYSCLK_0M83	SYSCLK operates at a nominal 0.83MHz	0x1
	SYSCLK_1M22	SYSCLK operates at a nominal 1.22MHz	0x2
	SYSCLK_1M59	SYSCLK operates at a nominal 1.59MHz	0x3
	SYSCLK_1M96	SYSCLK operates at a nominal 1.96MHz	0x4*
	SYSCLK_2M31	SYSCLK operates at a nominal 2.31MHz	0x5
	SYSCLK_2M97	SYSCLK operates at a nominal 2.97MHz	0x6
	SYSCLK_3M59	SYSCLK operates at a nominal 3.59MHz	0x7
	SYSCLK_4M17	SYSCLK operates at a nominal 4.17MHz	0x8
	SYSCLK_5M24	SYSCLK operates at a nominal 5.24MHz	0x9
	SYSCLK_6M16	SYSCLK operates at a nominal 6.16MHz	0xA
	SYSCLK_7M35	SYSCLK operates at a nominal 7.35MHz	0xB
	SYSCLK_8M68	SYSCLK operates at a nominal 8.68MHz	0xC
	SYSCLK_11M0	SYSCLK operates at a nominal 11.0MHz	0xD
	SYSCLK_11M1	SYSCLK operates at a nominal 11.1MHz	0xE
	SYSCLK_11M8	SYSCLK operates at a nominal 11.8MHz	0xF

13.2 CLOCK DISTRIBUTION

As noted in Section 13.1, “Clock Generation” on page 161, the system clock (SYSCLK) is used as the source of all of the other internal clocks and clock domains. The connections of the clock distribution network that provides SYSCLK in its original or divided forms can be seen in Figure 22.

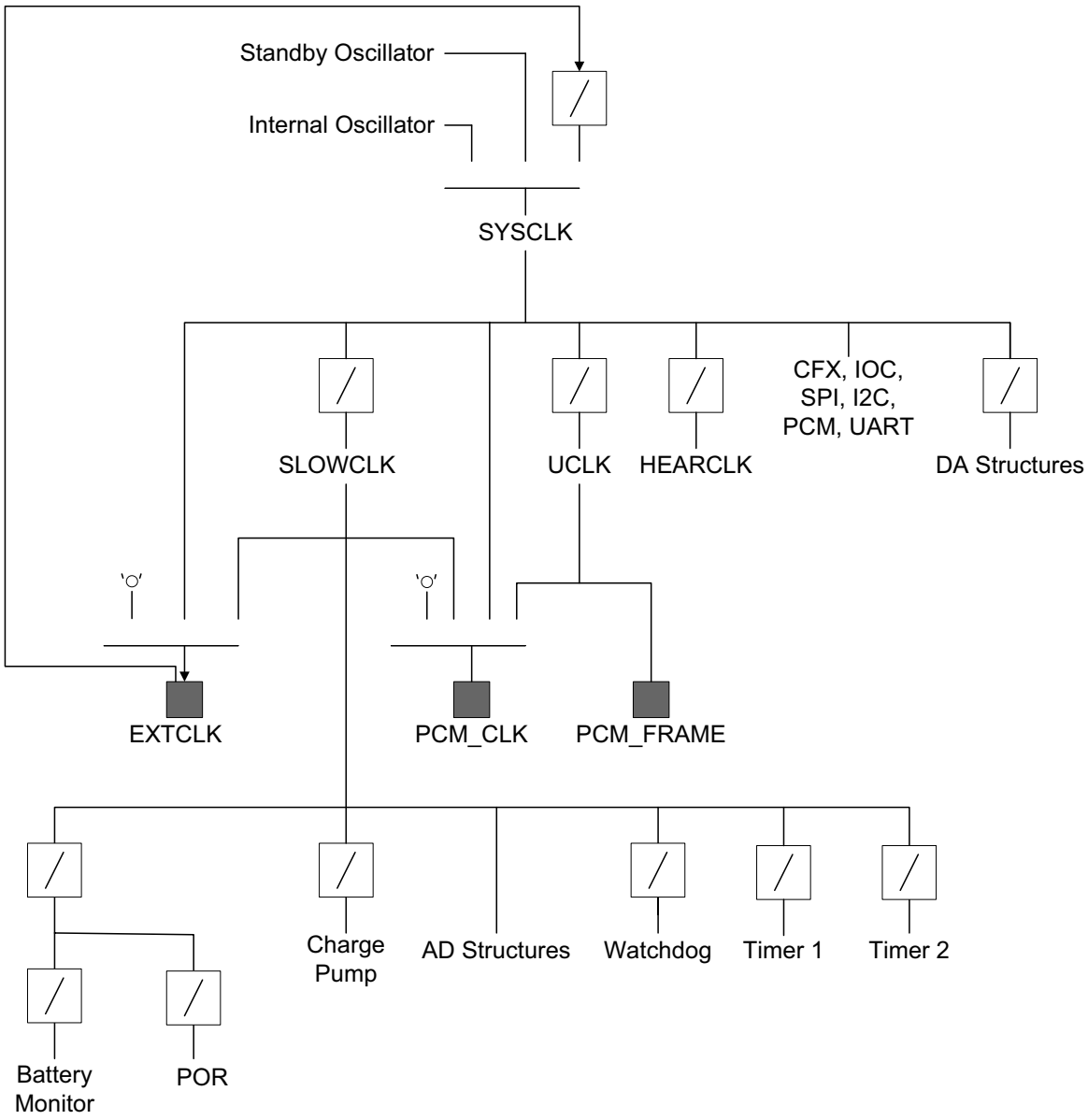


Figure 22: System Clock Distribution

Further information on each of the internal clocks and clock domains is available in the following subsections of this chapter. Some subsections of this manual also contain related information

where a clock is used for only one component of the BelaSigna 300 device and has no side effects that change the behavior of other system components.

13.2.1 System Clock (SYSCLK)

The system clock is the primary clock for the system and all other clocks in the BelaSigna 300 device are derived from this clock.

For devices with CID 201, this clock is configured to use the external clock by default. For devices with CID 102, this clock is configured to use the clock generated by the primary internal oscillator by default. For all BelaSigna 300 devices, this clock can be configured to use the clock generated by the primary internal oscillator, the standby oscillator, or the external clock. To select the standby oscillator as the source for the system clock, set the `CLK_CFG0_SYSCLK_SEL` bit from the `D_CLK_CFG_0` register. To configure the external clock as the source for the system clock, use the `CLK_CFG0_EXTCLK_SEL` bit field from the `D_CLK_CFG0` register (which is described further in Section 13.2.4, “External Clock (EXTCLK)” on page 165).

Note: Configuring the external clock as input takes precedence over configuring the system clock to run off of the standby oscillator.

13.2.2 Slow Clock (SLOWCLK)

The slow clock is used by the analog and timer related components of the system. These include the input and output stages, the general purpose and watchdog timers, and the various power components of the system.

This clock derives from the system clock and uses a 6-bit integer divisor to scale the system clock to the correct slow clock frequency. Set the value of this divisor by using the `CLK_CFG1_SLOWCLK_PRESCALE` bit field in the `D_CLK_CFG_1` register.

IMPORTANT: System behaviors have only been characterized for SLOWCLK frequencies between 100kHz and 3.84MHz. Use of SLOWCLK frequencies outside of this range might result in unexpected or incorrect behavior, and are not supported.

13.2.3 Accelerator Clock (HEARCLK)

The accelerator clock is only used as the clock for the accelerator. Normally this clock operates at the same frequency as the system clock; however to spread out the dynamic power consumption of the system or to ensure no conflicts in accessing shared memory and microcode memory, you might want to use a different clock frequency for the HEAR.

This clock derives from the system clock through scaling by a fractional divisor. Setting the `CLK_CFG0_HEARCLK_PRESCALE` bit field from the `D_CLK_CFG_0` register configures the clock frequency of this clock according to the following equation:

$$f_{HEARCLK} = \frac{f_{SYSCLK} \times 8}{(8 + CLK_CFG0_HEARCLK_PRESCALE)}$$

The double access mode setting configures the system to share system clock cycles between the CFX processor and the accelerator clock if the accelerator clock uses a prescaler. The system can then guarantee that no conflicts will occur when accessing the microcode memory or shared

memory bus. If the CLK_CFG0_HEARCLK_PRESCALE bit field is not set to 0 (i.e. the HEAR clock is running slower than the system clock), and double access mode has been selected, the CFX processor operates at a clock frequency defined by the following equation:

$$f_{\text{CFX Core}} = \frac{f_{\text{SYSCLK}} \times \text{CLK_CFG0_HEARCLK_PRESCALE}}{(8 + \text{CLK_CFG0_HEARCLK_PRESCALE})}$$

In this way, the accelerator clock frequency remains the same and the execution on the CFX processor is limited to those system clock cycles when the accelerator clock is not running.

To configure the double access mode setting, set the CLK_CFG1_DBL_ACCESS_MODE bit in the D_CLK_CFG_1 register.

An example use case for double access mode might use an accelerator clock prescaler of 8. This results in both the CFX DSP and HEAR Accelerator each operating at $f_{\text{SYSCLK}}/2$. If you then doubled the system clock frequency and applied an additional division by 2 to all derived clocks (including the slow clock), you would end up with the original number of clock cycles for both of the cores without the possibility of memory conflicts occurring between the cores.

13.2.4 External Clock (EXTCLK)

IMPORTANT: The external clock is the default clock source for BelaSigna 300 devices with CID 201. For these devices, using the external clock as an output clock is generally not advisable. If your application requires an output clock, we recommend using the user clock (Section 13.2.5, "User Clock (UCLK)") on these BelaSigna 300 devices.

The external clock in the BelaSigna 300 system can be used as both an input and an output clock source. To select the operating configuration for the external clock, set the CLK_CFG0_EXTCLK_SEL bit field of the D_CLK_CFG_0 register.

When configured as an input source, the external clock is passed through a prescaling integer division and is used as a source for the system clock. To select the divisor used for this division, set the CLK_CFG0_EXTCLK_PRESCALE bit in the D_CLK_CFG_0 register.

When configured as an output source, the external clock output pin can be driven by either the system clock (SYSCLK) or slow clock (SLOWCLK). The external clock output is not scaled by any factor in addition to those scaling factors that already apply to the source clock.

13.2.5 User Clock (UCLK)

User clock is an output clock that you can use as a source for the shared PCM/I^2S interface, or for an external device through the PCM frame pin. To select this clock as the source for the PCM_FRAME pad, set the CLK_CFG1_PCM_FRAME_PAD_SEL bit in the D_CLK_CFG_1 register.

User clock is derived from the system clock through a 12-bit integer division by the CLK_CFG1_UCLK_PRESCALE bit field in the D_CLK_CFG_1 register.

13.2.6 Control and Configuration Registers

Register Name	Register Description	Address
D_CLK_CFG_0	Digital clock configuration (first register)	0xE080
D_CLK_CFG_1	Digital clock configuration (second register)	0xE081

13.2.6.1 D_CLK_CFG_0 Settings

Bit Field	Field Name	Field Description
18	CLK_CFG0_SYSCLK_SEL	Select which of the oscillators to use
17:16	CLK_CFG0_EXTCLK_SEL	Select the behavior of the external clock pin
15:12	CLK_CFG0_EXTCLK_PRESCALE	Select a prescaling divisor for an external clock input
9:0	CLK_CFG0_HEARCLK_PRESCALE	Select a prescaling divisor (in steps of 1/8) for the accelerator clock

Field Name	Value Symbol	Value Description	Hex Value
CLK_CFG0_SYSCLK_SEL	SYSCLK_SEL_DEFAULT	Use the default internal oscillator	0x0*
	SYSCLK_SEL_STANDBY	Use the standby oscillator	0x1
CLK_CFG0_EXTCLK_SEL	EXTCLK_DISABLED	Disable the external clock	0x0*
	EXTCLK_OUT_SYSCLK	Enable the external clock as output (using SYSCLK as source)	0x1
	EXTCLK_OUT_SLOWCLK	Enable the external clock as output (using SLOWCLK as source)	0x2
	EXTCLK_IN_ENABLE	Enable the external clock as input	0x3

Field Name	Value Symbol	Value Description	Hex Value
CLK_CFG0_EXTCLK_PRESCALE	EXTCLK_PRESCALE_1	Divide input EXTCLK by 1	0x0*
	EXTCLK_PRESCALE_2	Divide input EXTCLK by 2	0x1
	EXTCLK_PRESCALE_3	Divide input EXTCLK by 3	0x2
	EXTCLK_PRESCALE_4	Divide input EXTCLK by 4	0x3
	EXTCLK_PRESCALE_5	Divide input EXTCLK by 5	0x4
	EXTCLK_PRESCALE_6	Divide input EXTCLK by 6	0x5
	EXTCLK_PRESCALE_7	Divide input EXTCLK by 7	0x6
	EXTCLK_PRESCALE_8	Divide input EXTCLK by 8	0x7
	EXTCLK_PRESCALE_9	Divide input EXTCLK by 9	0x8
	EXTCLK_PRESCALE_10	Divide input EXTCLK by 10	0x9
	EXTCLK_PRESCALE_11	Divide input EXTCLK by 11	0xA
	EXTCLK_PRESCALE_12	Divide input EXTCLK by 12	0xB
	EXTCLK_PRESCALE_13	Divide input EXTCLK by 13	0xC
	EXTCLK_PRESCALE_14	Divide input EXTCLK by 14	0xD
	EXTCLK_PRESCALE_15	Divide input EXTCLK by 15	0xE
	EXTCLK_PRESCALE_16	Divide input EXTCLK by 16	0xF
CLK_CFG0_HEARCLK_PRESCALE[1]	HEARCLK_PRESCALE_1_000	Set HEARCLK to SYSCLK/1	0x0*
	HEARCLK_PRESCALE_1_125	Set HEARCLK to SYSCLK/1.125	0x1
	HEARCLK_PRESCALE_1_250	Set HEARCLK to SYSCLK/1.25	0x2
	HEARCLK_PRESCALE_1_375	Set HEARCLK to SYSCLK/1.375	0x3
	HEARCLK_PRESCALE_1_500	Set HEARCLK to SYSCLK/1.5	0x4
	HEARCLK_PRESCALE_1_625	Set HEARCLK to SYSCLK/1.625	0x5
	HEARCLK_PRESCALE_1_750	Set HEARCLK to SYSCLK/1.75	0x6
	HEARCLK_PRESCALE_1_875	Set HEARCLK to SYSCLK/1.875	0x7
	HEARCLK_PRESCALE_2_000	Set HEARCLK to SYSCLK/2	0x8
	HEARCLK_PRESCALE_3_000	Set HEARCLK to SYSCLK/3	0x10
	HEARCLK_PRESCALE_4_000	Set HEARCLK to SYSCLK/4	0x18
	HEARCLK_PRESCALE_8_000	Set HEARCLK to SYSCLK/8	0x38
	HEARCLK_PRESCALE_16_000	Set HEARCLK to SYSCLK/16	0x78
	HEARCLK_PRESCALE_24_000	Set HEARCLK to SYSCLK/24	0xB8
	HEARCLK_PRESCALE_32_000	Set HEARCLK to SYSCLK/32	0xF8
	HEARCLK_PRESCALE_64_000	Set HEARCLK to SYSCLK/64	0x1F8
	HEARCLK_PRESCALE_128_000	Set HEARCLK to SYSCLK/128	0x3F8

1. A limited number of defines are provided for the CLK_CFG0_HEARCLK_PRESCALE bit field; however this field can take on any value in the range of 1 to 128.875, in steps of 1/8.

13.2.6.2 D_CLK_CFG_1 Settings

Bit Field	Field Name	Field Description
20	CLK_CFG1_DBL_ACCESS_MODE	Configure the device to share clock cycles between the accelerator clock and the CFX processor (double access mode)
19:8	CLK_CFG1_UCLK_PRESCALE	Select a prescaling divisor for user clock
5:0	CLK_CFG1_SLOWCLK_PRESCALE	Select a prescaling divisor for slow clock

Field Name	Value Symbol	Value Description	Hex Value
CLK_CFG1_DBL_ACCESS_MODE	DBL_ACCESS_MODE_DISABLE	Disable double access mode	0x0*
	DBL_ACCESS_MODE_ENABLE	Enable double access mode	0x1
CLK_CFG1_UCLK_PRESCALE[1]	UCLK_PRESCALE_1	Set UCLK to SYSCLK/1	0x0*
	UCLK_PRESCALE_2	Set UCLK to SYSCLK/2	0x1
	UCLK_PRESCALE_3	Set UCLK to SYSCLK/3	0x2
	UCLK_PRESCALE_4	Set UCLK to SYSCLK/4	0x3
	UCLK_PRESCALE_5	Set UCLK to SYSCLK/5	0x4
	UCLK_PRESCALE_6	Set UCLK to SYSCLK/6	0x5
	UCLK_PRESCALE_7	Set UCLK to SYSCLK/7	0x6
	UCLK_PRESCALE_8	Set UCLK to SYSCLK/8	0x7
	UCLK_PRESCALE_9	Set UCLK to SYSCLK/9	0x8
	UCLK_PRESCALE_10	Set UCLK to SYSCLK/10	0x9
	UCLK_PRESCALE_16	Set UCLK to SYSCLK/16	0xF
	UCLK_PRESCALE_32	Set UCLK to SYSCLK/32	0x1F
	UCLK_PRESCALE_64	Set UCLK to SYSCLK/64	0x3F
	UCLK_PRESCALE_128	Set UCLK to SYSCLK/128	0x7F
	UCLK_PRESCALE_256	Set UCLK to SYSCLK/256	0xFF
	UCLK_PRESCALE_512	Set UCLK to SYSCLK/512	0x1FF
	UCLK_PRESCALE_1024	Set UCLK to SYSCLK/1024	0x3FF
	UCLK_PRESCALE_2048	Set UCLK to SYSCLK/2048	0x7FF
	UCLK_PRESCALE_4096	Set UCLK to SYSCLK/4096	0xFFF

Field Name	Value Symbol	Value Description	Hex Value
CLK_CFG1_SLOWCLK_PRESCALE	SLOWCLK_PRESCALE_1	Set SLOWCLK to SYSCLK/1	0x0*
	SLOWCLK_PRESCALE_2	Set SLOWCLK to SYSCLK/2	0x1
	SLOWCLK_PRESCALE_3	Set SLOWCLK to SYSCLK/3	0x2
	SLOWCLK_PRESCALE_4	Set SLOWCLK to SYSCLK/4	0x3
	SLOWCLK_PRESCALE_5	Set SLOWCLK to SYSCLK/5	0x4
	SLOWCLK_PRESCALE_6	Set SLOWCLK to SYSCLK/6	0x5
	SLOWCLK_PRESCALE_7	Set SLOWCLK to SYSCLK/7	0x6
	SLOWCLK_PRESCALE_8	Set SLOWCLK to SYSCLK/8	0x7
	SLOWCLK_PRESCALE_9	Set SLOWCLK to SYSCLK/9	0x8
	SLOWCLK_PRESCALE_10	Set SLOWCLK to SYSCLK/10	0x9
	SLOWCLK_PRESCALE_11	Set SLOWCLK to SYSCLK/11	0xA
	SLOWCLK_PRESCALE_12	Set SLOWCLK to SYSCLK/12	0xB
	SLOWCLK_PRESCALE_13	Set SLOWCLK to SYSCLK/13	0xC
	SLOWCLK_PRESCALE_14	Set SLOWCLK to SYSCLK/14	0xD
	SLOWCLK_PRESCALE_15	Set SLOWCLK to SYSCLK/15	0xE
	SLOWCLK_PRESCALE_16	Set SLOWCLK to SYSCLK/16	0xF
	SLOWCLK_PRESCALE_17	Set SLOWCLK to SYSCLK/17	0x10
	SLOWCLK_PRESCALE_18	Set SLOWCLK to SYSCLK/18	0x11
	SLOWCLK_PRESCALE_19	Set SLOWCLK to SYSCLK/19	0x12
	SLOWCLK_PRESCALE_20	Set SLOWCLK to SYSCLK/20	0x13
	SLOWCLK_PRESCALE_21	Set SLOWCLK to SYSCLK/21	0x14
	SLOWCLK_PRESCALE_22	Set SLOWCLK to SYSCLK/22	0x15
	SLOWCLK_PRESCALE_23	Set SLOWCLK to SYSCLK/23	0x16
	SLOWCLK_PRESCALE_24	Set SLOWCLK to SYSCLK/24	0x17
	SLOWCLK_PRESCALE_25	Set SLOWCLK to SYSCLK/25	0x18
	SLOWCLK_PRESCALE_26	Set SLOWCLK to SYSCLK/26	0x19
	SLOWCLK_PRESCALE_27	Set SLOWCLK to SYSCLK/27	0x1A
	SLOWCLK_PRESCALE_28	Set SLOWCLK to SYSCLK/28	0x1B
	SLOWCLK_PRESCALE_29	Set SLOWCLK to SYSCLK/29	0x1C
	SLOWCLK_PRESCALE_30	Set SLOWCLK to SYSCLK/30	0x1D
	SLOWCLK_PRESCALE_31	Set SLOWCLK to SYSCLK/31	0x1E
	SLOWCLK_PRESCALE_32	Set SLOWCLK to SYSCLK/32	0x1F

Field Name	Value Symbol	Value Description	Hex Value
CLK_CFG1_SLOWCLK_PRESCALE (Continued)	SLOWCLK_PRESCALE_33	Set SLOWCLK to SYSCLK/33	0x20
	SLOWCLK_PRESCALE_34	Set SLOWCLK to SYSCLK/34	0x21
	SLOWCLK_PRESCALE_35	Set SLOWCLK to SYSCLK/35	0x22
	SLOWCLK_PRESCALE_36	Set SLOWCLK to SYSCLK/36	0x23
	SLOWCLK_PRESCALE_37	Set SLOWCLK to SYSCLK/37	0x24
	SLOWCLK_PRESCALE_38	Set SLOWCLK to SYSCLK/38	0x25
	SLOWCLK_PRESCALE_39	Set SLOWCLK to SYSCLK/39	0x26
	SLOWCLK_PRESCALE_40	Set SLOWCLK to SYSCLK/40	0x27
	SLOWCLK_PRESCALE_41	Set SLOWCLK to SYSCLK/41	0x28
	SLOWCLK_PRESCALE_42	Set SLOWCLK to SYSCLK/42	0x29
	SLOWCLK_PRESCALE_43	Set SLOWCLK to SYSCLK/43	0x2A
	SLOWCLK_PRESCALE_44	Set SLOWCLK to SYSCLK/44	0x2B
	SLOWCLK_PRESCALE_45	Set SLOWCLK to SYSCLK/45	0x2C
	SLOWCLK_PRESCALE_46	Set SLOWCLK to SYSCLK/46	0x2D
	SLOWCLK_PRESCALE_47	Set SLOWCLK to SYSCLK/47	0x2E
	SLOWCLK_PRESCALE_48	Set SLOWCLK to SYSCLK/48	0x2F
	SLOWCLK_PRESCALE_49	Set SLOWCLK to SYSCLK/49	0x30
	SLOWCLK_PRESCALE_50	Set SLOWCLK to SYSCLK/50	0x31
	SLOWCLK_PRESCALE_51	Set SLOWCLK to SYSCLK/51	0x32
	SLOWCLK_PRESCALE_52	Set SLOWCLK to SYSCLK/52	0x33
	SLOWCLK_PRESCALE_53	Set SLOWCLK to SYSCLK/53	0x34
	SLOWCLK_PRESCALE_54	Set SLOWCLK to SYSCLK/54	0x35
	SLOWCLK_PRESCALE_55	Set SLOWCLK to SYSCLK/55	0x36
	SLOWCLK_PRESCALE_56	Set SLOWCLK to SYSCLK/56	0x37
	SLOWCLK_PRESCALE_57	Set SLOWCLK to SYSCLK/57	0x38
	SLOWCLK_PRESCALE_58	Set SLOWCLK to SYSCLK/58	0x39
	SLOWCLK_PRESCALE_59	Set SLOWCLK to SYSCLK/59	0x3A
	SLOWCLK_PRESCALE_60	Set SLOWCLK to SYSCLK/60	0x3B
	SLOWCLK_PRESCALE_61	Set SLOWCLK to SYSCLK/61	0x3C
	SLOWCLK_PRESCALE_62	Set SLOWCLK to SYSCLK/62	0x3D
	SLOWCLK_PRESCALE_63	Set SLOWCLK to SYSCLK/63	0x3E
	SLOWCLK_PRESCALE_64	Set SLOWCLK to SYSCLK/64	0x3F

1. A limited number of defines have been provided for the CLK_CFG1_UCLK_PRESCALE bit field, however this field may take on any value in the range of 1 to 4096

14 Power Components

14.1 Power Supply

The power supply is a critical component of the BelaSigna 300 system, as system performance, particularly audio performance, is greatly affected by the supplied power.

The power supply includes:

- A regulated, low-noise power source for all analog components
- A regulated, noise-robust voltage source to be used for microphones
- A configurable regulated supply for all digital components
- An on-chip charge pump

Additionally the power supply also interfaces with components that support an ultra-low power soft power-down mode and power management to ensure safe system operation. See Section 14.4, “Soft Power Down Mode Tools” on page 178 and Section 14.2, “Power Management Unit” on page 176 for further details.

Figure 23 shows a block diagram of the main components of the power supply.The following sections describe each of the voltages used or provided by the power supply to ensure proper system operation.

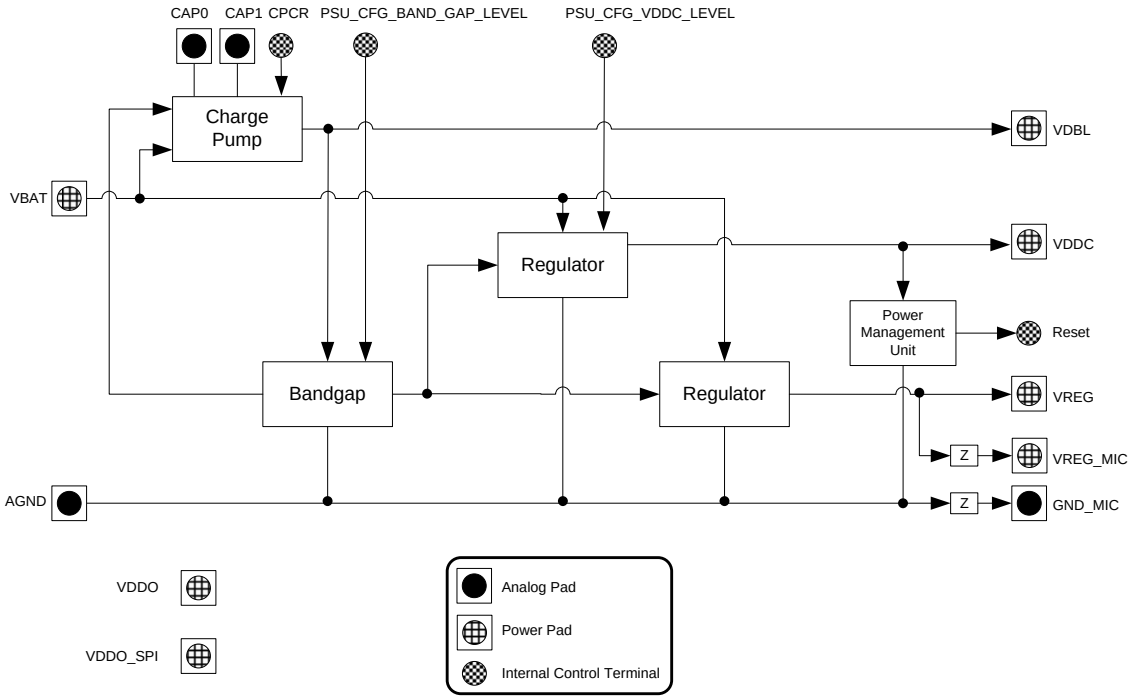


Figure 23: Power Supply Block Diagram

14.1.1 Battery Supply Voltage (VBAT)

The primary voltage supplied to a BelaSigna 300 device is known as the battery supply voltage because in a typical application, this voltage is supplied by a battery. As a result the nominal voltage level for VBAT should be 1.8 V.

14.1.2 Digital Output Supply Voltages (VDDO and VDDO_SPI)

The digital I/O supply is attached to all of the digital I/O pads on a BelaSigna 300 device. The internal system digital logic is attached to the pads through internal level translators that shift the voltage level from VDDC to VDDO. A separate digital output supply, VDDO_SPI, provides the voltage level for the SPI interface.

Devices with CID 201 connect the VDDO and VDDO_SPI signals to VBAT internally and devices with CID 102 should generally connect the VDDO and VDDO_SPI inputs externally to VBAT for proper functionality. These configurations allow all of the digital I/O pads of the BelaSigna 300 device to operate at a level appropriate for communications with external devices, including devices such as an EEPROM or other non-volatile memory storage.

14.1.3 Internal Band Gap Reference Voltage

The band gap reference voltage is a voltage that has been stabilized over temperature and process variations. This reference voltage is used in the generation of all of the regulated voltages

in the BelaSigna 300 system and provides a nominal 1V reference signal to all components using the reference voltage.

The reference voltage is configurable through the PSU_CFG_BAND_GAP_LEVEL bit field in the A_PSU_CFG register. This register allows you to specify any reference voltage between 0.90V and 1.20V in 20mV increments. The default reference voltage is 1.00V (corresponding to a setting of BAND_GAP_LEVEL_100_VOLT for the PSU_CFG_BAND_GAP_LEVEL bit field).

CAUTION: All BelaSigna 300 datasheet results have been generated using a band gap reference configuration of BAND_GAP_LEVEL_100_VOLT. All other configurations are not explicitly supported by ON Semiconductor. Only set this register if you are also configuring the internal digital supply voltage (see Section 14.1.4, "Internal Digital Supply Voltage (VDDC)").

14.1.4 Internal Digital Supply Voltage (VDDC)

The internal digital supply voltage is used as the supply voltage for all internal digital components, including being used as the interface voltage at the low side of the level translation circuitry attached to all of the external digital pads. VDDC is also provided as an output pad.

The internal digital supply voltage is supplied through an internal configurable voltage regulator. This regulator supplies a voltage that meets the regulated voltage specifications from the BelaSigna 300 datasheet. The output voltage from this regulator is configurable through the PSU_CFG_VDDC_LEVEL bit field in the A_PSU_CFG register. This register allows you to specify a range of output voltages from 0.75V and 1.11V using the defines provided in Section 14.1.8.1, "A_PSU_CFG Settings" on page 174.

IMPORTANT: The VDDC voltage level limits the range of the externally supplied clock for BelaSigna 300. The default setting for PSU_CFG_VDDC_LEVEL (VDDC_LEVEL_111_VOLT) was used to calculate the supply clock limitations provided in the BelaSigna 300 datasheet. We recommend that you consult with ON Semiconductor before changing this configuration to avoid configurations that are not compatible with your supplied clock.

14.1.5 Regulated Supply Voltage (VREG) and Regulated Output Supply Voltage (VREG_MIC)

The BelaSigna 300 device supplies a regulated reference supply voltage at the VREG pad to allow for additional external filtering of the regulated voltages within the system.

The BelaSigna 300 system also provides a regulated supply voltage to be used as a noise-robust supply for an external microphone. This voltage is available at the VREG_MIC output pad. Along with this voltage supply pad, an associated noise-robust ground pad has been supplied as GND_MIC to connect to the ground side of an external microphone. This supply voltage meets the regulated voltage specifications from the BelaSigna 300 datasheet.

14.1.6 Regulated Doubled Supply Voltage (VDBL)

The doubled supply voltage makes use of the internal charge pump to generate a regulated 2V signal that is used for all of the analog audio components. In particular this is used for the input stage of the system.

The internal charge pump uses an external capacitor that is periodically refreshed to maintain the 2V supply. The charge pump refresh frequency is derived from slow clock which assists the input stage in filtering out any noise generated by the dynamic current draw on this supply voltage.

The regulated doubled supply voltage is available at the VDBL output pad. This supply voltage meets the regulated voltage specifications from the BelaSigna 300 datasheet when the system is supplied with a VBAT supply voltage of 1.25V.

14.1.7 Output Receiver Supply Voltage (VBATRCVR)

To allow maximum flexibility, the direct-digital output driver in the output stage is powered from the VBATRCVR pad. Common configurations include connecting this pad externally to VBAT or VDBL based on your system needs.

The sigma-delta modulator in the output stage is a digital system component powered by VDDC (see Section 14.1.4, “Internal Digital Supply Voltage (VDDC)”).

14.1.8 Control and Configuration Registers

Register Name	Register Description	Address
A_PSU_CFG	Regulator output voltage configuration	0xE102

14.1.8.1 A_PSU_CFG Settings

Bit Field	Field Name	Field Description
7:4	PSU_CFG_BAND_GAP_LEVEL	Select the band gap reference voltage
3:0	PSU_CFG_VDDC_LEVEL	Select the output voltage

Field Name	Value Symbol	Value Description	Hex Value
PSU_CFG_BAND_GAP_LEVEL	BAND_GAP_LEVEL_090_VOLT	Set the band gap reference voltage to 0.90V	0x0
	BAND_GAP_LEVEL_092_VOLT	Set the band gap reference voltage to 0.92V	0x1
	BAND_GAP_LEVEL_094_VOLT	Set the band gap reference voltage to 0.94V	0x2
	BAND_GAP_LEVEL_096_VOLT	Set the band gap reference voltage to 0.96V	0x3
	BAND_GAP_LEVEL_098_VOLT	Set the band gap reference voltage to 0.98V	0x4
	BAND_GAP_LEVEL_100_VOLT	Set the band gap reference voltage to 1.00V	0x5*
	BAND_GAP_LEVEL_102_VOLT	Set the band gap reference voltage to 1.02V	0x6
	BAND_GAP_LEVEL_104_VOLT	Set the band gap reference voltage to 1.04V	0x7
	BAND_GAP_LEVEL_106_VOLT	Set the band gap reference voltage to 1.06V	0x8
	BAND_GAP_LEVEL_108_VOLT	Set the band gap reference voltage to 1.08V	0x9
	BAND_GAP_LEVEL_110_VOLT	Set the band gap reference voltage to 1.10V	0xA
	BAND_GAP_LEVEL_112_VOLT	Set the band gap reference voltage to 1.12V	0xB
	BAND_GAP_LEVEL_114_VOLT	Set the band gap reference voltage to 1.14V	0xC
	BAND_GAP_LEVEL_116_VOLT	Set the band gap reference voltage to 1.16V	0xD
	BAND_GAP_LEVEL_118_VOLT	Set the band gap reference voltage to 1.18V	0xE
	BAND_GAP_LEVEL_120_VOLT	Set the band gap reference voltage to 1.20V	0xF

Field Name	Value Symbol	Value Description	Hex Value
PSU_CFG_VDDC_LEVEL	VDDC_LEVEL_075_VOLT	Set the internal digital supply voltage to 0.75V	0x0
	VDDC_LEVEL_077_VOLT	Set the internal digital supply voltage to 0.77V	0x1
	VDDC_LEVEL_080_VOLT	Set the internal digital supply voltage to 0.80V	0x2
	VDDC_LEVEL_082_VOLT	Set the internal digital supply voltage to 0.82V	0x3
	VDDC_LEVEL_084_VOLT	Set the internal digital supply voltage to 0.84V	0x4
	VDDC_LEVEL_087_VOLT	Set the internal digital supply voltage to 0.87V	0x5
	VDDC_LEVEL_090_VOLT	Set the internal digital supply voltage to 0.90V	0x6
	VDDC_LEVEL_092_VOLT	Set the internal digital supply voltage to 0.92V	0x7
	VDDC_LEVEL_094_VOLT	Set the internal digital supply voltage to 0.94V	0x8
	VDDC_LEVEL_097_VOLT	Set the internal digital supply voltage to 0.97V	0x9
	VDDC_LEVEL_099_VOLT	Set the internal digital supply voltage to 0.99V	0xA
	VDDC_LEVEL_101_VOLT	Set the internal digital supply voltage to 1.01V	0xB
	VDDC_LEVEL_104_VOLT	Set the internal digital supply voltage to 1.04V	0xC
	VDDC_LEVEL_106_VOLT	Set the internal digital supply voltage to 1.06V	0xD
	VDDC_LEVEL_109_VOLT	Set the internal digital supply voltage to 1.09V	0xE
	VDDC_LEVEL_111_VOLT	Set the internal digital supply voltage to 1.11V	0xF*

14.2 POWER MANAGEMENT UNIT

The power management unit consists primarily of two components:

- A power supervisory circuit to ensure safe system operation under any battery condition
- A power-on reset component that ensures startup behavior and configuration

These components work together to ensure proper system behavior under all battery conditions. They also ensure that the system startup sequence behaves correctly as voltage fluctuations related to system startup might otherwise result in an indeterminate system state. Neither

component is user configurable. Further information on each of these components is provided in the following sections.

14.2.1 Power Supervisory

The power supervisory circuitry on an BelaSigna 300 device monitors both the battery supply voltage (VBAT) and the internal digital supply voltage (VDDC). This circuit is used to start the system when VBAT reaches a safe startup voltage, and to reset the system when either of the VBAT or VDDC voltages drops below a relevant voltage threshold. The relevant threshold voltages are supplied in Table 34.

TABLE 34: POWER SUPERVISORY VOLTAGE THRESHOLDS

Threshold	Voltage Level
VBAT Monitor Startup	0.70V
VBAT Startup	$0.82\text{V} \pm 50\text{mV}$
VBAT and VDDC Shutdown	$0.80\text{V} \pm 50\text{mV}$

When power is first supplied to the system the power supervisory circuitry starts to monitor the battery supply voltage at the VBAT monitor startup threshold. As the voltage continues to rise past the startup voltage threshold the system will start up using the power-on-reset module (see Section 14.2.2, “Power-On-Reset (POR)”) to ensure proper system startup behavior.

If either of VBAT or VDDC falls below their respective shutdown voltage threshold due to a voltage transient or dying battery the system will go into shutdown mode. In this mode the power supervisory circuitry will hold the system components in a reset state until such a time that the voltage recovers and the value of VBAT passes by the startup voltage threshold again.

Note: The voltage difference between the startup and shutdown voltage thresholds is used to prevent repeated system resets due to voltage oscillation around the shutdown voltage threshold.

14.2.2 Power-On-Reset (POR)

The BelaSigna 300 device uses a power-on-reset (POR) sequence to ensure proper system behavior during startup and proper system configuration after startup.

At the start of the POR sequence, audio output is disabled and all configuration and control registers are asynchronously reset to their default values (as specified in this manual). All CFX DSP registers are cleared and the contents of all RAM is unspecified at this point.

The POR sequence consists of two components: Voltage supply stabilization and boot ROM initialization. During the voltage supply stabilization component, the following steps are performed:

1. The internal regulators are enabled and allowed to stabilize.
2. The internal charge pump is enabled and allowed to stabilize.
3. SYSCLK is connected to all of the system components (as per normal system operation).

The boot ROM initialization component is then run. After approximately 210ms, the POR sequence is complete and the device will attempt to load data from the SPI interface (as there is generally an attached EEPROM that includes configuration and application information for the system attached to that interface).

As SYSCLK is connected to EXTCLK at the end of the boot ROM initialization sequence, an input clock signal must be provided to the external oscillator before the POR sequence will complete.

14.3 BATTERY MONITOR

The BelaSigna 300 device includes a battery monitor that can be used to signal a user application that the VBAT supply voltage has fallen below a specified voltage threshold.

To specify the value of this threshold voltage, set the value of the A_SUP_THRSH_DATA register. This register allows you to specify any voltage threshold between 0V and 2V in 7.8mV steps. The default voltage threshold is 1.0V, corresponding to a setting of 0x7F for the A_SUP_THRSH_DATA register.

If the supplied VBAT voltage stays below the threshold voltage, the battery monitor increments at an interval determined by the speed of the LSAD interface (and hence this interval is indirectly dependant on SLOWCLK as well). Under default system settings the update interval is 80ms and the exact battery monitor update frequency can be calculated using:

$$f_{\text{Battery Monitor}} = \frac{1}{1024} \times \left[\frac{f_{\text{SLOWCLK}}}{20 \times (\text{LSAD_CTRL_FREQ} + 1)} \right]$$

CAUTION: When operating in soft power down mode with the analog components disabled the battery monitor is also disabled because it depends on the LSAD components being active.

The count of the number of times that the supply voltage has dropped below the specified voltage threshold is provided through the A_SUP_COUNT_DATA register. This read-only register provides the current battery monitor count.

IMPORTANT: A_SUP_COUNT_DATA is reset when it is read.

14.3.1 Control and Configuration Registers

Register Name	Register Description	Address
A_SUP_THRSH_DATA	Set the battery monitor voltage threshold	0xE190
A_SUP_COUNT_DATA	Number of times the voltage has dropped below the battery monitor threshold	0xE191

14.4 SOFT POWER DOWN MODE TOOLS

The BelaSigna 300 device provides an ultra-low power mode where the CFX DSP, HEAR and other digital components of the system are still active but are being clocked from a SYSCLK that is using the standby clock and the analog circuitry is disabled. Since the CFX is still active, an event on the

GPIO interface or another system element can trigger a reactivation of the system from soft power down mode.

To reduce the power consumed during soft power down, the analog components of the BelaSigna 300 device may be disconnected from their voltage supplies. To disable power to these components, set the ANALOG_PWR_CTRL_ANALOG_DISABLE bit in the ANALOG_PWR_CTRL register.

As the digital interfaces are still operative in soft power down mode, disconnect system elements such as the IOC from the input and output stages when in soft power down mode to prevent unexpected behavior upon restoration of the normal clock. Also note that the I^2C debug interface and any other interface that is not explicitly disabled remains active, although they cannot operate at their typical communication frequencies because their interface clocks are scaled down proportionally with SYSCLK.

Use the following sequence of tasks to enter soft power down mode:

1. Switch the source for SYSCLK to the standby clock. For more information on changing the source of SYSCLK, see Section 13.2.1, “System Clock (SYSCLK)” on page 164.
2. Disable the analog blocks from the input and output stages by setting the ANALOG_PWR_CTRL_ANALOG_DISABLE bit in the ANALOG_PWR_CTRL control register as described above.

IMPORTANT: You must follow the above order because the power supplied to the primary oscillator is disabled along with the rest of the analog circuitry and cannot be disabled unless the system is operating using the standby clock. For similar reasons, when re-enabling the system in response to an event, enable the analog circuitry before restoring the normal SYSCLK.

14.4.1 Control and Configuration Registers

Register Name	Register Description	Address
A_ANALOG_PWR_CTRL	Enable/disable power supply to the analog portions of the system	0xE104

14.4.1.1 A_ANALOG_PWR_CTRL Settings

Bit Field	Field Name	Field Description
0	ANALOG_PWR_CTRL_ANALOG_DISABLE	Enable/disable power supplied to the analog components

Field Name	Value Symbol	Value Description	Hex Value
ANALOG_PWR_CTRL_ANALOG_DISABLE	ANALOG_ENABLE	Enable power to the analog components	0x0*
	ANALOG_DISABLE	Disable power to the analog components	0x1

This page left blank for printing purposes.

A Control and Configuration Registers

Refer to the appropriate following section for information on the control and configuration registers for a block. The sections are:

- For configuration and control of analog blocks, see Section A.1, “Analog Block Registers” on page 182.
- For configuration and control of general digital blocks, see Section A.2, “Digital Block Registers” on page 185.
- For configuration and control of the HEAR digital block, see Section A.3, “HEAR Digital Block Registers” on page 194.
- For configuration and control of the FIFOs, see Section A.4, “FIFO Configuration Registers” on page 197.

A.1 ANALOG BLOCK REGISTERS

Address	Register Name	Register Write	Register Read	Default	Description
P:0xE100	A_CLK_CTRL	(11:8) : CLK_CTRL_SYSCLK	(11:8) : CLK_CTRL_SYSCLK	0x4	Select the primary oscillator coarse setting
		(7:0) : CLK_CTRL_CCR_DATA	(7:0) : CLK_CTRL_CCR_DATA	0x80	Select the primary oscillator fine calibration setting
P:0xE102	A_PSU_CFG	(7:4): PSU_CFG_BAND_GAP_LEVEL	(7:4): PSU_CFG_BAND_GAP_LEVEL	0x5	Select the band gap reference voltage
		(3:0): PSU_CFG_VDDC_LEVEL	(3:0): PSU_CFG_VDDC_LEVEL	0xF	Select the output voltage
P:0xE104	A_ANALOG_PWR_CTRL	0 : ANALOG_PWR_CTRL_ANALOG_ DISABLE	0 : ANALOG_PWR_CTRL_ANALOG_ DISABLE	0x0	Enable/disable the analog portions of the system
P:0xE140	A_ADC_CFG	(22:21) : ADC_CFG_ADC3_DC_REMOVE	(22:21) : ADC_CFG_ADC3_DC_REMOVE	0x2	Select the cut-off for the ADC 3 DC removal filter
		(20:19) : ADC_CFG_ADC2_DC_REMOVE	(20:19) : ADC_CFG_ADC2_DC_REMOVE	0x2	Select the cut-off for the ADC 2 DC removal filter
		(18:17) : ADC_CFG_ADC1_DC_REMOVE	(18:17) : ADC_CFG_ADC1_DC_REMOVE	0x2	Select the cut-off for the ADC 1 DC removal filter
		(16:15) : ADC_CFG_ADC0_DC_REMOVE	(16:15) : ADC_CFG_ADC0_DC_REMOVE	0x2	Select the cut-off for the ADC 0 DC removal filter
		14 : ADC_CFG_HIGH_FREQ	14 : ADC_CFG_HIGH_FREQ	0x0	Select high or low frequency mode
		13 : ADC_CFG_LPF3_ENABLE	13 : ADC_CFG_LPF3_ENABLE	0x1	Enable/disable low-pass filter for channel 3
		12 : ADC_CFG_LPF2_ENABLE	12 : ADC_CFG_LPF2_ENABLE	0x1	Enable/disable low-pass filter for channel 2
		11 : ADC_CFG_LPF1_ENABLE	11 : ADC_CFG_LPF1_ENABLE	0x1	Enable/disable low-pass filter for channel 1
		10 : ADC_CFG_LPF0_ENABLE	10 : ADC_CFG_LPF0_ENABLE	0x1	Enable/disable low-pass filter for channel 0
		9 : ADC_CFG_LINEOUT3_ENABLE	9 : ADC_CFG_LINEOUT3_ENABLE	0x0	Enable/disable line-out 3
		8 : ADC_CFG_LINEOUT2_ENABLE	8 : ADC_CFG_LINEOUT2_ENABLE	0x0	Enable/disable line-out 2
		7 : ADC_CFG_LINEOUT1_ENABLE	7 : ADC_CFG_LINEOUT1_ENABLE	0x0	Enable/disable line-out 1
		6 : ADC_CFG_LINEOUT0_ENABLE	6 : ADC_CFG_LINEOUT0_ENABLE	0x0	Enable/disable line-out 0
		(5:0) : ADC_CFG_SAMPLE_FREQ	(5:0) : ADC_CFG_SAMPLE_FREQ	0x2	Configure the system sampling frequency
P:0xE141	A_INPUT_CTRL	15 : INPUT_CTRL_IDRX1_ENABLE	15 : INPUT_CTRL_IDRX1_ENABLE	0x0	Enable dynamic range extended input 1 (i.e. enable IDRX1)
		(14:13) : INPUT_CTRL_ADC3_IN_SEL	(14:13) : INPUT_CTRL_ADC3_IN_SEL	0x3	Select the input pad for channel 3
		12 : INPUT_CTRL_ADC3_ENABLE	12 : INPUT_CTRL_ADC3_ENABLE	0x0	Enable channel 3 (i.e. enable ADC3)
		(10:9) : INPUT_CTRL_ADC2_IN_SEL	(10:9) : INPUT_CTRL_ADC2_IN_SEL	0x2	Select the input pad for channel 2
		8 : INPUT_CTRL_ADC2_ENABLE	8 : INPUT_CTRL_ADC2_ENABLE	0x0	Enable channel 2 (i.e. enable ADC2)
		7 : INPUT_CTRL_IDRX0_ENABLE	7 : INPUT_CTRL_IDRX0_ENABLE	0x0	Enable dynamic range extended input 0 (i.e. enable IDRX0)
		(6:5) : INPUT_CTRL_ADC1_IN_SEL	(6:5) : INPUT_CTRL_ADC1_IN_SEL	0x1	Select the input pad for channel 1
		4 : INPUT_CTRL_ADC1_ENABLE	4 : INPUT_CTRL_ADC1_ENABLE	0x0	Enable channel 1 (i.e. enable ADC1)
		(2:1) : INPUT_CTRL_ADC0_IN_SEL	(2:1) : INPUT_CTRL_ADC0_IN_SEL	0x0	Select the input pad for channel 0
		0 : INPUT_CTRL_ADC0_ENABLE	0 : INPUT_CTRL_ADC0_ENABLE	0x0	Enable channel 0 (i.e. enable ADC0)

Address	Register Name	Register Write	Register Read	Default	Description
P:0xE142	A_IN_GAIN_CTRL	(14:12) : IN_GAIN_PGAIN3	(14:12) : IN_GAIN_PGAIN3	0x0	Select the preamplifier gain for channel 3
		(10:8) : IN_GAIN_PGAIN2	(10:8) : IN_GAIN_PGAIN2	0x0	Select the preamplifier gain for channel 2
		(6:4) : IN_GAIN_PGAIN1	(6:4) : IN_GAIN_PGAIN1	0x0	Select the preamplifier gain for channel 1
		(2:0) : IN_GAIN_PGAIN0	(2:0) : IN_GAIN_PGAIN0	0x0	Select the preamplifier gain for channel 0
P:0xE144	A_ADC_CUR_CTRL	(14:12) : ADC_CUR_CTRL_ADC3_CURRENT	(14:12) : ADC_CUR_CTRL_ADC3_CURRENT	0x3	Select the current supplied to ADC 3
		(10:8) : ADC_CUR_CTRL_ADC2_CURRENT	(10:8) : ADC_CUR_CTRL_ADC2_CURRENT	0x3	Select the current supplied to ADC 2
		(6:4) : ADC_CUR_CTRL_ADC1_CURRENT	(6:4) : ADC_CUR_CTRL_ADC1_CURRENT	0x3	Select the current supplied to ADC 1
		(2:0) : ADC_CUR_CTRL_ADC0_CURRENT	(2:0) : ADC_CUR_CTRL_ADC0_CURRENT	0x3	Select the current supplied to ADC 0
P:0xE145	A_ADC_GF01_CTRL	(23:12) : ADC_GF_CTRL_ADC1_GF	(23:12) : ADC_GF_CTRL_ADC1_GF	0x665	Select the ADC gain factor for channel 1
		(11:0) : ADC_GF_CTRL_ADC0_GF	(11:0) : ADC_GF_CTRL_ADC0_GF	0x665	Select the ADC gain factor for channel 0
P:0xE146	A_ADC_GF23_CTRL	(23:12) : ADC_GF_CTRL_ADC3_GF	(23:12) : ADC_GF_CTRL_ADC3_GF	0x665	Select the ADC gain factor for channel 3
		(11:0) : ADC_GF_CTRL_ADC2_GF	(11:0) : ADC_GF_CTRL_ADC2_GF	0x665	Select the ADC gain factor for channel 2
P:0xE147	A_FRAC_DELAY_0_CTRL	(10:8) : FRAC_DELAY_INTEGER	(10:8) : FRAC_DELAY_INTEGER	0x0	Select the coarse fractional delay for channel 0
		(5:0) : FRAC_DELAY_FRACTIONAL	(5:0) : FRAC_DELAY_FRACTIONAL	0x0	Select the fine fractional delay for channel 0
P:0xE148	A_FRAC_DELAY_1_CTRL	(10:8) : FRAC_DELAY_INTEGER	(10:8) : FRAC_DELAY_INTEGER	0x0	Select the coarse fractional delay for channel 1
		(5:0) : FRAC_DELAY_FRACTIONAL	(5:0) : FRAC_DELAY_FRACTIONAL	0x0	Select the fine fractional delay for channel 1
P:0xE149	A_FRAC_DELAY_2_CTRL	(10:8) : FRAC_DELAY_INTEGER	(10:8) : FRAC_DELAY_INTEGER	0x0	Select the coarse fractional delay for channel 2
		(5:0) : FRAC_DELAY_FRACTIONAL	(5:0) : FRAC_DELAY_FRACTIONAL	0x0	Select the fine fractional delay for channel 2
P:0xE14A	A_FRAC_DELAY_3_CTRL	(10:8) : FRAC_DELAY_INTEGER	(10:8) : FRAC_DELAY_INTEGER	0x0	Select the coarse fractional delay for channel 3
		(5:0) : FRAC_DELAY_FRACTIONAL	(5:0) : FRAC_DELAY_FRACTIONAL	0x0	Select the fine fractional delay for channel 3
P:0xE14B	A_BAND_SELECT_CTRL	(6:4) : BAND_SELECT_ADC3	(6:4) : BAND_SELECT_ADC3	0x0	Select the passband of filtered signal on ADC 3
		(2:0) : BAND_SELECT_ADC2	(2:0) : BAND_SELECT_ADC2	0x0	Select the passband of filtered signal on ADC 2
P:0xE14C	A_IDRX0_CFG	-	23 : IDRX_CFG_IDRX_SEL	-	Indicator of active ADC in the combined channel
		22 : IDRX_CFG_ADC_OFF_EN	22 : IDRX_CFG_ADC_OFF_EN	0x0	Allow the inactive ADC to be disabled
		(21:20) : IDRX_CFG_MAX_PG	(21:20) : IDRX_CFG_MAX_PG	0x3	Select the maximum available preamp gain
		(17:16) : IDRX_CFG_MIN_PG	(17:16) : IDRX_CFG_MIN_PG	0x0	Select the minimum available preamp gain
		(15:12) : IDRX_CFG_HIGH_LIMIT	(15:12) : IDRX_CFG_HIGH_LIMIT	0x7	Set the high signal limit for triggering channel changes
		(11:8) : IDRX_CFG_LOW_LIMIT	(11:8) : IDRX_CFG_LOW_LIMIT	0x7	Set the low signal limit for triggering channel changes
		(7:4) : IDRX_CFG_TIMER2	(7:4) : IDRX_CFG_TIMER2	0x3	Select a timeout period for IDRX timer 2
		(3:0) : IDRX_CFG_TIMER1	(3:0) : IDRX_CFG_TIMER1	0x3	Select a timeout period for IDRX timer 1

Address	Register Name	Register Write	Register Read	Default	Description
P:0xE14D	A_IDRX1_CFG	-	23 : IDRX_CFG_IDRX_SEL	-	Indicator of active ADC in the combined channel
		22 : IDRX_CFG_ADC_OFF_EN	22 : IDRX_CFG_ADC_OFF_EN	0x0	Allow the inactive ADC to be disabled
		(21:20) : IDRX_CFG_MAX_PG	(21:20) : IDRX_CFG_MAX_PG	0x3	Select the maximum available preamp gain
		(17:16) : IDRX_CFG_MIN_PG	(17:16) : IDRX_CFG_MIN_PG	0x0	Select the minimum available preamp gain
		(15:12) : IDRX_CFG_HIGH_LIMIT	(15:12) : IDRX_CFG_HIGH_LIMIT	0x7	Set the high signal limit for triggering channel changes
		(11:8) : IDRX_CFG_LOW_LIMIT	(11:8) : IDRX_CFG_LOW_LIMIT	0x7	Set the low signal limit for triggering channel changes
		(7:4) : IDRX_CFG_TIMER2	(7:4) : IDRX_CFG_TIMER2	0x3	Select a timeout period for IDRX timer 2
		(3:0) : IDRX_CFG_TIMER1	(3:0) : IDRX_CFG_TIMER1	0x3	Select a timeout period for IDRX timer 1
P:0xE150	A_OUTPUT_CTRL	2 : OUTPUT_CTRL_OD0_HIGHPOWER	2 : OUTPUT_CTRL_OD0_HIGHPOWER	0x0	Enable/disable high-power output mode
		0 : OUTPUT_CTRL_OD0_ENABLE	0 : OUTPUT_CTRL_OD0_ENABLE	0x0	Enable/disable the output
P:0xE151	A_O_DAC_CTRL	(5:0) : O_DAC_CTRL_OSRDAC	(5:0) : O_DAC_CTRL_OSRDAC	0x0	Prescaler for the system clock to achieve the desired oversampling ratio
P:0xE152	A_O_DAC_GF_CTRL	(17:16) : O_DAC_GF_CTRL	(17:16) : O_DAC_GF_CTRL	0x0	Configure when the output DAC gain factor is used
		(11:0) : O_DAC_GF	(11:0) : O_DAC_GF	0x400	Select the DAC gain factor
P:0xE180	A_LSAD_CTRL	11 : LSAD_CTRL_VREG_MODE	11 : LSAD_CTRL_VREG_MODE	0x0	Select the LSAD mode for the VREG monitor
		10 : LSAD_CTRL_VBAT_MODE	10 : LSAD_CTRL_VBAT_MODE	0x0	Select the LSAD mode for the VBAT monitor
		9 : LSAD_CTRL_VDDC_MODE	9 : LSAD_CTRL_VDDC_MODE	0x0	Select the LSAD mode for the VDDC monitor
		8 : LSAD_CTRL_CH4_MODE	8 : LSAD_CTRL_CH4_MODE	0x0	Select the LSAD mode for channel 4
		7 : LSAD_CTRL_CH3_MODE	7 : LSAD_CTRL_CH3_MODE	0x0	Select the LSAD mode for channel 3
		6 : LSAD_CTRL_CH2_MODE	6 : LSAD_CTRL_CH2_MODE	0x0	Select the LSAD mode for channel 2
		5 : LSAD_CTRL_CH1_MODE	5 : LSAD_CTRL_CH1_MODE	0x0	Select the LSAD mode for channel 1
		4 : LSAD_CTRL_CH0_MODE	4 : LSAD_CTRL_CH0_MODE	0x0	Select the LSAD mode for channel 0
		(3:0) : LSAD_CTRL_FREQ	(3:0) : LSAD_CTRL_FREQ	0x4	Configure the LSAD sampling frequency
P:0xE181	A_LSAD_CH0_DATA	-	(9:0) : A_LSAD_CH0_DATA	0x0	Value read from LSAD channel 0
P:0xE182	A_LSAD_CH1_DATA	-	(9:0) : A_LSAD_CH1_DATA	0x0	Value read from LSAD channel 1
P:0xE183	A_LSAD_CH2_DATA	-	(9:0) : A_LSAD_CH2_DATA	0x0	Value read from LSAD channel 2
P:0xE184	A_LSAD_CH3_DATA	-	(9:0) : A_LSAD_CH3_DATA	0x0	Value read from LSAD channel 3
P:0xE185	A_LSAD_CH4_DATA	-	(9:0) : A_LSAD_CH4_DATA	0x0	Value read from LSAD channel 4
P:0xE186	A_LSAD_VDDC_DATA	-	(9:0) : A_LSAD_VDDC_DATA	0x0	Value read from the VDDC LSAD channel
P:0xE187	A_LSAD_VBAT_DATA	-	(9:0) : A_LSAD_VBAT_DATA	0x0	Value read from the VBAT LSAD channel
P:0xE188	A_LSAD_VREG_DATA	-	(9:0) : A_LSAD_VREG_DATA	0x0	Value read from the VREG LSAD channel
P:0xE190	A_SUP_THRSH_DATA	(7:0) : A_SUP_THRSH_DATA	(7:0) : A_SUP_THRSH_DATA	0x7F	Set the battery monitor voltage threshold
P:0xE191	A_SUP_COUNT_DATA	-	(7:0) : A_SUP_COUNT_DATA	0x0	Number of voltage drops below the battery monitor voltage threshold

A.2 DIGITAL BLOCK REGISTERS

Address	Register Name	Register Write	Register Read	Default	Description
P:0xE000	D_SYSTEM_CTRL	7 : SYSTEM_CTRL_PRIORITY_HEAR	-	-	Give the HEAR priority
		6 : SYSTEM_CTRL_PRIORITY_CFX	6 : SYSTEM_CTRL_PRIORITY_CFX	0x0	Give the CFX priority, check if CFX has priority
		5 : SYSTEM_CTRL_HEAR_PAUSE	-	-	Pause processing on the HEAR
		4 : SYSTEM_CTRL_HEAR_RESUME	4 : SYSTEM_CTRL_HEAR_RUNNING	0x0	Resume/check status of processing on the HEAR
		3 : SYSTEM_CTRL_HEAR_RESET	-	-	Reset the HEAR
		2 : SYSTEM_CTRL_TIMER_2_START	2 : SYSTEM_CTRL_TIMER_2_BUSY	0x0	Start/check status of timer 2
		1 : SYSTEM_CTRL_TIMER_1_START	1 : SYSTEM_CTRL_TIMER_1_BUSY	0x0	Start/check status of timer 1
		0 : SYSTEM_CTRL_WATCHDOG_REFRESH	-	-	Refresh the watchdog timer
P:0xE001	D_WATCHDOG_CFG	(3:0) : WATCHDOG_CFG_TIMEOUT	(3:0) : WATCHDOG_CFG_TIMEOUT	0xB	Configure the watchdog timeout delay
P:0xE002	D_TIMER_1_CFG	16 : TIMER_CFG_START_STOP	16 : TIMER_CFG_START_STOP	0x0	Start or stop the timer
		15 : TIMER_CFG_MODE	15 : TIMER_CFG_MODE	0x0	Select a one-shot or free-running timer
		(14:12) : TIMER_CFG_PRESCALE	(14:12) : TIMER_CFG_PRESCALE	0x0	Prescale the clock used for the timer
		(11:0) : TIMER_CFG_DELAY	(11:0) : TIMER_CFG_DELAY	0x0	Number of prescaled timer cycles before a timeout
P:0xE003	D_TIMER_2_CFG	16 : TIMER_CFG_START_STOP	16 : TIMER_CFG_START_STOP	0x0	Start or stop the timer
		15 : TIMER_CFG_MODE	15 : TIMER_CFG_MODE	0x0	Select a one-shot or free-running timer
		(14:12) : TIMER_CFG_PRESCALE	(14:12) : TIMER_CFG_PRESCALE	0x0	Prescale the clock used for the timer
		(11:0) : TIMER_CFG_DELAY	(11:0) : TIMER_CFG_DELAY	0x0	Number of prescaled timer cycles before a timeout
P:0xE004	D_HEAR_CMDS_0_TO_7	(7:0) : D_HEAR_CMDS_0_TO_7	-	-	Issue commands 0-7 to the HEAR
P:0xE005	D_HEAR_CMDS_8_TO_15	(7:0) : D_HEAR_CMDS_8_TO_15	-	-	Issue commands 8-15 to the HEAR
P:0xE020	D_EXT_BREAK_CFG	(8:6) : EXT_BREAK_CFG_COMP_MODE	(8:6) : EXT_BREAK_CFG_COMP_MODE	0x0	Select the comparison mode
		(5:4) : EXT_BREAK_CFG_ADDR_COND	(5:4) : EXT_BREAK_CFG_ADDR_COND	0x0	Select the address condition
		(3:2) : EXT_BREAK_CFG_ACCESS_COND	(3:2) : EXT_BREAK_CFG_ACCESS_COND	0x0	Select the access or data condition
		(1:0) : EXT_BREAK_CFG_BUS_SELECT	(1:0) : EXT_BREAK_CFG_BUS_SELECT	0x0	Select the data bus to monitor for external breakpoints
P:0xE021	D_EXT_BREAK_DATA	(31:0) : D_EXT_BREAK_DATA	(31:0) : D_EXT_BREAK_DATA	0x0000	Data for setting external breakpoint conditions
P:0xE022	D_CRC_VALUE	(15:0) : D_CRC_VALUE	(15:0) : D_CRC_VALUE	0xFFFF	Value of CRC-CCITT for supplied data
P:0xE023	D_CRC_ADD_1	0 : D_CRC_ADD_1	-	-	Add 1 data bit to the CRC-CCITT
P:0xE024	D_CRC_ADD_8	(7:0) : D_CRC_ADD_8	-	-	Add 8 data bits to the CRC-CCITT
P:0xE025	D_CRC_ADD_16	(15:0) : D_CRC_ADD_16	-	-	Add 16 data bits to the CRC-CCITT
P:0xE026	D_CRC_ADD_24	(23:0) : D_CRC_ADD_24	-	-	Add 24 data bits to the CRC-CCITT
P:0xE027	D_CRC_ADD_32	(31:0) : D_CRC_ADD_32	-	-	Add 32 data bits to the CRC-CCITT

Address	Register Name	Register Write	Register Read	Default	Description
P:0xE030	D_CHIP_FAMILY	-	(7:0) : D_CHIP_FAMILY	0x3	Technology family (SK3) for the device
P:0xE031	D_CHIP_VERSION	-	(7:0) : D_CHIP_VERSION	N/A	Version number for the device
P:0xE032	D_CHIP_REVISION	-	(15:0) : D_CHIP_REVISION	N/A	Revision number for the device
P:0xE033	D_DEBUG_VERSION	-	(15:0) : D_DEBUG_VERSION	N/A	Version number for the debug port protocol
P:0xE034	D_DEBUG_CFG	-	3 : DEBUG_CFG_DBG_REG_WRITTEN	0x0	Indicate if the debug port data register has been written
		-	2 : DEBUG_CFG_DBG_REG_READ	0x0	Indicate if the debug port data register has been read
		-	1 : DEBUG_CFG_AUDIO_MODE	0x0	Indicate if the debug port is in audio mode
		0 : DEBUG_CFG_ACCESS_MODE	0 : DEBUG_CFG_ACCESS_MODE	0x1	Restrict/unrestrict access to the system
P:0xE035	D_DEBUG_DATA	(31:0) : D_DEBUG_DATA	(31:0) : D_DEBUG_DATA	0x0	The debug port data register
P:0xE040	D_INT_ENABLE	25 : INT_UART_TX	25 : INT_UART_TX	0x0	Enable/disable UART transmit interrupts
		24 : INT_UART_RX	24 : INT_UART_RX	0x0	Enable/disable UART receive interrupts
		23 : INT_FIFO_B3	23 : INT_FIFO_B3	0x0	Enable/disable interrupts for FIFO B3
		22 : INT_FIFO_B2	22 : INT_FIFO_B2	0x0	Enable/disable interrupts for FIFO B2
		21 : INT_FIFO_B1	21 : INT_FIFO_B1	0x0	Enable/disable interrupts for FIFO B1
		20 : INT_FIFO_B0	20 : INT_FIFO_B0	0x0	Enable/disable interrupts for FIFO B0
		19 : INT_FIFO_A3	19 : INT_FIFO_A3	0x0	Enable/disable interrupts for FIFO A3
		18 : INT_FIFO_A2	18 : INT_FIFO_A2	0x0	Enable/disable interrupts for FIFO A2
		17 : INT_FIFO_A1	17 : INT_FIFO_A1	0x0	Enable/disable interrupts for FIFO A1
		16 : INT_FIFO_A0	16 : INT_FIFO_A0	0x0	Enable/disable interrupts for FIFO A0
		15 : INT_HEAR_7	15 : INT_HEAR_7	0x0	Enable/disable interrupt #7 from the HEAR
		14 : INT_HEAR_6	14 : INT_HEAR_6	0x0	Enable/disable interrupt #6 from the HEAR
		13 : INT_HEAR_5	13 : INT_HEAR_5	0x0	Enable/disable interrupt #5 from the HEAR
		12 : INT_HEAR_4	12 : INT_HEAR_4	0x0	Enable/disable interrupt #4 from the HEAR
		11 : INT_HEAR_3	11 : INT_HEAR_3	0x0	Enable/disable interrupt #3 from the HEAR
		10 : INT_HEAR_2	10 : INT_HEAR_2	0x0	Enable/disable interrupt #2 from the HEAR
		9 : INT_HEAR_1	9 : INT_HEAR_1	0x0	Enable/disable interrupt #1 from the HEAR
		8 : INT_HEAR_0	8 : INT_HEAR_0	0x0	Enable/disable interrupt #0 from the HEAR
		7 : INT_GPIO	7 : INT_GPIO	0x0	Enable/disable GPIO interrupts
		6 : INT_PCM	6 : INT_PCM	0x0	Enable/disable PCM interrupts
		5 : INT_I2C	5 : INT_I2C	0x0	Enable/disable I^2C interrupts
		4 : INT_SPI	4 : INT_SPI	0x0	Enable/disable SPI interrupts
		3 : INT_TIMER_2	3 : INT_TIMER_2	0x0	Enable/disable interrupts for timer 2
		2 : INT_TIMER_1	2 : INT_TIMER_1	0x0	Enable/disable interrupts for timer 1
		1 : INT_WATCHDOG	1 : INT_WATCHDOG	0x0	Enable/disable watchdog timer interrupts
		0 : INT_MASTER	0 : INT_MASTER	0x0	Master enable/disable for all interrupts

Address	Register Name	Register Write	Register Read	Default	Description
P:0xE041	D_INT_STATUS	25 : INT_UART_TX	25 : INT_UART_TX	0x0	Acknowledge/status of UART transmit interrupt
		24 : INT_UART_RX	24 : INT_UART_RX	0x0	Acknowledge/status of UART receive interrupt
		23 : INT_FIFO_B3	23 : INT_FIFO_B3	0x0	Acknowledge/status of interrupt for FIFO B3
		22 : INT_FIFO_B2	22 : INT_FIFO_B2	0x0	Acknowledge/status of interrupt for FIFO B2
		21 : INT_FIFO_B1	21 : INT_FIFO_B1	0x0	Acknowledge/status of interrupt for FIFO B1
		20 : INT_FIFO_B0	20 : INT_FIFO_B0	0x0	Acknowledge/status of interrupt for FIFO B0
		19 : INT_FIFO_A3	19 : INT_FIFO_A3	0x0	Acknowledge/status of interrupt for FIFO A3
		18 : INT_FIFO_A2	18 : INT_FIFO_A2	0x0	Acknowledge/status of interrupt for FIFO A2
		17 : INT_FIFO_A1	17 : INT_FIFO_A1	0x0	Acknowledge/status of interrupt for FIFO A1
		16 : INT_FIFO_A0	16 : INT_FIFO_A0	0x0	Acknowledge/status of interrupt for FIFO A0
		15 : INT_HEAR_7	15 : INT_HEAR_7	0x0	Acknowledge/status of interrupt #7 from the HEAR
		14 : INT_HEAR_6	14 : INT_HEAR_6	0x0	Acknowledge/status of interrupt #6 from the HEAR
		13 : INT_HEAR_5	13 : INT_HEAR_5	0x0	Acknowledge/status of interrupt #5 from the HEAR
		12 : INT_HEAR_4	12 : INT_HEAR_4	0x0	Acknowledge/status of interrupt #4 from the HEAR
		11 : INT_HEAR_3	11 : INT_HEAR_3	0x0	Acknowledge/status of interrupt #3 from the HEAR
		10 : INT_HEAR_2	10 : INT_HEAR_2	0x0	Acknowledge/status of interrupt #2 from the HEAR
		9 : INT_HEAR_1	9 : INT_HEAR_1	0x0	Acknowledge/status of interrupt #1 from the HEAR
		8 : INT_HEAR_0	8 : INT_HEAR_0	0x0	Acknowledge/status of interrupt #0 from the HEAR
		7 : INT_GPIO	7 : INT_GPIO	0x0	Acknowledge/status of GPIO interrupt
		6 : INT_PCM	6 : INT_PCM	0x0	Acknowledge/status of PCM interrupt
		5 : INT_I2C	5 : INT_I2C	0x0	Acknowledge/status of I^2C interrupt
		4 : INT_SPI	4 : INT_SPI	0x0	Acknowledge/status of SPI interrupt
		3 : INT_TIMER_2	3 : INT_TIMER_2	0x0	Acknowledge/status of interrupt for timer 2
		2 : INT_TIMER_1	2 : INT_TIMER_1	0x0	Acknowledge/status of interrupt for timer 1
		1 : INT_WATCHDOG	1 : INT_WATCHDOG	0x0	Acknowledge/status of watchdog timer interrupt

Address	Register Name	Register Write	Register Read	Default	Description
P:0xE042	D_INT_PRIORITY	25 : INT_UART_TX	25 : INT_UART_TX	0x0	Select between high and low priority
		24 : INT_UART_RX	24 : INT_UART_RX	0x0	Select between high and low priority
		23 : INT_FIFO_B3	23 : INT_FIFO_B3	0x0	Select between high and low priority
		22 : INT_FIFO_B2	22 : INT_FIFO_B2	0x0	Select between high and low priority
		21 : INT_FIFO_B1	21 : INT_FIFO_B1	0x0	Select between high and low priority
		20 : INT_FIFO_B0	20 : INT_FIFO_B0	0x0	Select between high and low priority
		19 : INT_FIFO_A3	19 : INT_FIFO_A3	0x0	Select between high and low priority
		18 : INT_FIFO_A2	18 : INT_FIFO_A2	0x0	Select between high and low priority
		17 : INT_FIFO_A1	17 : INT_FIFO_A1	0x0	Select between high and low priority
		16 : INT_FIFO_A0	16 : INT_FIFO_A0	0x0	Select between high and low priority
		15 : INT_HEAR_7	15 : INT_HEAR_7	0x0	Select between high and low priority
		14 : INT_HEAR_6	14 : INT_HEAR_6	0x0	Select between high and low priority
		13 : INT_HEAR_5	13 : INT_HEAR_5	0x0	Select between high and low priority
		12 : INT_HEAR_4	12 : INT_HEAR_4	0x0	Select between high and low priority
		11 : INT_HEAR_3	11 : INT_HEAR_3	0x0	Select between high and low priority
		10 : INT_HEAR_2	10 : INT_HEAR_2	0x0	Select between high and low priority
		9 : INT_HEAR_1	9 : INT_HEAR_1	0x0	Select between high and low priority
		8 : INT_HEAR_0	8 : INT_HEAR_0	0x0	Select between high and low priority
		7 : INT_GPIO	7 : INT_GPIO	0x0	Select between high and low priority
		6 : INT_PCM	6 : INT_PCM	0x0	Select between high and low priority
		5 : INT_I2C	5 : INT_I2C	0x0	Select between high and low priority
		4 : INT_SPI	4 : INT_SPI	0x0	Select between high and low priority
		3 : INT_TIMER_2	3 : INT_TIMER_2	0x0	Select between high and low priority
		2 : INT_TIMER_1	2 : INT_TIMER_1	0x0	Select between high and low priority
		1 : INT_WATCHDOG	1 : INT_WATCHDOG	0x0	Select between high and low priority
P:0xE050	D_INT_EBL_0	(7:0) : D_INT_EBL_0	-	-	Enable (no disable) of interrupts 1 to 7 and master interrupt enable
P:0xE051	D_INT_EBL_1	(7:0) : D_INT_EBL_1	-	-	Enable (no disable) of interrupts 8 to 15
P:0xE052	D_INT_EBL_2	(7:0) : D_INT_EBL_2	-	-	Enable (no disable) of interrupts 16 to 23
P:0xE053	D_INT_EBL_3	(1:0) : D_INT_EBL_3	-	-	Enable (no disable) of interrupts 24 to 25
P:0xE054	D_INT_ACK_EBL_0	(7:0) : D_INT_ACK_EBL_0	-	-	Acknowledge and enable interrupts 1 to 7 and master interrupt enable
P:0xE055	D_INT_ACK_EBL_1	(7:0) : D_INT_ACK_EBL_1	-	-	Acknowledge and enable interrupts 8 to 15
P:0xE056	D_INT_ACK_EBL_2	(7:0) : D_INT_ACK_EBL_2	-	-	Acknowledge and enable interrupts 16 to 23
P:0xE057	D_INT_ACK_EBL_3	(1:0) : D_INT_ACK_EBL_3	-	-	Acknowledge and enable interrupts 24 to 25
P:0xE058	D_INT_DIS_0	(7:0) : D_INT_DIS_0	-	-	Disable (no enable) of interrupts 1 to 7 and master interrupt enable
P:0xE059	D_INT_DIS_1	(7:0) : D_INT_DIS_1	-	-	Disable (no enable) of interrupts 8 to 15

Address	Register Name	Register Write	Register Read	Default	Description
P:0xE05A	D_INT_DIS_2	(7:0) : D_INT_DIS_2	-	-	Disable (no enable) of interrupts 16 to 23
P:0xE05B	D_INT_DIS_3	(1:0) : D_INT_DIS_3	-	-	Disable (no enable) of interrupts 31 to 24
P:0xE05C	D_INT_ACK_0	(7:1) : D_INT_ACK_0	-	-	Acknowledge/status of interrupts 1 to 7
P:0xE05D	D_INT_ACK_1	(7:0) : D_INT_ACK_1	-	-	Acknowledge/status of interrupts 8 to 15
P:0xE05E	D_INT_ACK_2	(7:0) : D_INT_ACK_2	-	-	Acknowledge/status of interrupts 16 to 23
P:0xE05F	D_INT_ACK_3	(1:0) : D_INT_ACK_3	-	-	Acknowledge/status of interrupts 24 to 25
P:0xE060	D_INT_PRIO_0	(7:1) : D_INT_PRIO_0	-	-	Select priority for interrupts 1 to 7
P:0xE061	D_INT_PRIO_1	(7:0) : D_INT_PRIO_1	-	-	Select priority for interrupts 8 to 15
P:0xE062	D_INT_PRIO_2	(7:0) : D_INT_PRIO_2	-	-	Select priority for interrupts 16 to 23
P:0xE063	D_INT_PRIO_3	(1:0) : D_INT_PRIO_3	-	-	Select priority for interrupts 24 to 25
P:0xE080	D_CLK_CFG_0	18 : CLK_CFG0_SYSCLK_SEL	18 : CLK_CFG0_SYSCLK_SEL	0x0	Select the system oscillator source
		(17:16) : CLK_CFG0_EXTCLK_SEL	(17:16) : CLK_CFG0_EXTCLK_SEL	0x0	Select a configuration for the external clock
		(15:12) : CLK_CFG0_EXTCLK_PRESCALE	(15:12) : CLK_CFG0_EXTCLK_PRESCALE	0x0	Divider for an input external clock signal
		(9:0) : CLK_CFG0_HEARCLK_PRESCALE	(9:0) : CLK_CFG0_HEARCLK_PRESCALE	0x0	Prescaling factor for the HEAR clock
P:0xE081	D_CLK_CFG_1	(23:22) : CLK_CFG1_PCM_CLK_PAD_SEL	(23:22) : CLK_CFG1_PCM_CLK_PAD_SEL	0x0	Select the PCM clock pad functionality
		21 : CLK_CFG1_PCM_FRAME_PAD_SEL	21 : CLK_CFG1_PCM_FRAME_PAD_SEL	0x0	Select the PCM frame pad functionality
		20 : CLK_CFG1_DBL_ACCESS_MODE	20 : CLK_CFG1_DBL_ACCESS_MODE	0x0	Configure for double (shared) access mode
		(19:8) : CLK_CFG1_UCLK_PRESCALE	(19:8) : CLK_CFG1_UCLK_PRESCALE	0x0	Prescaling divider used for the user clock
		(5:0) : CLK_CFG1_SLOWCLK_PRESCALE	(5:0) : CLK_CFG1_SLOWCLK_PRESCALE	0x0	Prescaling divider used for the slow clock
P:0xE0C0	D_IOC_INPUT_CFG	(23:20) : IOC_INPUT_CFG_PCM1_STORE	(23:20) : IOC_INPUT_CFG_PCM1_STORE	0xF	Select where to store PCM 1 input
		(19:16) : IOC_INPUT_CFG_PCM0_STORE	(19:16) : IOC_INPUT_CFG_PCM0_STORE	0xF	Select where to store PCM 0 input
		(15:12) : IOC_INPUT_CFG_ADC3_STORE	(15:12) : IOC_INPUT_CFG_ADC3_STORE	0xF	Select where to store the ADC3 input
		(11:8) : IOC_INPUT_CFG_ADC2_STORE	(11:8) : IOC_INPUT_CFG_ADC2_STORE	0xF	Select where to store the ADC2 input
		(7:4) : IOC_INPUT_CFG_ADC1_STORE	(7:4) : IOC_INPUT_CFG_ADC1_STORE	0xF	Select where to store the ADC1 input
		(3:0) : IOC_INPUT_CFG_ADC0_STORE	(3:0) : IOC_INPUT_CFG_ADC0_STORE	0xF	Select where to store the ADC0 input

Address	Register Name	Register Write	Register Read	Default	Description
P:0xE0C1	D_IOC_FIFO_CFG	13 : IOC_FIFO_CFG_DAC_SATURATE	13 : IOC_FIFO_CFG_DAC_SATURATE	0x1	Saturate the input value to the DAC
		12 : IOC_FIFO_CFG_DAC_SHIFT	12 : IOC_FIFO_CFG_DAC_SHIFT	0x0	Select low or high bits for input to the DAC
		11 : IOC_FIFO_CFG_ADC3_SHIFT	11 : IOC_FIFO_CFG_ADC3_SHIFT	0x0	Select low or high bits filled by ADC3 input
		10 : IOC_FIFO_CFG_ADC2_SHIFT	10 : IOC_FIFO_CFG_ADC2_SHIFT	0x0	Select low or high bits filled by ADC2 input
		9 : IOC_FIFO_CFG_ADC1_SHIFT	9 : IOC_FIFO_CFG_ADC1_SHIFT	0x0	Select low or high bits filled by ADC1 input
		8 : IOC_FIFO_CFG_ADC0_SHIFT	8 : IOC_FIFO_CFG_ADC0_SHIFT	0x0	Select low or high bits filled by ADC0 input
		7 : IOC_FIFO_CFG_FIFOB3_DBL_ACC	7 : IOC_FIFO_CFG_FIFOB3_DBL_ACC	0x0	Enable/disable double access for FIFO B3
		6 : IOC_FIFO_CFG_FIFOB2_DBL_ACC	6 : IOC_FIFO_CFG_FIFOB2_DBL_ACC	0x0	Enable/disable double access for FIFO B2
		5 : IOC_FIFO_CFG_FIFOB1_DBL_ACC	5 : IOC_FIFO_CFG_FIFOB1_DBL_ACC	0x0	Enable/disable double access for FIFO B1
		4 : IOC_FIFO_CFG_FIFOB0_DBL_ACC	4 : IOC_FIFO_CFG_FIFOB0_DBL_ACC	0x0	Enable/disable double access for FIFO B0
		3 : IOC_FIFO_CFG_FIFOA3_DBL_ACC	3 : IOC_FIFO_CFG_FIFOA3_DBL_ACC	0x0	Enable/disable double access for FIFO A3
		2 : IOC_FIFO_CFG_FIFOA2_DBL_ACC	2 : IOC_FIFO_CFG_FIFOA2_DBL_ACC	0x0	Enable/disable double access for FIFO A2
		1 : IOC_FIFO_CFG_FIFOA1_DBL_ACC	1 : IOC_FIFO_CFG_FIFOA1_DBL_ACC	0x0	Enable/disable double access for FIFO A1
		0 : IOC_FIFO_CFG_FIFOA0_DBL_ACC	0 : IOC_FIFO_CFG_FIFOA0_DBL_ACC	0x0	Enable/disable double access for FIFO A0
P:0xE0C2	D_IOC_OUTPUT_CFG	15 : IOC_OUTPUT_CFG_PCM_SYNC_ADCDAC	15 : IOC_OUTPUT_CFG_PCM_SYNC_ADCDAC	0x0	Enable/disable synchronization of PCM and ADC signals
		14 : IOC_OUTPUT_CFG_PCM_DBL_ACC	14 : IOC_OUTPUT_CFG_PCM_DBL_ACC	0x0	Enable/disable double access for PCM
		13 : IOC_OUTPUT_CFG_PCM_STEREO	13 : IOC_OUTPUT_CFG_PCM_STEREO	0x0	Select PCM FIFO source as mono or stereo
		12 : IOC_OUTPUT_CFG_DAC_DBL_ACC	12 : IOC_OUTPUT_CFG_DAC_DBL_ACC	0x0	Enable/disable double access for DAC
		(11:8) : IOC_OUTPUT_CFG_PCM1_LOAD	(11:8) : IOC_OUTPUT_CFG_PCM1_LOAD	0xF	Select where to load PCM 1 output from
		(7:4) : IOC_OUTPUT_CFG_PCM0_LOAD	(7:4) : IOC_OUTPUT_CFG_PCM0_LOAD	0xF	Select where to load PCM 0 output from
		(3:0) : IOC_OUTPUT_CFG_DAC_LOAD	(3:0) : IOC_OUTPUT_CFG_DAC_LOAD	0xF	Select where to load DAC output from
P:0xE0D0	D_ADC0_DATA	-	(17:0) : D_ADC0_DATA	0x0	Most recent input data from ADC 0
P:0xE0D1	D_ADC1_DATA	-	(17:0) : D_ADC1_DATA	0x0	Most recent input data from ADC 1
P:0xE0D2	D_ADC2_DATA	-	(17:0) : D_ADC2_DATA	0x0	Most recent input data from ADC 2
P:0xE0D3	D_ADC3_DATA	-	(17:0) : D_ADC3_DATA	0x0	Most recent input data from ADC 3
P:0xE1C0	D_SPI_CFG	6 : SPI_CFG_PULLUP_DIS	6 : SPI_CFG_PULLUP_DIS	0x0	Configure the SPI serial input pull-up resistor
		5 : SPI_CFG_CLKPOL	5 : SPI_CFG_CLKPOL	0x0	Select SPI interface clock polarity
		4 : SPI_CFG_AUTO	4 : SPI_CFG_AUTO	0x0	Enable/disable automatic read/write mode
		3 : SPI_CFG_ENABLE	3 : SPI_CFG_ENABLE	0x0	Enable/disable the SPI interface
		(2:0) : SPI_CFG_PRESCALE	(2:0) : SPI_CFG_PRESCALE	0x0	Divider for the SPI speed from slow clock
P:0xE1C1	D_SPI_CTRL	7 : SPI_CTRL_START	7 : SPI_CTRL_BUSY	0x0	Start/Status of an SPI transaction
		6 : SPI_CTRL_RW	6 : SPI_CTRL_RW	0x0	Configure SPI for input or output
		5 : SPI_CTRL_CS_N	5 : SPI_CTRL_CS_N	0x0	Select the value of the chip select line
		(4:0) : SPI_CTRL_N_BITS	(4:0) : SPI_CTRL_N_BITS	0x0	Select how many bits are in a transaction
P:0xE1C2	D_SPI_DATA	(31:0) : D_SPI_DATA	(31:0) : D_SPI_DATA	0x0	Data used in an SPI transaction

Address	Register Name	Register Write	Register Read	Default	Description
P:0xE1C3	D_SPI_DATA_M	(31:0) : D_SPI_DATA_M	(31:0) : D_SPI_DATA_M	0x0	Mirror of D_SPI_DATA (No side effects)
P:0xE1C4	D_SPI_DATA_M_16LSB	-	(15:0) : D_SPI_DATA_M_16LSB	0x0	Mirror of bits (15:0) from D_SPI_DATA
P:0xE1C5	D_SPI_DATA_M_16MSB	-	(15:0) : D_SPI_DATA_M_16MSB	0x0	Mirror of bits (31:16) from D_SPI_DATA
P:0xE1C6	D_SPI_DATA_M_8LSB	-	(7:0) : D_SPI_DATA_M_8LSB	0x0	Mirror of bits (7:0) from D_SPI_DATA
P:0xE1C7	D_SPI_DATA_M_8_15BITS	-	(7:0) : D_SPI_DATA_M_8_15BITS	0x0	Mirror of bits (15:8) from D_SPI_DATA
P:0xE1C8	D_SPI_DATA_M_16_23BITS	-	(7:0) : D_SPI_DATA_M_16_23BITS	0x0	Mirror of bits (23:16) from D_SPI_DATA
P:0xE1C9	D_SPI_DATA_M_8MSB	-	(7:0) : D_SPI_DATA_M_8MSB	0x0	Mirror of bits (31:24) from D_SPI_DATA
P:0xE200	D_I2C_CFG	(19:12) : I2C_CFG_MASTER_SPEED	(19:12) : I2C_CFG_MASTER_SPEED	0x0	Configure I^2C master mode interface speed
		(10:4) : I2C_CFG_SLAVE_ID	(10:4) : I2C_CFG_SLAVE_ID	0x10	I^2C address for general purpose I^2C
		3 : I2C_CFG_STOP_INT	3 : I2C_CFG_STOP_INT	0x0	Enable/disable interrupts on stop conditions
		2 : I2C_CFG_AUTO_ACK	2 : I2C_CFG_AUTO_ACK	0x0	Enable/disable automatic acknowledgment
		0 : I2C_CFG_SLAVE_EN	0 : I2C_CFG_SLAVE_EN	0x0	Enable I^2C slave mode
P:0xE201	D_I2C_CTRL	-	10 : I2C_CTRL_STOP_DETECTED	0x0	Indicate that a stop condition has been detected
		-	9 : I2C_CTRL_DATA_EVENT	0x0	Indicate data has been received or is needed for transmission
		-	8 : I2C_CTRL_ERROR	0x0	Indicate if an I^2C bus error has occurred since the last stop condition, or if an I^2C watchdog error has occurred
		-	7 : I2C_CTRL_BUS_ERROR	0x0	Indicate if an I^2C bus error has occurred since the last stop condition
		-	6 : I2C_CTRL_BUFFER_FULL	0x0	Indicate if the I^2C buffer is full
		5: I2C_CTRL_RESET_POS	5 : I2C_CTRL_CLK_STRETCH	0x0	Reset the I^2C interface and bus; indicate if the SCL is being stretched
		4: I2C_CTRL_LAST_DATA_POS	4 : I2C_CTRL_LINE_FREE_POS	0x0	The next byte is the last byte the current data transfer; indicate if the bus lines are not in use
		3: I2C_CTRL_STOP_CMD_POS	3 : I2C_CTRL_ADDR_DATA	0x0	Issue a stop command (master mode only); indicate whether the current data is an address byte or a data byte
		-	2 : I2C_CTRL_RW	0x0	Indicate read or write
		1 : I2C_CTRL_NACK	1 : I2C_CTRL_GEN_CALL	0x0	I^2C slave mode NACK; indicate that transmission is to general call address
		0 : I2C_CTRL_ACK	0 : I2C_CTRL_ACK	0x1	I^2C slave mode ACK; I^2C master mode ACK
P:0xE202	D_I2C_DATA	(7:0) : D_I2C_DATA	(7:0) : D_I2C_DATA	0x0	Data used in an I^2C transaction
P:0xE203	D_I2C_DATA_M	(7:0) : D_I2C_DATA_M	(7:0) : D_I2C_DATA_M	0x0	Mirror of D_I2C_DATA (No side effects)
P:0xE204	D_DEBUG_ADDRESS	(6:0) : DEBUG_ADDR	(6:0) : DEBUG_ADDR	0x60	Selection of the debug port internally-programmable address
P:0xE205	D_I2C_ERROR_CNT	-	(7:0) : D_I2C_ERROR_CNT	0x0	Number of I^2C watchdog timeouts

Address	Register Name	Register Write	Register Read	Default	Description
P:0xE206	D_I2C_ADDR_START	(7:1) : D_I2C_START_ADDR	-	0x0	Address to send with the start condition over the I^2C bus
		0 : D_I2C_START_RW	-	0x0	Direction of the initiated I^2C transfer
P:0xE207	D_I2C_PIN_CFG	2 : I2C_CFG_PULLUP_DIS	2 : I2C_CFG_PULLUP_DIS	0x0	Configure the pull-up resistor connections
		1 : I2C_CFG_PULL_UP	1 : I2C_CFG_PULL_UP	0x0	Select 250kΩ or 1kΩ pull-up resistors
		0 : I2C_CFG_FILTER_EN	0 : I2C_CFG_FILTER_EN	0x0	Enable/disable low-pass filter for I^2C signals
P:0xE208	D_I2C_WATCHDOG_CFG	(2:0) : I2C_WATCHDOG_CFG_TIMEOUT	(2:0) : I2C_WATCHDOG_CFG_TIMEOUT	0x2	Configure the I^2C bus' watchdog timeout
P:0xE240	D_PCM_CFG	23 : PCM_CLK_LPF	23 : PCM_CLK_LPF	0x0	Enable/disable the low-pass filter for the PCM clock
		22 : PCM_PULLUP_DIS	22 : PCM_PULLUP_DIS	0x0	Configure the pull-up resistors for the PCM input signals
		21 : PCM_SATURATE_TX_DATA	21 : PCM_SATURATE_TX_DATA	0x0	Enable/disable saturation of transmit data
		20 : PCM_FIELD_TX_DATA	20 : PCM_FIELD_TX_DATA	0x0	Select what bits to use for transmit data
		19 : PCM_SHIFT_RX_DATA	19 : PCM_SHIFT_RX_DATA	0x0	Left/right shift received data
		(18:15) : PCM_WORD_SIZE	(18:15) : PCM_WORD_SIZE	0x0	Select the number of bits per PCM word
		(14:12) : PCM_MULTIWORD_CTRL	(14:12) : PCM_MULTIWORD_CTRL	0x0	Select the number of words per PCM frame
		11 : PCM_DATA_MGT	11 : PCM_DATA_MGT	0x0	Select data management by the CFX or IOC
		7 : PCM_TX_INT	7 : PCM_TX_INT	0x0	Enable/disable PCM transmit interrupts
		6 : PCM_RX_INT	6 : PCM_RX_INT	0x0	Enable/disable PCM receive interrupts
		5 : PCM_FRAME_WIDTH	5 : PCM_FRAME_WIDTH	0x0	Use a long/short PCM frame signal
		4 : PCM_FRAME_ALIGN	4 : PCM_FRAME_ALIGN	0x0	Align the PCM frame to the first/last bit
		3 : PCM_BIT_ORDER	3 : PCM_BIT_ORDER	0x0	Data is organized MSB/LSB bit first
		2 : PCM_SUBFRAME	2 : PCM_SUBFRAME	0x0	Enable/disable PCM subframes
		1 : PCM_ENABLE	1 : PCM_ENABLE	0x0	Enable/disable the PCM interface
		0 : PCM_SLAVE	0 : PCM_SLAVE	0x1	Use the PCM interface as master/slave
P:0xE241	D_PCM_DATA_0	(24:0) : D_PCM_DATA_0	(24:0) : D_PCM_DATA_0	0x0	Data used in a PCM transaction (channel 0)
P:0xE242	D_PCM_DATA_1	(24:0) : D_PCM_DATA_1	(24:0) : D_PCM_DATA_1	0x0	Data used in a PCM transaction (channel 1)
P:0xE243	D_PCM_CTRL	-	1 : PCM_TX_INT_STATUS	0x0	Indicate if a PCM transmit interrupt has been generated
		-	0: PCM_RX_INT_STATUS	0x0	Indicate if a PCM receive interrupt has been generated

Address	Register Name	Register Write	Register Read	Default	Description
P:0xE280	D_GPIO_CFG	(22:20) : GPIO_CFG_INT_SRC	(22:20) : GPIO_CFG_INT_SRC	0x0	Select the trigger source for GPIO interrupts
		(18:16) : GPIO_CFG_INT_EVENT	(18:16) : GPIO_CFG_INT_EVENT	0x0	Select the trigger type for GPIO interrupts
		12 : GPIO_CFG_INPUT_DIS_GPIO4	12 : GPIO_CFG_INPUT_DIS_GPIO4	0x0	Enable or disable input mode for GPIO 4
		11: GPIO_CFG_INPUT_DIS_GPIO3	11: GPIO_CFG_INPUT_DIS_GPIO3	0x0	Enable or disable input mode for GPIO 3
		10 : GPIO_CFG_INPUT_DIS_GPIO2	10 : GPIO_CFG_INPUT_DIS_GPIO2	0x0	Enable or disable input mode for GPIO 2
		9 : GPIO_CFG_INPUT_DIS_GPIO1	9 : GPIO_CFG_INPUT_DIS_GPIO1	0x0	Enable or disable input mode for GPIO 1
		8 : GPIO_CFG_PULLUP_DIS_GPIO0	8 : GPIO_CFG_PULLUP_DIS_GPIO0	0x0	Configure the pull-up resistor for GPIO 0
		4 : GPIO_CFG_DIR_CFG_GPIO4	4 : GPIO_CFG_DIR_CFG_GPIO4	0x1	Select input/output direction for GPIO 4
		3 : GPIO_CFG_DIR_CFG_GPIO3	3 : GPIO_CFG_DIR_CFG_GPIO3	0x1	Select input/output direction for GPIO 3
		2 : GPIO_CFG_DIR_CFG_GPIO2	2 : GPIO_CFG_DIR_CFG_GPIO2	0x1	Select input/output direction for GPIO 2
		1 : GPIO_CFG_DIR_CFG_GPIO1	1 : GPIO_CFG_DIR_CFG_GPIO1	0x1	Select input/output direction for GPIO 1
		0 : GPIO_CFG_DIR_CFG_GPIO0	0 : GPIO_CFG_DIR_CFG_GPIO0	0x1	Select input/output direction for GPIO 0
P:0xE281	D_GPIO_PIN_CFG	(9:8) : GPIO_PIN_CFG_GPIO4	(9:8) : GPIO_PIN_CFG_GPIO4	0x0	Select what GPIO 4 functionality is used
		(7:6) : GPIO_PIN_CFG_GPIO3	(7:6) : GPIO_PIN_CFG_GPIO3	0x0	Select what GPIO 3 functionality is used
		(5:4) : GPIO_PIN_CFG_GPIO2	(5:4) : GPIO_PIN_CFG_GPIO2	0x0	Select what GPIO 2 functionality is used
		(3:2) : GPIO_PIN_CFG_GPIO1	(3:2) : GPIO_PIN_CFG_GPIO1	0x0	Select what GPIO 1 functionality is used
		(1:0) : GPIO_PIN_CFG_GPIO0	(1:0) : GPIO_PIN_CFG_GPIO0	0x0	Select what GPIO 0 functionality is used
P:0xE282	D_GPIO_DATA	4 : GPIO_DATA_GPIO4	4 : GPIO_DATA_GPIO4	-	Value of input or output from GPIO 4
		3 : GPIO_DATA_GPIO3	3 : GPIO_DATA_GPIO3	-	Value of input or output from GPIO 3
		2 : GPIO_DATA_GPIO2	2 : GPIO_DATA_GPIO2	-	Value of input or output from GPIO 2
		1 : GPIO_DATA_GPIO1	1 : GPIO_DATA_GPIO1	-	Value of input or output from GPIO 1
		0 : GPIO_DATA_GPIO0	0 : GPIO_DATA_GPIO0	-	Value of input or output from GPIO 0
P:0xE2C0	D_UART_CFG	(23:8) : UART_CFG_PRESCALE	(23:8) : UART_CFG_PRESCALE	0x0	Prescaler used to define the UART baud rate
		4 : UART_CFG_PRESCALE_ENABLE	4 : UART_CFG_PRESCALE_ENABLE	0x0	Enable/disable additional prescaling by 12
		0 : UART_CFG_ENABLE	0 : UART_CFG_ENABLE	0x0	Enable/disable the UART interface
P:0xE2C1	D_UART_DATA	(7:0) D_UART_DATA	(7:0) D_UART_DATA	0x0	Data used in a UART transaction

A.3 HEAR DIGITAL BLOCK REGISTERS

Address	Register Name	Register Write	Register Read	Default	Description
X:0x9F80	D_HEAR_FC_ENABLE	23 : HEAR_FC_FIFO_B3	23 : HEAR_FC_FIFO_B3	0x0	Enable the function chain linked to FIFO B3
		22 : HEAR_FC_FIFO_B2	22 : HEAR_FC_FIFO_B2	0x0	Enable the function chain linked to FIFO B2
		21 : HEAR_FC_FIFO_B1	21 : HEAR_FC_FIFO_B1	0x0	Enable the function chain linked to FIFO B1
		20 : HEAR_FC_FIFO_B0	20 : HEAR_FC_FIFO_B0	0x0	Enable the function chain linked to FIFO B0
		19 : HEAR_FC_FIFO_A3	19 : HEAR_FC_FIFO_A3	0x0	Enable the function chain linked to FIFO A3
		18 : HEAR_FC_FIFO_A2	18 : HEAR_FC_FIFO_A2	0x0	Enable the function chain linked to FIFO A2
		17 : HEAR_FC_FIFO_A1	17 : HEAR_FC_FIFO_A1	0x0	Enable the function chain linked to FIFO A1
		16 : HEAR_FC_FIFO_A0	16 : HEAR_FC_FIFO_A0	0x0	Enable the function chain linked to FIFO A0
		15 : HEAR_FC_CMD15	15 : HEAR_FC_CMD15	0x0	Enable the command 15 function chain
		14 : HEAR_FC_CMD14	14 : HEAR_FC_CMD14	0x0	Enable the command 14 function chain
		13 : HEAR_FC_CMD13	13 : HEAR_FC_CMD13	0x0	Enable the command 13 function chain
		12 : HEAR_FC_CMD12	12 : HEAR_FC_CMD12	0x0	Enable the command 12 function chain
		11 : HEAR_FC_CMD11	11 : HEAR_FC_CMD11	0x0	Enable the command 11 function chain
		10 : HEAR_FC_CMD10	10 : HEAR_FC_CMD10	0x0	Enable the command 10 function chain
		9 : HEAR_FC_CMD9	9 : HEAR_FC_CMD9	0x0	Enable the command 9 function chain
		8 : HEAR_FC_CMD8	8 : HEAR_FC_CMD8	0x0	Enable the command 8 function chain
		7 : HEAR_FC_CMD7	7 : HEAR_FC_CMD7	0x0	Enable the command 7 function chain
		6 : HEAR_FC_CMD6	6 : HEAR_FC_CMD6	0x0	Enable the command 6 function chain
		5 : HEAR_FC_CMD5	5 : HEAR_FC_CMD5	0x0	Enable the command 5 function chain
		4 : HEAR_FC_CMD4	4 : HEAR_FC_CMD4	0x0	Enable the command 4 function chain
		3 : HEAR_FC_CMD3	3 : HEAR_FC_CMD3	0x0	Enable the command 3 function chain
		2 : HEAR_FC_CMD2	2 : HEAR_FC_CMD2	0x0	Enable the command 2 function chain
		1 : HEAR_FC_CMD1	1 : HEAR_FC_CMD1	0x0	Enable the command 1 function chain
		0 : HEAR_FC_CMD0	0 : HEAR_FC_CMD0	0x0	Enable the command 0 function chain

Address	Register Name	Register Write	Register Read	Default	Description
X:0x9F81	D_HEAR_FC_PRIORITY	23 : HEAR_FC_FIFO_B3	23 : HEAR_FC_FIFO_B3	0x0	Select between high and low priority
		22 : HEAR_FC_FIFO_B2	22 : HEAR_FC_FIFO_B2	0x0	Select between high and low priority
		21 : HEAR_FC_FIFO_B1	21 : HEAR_FC_FIFO_B1	0x0	Select between high and low priority
		20 : HEAR_FC_FIFO_B0	20 : HEAR_FC_FIFO_B0	0x0	Select between high and low priority
		19 : HEAR_FC_FIFO_A3	19 : HEAR_FC_FIFO_A3	0x0	Select between high and low priority
		18 : HEAR_FC_FIFO_A2	18 : HEAR_FC_FIFO_A2	0x0	Select between high and low priority
		17 : HEAR_FC_FIFO_A1	17 : HEAR_FC_FIFO_A1	0x0	Select between high and low priority
		16 : HEAR_FC_FIFO_A0	16 : HEAR_FC_FIFO_A0	0x0	Select between high and low priority
		15 : HEAR_FC_CMD15	15 : HEAR_FC_CMD15	0x0	Select between high and low priority
		14 : HEAR_FC_CMD14	14 : HEAR_FC_CMD14	0x0	Select between high and low priority
		13 : HEAR_FC_CMD13	13 : HEAR_FC_CMD13	0x0	Select between high and low priority
		12 : HEAR_FC_CMD12	12 : HEAR_FC_CMD12	0x0	Select between high and low priority
		11 : HEAR_FC_CMD11	11 : HEAR_FC_CMD11	0x0	Select between high and low priority
		10 : HEAR_FC_CMD10	10 : HEAR_FC_CMD10	0x0	Select between high and low priority
		9 : HEAR_FC_CMD9	9 : HEAR_FC_CMD9	0x0	Select between high and low priority
		8 : HEAR_FC_CMD8	8 : HEAR_FC_CMD8	0x0	Select between high and low priority
		7 : HEAR_FC_CMD7	7 : HEAR_FC_CMD7	0x0	Select between high and low priority
		6 : HEAR_FC_CMD6	6 : HEAR_FC_CMD6	0x0	Select between high and low priority
		5 : HEAR_FC_CMD5	5 : HEAR_FC_CMD5	0x0	Select between high and low priority
		4 : HEAR_FC_CMD4	4 : HEAR_FC_CMD4	0x0	Select between high and low priority
		3 : HEAR_FC_CMD3	3 : HEAR_FC_CMD3	0x0	Select between high and low priority
		2 : HEAR_FC_CMD2	2 : HEAR_FC_CMD2	0x0	Select between high and low priority
		1 : HEAR_FC_CMD1	1 : HEAR_FC_CMD1	0x0	Select between high and low priority
		0 : HEAR_FC_CMD0	0 : HEAR_FC_CMD0	0x0	Select between high and low priority

Address	Register Name	Register Write	Register Read	Default	Description
X:0x9F87	D_HEAR_FC_STATUS	-	23 : HEAR_FC_FIFO_B3	0x0	Status of FIFO B3 function chain
		-	22 : HEAR_FC_FIFO_B2	0x0	Status of FIFO B2 function chain
		-	21 : HEAR_FC_FIFO_B1	0x0	Status of FIFO B1 function chain
		-	20 : HEAR_FC_FIFO_B0	0x0	Status of FIFO B0 function chain
		-	19 : HEAR_FC_FIFO_A3	0x0	Status of FIFO A3 function chain
		-	18 : HEAR_FC_FIFO_A2	0x0	Status of FIFO A2 function chain
		-	17 : HEAR_FC_FIFO_A1	0x0	Status of FIFO A1 function chain
		-	16 : HEAR_FC_FIFO_A0	0x0	Status of FIFO A0 function chain
		-	15 : HEAR_FC_CMD15	0x0	Status of command 15 function chain
		-	14 : HEAR_FC_CMD14	0x0	Status of command 14 function chain
		-	13 : HEAR_FC_CMD13	0x0	Status of command 13 function chain
		-	12 : HEAR_FC_CMD12	0x0	Status of command 12 function chain
		-	11 : HEAR_FC_CMD11	0x0	Status of command 11 function chain
		-	10 : HEAR_FC_CMD10	0x0	Status of command 10 function chain
		-	9 : HEAR_FC_CMD9	0x0	Status of command 9 function chain
		-	8 : HEAR_FC_CMD8	0x0	Status of command 8 function chain
		-	7 : HEAR_FC_CMD7	0x0	Status of command 7 function chain
		-	6 : HEAR_FC_CMD6	0x0	Status of command 6 function chain
		-	5 : HEAR_FC_CMD5	0x0	Status of command 5 function chain
		-	4 : HEAR_FC_CMD4	0x0	Status of command 4 function chain
		-	3 : HEAR_FC_CMD3	0x0	Status of command 3 function chain
		-	2 : HEAR_FC_CMD2	0x0	Status of command 2 function chain
		-	1 : HEAR_FC_CMD1	0x0	Status of command 1 function chain
		-	0 : HEAR_FC_CMD0	0x0	Status of command 0 function chain
X:0x9F90	D_HEAR_BFP	(21:16) : HEAR_BFP_BLOCK_EXP	(21:16) : HEAR_BFP_BLOCK_EXP	0x0	Stored block exponent for the data
		15 : HEAR_BFP_FMD_MODE	15 : HEAR_BFP_FMD_MODE	0x0	Enable/disable FMD mode
		(12:8) : HEAR_BFP_BLOCK_MAX_MAG	(12:8) : HEAR_BFP_BLOCK_MAX_MAG	0x0	Maximum magnitude for a block of data
		(5:4) : HEAR_BFP_SCALE_FACTOR	(5:4) : HEAR_BFP_SCALE_FACTOR	0x0	Select a scaling factor for BFP
		(1:0) : HEAR_BFP_FORMAT	(1:0) : HEAR_BFP_FORMAT	0x0	Select a format based on the number of bits
X:0x9F91	D_HEAR_BFP_BLOCK_EXP	(5:0) : HEAR_BFP_BLOCK_EXP	(5:0) : HEAR_BFP_BLOCK_EXP	0x0	Mirror of the block exponent for the data

A.4 FIFO CONFIGURATION REGISTERS

Address	Register Name	Register Write	Register Read	Default	Description
X:0x9F00	D_FIFO_A0_START	(10:5) : D_FIFO_A0_START	(10:5) : D_FIFO_A0_START	0x0	Start offset in A memory for FIFO A0
X:0x9F01	D_FIFO_A0_END	(10:5) : D_FIFO_A0_END	(10:5) : D_FIFO_A0_END	0x0	End offset in A memory for FIFO A0
X:0x9F02	D_FIFO_A0_SIZE	(10:5) : D_FIFO_A0_SIZE	(10:5) : D_FIFO_A0_SIZE	0x0	Total size of A memory used in FIFO A0
X:0x9F04	D_FIFO_A0_BASE_PTR	(10:1) : D_FIFO_A0_BASE_PTR	(10:1) : D_FIFO_A0_BASE_PTR	0x0	Pointer to the base of FIFO A0
X:0x9F05	D_FIFO_A0_IOBLOCK_PTR	(10:1) : D_FIFO_A0_IOBLOCK_PTR	(10:1) : D_FIFO_A0_IOBLOCK_PTR	0x0	Pointer to the latest I/O block in FIFO A0
X:0x9F08	D_FIFO_A0_BLOCK_SIZE	(9:0) : D_FIFO_A0_BLOCK_SIZE	(9:0) : D_FIFO_A0_BLOCK_SIZE	0x0	Size of one block of samples for FIFO A0
X:0x9F0C	D_FIFO_A0_BLOCK_CNT	(9:0) : D_FIFO_A0_BLOCK_CNT	(9:0) : D_FIFO_A0_BLOCK_CNT	0x0	Next read/write location in current sample block
X:0x9F0E	D_FIFO_A0_ACCESS	(23:0) : D_FIFO_A0_ACCESS	(23:0) : D_FIFO_A0_ACCESS	-	Smart FIFO access for FIFO A0
X:0x9F10	D_FIFO_A1_START	(10:5) : D_FIFO_A1_START	(10:5) : D_FIFO_A1_START	0x0	Start offset in A memory for FIFO A1
X:0x9F11	D_FIFO_A1_END	(10:5) : D_FIFO_A1_END	(10:5) : D_FIFO_A1_END	0x0	End offset in A memory for FIFO A1
X:0x9F12	D_FIFO_A1_SIZE	(10:5) : D_FIFO_A1_SIZE	(10:5) : D_FIFO_A1_SIZE	0x0	Total size of A memory used in FIFO A1
X:0x9F14	D_FIFO_A1_BASE_PTR	(10:1) : D_FIFO_A1_BASE_PTR	(10:1) : D_FIFO_A1_BASE_PTR	0x0	Pointer to the base of FIFO A1
X:0x9F15	D_FIFO_A1_IOBLOCK_PTR	(10:1) : D_FIFO_A1_IOBLOCK_PTR	(10:1) : D_FIFO_A1_IOBLOCK_PTR	0x0	Pointer to the latest I/O block in FIFO A1
X:0x9F18	D_FIFO_A1_BLOCK_SIZE	(9:0) : D_FIFO_A1_BLOCK_SIZE	(9:0) : D_FIFO_A1_BLOCK_SIZE	0x0	Size of one block of samples for FIFO A1
X:0x9F1C	D_FIFO_A1_BLOCK_CNT	(9:0) : D_FIFO_A1_BLOCK_CNT	(9:0) : D_FIFO_A1_BLOCK_CNT	0x0	Next read/write location in current sample block
X:0x9F1E	D_FIFO_A1_ACCESS	(23:0) : D_FIFO_A1_ACCESS	(23:0) : D_FIFO_A1_ACCESS	-	Smart FIFO access for FIFO A1
X:0x9F20	D_FIFO_A2_START	(10:5) : D_FIFO_A2_START	(10:5) : D_FIFO_A2_START	0x0	Start offset in A memory for FIFO A2
X:0x9F21	D_FIFO_A2_END	(10:5) : D_FIFO_A2_END	(10:5) : D_FIFO_A2_END	0x0	End offset in A memory for FIFO A2
X:0x9F22	D_FIFO_A2_SIZE	(10:5) : D_FIFO_A2_SIZE	(10:5) : D_FIFO_A2_SIZE	0x0	Total size of A memory used in FIFO A2
X:0x9F24	D_FIFO_A2_BASE_PTR	(10:1) : D_FIFO_A2_BASE_PTR	(10:1) : D_FIFO_A2_BASE_PTR	0x0	Pointer to the base of FIFO A2
X:0x9F25	D_FIFO_A2_IOBLOCK_PTR	(10:1) : D_FIFO_A2_IOBLOCK_PTR	(10:1) : D_FIFO_A2_IOBLOCK_PTR	0x0	Pointer to the latest I/O block in FIFO A2
X:0x9F28	D_FIFO_A2_BLOCK_SIZE	(9:0) : D_FIFO_A2_BLOCK_SIZE	(9:0) : D_FIFO_A2_BLOCK_SIZE	0x0	Size of one block of samples for FIFO A2
X:0x9F2C	D_FIFO_A2_BLOCK_CNT	(9:0) : D_FIFO_A2_BLOCK_CNT	(9:0) : D_FIFO_A2_BLOCK_CNT	0x0	Next read/write location in current sample block
X:0x9F2E	D_FIFO_A2_ACCESS	(23:0) : D_FIFO_A2_ACCESS	(23:0) : D_FIFO_A2_ACCESS	-	Smart FIFO access for FIFO A2
X:0x9F30	D_FIFO_A3_START	(10:5) : D_FIFO_A3_START	(10:5) : D_FIFO_A3_START	0x0	Start offset in A memory for FIFO A3
X:0x9F31	D_FIFO_A3_END	(10:5) : D_FIFO_A3_END	(10:5) : D_FIFO_A3_END	0x0	End offset in A memory for FIFO A3
X:0x9F32	D_FIFO_A3_SIZE	(10:5) : D_FIFO_A3_SIZE	(10:5) : D_FIFO_A3_SIZE	0x0	Total size of A memory used in FIFO A3

Address	Register Name	Register Write	Register Read	Default	Description
X:0x9F34	D_FIFO_A3_BASE_PTR	(10:1) : D_FIFO_A3_BASE_PTR	(10:1) : D_FIFO_A3_BASE_PTR	0x0	Pointer to the base of FIFO A3
X:0x9F35	D_FIFO_A3_IOBLOCK_PTR	(10:1) : D_FIFO_A3_IOBLOCK_PTR	(10:1) : D_FIFO_A3_IOBLOCK_PTR	0x0	Pointer to the latest I/O block in FIFO A3
X:0x9F38	D_FIFO_A3_BLOCK_SIZE	(9:0) : D_FIFO_A3_BLOCK_SIZE	(9:0) : D_FIFO_A3_BLOCK_SIZE	0x0	Size of one block of samples for FIFO A3
X:0x9F3C	D_FIFO_A3_BLOCK_CNT	(9:0) : D_FIFO_A3_BLOCK_CNT	(9:0) : D_FIFO_A3_BLOCK_CNT	0x0	Next read/write location in current sample block
X:0x9F3E	D_FIFO_A3_ACCESS	(23:0) : D_FIFO_A3_ACCESS	(23:0) : D_FIFO_A3_ACCESS	-	Smart FIFO access for FIFO A3
X:0x9F40	D_FIFO_B0_START	(10:5) : D_FIFO_B0_START	(10:5) : D_FIFO_B0_START	0x0	Start offset in B memory for FIFO B0
X:0x9F41	D_FIFO_B0_END	(10:5) : D_FIFO_B0_END	(10:5) : D_FIFO_B0_END	0x0	End offset in B memory for FIFO B0
X:0x9F42	D_FIFO_B0_SIZE	(10:5) : D_FIFO_B0_SIZE	(10:5) : D_FIFO_B0_SIZE	0x0	Total size of B memory used in FIFO B0
X:0x9F44	D_FIFO_B0_BASE_PTR	(10:1) : D_FIFO_B0_BASE_PTR	(10:1) : D_FIFO_B0_BASE_PTR	0x0	Pointer to the base of FIFO B0
X:0x9F45	D_FIFO_B0_IOBLOCK_PTR	(10:1) : D_FIFO_B0_IOBLOCK_PTR	(10:1) : D_FIFO_B0_IOBLOCK_PTR	0x0	Pointer to the latest I/O block in FIFO B0
X:0x9F48	D_FIFO_B0_BLOCK_SIZE	(9:0) : D_FIFO_B0_BLOCK_SIZE	(9:0) : D_FIFO_B0_BLOCK_SIZE	0x0	Size of one block of samples for FIFO B0
X:0x9F4C	D_FIFO_B0_BLOCK_CNT	(9:0) : D_FIFO_B0_BLOCK_CNT	(9:0) : D_FIFO_B0_BLOCK_CNT	0x0	Next read/write location in current sample block
X:0x9F4E	D_FIFO_B0_ACCESS	(23:0) : D_FIFO_B0_ACCESS	(23:0) : D_FIFO_B0_ACCESS	-	Smart FIFO access for FIFO B0
X:0x9F50	D_FIFO_B1_START	(10:5) : D_FIFO_B1_START	(10:5) : D_FIFO_B1_START	0x0	Start offset in B memory for FIFO B1
X:0x9F51	D_FIFO_B1_END	(10:5) : D_FIFO_B1_END	(10:5) : D_FIFO_B1_END	0x0	End offset in B memory for FIFO B1
X:0x9F52	D_FIFO_B1_SIZE	(10:5) : D_FIFO_B1_SIZE	(10:5) : D_FIFO_B1_SIZE	0x0	Total size of B memory used in FIFO B1
X:0x9F54	D_FIFO_B1_BASE_PTR	(10:1) : D_FIFO_B1_BASE_PTR	(10:1) : D_FIFO_B1_BASE_PTR	0x0	Pointer to the base of FIFO B1
X:0x9F55	D_FIFO_B1_IOBLOCK_PTR	(10:1) : D_FIFO_B1_IOBLOCK_PTR	(10:1) : D_FIFO_B1_IOBLOCK_PTR	0x0	Pointer to the latest I/O block in FIFO B1
X:0x9F58	D_FIFO_B1_BLOCK_SIZE	(9:0) : D_FIFO_B1_BLOCK_SIZE	(9:0) : D_FIFO_B1_BLOCK_SIZE	0x0	Size of one block of samples for FIFO B1
X:0x9F5C	D_FIFO_B1_BLOCK_CNT	(9:0) : D_FIFO_B1_BLOCK_CNT	(9:0) : D_FIFO_B1_BLOCK_CNT	0x0	Next read/write location in current sample block
X:0x9F5E	D_FIFO_B1_ACCESS	(23:0) : D_FIFO_B1_ACCESS	(23:0) : D_FIFO_B1_ACCESS	-	Smart FIFO access for FIFO B1
X:0x9F60	D_FIFO_B2_START	(10:5) : D_FIFO_B2_START	(10:5) : D_FIFO_B2_START	0x0	Start offset in B memory for FIFO B2
X:0x9F61	D_FIFO_B2_END	(10:5) : D_FIFO_B2_END	(10:5) : D_FIFO_B2_END	0x0	End offset in B memory for FIFO B2
X:0x9F62	D_FIFO_B2_SIZE	(10:5) : D_FIFO_B2_SIZE	(10:5) : D_FIFO_B2_SIZE	0x0	Total size of B memory used in FIFO B2
X:0x9F64	D_FIFO_B2_BASE_PTR	(10:1) : D_FIFO_B2_BASE_PTR	(10:1) : D_FIFO_B2_BASE_PTR	0x0	Pointer to the base of FIFO B2
X:0x9F65	D_FIFO_B2_IOBLOCK_PTR	(10:1) : D_FIFO_B2_IOBLOCK_PTR	(10:1) : D_FIFO_B2_IOBLOCK_PTR	0x0	Pointer to the latest I/O block in FIFO B2
X:0x9F68	D_FIFO_B2_BLOCK_SIZE	(9:0) : D_FIFO_B2_BLOCK_SIZE	(9:0) : D_FIFO_B2_BLOCK_SIZE	0x0	Size of one block of samples for FIFO B2
X:0x9F6C	D_FIFO_B2_BLOCK_CNT	(9:0) : D_FIFO_B2_BLOCK_CNT	(9:0) : D_FIFO_B2_BLOCK_CNT	0x0	Next read/write location in current sample block

Address	Register Name	Register Write	Register Read	Default	Description
X:0x9F6E	D_FIFO_B2_ACCESS	(23:0) : D_FIFO_B2_ACCESS	(23:0) : D_FIFO_B2_ACCESS	-	Smart FIFO access for FIFO B2
X:0x9F70	D_FIFO_B3_START	(10:5) : D_FIFO_B3_START	(10:5) : D_FIFO_B3_START	0x0	Start offset in B memory for FIFO B3
X:0x9F71	D_FIFO_B3_END	(10:5) : D_FIFO_B3_END	(10:5) : D_FIFO_B3_END	0x0	End offset in B memory for FIFO B3
X:0x9F72	D_FIFO_B3_SIZE	(10:5) : D_FIFO_B3_SIZE	(10:5) : D_FIFO_B3_SIZE	0x0	Total size of B memory used in FIFO B3
X:0x9F74	D_FIFO_B3_BASE_PTR	(10:1) : D_FIFO_B3_BASE_PTR	(10:1) : D_FIFO_B3_BASE_PTR	0x0	Pointer to the base of FIFO B3
X:0x9F75	D_FIFO_B3_IOBLOCK_PTR	(10:1) : D_FIFO_B3_IOBLOCK_PTR	(10:1) : D_FIFO_B3_IOBLOCK_PTR	0x0	Pointer to the latest I/O block in FIFO B3
X:0x9F78	D_FIFO_B3_BLOCK_SIZE	(9:0) : D_FIFO_B3_BLOCK_SIZE	(9:0) : D_FIFO_B3_BLOCK_SIZE	0x0	Size of one block of samples for FIFO B3
X:0x9F7C	D_FIFO_B3_BLOCK_CNT	(9:0) : D_FIFO_B3_BLOCK_CNT	(9:0) : D_FIFO_B3_BLOCK_CNT	0x0	Next read/write location in current sample block
X:0x9F7E	D_FIFO_B3_ACCESS	(23:0) : D_FIFO_B3_ACCESS	(23:0) : D_FIFO_B3_ACCESS	-	Smart FIFO access for FIFO B3

This page left blank for printing purposes.

B Sampling Frequency Reference

Table 35 is a lookup table that provides a summary of the possible sampling frequencies for a set of common SLOWCLK frequencies. All sampling frequency values are provided in Hz.

TABLE 35: SUMMARY OF POSSIBLE SAMPLING FREQUENCIES

Bit-Field Setting For: ADC_CFG_SAMPLE_FREQ	Desired SLOWCLK Frequency:				
	640kHz	1.28MHz	1.92MHz	2.56MHz	3.84MHz
0x00	10000.00	20000.00	30000.00	40000.00	60000.00
0x01	8888.89	17777.78	26666.67	35555.56	53333.33
0x02	8000.00	16000.00	24000.00	32000.00	48000.00
0x03	7272.73	14545.45	21818.18	29090.91	43636.36
0x04	6666.67	13333.33	20000.00	26666.67	40000.00
0x05	6153.85	12307.69	18461.54	24615.38	36923.08
0x06	5714.29	11428.57	17142.86	22857.14	34285.71
0x07	5333.33	10666.67	16000.00	21333.33	32000.00
0x08	5000.00	10000.00	15000.00	20000.00	30000.00
0x09	4705.88	9411.76	14117.65	18823.53	28235.29
0x0A	4444.44	8888.89	13333.33	17777.78	26666.67
0x0B	4210.53	8421.05	12631.58	16842.11	25263.16
0x0C	4000.00	8000.00	12000.00	16000.00	24000.00
0x0D	3809.52	7619.05	11428.57	15238.10	22857.14
0x0E	3636.36	7272.73	10909.09	14545.45	21818.18
0x0F	3478.26	6956.52	10434.78	13913.04	20869.57
0x10	3333.33	6666.67	10000.00	13333.33	20000.00
0x11	3200.00	6400.00	9600.00	12800.00	19200.00
0x12	3076.92	6153.85	9230.77	12307.69	18461.54
0x13	2962.96	5925.93	8888.89	11851.85	17777.78
0x14	2857.14	5714.29	8571.43	11428.57	17142.86
0x15	2758.62	5517.24	8275.86	11034.48	16551.72
0x16	2666.67	5333.33	8000.00	10666.67	16000.00
0x17	2580.65	5161.29	7741.94	10322.58	15483.87
0x18	2500.00	5000.00	7500.00	10000.00	15000.00
0x19	2424.24	4848.48	7272.73	9696.97	14545.45
0x1A	2352.94	4705.88	7058.82	9411.76	14117.65

TABLE 35: SUMMARY OF POSSIBLE SAMPLING FREQUENCIES (CONTINUED)

Bit-Field Setting For: ADC_CFG_SAMPLE_FREQ	Desired SLOWCLK Frequency:				
	640kHz	1.28MHz	1.92MHz	2.56MHz	3.84MHz
0x1B	2285.71	4571.43	6857.14	9142.86	13714.29
0x1C	2222.22	4444.44	6666.67	8888.89	13333.33
0x1D	2162.16	4324.32	6486.49	8648.65	12972.97
0x1E	2105.26	4210.53	6315.79	8421.05	12631.58
0x1F	2051.28	4102.56	6153.85	8205.13	12307.69
0x20	2000.00	4000.00	6000.00	8000.00	12000.00
0x21	1951.22	3902.44	5853.66	7804.88	11707.32
0x22	1904.76	3809.52	5714.29	7619.05	11428.57
0x23	1860.47	3720.93	5581.40	7441.86	11162.79
0x24	1818.18	3636.36	5454.55	7272.73	10909.09
0x25	1777.78	3555.56	5333.33	7111.11	10666.67
0x26	1739.13	3478.26	5217.39	6956.52	10434.78
0x27	1702.13	3404.26	5106.38	6808.51	10212.77
0x28	1666.67	3333.33	5000.00	6666.67	10000.00
0x29	1632.65	3265.31	4897.96	6530.61	9795.92
0x2A	1600.00	3200.00	4800.00	6400.00	9600.00
0x2B	1568.63	3137.25	4705.88	6274.51	9411.76
0x2C	1538.46	3076.92	4615.38	6153.85	9230.77
0x2D	1509.43	3018.87	4528.30	6037.74	9056.60
0x2E	1481.48	2962.96	4444.44	5925.93	8888.89
0x2F	1454.55	2909.09	4363.64	5818.18	8727.27
0x30	1428.57	2857.14	4285.71	5714.29	8571.43
0x31	1403.51	2807.02	4210.53	5614.04	8421.05
0x32	1379.31	2758.62	4137.93	5517.24	8275.86
0x33	1355.93	2711.86	4067.80	5423.73	8135.59
0x34	1333.33	2666.67	4000.00	5333.33	8000.00
0x35	1311.48	2622.95	3934.43	5245.90	7868.85
0x36	1290.32	2580.65	3870.97	5161.29	7741.94
> 0x36	1269.84	2539.68	3809.52	5079.37	7619.05

C Glossary

The following abbreviations and terms are used in this manual:

breakpoints, external

These breakpoints allow the configuration of a breakpoint event based on accesses to data and the value of the written data.

breakpoints, hardware

These breakpoints are configured as pointers to P-Memory. They are triggered when the instruction pointed to by the hardware breakpoint is issued for CFX processor execution.

breakpoints, software

These breakpoints are included in code for the CFX processor as `breakpoint` instructions.

CFX Digital Signal Processor

the BelaSigna 300 processing core that is used to configure the system and coordinate the flow of signal data progressing through the system

CID

chip identifier

CRC

cyclic redundancy code

FIFO

A section of memory that is treated as a circular buffer where components that use this memory use pointers to handle data in a first-in, first-out manner. A *hardware FIFO* is a FIFO instance where its registers are controlled by the IOC. A *software FIFO* is a FIFO instance where its registers are controlled by an application on the CFX or microcode configuration on the HEAR.

GPIO

general purpose input/output

HEAR Configurable Accelerator

the BelaSigna 300 core that is a microcode configurable signal processing engine used to execute a variety of signal processing functions; designed to work with the CFX to reduce the signal processing load on the CFX

I^2C

Industry standardized Inter-IC Communication protocol. Support for a limited subset of this communication interface was called *TWSS* in previous EDKs.

I^2S
Industry standardized Inter-IC Sound protocol

IOC
input/output controller

LSAD
low-speed analog-to-digital

LSB
least significant bit

LUT
look up table

MSB
most significant bit

NVM
non-volatile memory

PCM
pulse code modulation

POR
power-on reset

RAM
random access memory

ROM
read-only memory

SCL
serial clock signal (part of I^2C bus)

SDA
serial data signal (part of I^2C bus)

SPI
serial peripheral interface

TWSS
two wire synchronous serial interface

UART
universal asynchronous receiver-transmitter

This page left blank for printing purposes.

Global Headquarters
ON Semiconductor
5005 East McDowell Road
Phoenix, Arizona 85008 USA
Tel: +1 602 244 6600

Website
www.onsemi.com

Copyright © 2011 Semiconductor Components Industries, LLC d/b/a ON Semiconductor. ON Semiconductor and the ON logo are registered trademarks of Semiconductor Components Industries, LLC (ON Semiconductor). BELASIGNA and EZAIRO are either trademarks or registered trademarks of ON Semiconductor. All other brands, product names and company names mentioned herein may be trademarks or registered trademarks of their respective holders.